
INET Framework Tutorials

Release 4.6.0

Feb 27, 2026

CONTENTS

1	Wireless Tutorial	3
2	IPv4 Network Configurator Tutorial	31
3	Queueing Tutorial	79
4	Regression Testing and Fingerprints Tutorial	141
5	RIP Routing Tutorial	165
6	BGP Routing Tutorial	199
7	OSPF Routing Tutorial	247

The following tutorials explain via step-by-step examples how to use various features of the INET Framework. Each tutorial contains a series of simulation models of increasing complexity.

You can browse the tutorials pages on this web site. The source code of the simulations (NED, ini and other files) and the web pages are in the *tutorials/* subdirectory of the [INET repository](#).

Recently released tutorials:

- [*Regression Testing and Fingerprints Tutorial*](#) (released on: 2020-11-24)

The following tutorials are available:

WIRELESS TUTORIAL

Learn to create wireless simulations using the INET Framework.

Contents:

1.1 Getting Started

In this tutorial, we show you how to build wireless simulations in the INET Framework. The tutorial contains a series of simulation models numbered from 1 through 14. The models are of increasing complexity – they start from the basics, and in each step, they introduce new INET features and concepts related to wireless communication networks.

In the tutorial, each step is a separate configuration in the same `omnetpp.ini` file. Steps build on each other; they extend the configuration of the previous step by adding a few new lines. Consecutive steps mostly share the same network, defined in NED.

This is an advanced tutorial, and it assumes that you are familiar with creating and running simulations in OMNeT++ and INET. If you aren't, you can check out the TicToc Tutorial to get started with using OMNeT++.

1.1.1 Try It Yourself

If you already have INET and OMNeT++ installed, start the IDE by typing `omnetpp`, import the INET project into the IDE, then navigate to the `inet/tutorials/wireless` folder in the *Project Explorer*. There, you can view and edit the tutorial files, run simulations, and analyze results.

Otherwise, there is an easy way to install INET and OMNeT++ using `opp_env`, and run the simulation interactively. Ensure that `opp_env` is installed on your system, then execute:

```
$ opp_env run inet-4.6 --init -w inet-workspace --install --build-modes=release --  
↪ chdir \  
-c 'cd inet-4.6.*/tutorials/wireless && inet'
```

This command creates an `inet-workspace` directory, installs the appropriate versions of INET and OMNeT++ within it, and launches the `inet` command in the `showcase` directory for interactive simulation.

Alternatively, for a more hands-on experience, you can first set up the workspace and then open an interactive shell:

```
$ opp_env install --init -w inet-workspace --build-modes=release inet-4.6  
$ cd inet-workspace  
$ opp_env shell
```

Inside the shell, start the IDE by typing `omnetpp`, import the INET project, then start exploring.

1.1.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.2 Step 1. Two hosts communicating wirelessly

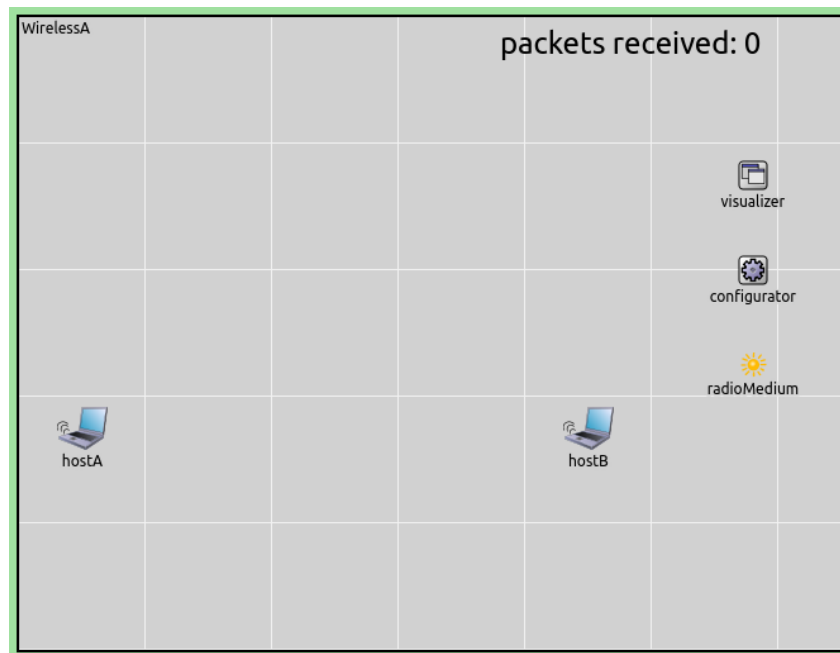
1.2.1 Goals

In the first step, we want to create a network that contains two hosts, with one host sending a UDP data stream wirelessly to the other. Our goal is to keep the physical layer and lower layer protocol models as simple as possible.

We'll make the model more realistic in later steps.

1.2.2 The model

In this step, we'll use the model depicted below.



Here is the NED source of the network:

```
network WirelessA
{
    parameters:
        @display("bgb=650,500;bpg=100,1,gre95");
        @figure[title](type=label; pos=0,-1; anchor=sw; color=darkblue);

        @figure[rcvdPkText](type=indicatorText; pos=380,20; anchor=w; font=,18;
↪textFormat="packets received: %g"; initialValue=0);
        @statistic[packetReceived](source=hostB.app[0].packetReceived;
↪record=figure(count); targetFigure=rcvdPkText);

    submodules:
        visualizer: <default(firstAvailableOrEmpty("IntegratedCanvasVisualizer"))>
↪like IIntegratedVisualizer if typename != "" {
            @display("p=580,125");
        }
        configurator: Ipv4NetworkConfigurator {
            @display("p=580,200");
        }
        radioMedium: RadioMedium {
            @display("p=580,275");
        }
}
```

(continues on next page)

(continued from previous page)

```

hostA: <default("WirelessHost")> like INetworkNode {
    @display("p=50,325");
}
hostB: <default("WirelessHost")> like INetworkNode {
    @display("p=450,325");
}
}

```

We'll explain the above NED file below.

The playground

The model contains a playground of the size 500x650 meters, with two hosts spaced 400 meters apart. (The distance will be relevant in later steps.) These numbers are set via display strings.

The modules that are present in the network in addition to the hosts are responsible for tasks like visualization, configuring the IP layer, and modeling the physical radio channel. We'll return to them later.

The hosts

In INET, hosts are usually represented with the `StandardHost` NED type, which is a generic template for TCP/IP hosts. It contains protocol components like TCP, UDP and IP, slots for plugging in application models, and various network interfaces (NICs). `StandardHost` has some variations in INET, for example, `WirelessHost`, which is basically a `StandardHost` preconfigured for wireless scenarios.

As you can see, the hosts' type is parametric in this NED file (defined via a `hostType` parameter and the `INetworkNode` module interface). This is done so that in later steps we can replace hosts with a different NED type. The actual NED type here is `WirelessHost` (given near the top of the NED file), and later steps will override this setting using `omnetpp.ini`.

Address assignment

IP addresses are assigned to hosts by an `Ipv4NetworkConfigurator` module, which appears as the `configurator` submodule in the network. The hosts also need to know each others' MAC addresses to communicate, which in this model is taken care of by using per-host `GlobalArp` modules instead of real ARP.

Traffic model

In the model, host A generates UDP packets that are received by host B. To this end, host A is configured to contain a `UdpBasicApp` module, which generates 1000-byte UDP messages at random intervals with exponential distribution, the mean of which is 12ms. Therefore the app is going to generate 100 kbyte/s (800 kbps) UDP traffic, not counting protocol overhead. Host B contains a `UdpSink` application that just discards received packets.

The model also displays the number of packets received by host B. The text is added by the `@figure[rcvdPkText]` line, and the subsequent line arranges the figure to be updated during the simulation.

Physical layer modeling

Let us concentrate on the module called `radioMedium`. All wireless simulations in INET need a radio medium module. This module represents the shared physical medium where communication takes place. It is responsible for taking signal propagation, attenuation, interference, and other physical phenomena into account.

INET can model the wireless physical layer at various levels of detail, realized with different radio medium modules. In this step, we use `UnitDiskRadioMedium`, which is the simplest model. It implements a variation of unit disc radio, meaning that physical phenomena like signal attenuation are ignored, and the communication range is simply specified in meters. Transmissions within range are always correctly received unless collisions occur. Modeling collisions (overlapping transmissions causing reception failure) and interference range (a range where the signal cannot be received correctly, but still collides with other signals causing their reception to fail) are optional.

NOTE: Naturally, this model of the physical layer has little correspondence to reality. However, it has its uses in the simulation. Its simplicity and consequent predictability are an advantage in scenarios where realistic modeling of

the physical layer is not a primary concern, for example in the modeling of ad-hoc routing protocols. Simulations using `UnitDiskRadioMedium` also run faster than more realistic ones, due to the low computational cost.

In hosts, network interface cards are represented by NIC modules. Radio is part of wireless NIC modules. There are various radio modules, and one must always use one that is compatible with the medium module. In this step, hosts contain `GenericUnitDiskRadio` as part of `AckingWirelessInterface`.

In this model, we configure the chosen physical layer model (`UnitDiskRadioMedium` and `GenericUnitDiskRadio`) as follows. The communication range is set to 500m. Modeling packet losses due to collision (termed “interference” in this model) is turned off, resulting in pairwise independent duplex communication channels. The radio data rates are set to 1 Mbps. These values are set in `omnetpp.ini` with the `communicationRange`, `ignoreInterference`, and `bitrate` parameters of the appropriate modules.

MAC layer

NICs modules also contain an L2 (i.e. data link layer) protocol. The MAC protocol in `AckingWirelessInterface` is configurable, the default choice being `MultipleAccessMac`. `MultipleAccessMac` implements a trivial MAC layer which only provides encapsulation/decapsulation but no real medium access protocol. There is virtually no medium access control: packets are transmitted as soon as the previous packet has completed transmission. `MultipleAccessMac` also contains an optional out-of-band acknowledgment mechanism which we turn off here.

The configuration:

```
[Config Wireless01]
description = Two hosts communicating wirelessly
network = WirelessA
sim-time-limit = 20s

*.radioMedium.signalAnalogRepresentation = "unitDisk"
*.host*.ipv4.arp.typename = "GlobalArp"

*.hostA.numApps = 1
*.hostA.app[0].typename = "UdpBasicApp"
*.hostA.app[0].destAddresses = "hostB"
*.hostA.app[0].destPort = 5000
*.hostA.app[0].messageLength = 1000B
*.hostA.app[0].sendInterval = exponential(12ms)
*.hostA.app[0].packetName = "UDPData"

*.hostB.numApps = 1
*.hostB.app[0].typename = "UdpSink"
*.hostB.app[0].localPort = 5000

*.host*.wlan[0].typename = "AckingWirelessInterface"
*.host*.wlan[0].mac.useAck = false
*.host*.wlan[0].mac.fullDuplex = false
*.host*.wlan[0].radio.signalAnalogRepresentation = "unitDisk"
*.host*.wlan[0].radio.transmitter.analogModel.communicationRange = 500m
*.host*.wlan[0].radio.receiver.ignoreInterference = true
*.host*.wlan[0].mac.headerLength = 23B

*.host*.*.bitrate = 1Mbps
```

1.2.3 Results

When we run the simulation, here’s what happens. Host A’s `UdpBasicApp` generates UDP packets at random intervals. These packets are sent down via UDP and IPv4 to the network interface for transmission. The network interface queues packets and transmits them as soon as it can. As long as there are packets in the network interface’s transmission queue, packets are transmitted back-to-back, with no gaps between subsequent packets.

These events can be followed on OMNeT++'s Qtenv runtime GUI. The following image has been captured from Qtenv, and shows the inside of host A during the simulation. One can see a UDP packet being sent down from the `udpApp` submodule, traversing the intermediate protocol layers, and being transmitted by the `wlan` interface.

The next animation shows the communication between the hosts, using OMNeT++'s default "message sending" animation.

When the simulation concludes at $t=25s$, the packet count meter indicates that around 2000 packets were sent. A packet with overhead is 1028 bytes, which means the transmission rate was around 660 kbps.

Number of packets received by host B: 2017

Sources: `omnetpp.ini`, `WirelessA.ned`

1.2.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.3 Step 2. Setting up some animations

1.3.1 Goals

To facilitate understanding, we will visualize certain aspects of the simulation throughout this tutorial. In this step, we will focus on the physical layer, most notably radio transmissions and signal propagation.

1.3.2 The model

Visualization support in INET is implemented as separate modules that are optional parts of a network model. There are several kinds of visualizers responsible for showing various aspects of the simulation. Visualizers are parameterizable, and some visualizers are themselves composed of several optional components.

The `visualizer` submodule in this network is an `IntegratedCanvasVisualizer`, which is a compound module that contains all typically useful visualizers as submodules. It can display physical objects in the physical environment, movement trail, discovered network connectivity, discovered network routes, ongoing transmissions, ongoing receptions, propagating radio signals, statistics, and more.

We enable several kinds of visualizations: communication range, signal propagation, and recent successful physical layer transmissions.

The visualization of communication range is enabled using the `displayCommunicationRange` parameter of the radio module in host A. It displays a circle around the host, which represents the maximum distance where successful transmission is still possible with some hosts in the network.

The visualization of signal propagation is enabled with the `displaySignals` parameter of `MediumCanvasVisualizer`. It displays transmissions as colored rings emanating from hosts. Since this is sufficient to represent radio signals visually, it is advisable to turn off message animations in the Tkenv/Qtdev preferences dialog.

The visualization of recent successful physical layer transmissions is enabled with the `displayLinks` parameter of the `physicalLinkVisualizer` submodule. Additionally, its `packetFilter` parameter is set to only display packets whose names begin with "UDPData". Matching successful transmissions are displayed with dotted dark yellow arrows that fade with time. When a packet is successfully received by the physical layer, the arrow between the transmitter and receiver hosts is created or reinforced. The arrows visible at any given time indicate recent successful communication patterns.

Configuration:

```
[Config Wireless02]
description = Setting up some animations
extends = Wireless01

*.hostA.wlan[0].radio.displayCommunicationRange = true
```

(continues on next page)

(continued from previous page)

```

*.visualizer.sceneVisualizer.descriptionFigure = "title"

*.visualizer.mediumVisualizer.displaySignals = true

*.visualizer.physicalLinkVisualizer.displayLinks = true
*.visualizer.physicalLinkVisualizer.packetFilter = "UDPData*"

```

1.3.3 Results

The most notable change is the bubble animations representing radio signals. Each transmission starts with displaying a growing colored disk centered at the transmitter. The outer edge of the disk indicates the propagation of the radio signal's first bit. When the transmission ends, the disk becomes a ring and the inner edge appears at the transmitter. The growing inner edge of the ring indicates the propagation of the radio signal's last bit. The reception starts when the outer edge reaches the receiver, and it finishes when the inner edge arrives.

Note that when the inner edge appears, the outer edge is already far away from the transmitter. The explanation for that is that in this network, transmission durations are much longer than propagation times.

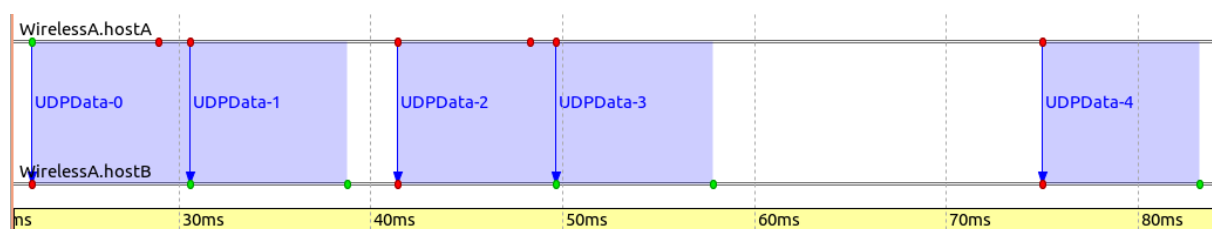
The UDP application generates packets at a rate so that there are back-to-back transmissions. Back-to-back means the first bit of a transmission immediately follows the last bit of the previous transmission. This can be seen in the animation, as there are no gaps between the colored transmission rings. Sometimes the transmission stops for a while, indicating that the transmission queue became empty.

The blue circle around host A depicts the communication range, and it clearly shows that host B is within the range, therefore successful communication is possible.

The dotted arrow that appears between host A and host B indicates successful communication at the physical layer. The arrow is created after a packet reception is successfully completed, when the packet is passed up to the link layer. The arrow is displayed when the reception of the first packet at host B is over.

Frame exchanges may also be visualized using the Sequence Chart tool in the OMNeT++ IDE. The following image was obtained by recording an event log (.elog) file from the simulation, opening it in the IDE, and tweaking the settings in the Sequence Chart tool.

The transmission of packet UDPData-0 starts at around 23ms and completes at around 30ms. The signal propagation takes a nonzero amount of time, but it's such a small value compared to the transmission duration that it's not visible in this image. (The arrow signifying the beginning of the transmission appears to be vertical, one needs to zoom in along the time axis to see that in fact, it is not. In a later step, we will see that it is possible to configure the Sequence Chart tool to represent time in a non-linear way.) The chart also indicates that UDPData-0 and UDPData-1 are transmitted back-to-back because there's no gap between them. UDPData-2 and UDPData-3 are also transmitted back-to-back.



Number of packets received by host B: 2017

Sources: omnetpp.ini, WirelessA.ned

1.3.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.4 Step 3. Adding more nodes and decreasing the communication range

1.4.1 Goals

Later in this tutorial, we'll want to turn our model into an ad-hoc network and experiment with routing. To this end, we add three more wireless nodes and reduce the communication range so that our two original hosts cannot reach one another directly. In later steps, we'll set up routing and use the extra nodes as relays.

1.4.2 The model

We need to add three more hosts. This could be done by copying and editing the network used in the previous steps, but instead, we extend `WirelessA` into `WirelessB` using the inheritance feature of NED:

```
network WirelessB extends WirelessA
{
    submodules:
        hostR1: <default("WirelessHost")> like INetworkNode {
            @display("p=250,300");
        }
        hostR2: <default("WirelessHost")> like INetworkNode {
            @display("p=150,450");
        }
        hostR3: <default("WirelessHost")> like INetworkNode {
            @display("p=350,450");
        }
}
```

We decrease the communication range of the radios of all hosts to 250 meters. This will make direct communication between hosts A and B impossible because their distance is 400 meters. The recently added hosts are in the correct positions to relay data between hosts A and B, but routing is not yet configured. The result is that hosts A and B will not be able to communicate at all.

The configuration:

```
[Config Wireless03]
description = Adding more nodes and decreasing the communication range
extends = Wireless02
network = WirelessB

*.host*.wlan[0].radio.transmitter.analogModel.communicationRange = 250m
*.hostR1.wlan[0].radio.displayCommunicationRange = true
```

1.4.3 Results

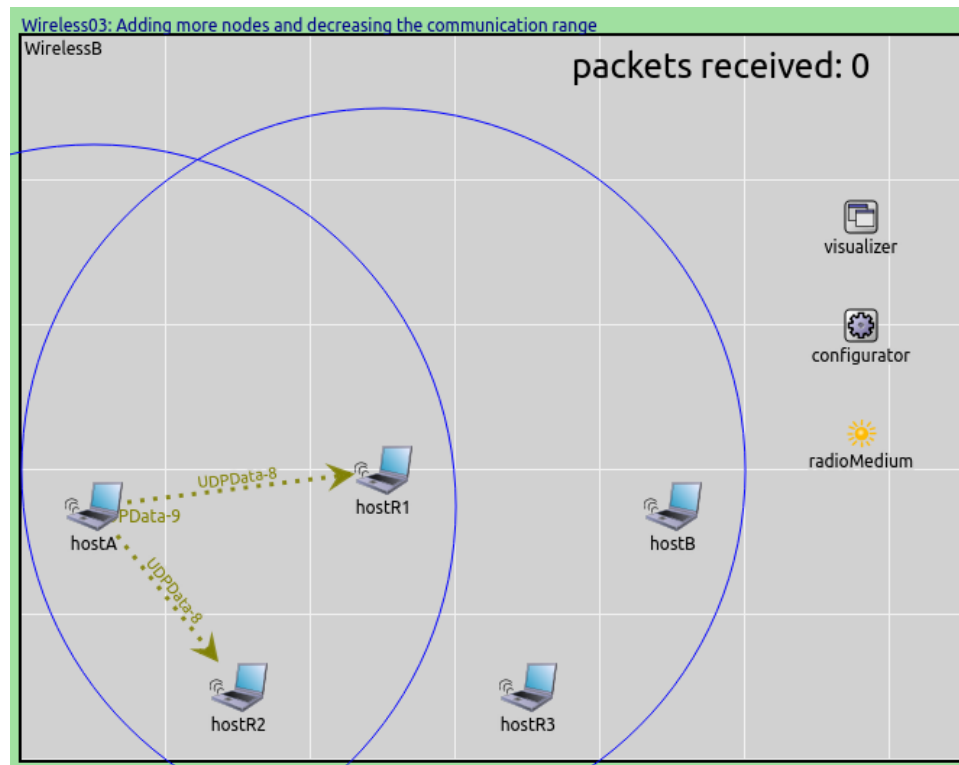
When we run the simulation, blue circles confirm that hosts R1 and R2 are the only hosts in the communication range of host A. Therefore, they are the only ones that receive host A's transmissions. This is indicated by the dotted arrows connecting host A to R1 and R2, respectively, representing recent successful receptions in the physical layer.

Host B is in the transmission range of host R1, and R1 could potentially relay A's packets, but it drops them because routing is not configured yet (it will be configured in a later step). Therefore, no packets are received by host B.

Host R1's MAC submodule logs indicate that it is discarding the received packets, as they are not addressed to it:

Number of packets received by host B: 0

Sources: omnetpp.ini, WirelessB.ned



```

** Event #14974 t=3.518101302943 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-290 (inet::IdealMacFrame, id=9439)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 14974 3.518101302943: Frame 'UDPData-290' not destined to us, discarding
** Event #15018 t=3.526325302943 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-291 (inet::IdealMacFrame, id=9471)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 15018 3.526325302943: Frame 'UDPData-291' not destined to us, discarding
** Event #15054 t=3.534549302943 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-292 (inet::IdealMacFrame, id=9480)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 15054 3.534549302943: Frame 'UDPData-292' not destined to us, discarding
** Event #15124 t=3.575921979275 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-293 (inet::IdealMacFrame, id=9512)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 15124 3.575921979275: Frame 'UDPData-293' not destined to us, discarding
** Event #15193 t=3.614801534878 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-294 (inet::IdealMacFrame, id=9544)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 15193 3.614801534878: Frame 'UDPData-294' not destined to us, discarding
** Event #15262 t=3.643246185845 WirelessB.hostR1.wlan[0].mac (IdealMac, id=114) on UDPData-295 (inet::IdealMacFrame, id=9576)
INFO (IdealMac)WirelessB.hostR1.wlan[0].mac 15262 3.643246185845: Frame 'UDPData-295' not destined to us, discarding

```

1.4.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.5 Step 4. Setting up static routing

1.5.1 Goals

In this step, we set up routing so that packets can flow from host A to B. For this to happen, the intermediate nodes will need to act as routers. As we still want to keep things simple, we'll use statically added routes that remain unchanged throughout the simulation.

We also configure visualization so that we can see the paths packets take when traveling from host A to B.

1.5.2 The model

Setting up routing

For the recently added hosts to act as routers, IPv4 forwarding needs to be enabled. This can be done by setting the `forwarding` parameter of `StandardHost`.

We also need to set up static routing. Static configuration in the INET Framework is often done by configurator modules. Static IPv4 configuration, including address assignment and adding routes, is usually done using the `Ipv4NetworkConfigurator` module. The model already has an instance of this module, the `configurator` submodule. The configurator can be configured using an XML specification and some additional parameters. Here, the XML specification is provided as a string constant inside the ini file.

Without going into details about the contents of the XML configuration string and other configurator parameters, we tell the configurator to assign IP addresses in the 10.0.0.x range, and to create routes based on the estimated packet error rate of links between the nodes. (The configurator looks at the wireless network as a full graph. Links with high error rates will have high costs, and links with low error rates will have low costs. Routes are formed such as to minimize their costs. In the case of the `GenericUnitDiskRadio` model, the error rate is 1 for nodes that are out of range and a very small value for ones in range. The result will be that nodes that are out of range of each other will send packets to intermediate nodes that can forward them.)

Visualization

The `IntegratedCanvasVisualizer` we use as the `visualizer` submodule in this network contains a `networkRouteVisualizer` module, which is able to render packet paths. This module displays paths where a packet has been recently sent between the network layers of the two end hosts. The path is displayed as a colored arrow that goes through the visited hosts. The path continually fades, and then it disappears after a certain amount of time unless it is reinforced by another packet.

The network route visualizer is activated by setting its `displayRoutes` parameter to `true`. Its `packetFilter` parameter specifies which packets it should take into account. By default, it is set to `*`, which means all packets. Our UDP application generates packets with the name `UDPData-0`, `UDPData-1`, etc, so we set the packet filter to `UDPData*` in order to filter out other types of packets that will appear in later steps.

Configuration:

```
[Config Wireless04]
description = Setting up static routing
extends = Wireless03

*.host*.forwarding = true

*.configurator.config = xml("<config><interface hosts='**' address='10.0.0.x' netmask=
↪ '255.255.255.0' /><autoroute metric='errorRate' /></config>")
*.configurator.optimizeRoutes = false
*.host*.ipv4.routingTable.netmaskRoutes = ""
```

(continues on next page)

(continued from previous page)

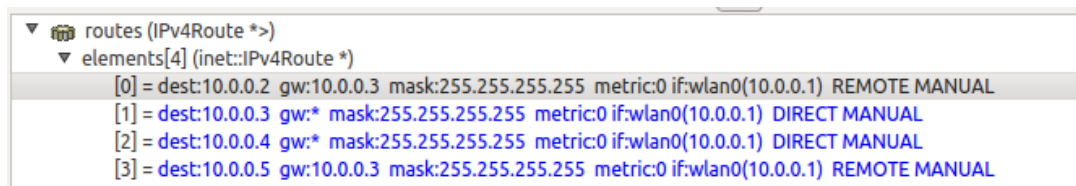
```

*.visualizer.physicalLinkVisualizer.displayLinks = true
*.visualizer.dataLinkVisualizer.displayLinks = true
*.visualizer.networkRouteVisualizer.displayRoutes = true
*.visualizer.*LinkVisualizer.lineShift = 0
*.visualizer.networkRouteVisualizer.lineShift = 0
*.visualizer.networkRouteVisualizer.packetFilter = "UDPData*"

```

1.5.3 Results

Routing tables are stored in the `routingTable` submodules of hosts, and can be inspected in the runtime GUI. The routing table of host A (10.0.0.1) can be seen in the following image. It tells that host B (10.0.0.2) can be reached via host R1 (10.0.0.3), as specified by the gateway (gw) value.



When the first packet sent by host A arrives at host R1, a dotted dark yellow arrow appears between the two hosts indicating a successful physical layer exchange, as it was noted earlier. A few events later but still at the same simulation time, a cyan-colored arrow appears on top of the dotted one. The cyan arrow represents a successful exchange between the two data link layers of the same hosts. As opposed to the previous step, this happens because according to the routing table of host A, a packet destined to host B, has to be sent to host R1 (the gateway). As the packet reaches the network layer of host R1, it is immediately routed according to the routing table of this host directly towards host B. So when the first packet arrives at host B, first a dotted arrow appears, then a cyan arrow appears on top of that, similarly to the host R1 case. Still at the same simulation time, the packet leaves the network layer of host B towards the UDP protocol component. At this moment a new polyline arrow appears between host A and host B going through host R1. This blue arrow represents the route the packet has taken from first entering the network layer at host A until it left the network layer at host B.

Note that there are dotted arrows leading to host R2 and R3 even though they don't transmit. This is because they receive the transmissions at the physical layer, but they discard the packets at the link layer because it is not addressed to them.

Note that the number of packets received by host B has dropped to about half of what we saw in step 2. This is so because R1's NIC operates in half-duplex mode (it can only transmit or receive at any time, but not both), so it can only relay packets at half the rate that host A emits.

Number of packets received by host B: 1125

Sources: `omnetpp.ini`, `WirelessB.ned`

1.5.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.6 Step 5. Taking interference into account

1.6.1 Goals

In this step, we make our model of the physical layer a little bit more realistic. First, we turn on interference modeling in the unit disc radio. By interference, we mean that if two signals collide (arrive at the receiver at the same time), they will become garbled and the reception will fail. (Remember that so far we were essentially modeling pairwise duplex communication.)

Second, we'll set the interference range of the unit disc radio to be 500m, twice as much as the communication range. The interference range parameter acknowledges the fact that radio signals become weaker with distance,

and there is a range where they can no longer be received correctly, but they are still strong enough to interfere with other signals, that is, they can cause the reception to fail. (For completeness, there is a third range called detection range, where signals are too weak to cause interference, but can still be detected by the receiver.)

Of course, this change reduces the throughput of the communication channel, so we expect the number of packets that go through to drop.

1.6.2 The model

To turn on interference modeling, we set the `ignoreInterference` parameter in the receiver part of the `GenericUnitDiskRadio` module to `false`. Interference range is defined by the `interferenceRange` parameter of the `GenericUnitDiskRadio` module's transmitter part, so we set that to 500m.

We expect that although host B will not be able to receive host A's transmissions, those transmissions will still cause interference with other transmissions (e.g., R1's) at host B.

Regarding visualization, we turn off the arrows indicating successful data link layer receptions because in our case, they do not add much value beyond displaying the network routes.

```
[Config Wireless05]
description = Taking interference into account
extends = Wireless04

*.host*.wlan[0].radio.receiver.ignoreInterference = false
*.host*.wlan[0].radio.transmitter.analogModel.interferenceRange = 500m

*.hostA.wlan[0].radio.displayInterferenceRange = true

*.visualizer.dataLinkVisualizer.packetFilter = ""
```

1.6.3 Results

Host A is transmitting packets generated by its UDP Application module. R1 is at the right position to act as a relay between A and B, and it retransmits A's packets to B as soon as they are received. Most of the time, A and R1 are transmitting simultaneously, causing two signals to be present at the receiver of host B, which results in collisions. When the gap between host A's two successive transmissions is large enough, R1 can transmit a packet to B without collision.

When the packet `UDPData-54` from host A arrives at host R1, it gets routed towards host B, so host R1 immediately starts transmitting it. Unfortunately, at the same time, host A is already transmitting the next packet (`UDPData-55`) back-to-back with the previous one. Thus, the reception of `UDPData-54` from host R1 at host B fails due to a collision. The colliding packets are `UDPData-55` from host A and `UDPData-54` from host R1. This is indicated by the lack of appearance of a blue polyline arrow between host A and host B.

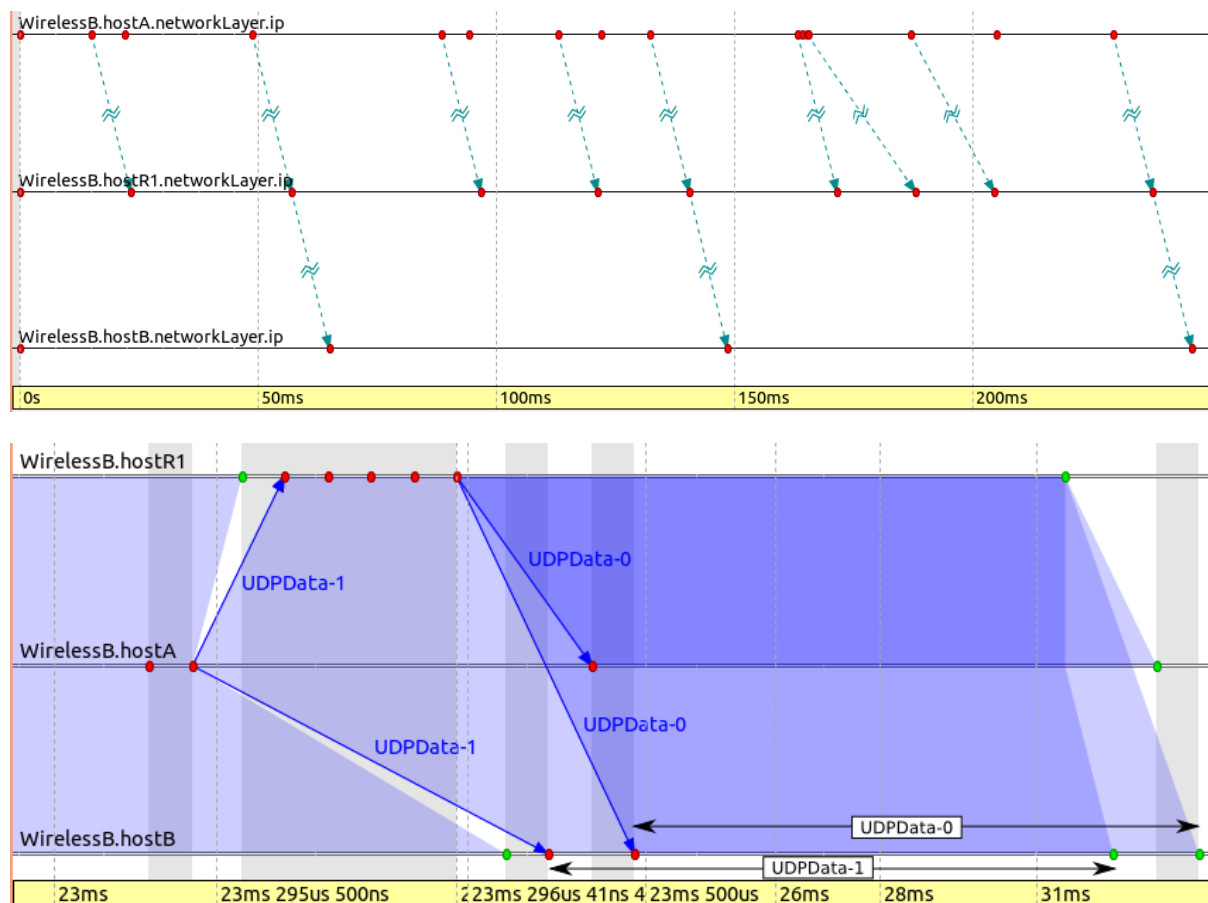
Luckily, the next packet (`UDPData-56`) from host A is not part of a back-to-back transmission. When it gets routed and retransmitted at host R1, there's no other transmission from host A to collide with at host B. Interestingly, the first bit of the transmission from host R1 occurs just after the last bit of the transmission from host A. This happens because the processing, including the routing decision, at host R1 takes zero amount of time. The successful transmission is indicated by the appearance of the blue polyline arrow between host A and host B.

This is shown in the following animation:

As expected, the number of packets received by host B is low. The following sequence chart illustrates packet traffic between hosts A, R1, and B's network layer. The image indicates that host B only occasionally receives packets successfully; most packets sent by R1 do not make it to host B's IP layer.

The following sequence chart shows host R1's and host A's signals overlapping at host B.

NOTE: On this and all other sequence charts in this tutorial, grey vertical strips are constant-time zones: all events within the same grey area have the same simulation time. Also, time is mapped to the horizontal axis using a nonlinear transformation so that both small and large time intervals can be depicted on the same chart. A larger distance along the horizontal axis does not necessarily correspond to a larger simulation time interval because the more events there are in an interval, the more it is inflated to make the events visible and discernible on the chart.



To minimize interference, some kind of media access protocol is needed to govern which host can transmit and when.

Number of packets received by host B: 183

Sources: `omnetpp.ini`, `WirelessB.ned`

1.6.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.7 Step 6. Using CSMA to better utilize the medium

1.7.1 Goals

In this step, we try to increase the utilization of the communication channel by choosing a medium access control (MAC) protocol that is better suited for wireless communication.

In the previous step, nodes transmitted on the channel immediately when they had something to send, without first listening for ongoing transmissions. This resulted in a lot of collisions and lost packets. We improve the communication by using the CSMA protocol, which is based on the “sense before transmit” (or “listen before talk”) principle.

CSMA (carrier sense multiple access) is a probabilistic MAC protocol in which a node verifies the absence of other traffic before transmitting on the shared transmission medium. CSMA has several variants; we’ll use CSMA/CA (where CA stands for collision avoidance). In this protocol, a node that has data to send first waits for the channel to become idle, and then it also waits for a random backoff period. If the channel is still idle during the backoff, the node can actually start transmitting. Otherwise, the procedure starts over, possibly with an updated range for the backoff period. We expect that the use of CSMA will improve throughput, as there will be fewer collisions, and the medium will be utilized better.

1.7.2 The model

To use CSMA, we need to replace `AckingWirelessInterface` in the hosts with `WirelessInterface`. `WirelessInterface` is a generic NIC with both the radio and the MAC module left open, so we specify `GenericUnitDiskRadio` for its `radioType` parameter, and `CsmaCaMac` for `macType`.

The `CsmaCaMac` module implements CSMA/CA with optional acknowledgments and a retry mechanism. It has a number of parameters for tweaking its operation. With the appropriate parameters, it can approximate basic 802.11b ad-hoc mode operation. Parameters include:

- acknowledgments on/off
- bit rate (this is used for both data and ACK frames)
- protocol overhead: MAC header length, ACK frame length
- backoff parameters: minimum/maximum contention window (in slots), slot time, maximum retry count
- timing: interval to wait before transmitting ACK frame (SIFS) and before data frames in addition to the backoff slots (DIFS)

For now, we do not use an acknowledgment (sending of ACK packets), so we can see purely the effect of “listen before talk” and waiting a random backoff period before each transmission (this is the default behavior for the MAC). In the absence of ACKs, the MAC has to assume that all its transmissions are successful, so no frame is ever retransmitted.

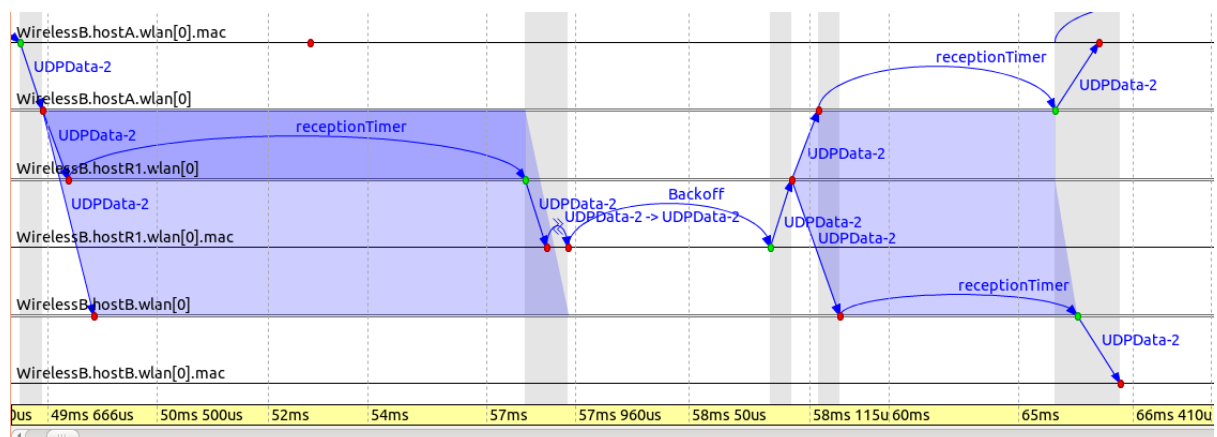
```
[Config Wireless06]
description = Using CSMA to better utilize the medium
extends = Wireless05

*.host*.wlan[0].typename = "WirelessInterface"
*.host*.wlan[0].radio.typename = "GenericRadio"
*.host*.wlan[0].mac.typename = "CsmaCaMac"
*.host*.wlan[0].mac.ackTimeout = 300us
*.host*.wlan[0].queue.typename = "DropTailQueue"
*.host*.wlan[0].queue.packetCapacity = -1
```

1.7.3 Results

The effect of CSMA can be seen in the animation below. The first two packets are sent by host A, and after waiting for a backoff period, host R1 retransmits both packets. This time, host B receives them correctly, because only host R1 is transmitting.

The following sequence chart displays that after receiving the `UDPData-2` packet, host R1 transmits it after the backoff period timer has expired.



It is already apparent by watching the simulation that there are much fewer collisions this time. The numbers also confirm that CSMA has worked: nearly eight times as many packets are received by host B than in the previous

step.

Number of packets received by host B: 1374

Sources: omnetpp.ini, WirelessB.ned

1.7.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.8 Step 7. Turning on ACKs in CSMA

1.8.1 Goals

In this step, we try to make link layer communication more reliable by adding acknowledgments to the MAC protocol.

1.8.2 The model

We turn on acknowledgments by setting the `useAcks` parameter of `CsmaCaMac` to `true`. This change will make the operation of the MAC module both more interesting and more complicated.

On the receiver side, the change is quite simple: when the MAC correctly receives a data frame addressed to it, it responds with an ACK frame after a fixed-length gap (SIFS). If the originator of the data frame does not receive the ACK correctly within due time, it will initiate a retransmission. The contention window (from which the random backoff period is drawn) will be doubled for each retransmission until it reaches the maximum (and then it will stay constant for further retransmissions). After a given number of unsuccessful retries, the MAC will give up, discard the data frame, and will take the next data frame from the queue. The next frame will start with a clean slate (i.e. the contention window and the retry count will be reset).

This operation roughly corresponds to the basic IEEE 802.11b MAC ad-hoc mode operation.

Note that when ACKs (in contrast to data frames) are lost, retransmissions will introduce duplicates in the packet stream the MAC sends up to the higher layer protocols in the receiver host. This could be eliminated by adding sequence numbers to frames and maintaining per-sender sequence numbers in each receiver, but the `CsmaCaMac` module does not contain such a duplicate detection algorithm in order to keep its code simple and accessible.

Another detail worth mentioning is that when a frame exceeds the maximum number of retries and is discarded, the MAC emits a link break signal. This signal may be interpreted by routing protocols such as AODV as a sign that a route has become broken, and a new one needs to be found.

```
[Config Wireless07]
description = Turning on ACKs in CSMA
extends = Wireless06

*.host*.wlan[0].mac.useAck = true
```

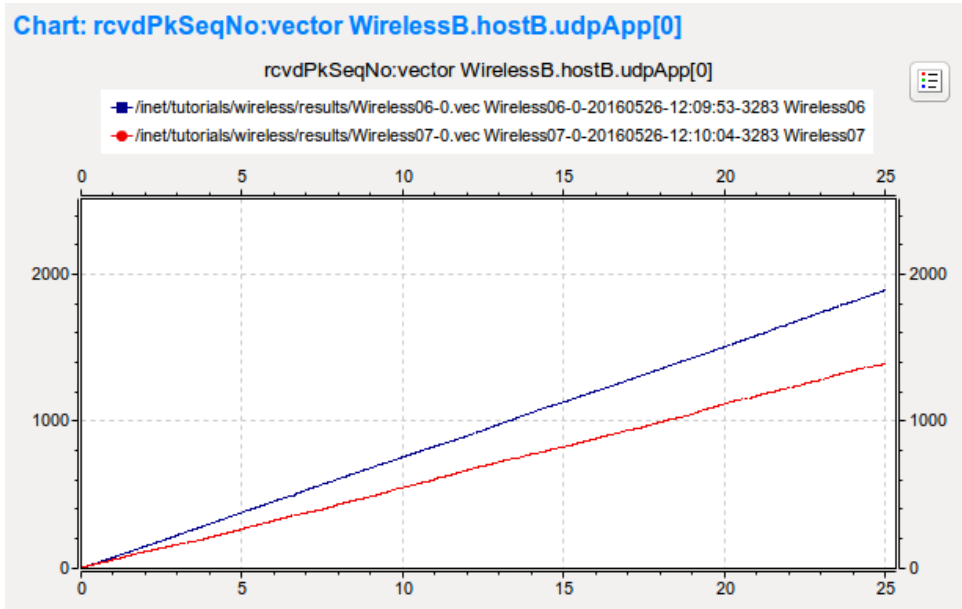
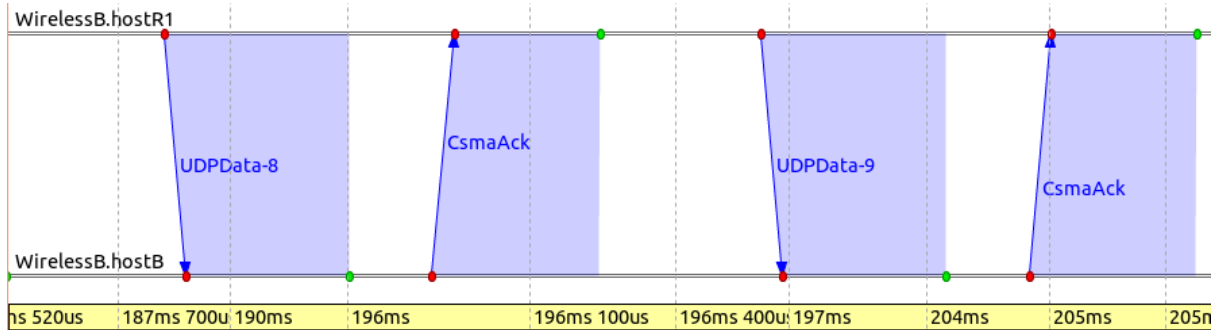
1.8.3 Results

With ACKs enabled, each successfully received packet is acknowledged. The animation below depicts a packet transmission from host R1, and the corresponding ACK from host B.

The UDPData + ACK sequences can be seen in the sequence chart below:

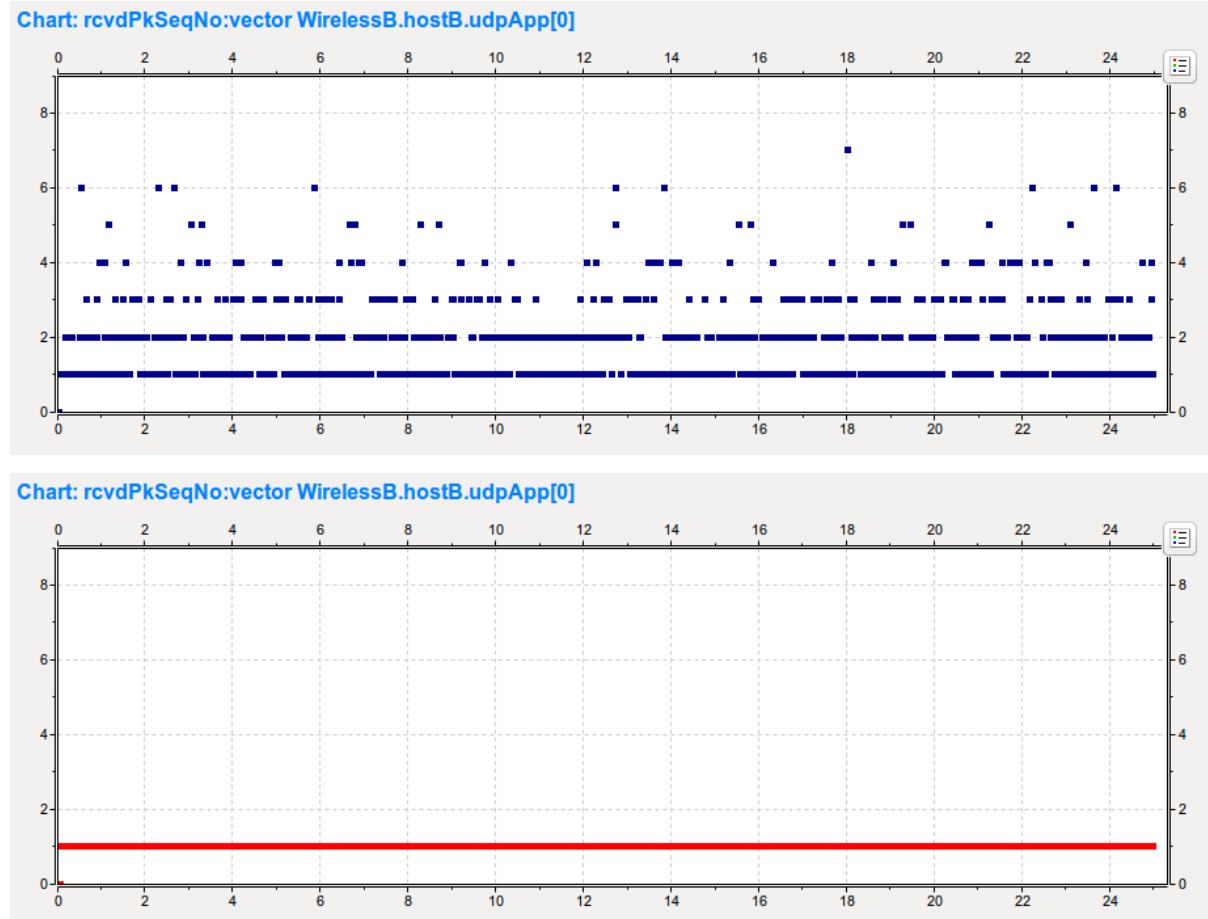
In the following chart, the UDPData packet sequence numbers that are received by host B's UDPApp, are plotted against time. This chart contains the statistics of the previous step (ACK off, blue) and the current step (ACK on, red).

When ACKs are turned on, each successfully received UDPData packet has to be acknowledged before the next one can be sent. Lost packets are retransmitted until an ACK arrives, implementing a kind of reliable transport. Because of this, the sequence numbers are sequential. In the case of ACKs turned off, lost packets are not retransmitted,



which results in gaps in the sequence numbers. The blue curve is steeper because the sequence numbers grow more rapidly. This is in contrast to the red curve, where the sequence numbers grow only by one.

The next two charts display the difference between the subsequently received UDPData packet sequence numbers. The first one (blue) is for the previous step. The difference is mostly 1, which means the packets are received sequentially. However, often there are gaps in the sequence numbers, signifying lost packets. The second one (red) is for the current step, where the difference is always 1, as there are no lost packets.



Number of packets received by host B: 1393

Sources: omnetpp.ini, WirelessB.ned

1.8.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.9 Step 8. Modeling energy consumption

1.9.1 Goals

Wireless ad-hoc networks often operate in an energy-constrained environment, and thus it is useful to be able to model the energy consumption of the devices. Consider, for example, wireless motes that operate on battery. The mote's activity has to be planned so that the battery lasts until it can be recharged or replaced.

In this step, we augment the nodes with components so that we can model (and measure) their energy consumption. For simplicity, we ignore energy constraints and just install infinite energy sources into the nodes.

1.9.2 The model

Energy consumption model

In a real system, there are energy consumers like the radio, and energy sources like a battery or the mains. In the INET model of the radio, energy consumption is represented by a separate component. This energy consumption model maps the activity of the core parts of the radio (the transmitter and the receiver) to power (energy) consumption. The core parts of the radio themselves do not contain anything about power consumption; they only expose their state variables. This allows one to switch to arbitrarily complex (or simple) power consumption models, without affecting the operation of the radio. The energy consumption model can be specified in the `energyConsumerType` parameter of the `Radio` module.

Here, we set the energy consumption model in the node radios to `StateBasedEpEnergyConsumer`. `StateBasedEpEnergyConsumer` models radio power consumption based on states like radio mode, transmitter and receiver state. Each state has a constant power consumption that can be set by a parameter. Energy use depends on how much time the radio spends in a particular state.

To go into a little bit more detail: the radio maintains two state variables, *receive state* and *transmit state*. At any given time, the radio mode (one of *off*, *sleep*, *switching*, *receiver*, *transmitter*, and *transceiver*) decides which of the two state variables are valid. The receive state may be *idle*, *busy*, or *receiving*, the former two referring to the channel state. When it is *receiving*, a sub-state stores which part of the signal it is receiving: the *preamble*, the (physical layer) *header*, the *data*, or *any* (we don't know/care). Similarly, the transmit state may be *idle* or *transmitting*, and a sub-state stores which part of the signal is being transmitted (if any).

`StateBasedEpEnergyConsumer` expects the consumption in various states to be specified in watts in parameters like `sleepPowerConsumption`, `receiverBusyPowerConsumption`, `transmitterTransmittingPreamblePowerConsumption`, and so on.

Measuring energy consumption

Hosts contain an energy storage component that basically models an energy source like a battery or the mains. INET contains several energy storage models, and the desired one can be selected in the `energyStorageType` parameter of the host. Also, the radio's energy consumption model is preconfigured to draw energy from the host's energy storage. (Hosts with more than one energy storage component are also possible.)

In this model, we use `IdealEpEnergyStorage` in hosts. `IdealEpEnergyStorage` provides an infinite amount of energy, can't be fully charged or depleted. We use this because we want to concentrate on the power consumption, not the storage.

The energy storage module contains an `energyBalance` watched variable that can be used to track energy consumption. Also, energy consumption over time can be obtained by plotting the `residualEnergyCapacity` statistic.

Visualization

The visualization of radio signals as expanding bubbles is no longer needed, so we turn it off.

Configuration:

```
[Config Wireless08]
description = Modeling energy consumption
extends = Wireless07

*.host*.wlan[0].radio.energyConsumer.typeName = "StateBasedEpEnergyConsumer"
*.host*.wlan[0].radio.energyConsumer.offPowerConsumption = 0mW
*.host*.wlan[0].radio.energyConsumer.sleepPowerConsumption = 1mW
*.host*.wlan[0].radio.energyConsumer.switchingPowerConsumption = 1mW
*.host*.wlan[0].radio.energyConsumer.receiverIdlePowerConsumption = 2mW
*.host*.wlan[0].radio.energyConsumer.receiverBusyPowerConsumption = 5mW
*.host*.wlan[0].radio.energyConsumer.receiverReceivingPowerConsumption = 10mW
*.host*.wlan[0].radio.energyConsumer.transmitterIdlePowerConsumption = 2mW
*.host*.wlan[0].radio.energyConsumer.transmitterTransmittingPowerConsumption = 100mW
```

(continues on next page)

(continued from previous page)

```

*.host*.energyStorage.typename = "IdealEpEnergyStorage"

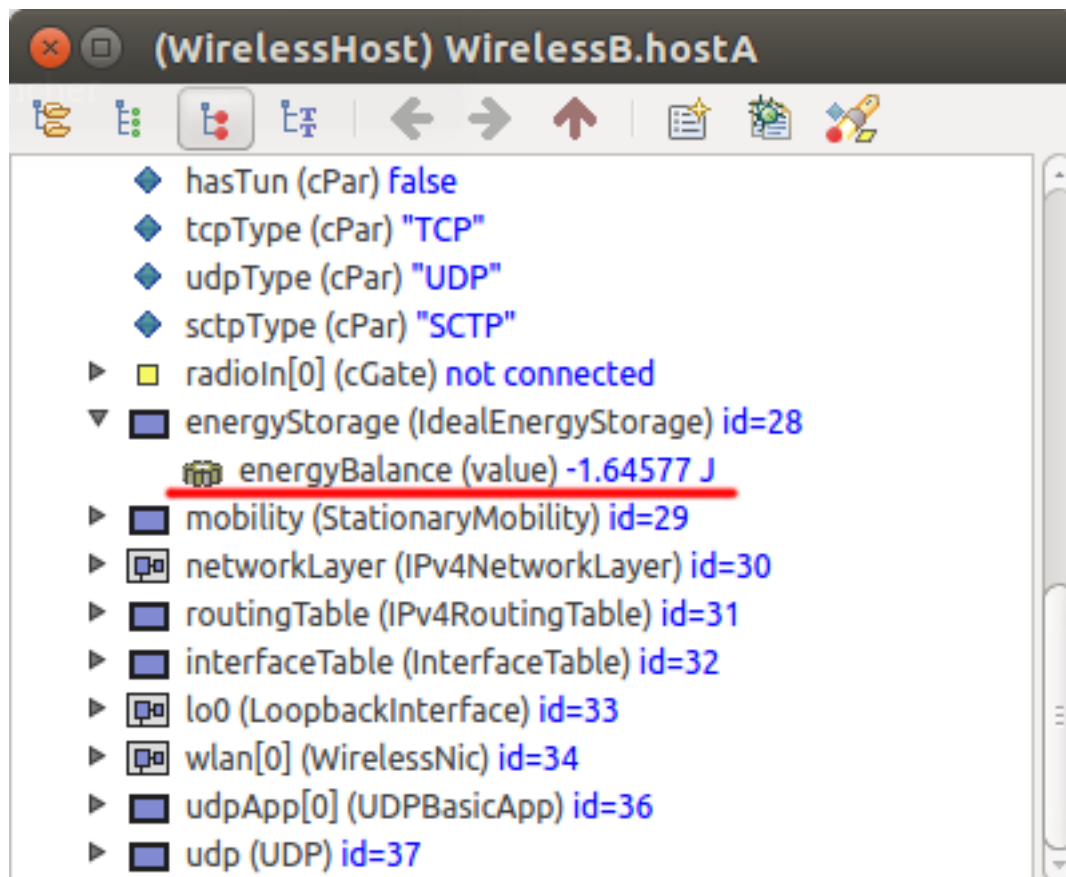
*.host*.wlan[0].radio.displayInterferenceRange = false
*.hostR1.wlan[0].radio.displayCommunicationRange = false

*.visualizer.mediumVisualizer.displaySignals = false

```

1.9.3 Results

The image below shows host A's `energyBalance` variable at the end of the simulation. The negative energy value signifies the consumption of energy.



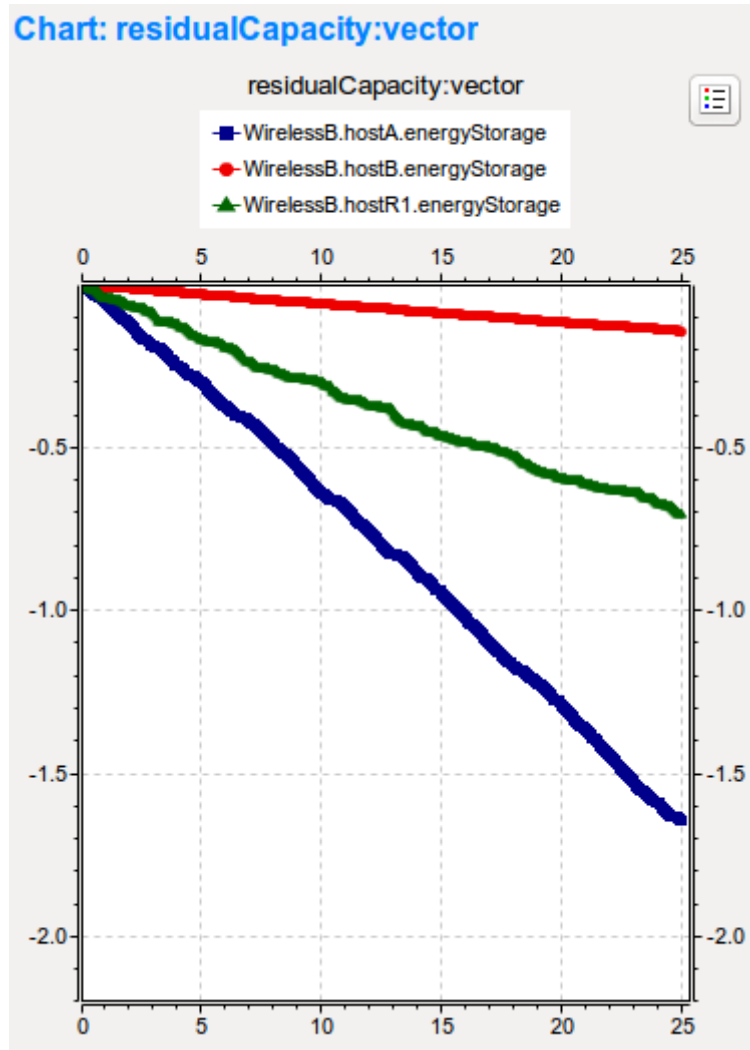
The `residualCapacity` statistic of hosts A, R1 and B are plotted on the following diagram. The diagram shows that host A has consumed the most power because it transmitted more than the other nodes.

Number of packets received by host B: 1393

Sources: `omnetpp.ini`, `WirelessB.ned`

1.9.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.



1.10 Step 9. Configuring node movements

1.10.1 Goals

In this step, we make the model more interesting by adding node mobility. Namely, we make the intermediate nodes travel north during the simulation. After a while, they will move out of the range of host A (and B), breaking the communication path.

1.10.2 The model

Mobility

In the INET Framework, node mobility is handled by the `mobility` submodule of hosts. Several mobility module types exist that can be plugged into a host. The movement trail may be deterministic (such as line, rectangle or circle), probabilistic (e.g. random waypoint), scripted (e.g. a “turtle” script) or trace-driven. There are also individual and group mobility models.

Here we install `LinearMobility` into the intermediate nodes. `LinearMobility` implements movement along a line, where the heading and speed are parameters. We configure the nodes to move north at the speed of 12 m/s.

Other changes

So far, L2 queues (the queues in the wireless interfaces) have been unlimited, that is, no packet would be dropped due to congestion. Meanwhile, there was indeed congestion in host R1 and host A, because the application in host A was generating packets at a higher rate than what the network could carry.

From this step on, we limit the queue lengths at 10 packets. This allows the network to react faster to topology changes caused by node movements because queues will not be clogged up by old packets. However, as a consequence of packet drops, we expect that the sequence numbers of packets received by host B will no longer be continuous.

We also update the visualization settings and turn on an option that will cause mobile nodes to leave a trail as they move. We also enable a visualizer option that will display the velocity vector of the moving nodes.

The configuration:

```
[Config Wireless09]
description = Configuring node movements
extends = Wireless08

*.hostR*.mobility.typename = "LinearMobility"
*.hostR*.mobility.speed = 12mps
*.hostR*.mobility.initialMovementHeading = 270deg

*.host*.wlan[0].queue.packetCapacity = 10

*.visualizer.mobilityVisualizer.displayVelocities = true
*.visualizer.mobilityVisualizer.displayMovementTrails = true
```

1.10.3 Results

It is advisable to run the simulation in Fast mode, because the nodes move very slowly if viewed in Normal mode.

It can be seen in the animation below as host R1 leaves host A’s communication range at around 11 seconds. After that, the communication path is broken. Traffic could be routed through R2 and R3, but that does not happen because the routing tables are static and have been configured according to the initial positions of the nodes. When the communication path breaks, the blue arrow that represents successful network layer communication paths fades away, because there are no more packets to reinforce it.

As mentioned before, a communication path could be established between host A and B by routing traffic through hosts R2 and R3. To reconfigure routes according to the changing topology of the network, an ad-hoc routing protocol is required.

Number of packets received by host B: 547

Sources: omnetpp.ini, WirelessB.ned

1.10.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.11 Step 10. Configuring ad-hoc routing (AODV)

1.11.1 Goals

In this step, we configure a routing protocol that adapts to the changing network topology and will arrange packets to be routed through R2 and R3 as R1 departs.

We'll use AODV (ad hoc on-demand distance vector routing). It is a reactive routing protocol, which means its maintenance of the routing tables is driven by demand. This is in contrast to proactive routing protocols which keep routing tables up to date all the time (or at least try to).

1.11.2 The model

Let's configure ad-hoc routing with AODV. As AODV will manage the routing tables, we don't need the statically added routes anymore. We only need `Ipv4NetworkConfigurator` to assign the IP addresses and turn all other functions off.

More importantly, we change the hosts to be instances of `AodvRouter`. `AodvRouter` is like `WirelessHost`, but with an added `AodvRouting` submodule. This change turns each node into an AODV router.

AODV stands for Ad hoc On-Demand Distance Vector. In AODV, routes are established as they are needed. Once established, a route is maintained as long as it is needed.

In AODV, the network is silent until a connection is needed. At that point, the network node that needs a connection broadcasts a request for connection. Other AODV nodes forward this message and record the node that they heard it from, creating an explosion of temporary routes back to the needy node. When a node receives such a message and already has a route to the desired node, it sends a message backward through a temporary route to the requesting node. The needy node then begins using the route that has the least number of hops through other nodes. Unused entries in the routing tables are recycled after a time.

The message types defined by AODV are Route Request (RREQ), Route Reply (RREP), and Route Error (RERRs). We expect to see these messages at the start of the simulation (when an initial route needs to be established), and later when the topology of the network changes due to the movement of nodes.

The configuration:

```
[Config Wireless10]
description = Configuring ad-hoc routing (AODV)
extends = Wireless09

*.configurator.addStaticRoutes = false

*.host*.typename = "AodvRouter"

*.hostB.wlan[0].radio.displayCommunicationRange = true

*.visualizer.dataLinkVisualizer.packetFilter = "AODV"
```

1.11.3 Results

Host R1 moves out of communication range of hosts A and B. The route that was established through R1 is broken. Hosts R2 and R3 are in the right position to relay host A's packets to host B, and the AODV protocol reconfigures

the route to go through R2 and R3. The UDP stream is re-established using these intermediate hosts to relay host A's packets to host B. This can be seen in the animation below.

AODV detects when a route is no longer able to pass packets. When the link is broken, there are no ACKs arriving at host A, so its AODV submodule triggers the reconfiguration of the route. To detect possible new routes and establish one, there is a burst of AODV transmissions between the hosts, and a new route is established through hosts R2 and R3. This detection and reconfiguration take very little time. After the AODV packet burst, the arrows displaying it fade quickly, and the UDP traffic continues.

The AODV route discovery messages can be seen in the following packet log:

Event#	Time	Src/Dest	Name	Info
#52586	15.97128884804	hostR2 --> hostR1	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.4.654 > 255.255.255.255.654: (32)
#52586	15.97128884804	hostR2 --> hostR3	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.4.654 > 255.255.255.255.654: (32)
#52611	15.974990321984	hostR1 --> hostA	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.3.654 > 255.255.255.255.654: (32)
#52611	15.974990321984	hostR1 --> hostB	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.3.654 > 255.255.255.255.654: (32)
#52611	15.974990321984	hostR1 --> hostR2	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.3.654 > 255.255.255.255.654: (32)
#52611	15.974990321984	hostR1 --> hostR3	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.3.654 > 255.255.255.255.654: (32)
#52632	15.976610586387	hostR3 --> hostA	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.5.654 > 255.255.255.255.654: (32)
#52632	15.976610586387	hostR3 --> hostB	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.5.654 > 255.255.255.255.654: (32)
#52632	15.976610586387	hostR3 --> hostR1	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.5.654 > 255.255.255.255.654: (32)
#52632	15.976610586387	hostR3 --> hostR2	AODV-RREQ	inet::AODVRREQ:24 bytes UDP: 10.0.0.5.654 > 255.255.255.255.654: (32)
#52657	15.978946987379	hostB --> hostA	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.2.654 > 10.0.0.5.654: (28)
#52657	15.978946987379	hostB --> hostR1	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.2.654 > 10.0.0.5.654: (28)
#52657	15.978946987379	hostB --> hostR2	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.2.654 > 10.0.0.5.654: (28)
#52657	15.978946987379	hostB --> hostR3	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.2.654 > 10.0.0.5.654: (28)
#52674	15.979523388423	hostR3 --> hostA	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52674	15.979523388423	hostR3 --> hostB	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52674	15.979523388423	hostR3 --> hostR1	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52674	15.979523388423	hostR3 --> hostR2	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52691	15.980035375743	hostR3 --> hostA	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.5.654 > 10.0.0.4.654: (28)
#52691	15.980035375743	hostR3 --> hostB	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.5.654 > 10.0.0.4.654: (28)
#52691	15.980035375743	hostR3 --> hostR1	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.5.654 > 10.0.0.4.654: (28)
#52691	15.980035375743	hostR3 --> hostR2	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.5.654 > 10.0.0.4.654: (28)
#52708	15.980612042871	hostR2 --> hostA	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52708	15.980612042871	hostR2 --> hostB	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52708	15.980612042871	hostR2 --> hostR1	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52708	15.980612042871	hostR2 --> hostR3	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#52726	15.982451840523	hostR2 --> hostA	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.4.654 > 10.0.0.1.654: (28)
#52726	15.982451840523	hostR2 --> hostB	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.4.654 > 10.0.0.1.654: (28)
#52726	15.982451840523	hostR2 --> hostR1	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.4.654 > 10.0.0.1.654: (28)
#52726	15.982451840523	hostR2 --> hostR3	AODV-RREP	inet::AODVRREP:20 bytes UDP: 10.0.0.4.654 > 10.0.0.1.654: (28)
#53149	15.983028241644	hostA --> hostB	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#53149	15.983028241644	hostA --> hostR1	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#53149	15.983028241644	hostA --> hostR2	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#53149	15.983028241644	hostA --> hostR3	CSMA-Ack	inet::CSMAFrame:5 bytes duration=0.04ms
#53162	15.984988241644	hostA --> hostB	UDPData-879	omnetpp::cPacket:1000 bytes UDP: 10.0.0.1.1025 > 10.0.0.2.5000:
#53162	15.984988241644	hostA --> hostR1	UDPData-879	omnetpp::cPacket:1000 bytes UDP: 10.0.0.1.1025 > 10.0.0.2.5000:
#53162	15.984988241644	hostA --> hostR2	UDPData-879	omnetpp::cPacket:1000 bytes UDP: 10.0.0.1.1025 > 10.0.0.2.5000:

The number of successfully received packets at host B has roughly doubled compared to the previous step. This is because the flow of packets doesn't stop when host R1 gets out of communication range of host A. Although the AODV protocol adds some overhead, in this simulation, it is not significant, and the number of received packets still increases substantially.

Number of packets received by host B: 956

Sources: omnetpp.ini, WirelessB.ned

1.11.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.12 Step 11. Adding obstacles to the environment

1.12.1 Goals

In an attempt to make our simulation both more realistic and more interesting, we add some obstacles to the playground.

In the real world, objects like walls, trees, buildings, and hills act as obstacles to radio signal propagation. They absorb and reflect radio waves, reducing signal quality and decreasing the chance of successful reception.

In this step, we add a concrete wall to the model that sits between hosts A and R1 and see what happens. Since our model still uses the unit disk radio and unit disk wireless medium models that do not model physical phenomena,

obstacle modeling will be very simple: all obstacles completely absorb radio signals, making reception behind them impossible.

1.12.2 The model

First, we need to represent obstacles. In INET, obstacles are managed as part of the `PhysicalEnvironment` module, so we need to add an instance to the `WirelessB` network:

```
network WirelessC extends WirelessB
{
    submodules:
        physicalEnvironment: PhysicalEnvironment {
            @display("p=580,425");
        }
}
```

Obstacles are described in an XML file. An obstacle is defined by its shape, location, orientation, and material. It may also have a name, and one can define how it should be rendered (color, line width, opacity, etc.) The XML format allows one to use predefined shapes like cuboid, prism, polyhedron or sphere, and also to define new shapes that may be reused for any number of obstacles. It is similar for materials: there are predefined materials like concrete, brick, wood, glass, forest, and one can also define new materials. A material is defined with its physical properties like resistivity, relative permittivity, and relative permeability. These properties are used in the computations of dielectric loss tangent, refractive index, and signal propagation speed, and ultimately in the computation of signal loss.

Our wall is defined in `walls.xml`, and the file name is given to `PhysicalEnvironment` in its `config` parameter. The file contents:

```
<environment>
  <object position="min 130 300 0" orientation="0 0 0" shape="cuboid 5 100 4"
    material="concrete" fill-color="203 65 84" opacity="0.8"/>
</environment>
```

Having obstacles is not enough in itself; we also need to teach the model of the wireless medium to take them into account. This is done by specifying an obstacle loss model. Since our model contains `UnitDiskRadioMedium`, we specify `IdealObstacleLoss`. With `IdealObstacleLoss`, obstacles completely block radio signals, making reception behind them impossible.

The `IntegratedCanvasVisualizer` we use as the visualizer submodule in the network contains two submodules related to obstacles: `physicalEnvironmentVisualizer` displays the obstacles themselves, and `obstacleLossVisualizer` is responsible for visualizing the obstacle loss of individual signals.

The wall in our simulation is at an elevation of 0m and is 4m high. So far the hosts (more precisely, their antennas) were at 0m elevation, the default setting; we change this to 1.7m so that the wall definitely blocks their signals.

The configuration:

```
[Config Wireless11]
description = Adding obstacles to the environment
extends = Wireless10
network = WirelessC

*.host*.mobility.initialZ = 1.7m

*.physicalEnvironment.config = xmldoc("walls.xml")
*.radioMedium.obstacleLoss.typeName = "IdealObstacleLoss"
```

1.12.3 Results

At the beginning of the simulation, the initial route that was established in previous steps (A-R1-B) cannot be established, because the wall is between host A and R1. The wall is completely blocking transmissions, therefore AODV establishes the A-R2-R1-B route. Host R2's transmission is cut when R2 moves behind the wall. This time, however, host R1 is available to relay host A's transmissions to host B. A new route is formed, and traffic continues to use this route until host R1 moves out of communication range. After that, the A-R2-R3-B route is used, as seen in the previous steps.

Number of packets received by host B: 784

Sources: omnetpp.ini, WirelessC.ned, walls.xml

1.12.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.13 Step 12. Changing to a more realistic radio model

1.13.1 Goals

After so many steps, we let go of the unit disk radio model and introduce a more realistic one. Our new radio will use an APSK modulation scheme, but still without other techniques like forward error correction, interleaving or spreading. We also want our model of the radio channel to simulate attenuation and obstacle loss.

1.13.2 The model

Switching to APSK radio

In this step, we replace `GenericUnitDiskRadio` with `ApskScalarRadio`. `ApskScalarRadio` models a radio with an APSK (amplitude and phase-shift keying) modulation scheme. By default it uses BPSK, but QPSK, QAM-16, QAM-64, QAM-256 and several other modulations can also be configured. (Modulation is a parameter of the radio's transmitter component.)

Since we are moving away from the “unit disc radio” type of abstraction, we need to specify the carrier frequency, signal bandwidth and transmission power of the radios. Together with other parameters, they will allow the radio channel and the receiver models to compute path loss, SNIR, bit error rate, and other values, and ultimately determine the success of the reception.

`ApskScalarRadio` also adds realism in that it simulates that the data are preceded by a preamble and a physical layer header. Their lengths are also parameters (and may be set to zero when not needed.)

NOTE: When choosing the preamble and the physical layer header lengths, we needed to take care that the `ackTimeout` parameter of `CsmaCaMac` is still valid. (The preamble and the physical layer header contribute to the duration of the ACK frame as well.)

Physical parameters of the receiver are important, too. We configure the following receiver parameters: - sensitivity [dBm]: if the signal power is below this threshold, reception is not possible (i.e. the receiver cannot go from the *channel busy* state to *receiving*) - energy detection threshold [dBm]: if reception power is below this threshold, no signal is detected and the channel is sensed to be empty (this is significant for the “carrier sense” part of CSMA) - SNIR threshold [dB]: reception is not successful if the SNIR is below this threshold

The concrete values in the inifile were chosen to approximately reproduce the communication and interference ranges used in the previous steps.

Setting up the wireless channel

Since we switched the radio to `ApskScalarRadio`, we also need to change the medium to `ScalarRadioMedium`. In general, one always needs to use a medium that is compatible with the given radio. (With `GenericUnitDiskRadio`, we also used `UnitDiskRadioMedium`.)

`ScalarRadioMedium` has “slots” to plug in various propagation models, path loss models, obstacle loss models, analog models, and background noise models. Here we make use of the fact that the default background noise model is homogeneous isotropic white noise, and set up the noise level to a nonzero value (-90dBm).

Configuration:

```
[Config Wireless12]
description = Changing to a more realistic radio model
extends = Wireless11

*.radioMedium.signalAnalogRepresentation = "scalar"
*.radioMedium.backgroundNoise.power = -90dBm
*.radioMedium.mediumLimitCache.centerFrequency = 2GHz

*.host*.wlan[0].radio.typename = "ApskRadio"
*.host*.wlan[0].radio.signalAnalogRepresentation = "scalar"
*.host*.wlan[0].radio.centerFrequency = 2GHz
*.host*.wlan[0].radio.bandwidth = 2MHz
*.host*.wlan[0].radio.transmitter.power = 1.4mW
*.host*.wlan[0].radio.transmitter.preambleDuration = 10us
*.host*.wlan[0].radio.transmitter.headerLength = 8B
*.host*.wlan[0].radio.receiver.sensitivity = -85dBm
*.host*.wlan[0].radio.receiver.energyDetection = -85dBm
*.host*.wlan[0].radio.receiver.snirThreshold = 4dB

*.hostR1.wlan[*].radio.bitErrorRate.result-recording-modes = default,+vector
```

1.13.3 Results

What happens is about the same as in the previous step. At first, host A's packets are relayed by host R2 until it moves so that the wall separates them. The connection is re-established when host R1 moves out from behind the wall. Then it gets out of communication range, and the new route goes through hosts R2 and R3.

In this model, more physical effects are simulated than in previous steps. There are radio signal attenuation, background noise and a more realistic radio model. The blue circles representing the communication range is an approximation. There is no distinct distance where receptions fail, as in the case of `GenericUnitDiskRadio`.

In host A, the MAC receives the packet `UDPData-408` from the radio. The MAC drops the packet because of bit errors; this can be seen in the following log:

```
** Event #19913 t=5.479322414577 WirelessC.hostA.wlan[0].mac (CsmaCaMac, id=59) on UDPData-408 (inet::CsmaCaMacDataFrame, id=966679)
received message from lower layer: (inet::CsmaCaMacDataFrame)UDPData-408
Self address: 0A-AA-00-00-00-01, receiver address: 0A-AA-00-00-00-02, received frame is for us: 0
FSM CsmaCaMac State Machine: processing event in state RECEIVE
FSM CsmaCaMac State Machine: leaving state RECEIVE
FSM CsmaCaMac State Machine: condition "(isLowerMessage(msg) && frame->hasBitError())" holds, taking transition "Receive-Bit-Error" to state IDLE
FSM CsmaCaMac State Machine: done processing associated actions
FSM CsmaCaMac State Machine: entering state IDLE
FSM CsmaCaMac State Machine: processing event in state IDLE
FSM CsmaCaMac State Machine: leaving state IDLE
FSM CsmaCaMac State Machine: condition "(isUpperMessage(msg) && isMediumFree() && useAck)" holds, taking transition "Start-Difs" to state WAITDIFS
FSM CsmaCaMac State Machine: done processing associated actions
FSM CsmaCaMac State Machine: entering state WAITDIFS
scheduling DIFS timer
```

Number of packets received by host B: 665

Sources: `omnetpp.ini`, `WirelessC.ned`

1.13.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.14 Step 13. Configuring a more accurate path loss model

1.14.1 Goals

By default, the medium uses the free-space path loss model, which assumes a line-of-sight path, with no obstacles nearby to cause reflection or diffraction. Since our wireless hosts move on the ground, a more accurate path loss model would be the two-ray ground reflection model that calculates with one reflection from the ground.

1.14.2 The model

It has been mentioned that `ScalarRadioMedium` relies on various subcomponents for computing path loss, obstacle loss, and background noise, among others. Installing the two-ray ground reflection model is just a matter of changing its `pathLossType` parameter from the default `FreeSpacePathLoss` to `TwoRayGroundReflection`. (Further options include `RayleighFading`, `RicianFading`, `LogNormalShadowing`, and some others.)

The two-ray ground reflection model uses the altitudes of the transmitter and the receiver antennas above the ground as input. To compute the altitude, we need the hosts' (x,y,z) positions and the ground's elevation at those points. The z coordinates of hosts have been set to 1.7m in an earlier step. The ground's elevation is defined by the ground model, which is part of the physical environment model.

In this model, we'll use `FlatGround` for the ground model, and specify it to the `physicalEnvironment` module. (Note that we added `physicalEnvironment` to the network when we introduced obstacles.) The ground's elevation is the `elevation` parameter of `FlatGround`. We set this parameter to 0m.

```
[Config Wireless13]
description = Configuring a more accurate pathloss model
extends = Wireless12

*.physicalEnvironment.ground.typename = "FlatGround"
*.physicalEnvironment.ground.elevation = 0m

*.radioMedium.pathLoss.typename = "TwoRayGroundReflection"
```

1.14.3 Results

The image below shows the bit error rate of host R1's radio as a function of time. The bit error rate is shown when free space path loss is used, and when using the two-ray ground reflection model. The interval shown here corresponds to the time in the simulation when host R1 is not cut off from host A by the wall anymore, and also still in communication range of host A and B. In the interval that is shown, from around 5 seconds to 11 seconds, the distance between host R1 and hosts A and B is increasing, which results in an increase in the bit error rate as well. There is no significant difference between the free space propagation path loss and the two-ray ground reflection path loss at first. The two curves separate towards the end of the displayed interval. As expected, in the case of the two-ray ground reflection model, the bit error rate is greater.

Number of packets received by host B: 679

Sources: omnetpp.ini, WirelessC.ned

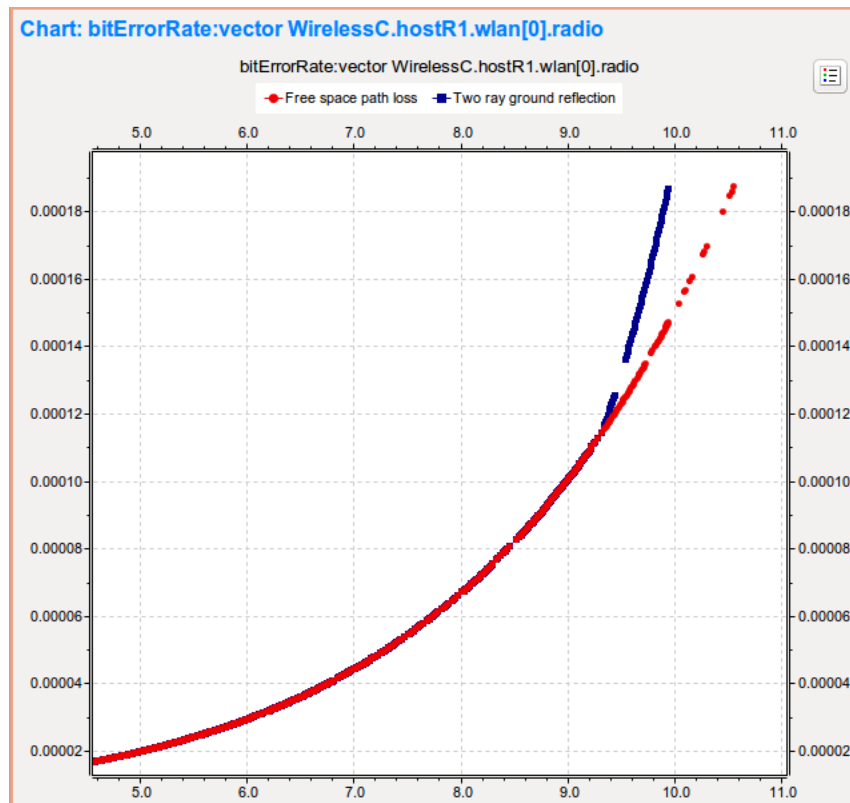
1.14.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.15 Step 14. Introducing antenna gain

1.15.1 Goals

In the previous steps, we have assumed an isotropic antenna for the radio, with a gain of 1 (0dB). Here we want to enhance the simulation by taking antenna gain into account.



1.15.2 The model

For simplicity, we configure the hosts to use `ConstantGainAntenna`. `ConstantGainAntenna` is an abstraction: it models an antenna that has a constant gain in the directions relevant for the simulation, regardless of how such an antenna could be implemented in real life. For example, if all nodes of a simulated wireless network are on the same plane, `ConstantGainAntenna` could correspond to an omnidirectional antenna such as a vertical dipole. (INET contains support for directional antennas as well.)

```
[Config Wireless14]
description = Introducing antenna gain
extends = Wireless13

*.host*.wlan[0].radio.antenna.typeName = "ConstantGainAntenna"
*.host*.wlan[0].radio.antenna.gain = 3dB
```

1.15.3 Results

With the added antenna gain, the transmissions are powerful enough to require only two hops to get to host B every time, as opposed to the previous step, where it sometimes required three. Therefore, at the beginning of the simulation, host R1 can reach host B directly. Also, host R1 goes out of host A's communication range only at the very end of the simulation. When this happens, host A's transmission is routed through host R2, which is again just two hops.

Number of packets received by host B: 1045

Sources: omnetpp.ini, WirelessC.ned

1.15.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

1.16 Conclusion

Congratulations, you've made it! You should now be familiar with the basic concepts of modeling wireless networks with the INET Framework.

We are planning further tutorials to touch on topics that have not made it into the present one. These topics include using multi-dimensional signal modeling, bit-precise signal modeling, forward error correction, scrambling, interleaving, and run-time optimization by using MAC address and range filtering.

A separate tutorial will be devoted to 802.11 simulations.

IPV4 NETWORK CONFIGURATOR TUTORIAL

This tutorial shows how to configure IP addresses and routing tables (that is, achieve static autoconfiguration) in wired and wireless networks in the INET Framework using the `Ipv4NetworkConfigurator` module.

Contents:

2.1 Getting Started

In INET simulations, configurator modules are commonly used for assigning IP addresses to network nodes and for setting up their routing tables. There are various configurators modules in INET; this tutorial covers the most generic and most featureful one, `Ipv4NetworkConfigurator`. `Ipv4NetworkConfigurator` supports automatic and manual network configuration, and their combinations. By default, the configuration is fully automatic. The user can also specify parts (or all) of the configuration manually, and the rest will be configured automatically by the configurator. The configurator's various features can be turned on and off with parameters. The details of the configuration, such as IP addresses and routes, can be specified in an XML file.

This is an advanced tutorial, and it assumes that you are familiar with creating and running simulations in OMNeT++ and INET. If you aren't, you can check out the TicToc Tutorial to get started with using OMNeT++.

2.1.1 Try It Yourself

If you already have INET and OMNeT++ installed, start the IDE by typing `omnetpp`, import the INET project into the IDE, then navigate to the `inet/tutorials/configurator` folder in the *Project Explorer*. There, you can view and edit the tutorial files, run simulations, and analyze results.

Otherwise, there is an easy way to install INET and OMNeT++ using `opp_env`, and run the simulation interactively. Ensure that `opp_env` is installed on your system, then execute:

```
$ opp_env run inet-4.6 --init -w inet-workspace --install --build-modes=release --  
→chdir \  
-c 'cd inet-4.6.*/tutorials/configurator && inet'
```

This command creates an `inet-workspace` directory, installs the appropriate versions of INET and OMNeT++ within it, and launches the `inet` command in the `showcase` directory for interactive simulation.

Alternatively, for a more hands-on experience, you can first set up the workspace and then open an interactive shell:

```
$ opp_env install --init -w inet-workspace --build-modes=release inet-4.6  
$ cd inet-workspace  
$ opp_env shell
```

Inside the shell, start the IDE by typing `omnetpp`, import the INET project, then start exploring.

2.1.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

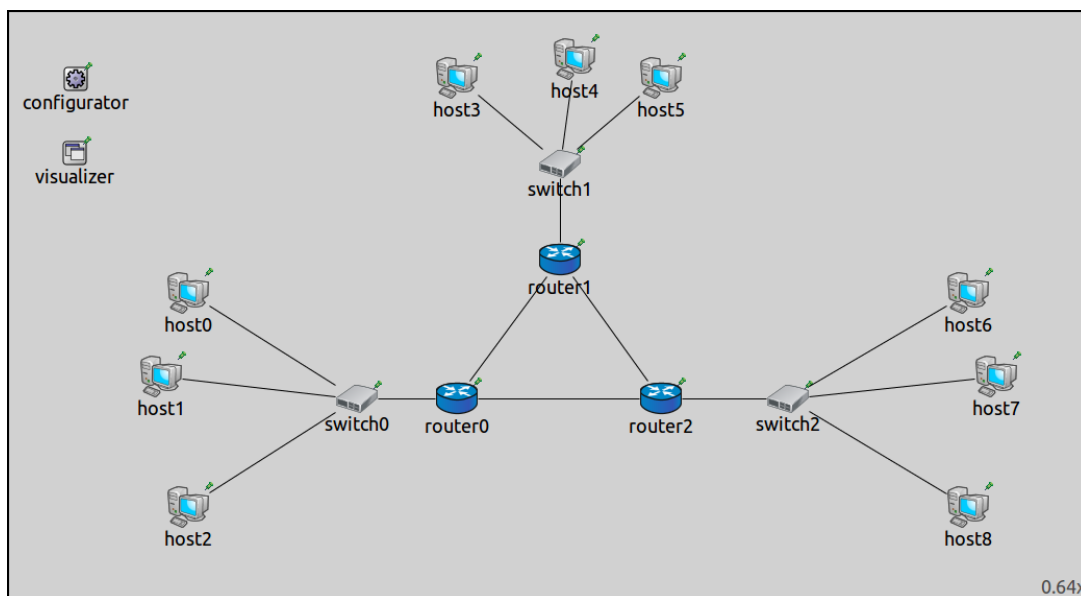
2.2 Step 1. Fully automatic IP address assignment

2.2.1 Goals

In the first step, we want to automatically assign addresses in a wired network. To make the task a little bit more complicated, the network won't be a single LAN, but several LANs connected via routers. We want to place nodes on each subnet into different subnets, but otherwise, we don't care how addresses are assigned. We also ignore the question of filling routing tables for now – it will be covered in later steps.

2.2.2 The network

The network we want to configure is `ConfiguratorA`, defined in `ConfiguratorA.ned`. It looks like this:



The network contains three routers, each connected to the other two. There are three subnetworks with `StandardHosts`, connected to the routers by Ethernet switches. It also contains an instance of `Ipv4NetworkConfigurator`.

2.2.3 The configuration

In many scenarios, including this one, the `Ipv4NetworkConfigurator` module can properly configure the network using just the default settings. Thus, the configuration in `omnetpp.ini` for this step is basically empty:

```
[Config Step1]
sim-time-limit = 500s
network = ConfiguratorA
description = "Fully automatic IP address assignment"
```

The configurator has several parameters that affect its operation, but for now, all of them are left at their default settings. Let's briefly review the ones pertaining to IP address assignment:

- `assignAddresses = default(true)` controls whether the configurator should perform auto address assignment, i.e. assign IP addresses to interfaces. Details can be specified in the XML configuration.
- `assignDisjunctSubnetAddresses = default(true)` controls whether the configurator should assign different address prefixes and netmasks to nodes on different links. (Nodes are considered to be on the same link if they can reach each other directly, or through L2 devices only.)

An XML configuration can be supplied with the `config` parameter. When the user doesn't specify an XML configuration (such as in this step), the configurator uses the following default:

```
config = default(xml("<config><interface hosts='**' address='10.x.x.x' netmask='255.x.x.x' /></config>"))
```

It tells the configurator to assign IP addresses to all interfaces of all hosts, from the address range 10.0.0.0 - 10.255.255.255 and netmask range 255.0.0.0 - 255.255.255.255.

2.2.4 Setting up logging and visualization

The configurator has several boolean parameters named `dump...`, which cause it to dump certain parts of the configuration information into the module log. The one that corresponds to assigned IP addresses is called `dumpAddresses`. These parameters are false by default, but we set them to true in the `General` section of `omnetpp.ini` to facilitate experimenting with the configurator.

Network interface information, such as interface names and IP addresses, can be visualized using the `InterfaceTableCanvasVisualizer` module, which is already included in the network as a submodule of `IntegratedCanvasVisualizer`. We also turn on interface visualization but tell the visualizer to ignore switches and access points, as they don't have IP addresses.

Other settings in the `General` section, such as the WiFi bit rate, are not relevant to the topic of the tutorial.

```
[General]
description = "(abstract)"

# Configurator settings
*.configurator.dumpAddresses = true
*.configurator.dumpTopology = true
*.configurator.dumpLinks = true
*.configurator.dumpRoutes = true

# Routing settings
*.ipv4.arp.typename = "GlobalArp"
*.ipv4.routingTable.netmaskRoutes = ""

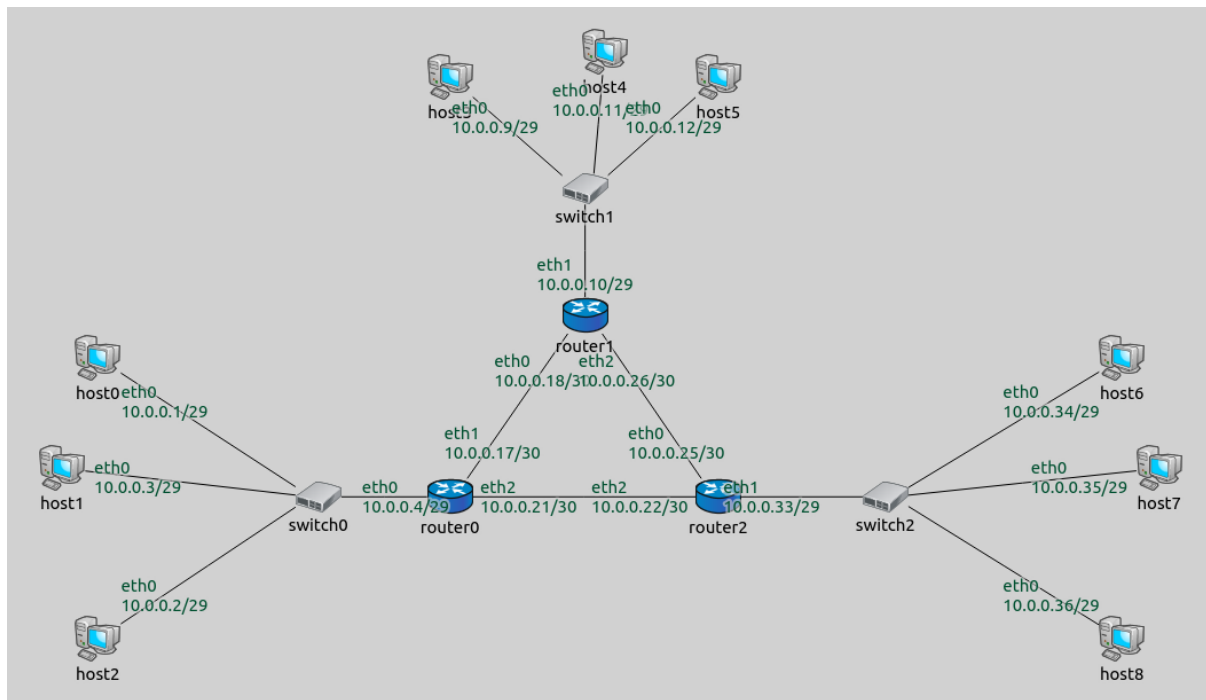
# Visualizer settings
*.visualizer.interfaceTableVisualizer.displayInterfaceTables = true
*.visualizer.interfaceTableVisualizer.nodeFilter = "not (*switch* or *Switch* or *AP*)"
↪ "
```

2.2.5 Results

The IP addresses assigned to interfaces by the configurator are shown on the image below. The switches and hosts connected to the individual routers are considered to be on the same link. Note that the configurator assigned addresses sequentially starting from 10.0.0.1 while making sure that different subnets got different address prefixes and netmasks, as instructed by the `assignDisjunctSubnetAddresses` parameter.

Note that the configurator assigned a 29-bit netmask to the hosts and the router interfaces connecting to the switches, and a 30-bit netmask to the other router interfaces. Three hosts and the router's interface towards a switch as a group have four interfaces, thus, a 30-bit netmask with a 2-bit host identifier would have sufficed. However, the configurator doesn't assign addresses where the host identifier is all-zeros or all-ones (as they commonly refer to the subnet itself and the subnet broadcast address.)

Sources: `omnetpp.ini`, `ConfiguratorA.ned`



2.2.6 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.3 Step 2. Manually overriding individual IP addresses

2.3.1 Goals

Automatic address assignment is great, but it is sometimes also useful to be able to manually specify IP addresses for certain nodes or interfaces. This step shows how to do that while retaining automatic assignment for the rest of the network.

2.3.2 The model

This step uses the [ConfiguratorA](#) network from the previous step. We will assign the 10.0.0.50 address to host1, and 10.0.0.100 to host3. The configurator will automatically assign addresses to the rest of the nodes.

Configuration

The configuration in `omnetpp.ini` for this step is the following:

```
[Config Step2]
sim-time-limit = 500s
network = ConfiguratorA
description = "Manually overriding individual IP addresses"

# Using inline XML configuration
*.configurator.config = xml("<config> \
                                <interface hosts='host3' names='eth0' address='10.0.0.
↵100' /> \
                                <interface hosts='host1' names='eth0' address='10.0.0.
↵50' /> \
                                <interface hosts='*' address='10.x.x.x' netmask='255.
↵x.x.x' /> \
                                </config>")
```

The value for the `config` parameter can be supplied either inline using the `xml()` function, or in an external XML file using the `xmldoc()` function (with the file name as the argument). In this step, the XML configuration is supplied to the configurator as inline XML.

In the above XML configuration, the first two rules state that `host3`'s `eth0` interface should get the address 10.0.0.100, and `host1`'s `eth0` interface should get the address 10.0.0.50. The third rule is the exact copy of the default configuration and tells the configurator to assign the rest of the addresses automatically.

Note that the XML configuration contains `<config>` as root element. Under this root element, there can be multiple configuration elements, such as the `<interface>` elements here.

The `<interface>` element

An `<interface>` element configures a network interface or a set of interfaces, and it has two groups of attributes: selector attributes (`hosts`, `names`, `towards`, `among`, etc.) that define which interface(s) are to be configured, and parameter attributes (`address`, `netmask`, `metric`, etc.) that specify values to be set on those interfaces. The attributes used in the configuration above are:

- The `hosts` selector attribute selects hosts. The selection pattern can be a full path (e.g. `".host0"`) or a module name anywhere in the hierarchy (e.g. `"host0"`). Only interfaces in the selected hosts will be affected by the `<interface>` element.
- The `names` selector attribute selects interfaces by name. Only the interfaces that match the specified names will be selected (e.g. `"eth0"`).
- The `address` parameter attribute specifies the addresses to be assigned. Address templates can be used, where an `x` in place of an octet means that the value should be selected by the configurator automatically. The value `""` means that no address will be assigned. Unconfigured interfaces will still have allocated addresses in their subnets, so they can be easily configured later dynamically.
- The `netmask` parameter attribute specifies the netmasks to be assigned. Address templates can be used here as well.

All attributes are optional. There are many other attributes available. For the complete list, please refer to the configurator's NED documentation.

The order of configuration elements is important, but the configurator doesn't assign addresses in the order of XML `<interface>` elements. It iterates interfaces in the network, and for each interface, the first matching rule in the XML configuration will take effect. Thus, statements that are positioned earlier in the configuration take precedence over those that come later.

When an XML configuration is supplied, it must contain `<interface>` elements in order to assign addresses at all. To make sure the configurator automatically assigns addresses to all interfaces, a rule similar to the one in the default configuration has to be included (unless the intention is to leave some interfaces unassigned.) The default rule should be the *last one* among the interface rules so that more specific rules can override it.

Note that after applying a manually specified address, auto address assignment will continue from that address. This may or may not be what you want.

2.3.3 Results

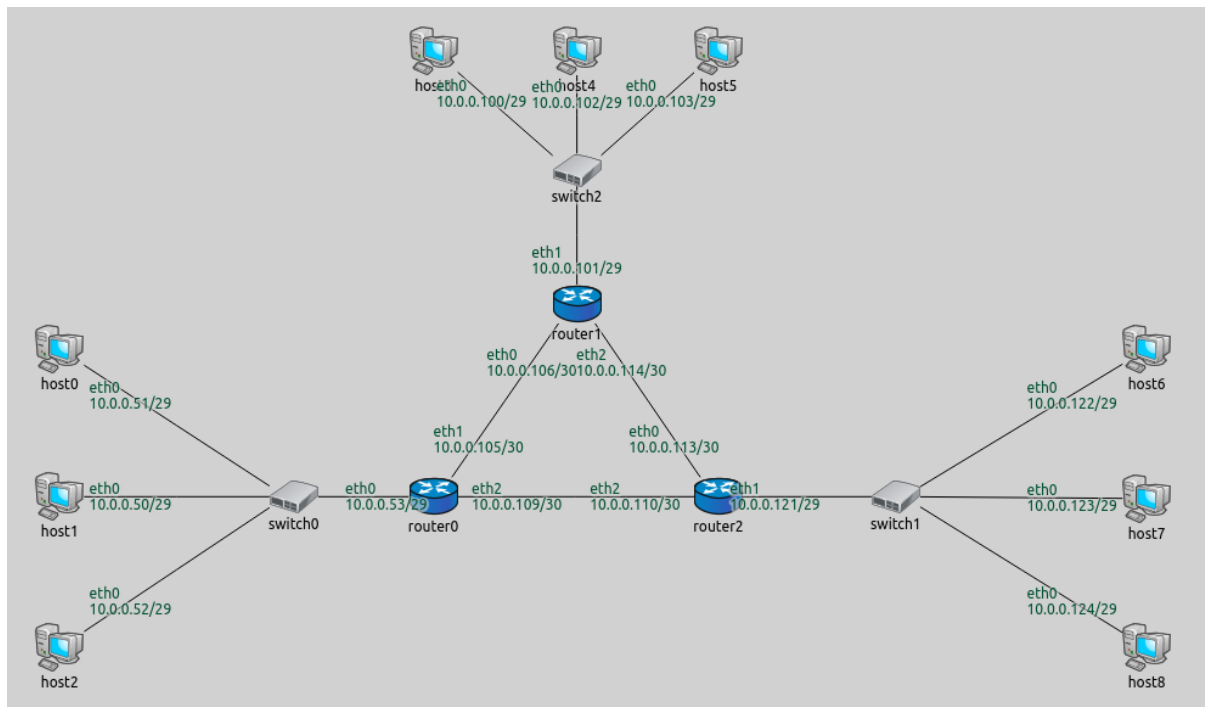
The assigned addresses are shown in the following image.

As in the previous step, the configurator assigned disjunct subnet addresses. Note that the configurator still assigned addresses sequentially, i.e. after setting the 10.0.0.100 address to `host3`, it didn't go back to the beginning of the address pool when assigning the remaining addresses.

Sources: `omnetpp.ini`, `ConfiguratorA.ned`

2.3.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.



2.4 Step 3. Automatically assigning IP addresses to a subnet from a given range

2.4.1 Goals

Complex networks often contain several subnetworks, and the user may want to assign specific IP address ranges to them. This step demonstrates how to assign a range of IP addresses to subnets.

2.4.2 The model

This step uses the [ConfiguratorA](#) network, as in the previous two steps. One switch and the connected hosts as a group will be on the same subnet, and there are three such groups in the network.

The configuration is as follows:

```
[Config Step3]
sim-time-limit = 500s
network = ConfiguratorA
description = "Automatically assigning IP addresses to a subnet from a given range"

*.configurator.config = xmldoc("step3.xml")
```

This time the XML configuration is supplied in an external file (step3.xml), using the `xmldoc()` function:

```
<config>
  <interface hosts="host{0-2}" address="10.0.0.x"/>
  <interface hosts="host{3-5}" address="10.0.1.x"/>
  <interface hosts="host{6-8}" address="10.0.2.x"/>
  <interface hosts="router0" towards="host0" address="10.0.0.x"/>
  <interface hosts="router1" towards="host3" address="10.0.1.x"/>
  <interface hosts="router2" towards="host6" address="10.0.2.x"/>
  <interface hosts="router*" address="10.1.x.x"/>
</config>
```

A brief explanation:

- The first three entries assign IP addresses with different network prefixes to hosts in the three different subnets.
- The next three entries specify, for each router, that its interface connecting to a subnet should belong to that subnet. These entries use the `towards` selector, which selects the interfaces that are connected towards the specified host or hosts.
- The last entry sets the network prefix of interfaces of all routers to be 10.1.x.x. The routers' interfaces facing the subnets were assigned addresses by the previous rules, so this rule only affects the interfaces facing the other routers.

These seven rules assign addresses to all interfaces in the network, thus, a default rule is not required.

Variations

The same effect can be achieved in more than one way. Here is an alternative XML configuration (step3alt1.xml) that results in the same address assignment:

```
<config>
  <interface among="host{0-2} router0" address="10.0.0.x"/>
  <interface among="host{3-5} router1" address="10.0.1.x"/>
  <interface among="host{6-8} router2" address="10.0.2.x"/>
  <interface hosts="router*" address="10.1.x.x"/>
</config>
```

The `among` selector selects the interfaces of the specified hosts towards the specified hosts (the statement `among="X Y Z"` is the same as `hosts="X Y Z" towards="X Y Z"`).

Another alternative XML configuration is the following:

```
<config>
  <interface hosts="host0" address="10.0.0.x"/>
  <interface hosts="host3" address="10.0.1.x"/>
  <interface hosts="host6" address="10.0.2.x"/>
  <interface among="router*" address="10.1.x.x"/>
  <interface hosts="*" address="10.x.x.x" netmask="255.x.x.x"/>
</config>
```

This one assigns an address to one host in each of the three subnets. It assigns addresses to the interfaces of the routers facing the other routers, and includes a copy of the default configuration. Because `assignDisjunctSubnetAddresses=true`, the configurator puts the unspecified hosts and the subnet-facing router interfaces into the same subnet as the specified host.

2.4.3 Results

The assigned addresses are shown in the following image.

The addresses are assigned as intended. This is useful because it is easy to recognize which group a node belongs to just by looking at its address (e.g., in the logs).

Sources: `omnetpp.ini`, `ConfiguratorA.ned`

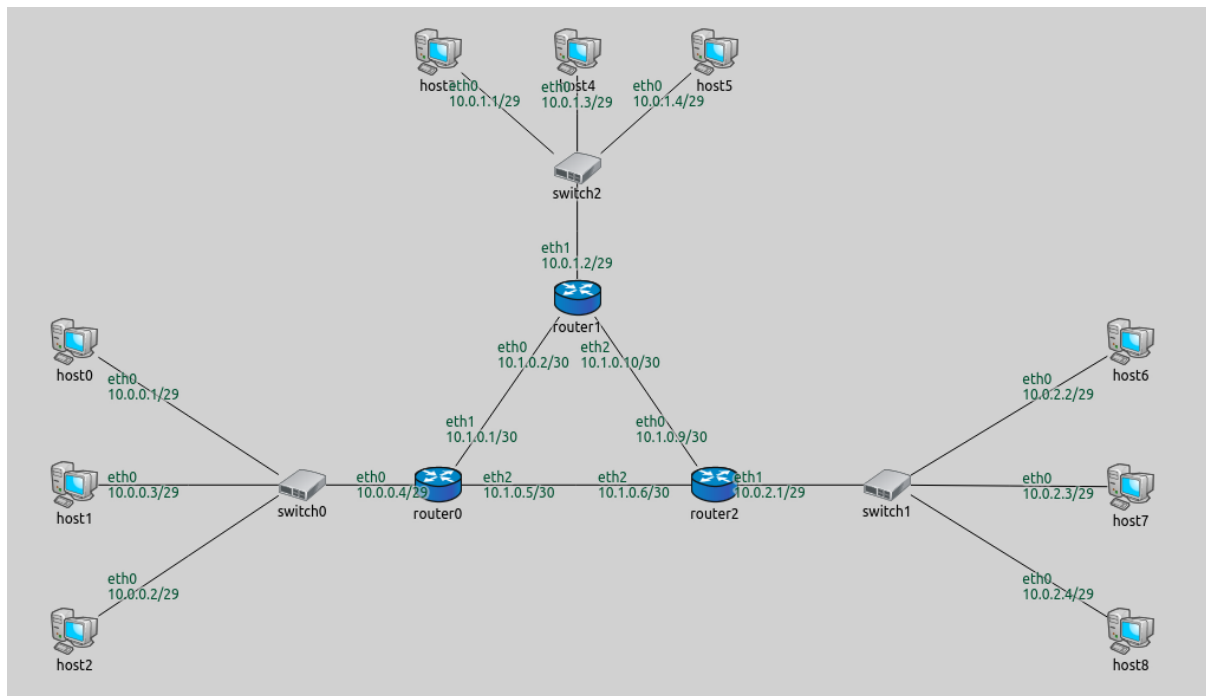
2.4.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.5 Step 4. Fully automatic static routing table configuration

2.5.1 Goals

Just as with IP addresses, in many cases, the configurator sets up routes in a network properly without any user input. This step demonstrates the default configuration for routing.



2.5.2 The model

This step uses the same network as the previous steps, [ConfiguratorA](#). The configuration for this step in `omnetpp.ini` is the following:

```
[Config Step4]
sim-time-limit = 500s
network = ConfiguratorA
description = "Fully automatic static routing table configuration"

*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[0].destAddr = "host7"

*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "host7"
```

The configuration leaves the configurator's parameters at their defaults. The configurator's parameters concerning static routing are the following:

- `addStaticRoutes = default(true)`: This is the main switch that turns on automatically filling routing tables. When this setting is turned off, only routes specified in the XML configuration are added, and all the following parameters are ignored.
- `addDefaultRoutes = default(true)`: Add a default route if all routes from a node go through the same gateway. This is often the case with hosts, which usually connect to a network via a single interface.
- `addSubnetRoutes = default(true)`: Optimize routing tables by adding routes towards subnets instead of individual interfaces.
- `optimizeRoutes = default(true)`: Optimize routing tables by merging entries where possible.

Adding traffic

We add some traffic to the network so that we can see which way packets are routed. We use ping applications for this purpose throughout the tutorial.

In this step, we configure `host1` to ping `host7` in order to see the route between the two hosts.

Logging and visualization settings

The `RoutingTableCanvasVisualizer` module (present in the network as a submodule of `IntegratedCanvasVisualizer`) can be used to visualize IP routes in the network. Routes are visualized with arrows. In general, an arrow indicates an entry in the source host's routing table. It points to the host that is the next hop or gateway for that routing table entry. The visualization is activated by setting the `RoutingTableVisualizer`'s `displayRoutingTables` parameter to `true`. The set of routes to be visualized is selected with the visualizer's `destinationFilter` and `nodeFilter` parameters. All routes leading towards the selected set of destinations from the selected set of source nodes are indicated by arrows. The default setting for both parameters is `"*"`, which visualizes all routes going from every node to every other node. Visualizing routes from all nodes to all destinations can often make the screen cluttered. In this step, the `destinationFilter` is set to visualize all routes heading towards `host7`.

The visualizer annotates the arrows with information about the visualized route by default. This feature can be customized and toggled on and off with the visualizer's parameters (discussed in later steps.)

Additional settings pertaining to routing are specified in the `General` configuration in `omnetpp.ini`:

```
[General]
description = "(abstract)"

# Configurator settings
*.configurator.dumpAddresses = true
*.configurator.dumpTopology = true
*.configurator.dumpLinks = true
*.configurator.dumpRoutes = true

# Routing settings
*.*.ipv4.arp.typename = "GlobalArp"
*.*.ipv4.routingTable.netmaskRoutes = ""

# Visualizer settings
*.visualizer.interfaceTableVisualizer.displayInterfaceTables = true
*.visualizer.interfaceTableVisualizer.nodeFilter = "not (*switch* or *Switch* or *AP*)"
↪ "
```

The `dumpTopology`, `dumpLinks`, and `dumpRoutes` parameters are set to `true`. These parameter settings instruct the configurator to print to the module output the topology of the network, the recognized network links, and the routing tables of all nodes, respectively. Topology describes which nodes are connected to which nodes. Hosts that can directly reach each other (i.e. the next hop is the destination), are considered to be on the same link.

Additional configuration

The `General` configuration also sets up `GlobalArp` to keep the packet exchanges simpler. `GlobalArp` fills the ARP tables of all nodes in advance, so when the simulation begins, no ARP exchanges are necessary.

The `*.*.routingTable.netmaskRoutes = ""` line keeps the routing table modules from adding netmask routes to the routing tables. Netmask routes mean that nodes with the same netmask but different IP addresses should reach each other directly. These routes are also added by the configurator, so `netmaskRoutes` is turned off to avoid duplicate routes.

2.5.3 Results

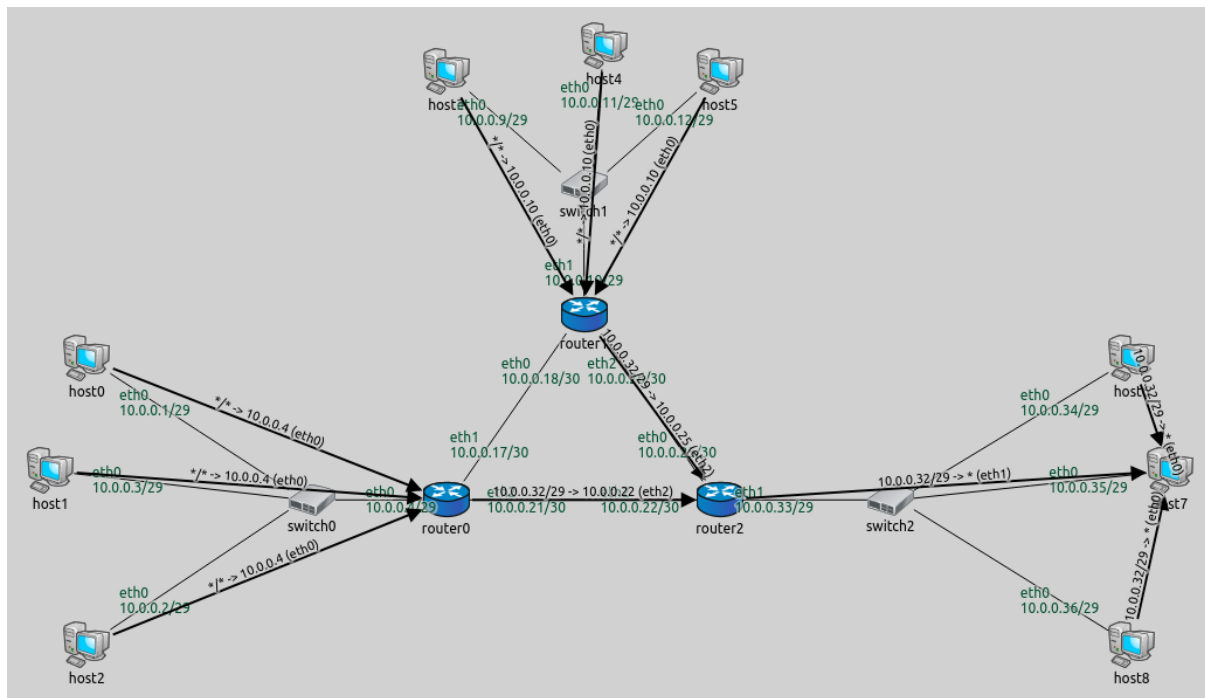
The visualized routes are displayed on the following image:

Routes from all nodes to `host7` are visualized. Note that arrows don't go through switches, because they are not the next hop. When routes are concerned, switches are transparent L2 devices.

The routing tables are the following (routes visualized on the image above are highlighted):

```
Node ConfiguratorB.host0 (hosts 1-2 are similar)
-- Routing table --
```

(continues on next page)



(continued from previous page)

Destination	Netmask	Gateway	Iface	Metric
10.0.0.0	255.255.255.248	*	eth0 (10.0.0.1)	0
*	*	10.0.0.4	eth0 (10.0.0.1)	0

Node ConfiguratorB.host3 (hosts 4-5 are similar)

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.8	255.255.255.248	*	eth0 (10.0.0.9)	0
*	*	10.0.0.10	eth0 (10.0.0.9)	0

Node ConfiguratorB.host6 (hosts 7-8 are similar)

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.32	255.255.255.248	*	eth0 (10.0.0.34)	0
*	*	10.0.0.33	eth0 (10.0.0.34)	0

Node ConfiguratorB.router0

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.18	255.255.255.255	*	eth1 (10.0.0.17)	0
10.0.0.22	255.255.255.255	*	eth2 (10.0.0.21)	0
10.0.0.25	255.255.255.255	10.0.0.22	eth2 (10.0.0.21)	0
10.0.0.0	255.255.255.248	*	eth0 (10.0.0.4)	0
10.0.0.32	255.255.255.248	10.0.0.22	eth2 (10.0.0.21)	0
10.0.0.0	255.255.255.224	10.0.0.18	eth1 (10.0.0.17)	0

Node ConfiguratorB.router1

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.17	255.255.255.255	*	eth0 (10.0.0.18)	0
10.0.0.22	255.255.255.255	10.0.0.25	eth2 (10.0.0.26)	0
10.0.0.25	255.255.255.255	*	eth2 (10.0.0.26)	0
10.0.0.8	255.255.255.248	*	eth1 (10.0.0.10)	0

(continues on next page)

(continued from previous page)

10.0.0.32	255.255.255.248	10.0.0.25	eth2 (10.0.0.26)	0
10.0.0.0	255.255.255.224	10.0.0.17	eth0 (10.0.0.18)	0
Node ConfiguratorB.router2				
-- Routing table --				
Destination	Netmask	Gateway	Iface	Metric
10.0.0.18	255.255.255.255	10.0.0.26	eth0 (10.0.0.25)	0
10.0.0.21	255.255.255.255	*	eth2 (10.0.0.22)	0
10.0.0.26	255.255.255.255	*	eth0 (10.0.0.25)	0
10.0.0.8	255.255.255.248	10.0.0.26	eth0 (10.0.0.25)	0
10.0.0.32	255.255.255.248	*	eth1 (10.0.0.33)	0
10.0.0.0	255.255.255.224	10.0.0.21	eth2 (10.0.0.22)	0

The * for the gateway means that the gateway is the same as the destination. Hosts have a routing table entry to reach other nodes on the same subnet directly. They also have a default route with the router as the gateway for packets sent to outside-of-subnet addresses. Routers have three rules in their routing tables for reaching the other routers, specifically, those interfaces of the other routers that are not facing the hosts.

Below is a video of host1 pinging host7.

Sources: omnetpp.ini, ConfiguratorA.ned

2.5.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.6 Step 5. Manually overriding individual routes

2.6.1 Goals

Automatic configuration can fill the routing tables correctly, but sometimes the user might want to manually override some of the routes. This step consists of two parts:

- In **Part A** we will override the routes to just one specific host
- In **Part B** we will override routes to a set of hosts

2.6.2 Part A - Overriding routes to a specific host

Both parts in this step use the [ConfiguratorA](#) network (displayed below), just as in the previous steps. In this part, we will override the routes going from the subnet of router0 to host7. With the automatic configuration, packets from router0's subnet would go through router2 to reach host7 (as in the previous step.) We want them to go through router1 instead.

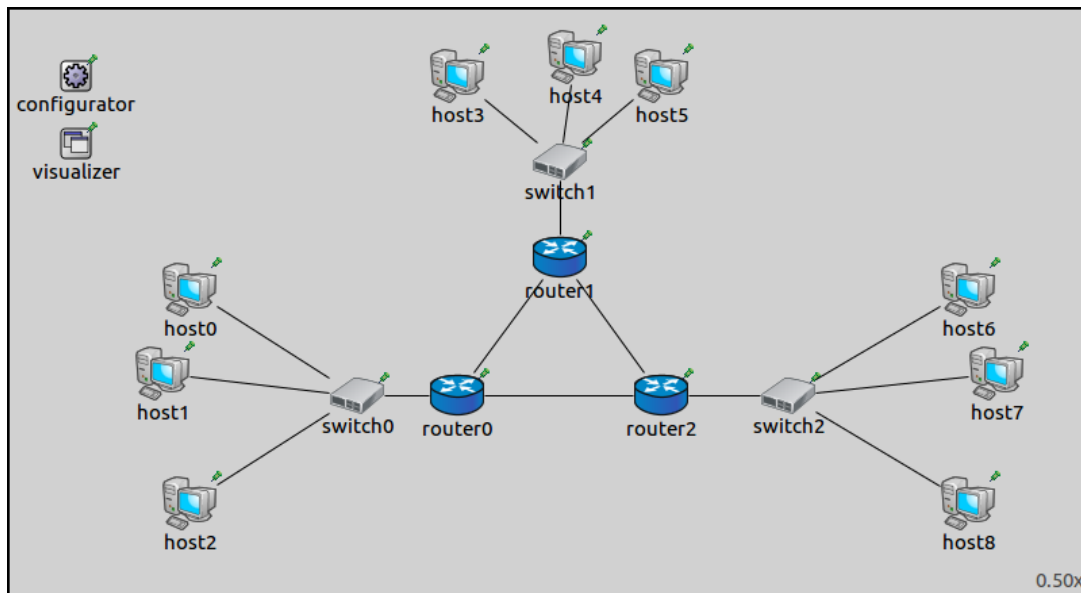
Configuration

The configuration in omnetpp.ini is the following:

```
[Config Step5A]
sim-time-limit = 500s
extends = Step4
description = "Manually overriding individual routes - route to a specific host"

*.configurator.config = xmldoc("step5a.xml")

*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[*].destAddr = "host6"
*.host0.app[*].startTime = 0.6s
```



A ping application is added to **host0**, in addition to the one in **host1** added in Step 4. The new app in **host0** pings **host6** to demonstrate that only packets sent to **host7** are affected by the route override.

For the routes to go through **router1**, the routing table of **router0** has to be altered. The new rules should dictate that packets with the destination of **host7** (10.0.0.35) should be routed towards **router2**. The XML configuration in **step5a.xml** is the following:

```
<config>
  <interface hosts="*" address="10.x.x.x" netmask="255.x.x.x"/>
  <route hosts="router0" destination="10.0.0.35" netmask="255.255.255.255"
    ↪gateway="10.0.0.18" interface="eth1" metric="0"/>
</config>
```

The **<route>** element describes a routing table entry for one or more nodes in the network. The **hosts** optional selector attribute specifies which hosts' routing tables should be affected. There are five additional optional parameter attributes. These are the same as in real life routing tables: **address**, **netmask**, **gateway**, **interface**, **metric**.

The **<route>** element in this XML configuration adds the following rule to **router0**'s routing table: Packets with the destination of 10.0.0.35/32 should use the interface **eth1** and the gateway 10.0.0.18 (**router1**.) The concrete IP addresses were obtained by setting up the network without the **<route>** element first and inspecting the result.

Caveats

First, note that adding a route manually does not erase the original route. Therefore, the new route will only take effect if it is "stronger" than the automatically added one, for example, it has a longer matching prefix, better metric, or (given the previous ones are equal) occurs earlier in the routing table. Luckily, the last condition usually holds (manually added routes are added at the top), so manually added routes do take effect unless they are explicitly weaker than the original ones.

Second, note that the **<route>** element refers to addresses (e.g. 10.0.0.35) which were automatically assigned by the **<interface>** element. It is valid to do so because the assignment of IP addresses is deterministic, that is, given the same input, it will always produce the same result. However, if you change the network topology, for example, add, remove, or reorder hosts, addresses might be assigned in a different way. The consequence may be that addresses in the **<route>** element no longer exist in the modified network or they refer to different hosts/routers than originally intended, i.e. the configuration will silently break.

One solution to make the configuration more robust is to explicitly assign the addresses in question, using extra **<interface>** elements. (Note, however, that adding an **<interface>** element might also affect the automatically assigned addresses, so it makes sense to add all **<interface>** rules together at once, instead of one-by-one.)

Another solution would be to let `<route>` elements refer to addresses symbolically, i.e. be able to formally express “the address of router1’s interface that faces router0” instead of spelling out “10.0.0.18”. However, support for such mini-language has not yet been added to the configurator.

Results

The routing table of `router0` (manually added route highlighted):

Node ConfiguratorB.router0					
-- Routing table --					
Destination	Netmask	Gateway	Iface	Metric	
10.0.0.18	255.255.255.255	*	eth1 (10.0.0.17)	0	
10.0.0.22	255.255.255.255	*	eth2 (10.0.0.21)	0	
10.0.0.25	255.255.255.255	10.0.0.22	eth2 (10.0.0.21)	0	
10.0.0.35	255.255.255.255	10.0.0.18	eth1 (10.0.0.17)	0	
10.0.0.0	255.255.255.248	*	eth0 (10.0.0.4)	0	
10.0.0.32	255.255.255.248	10.0.0.22	eth2 (10.0.0.21)	0	
10.0.0.0	255.255.255.224	10.0.0.18	eth1 (10.0.0.17)	0	

The routing table of `router0` in the previous step had six entries. Now it has seven, as the rule specified in the XML configuration has been added (highlighted). This and the second to last rule both match packets to `host7`, but the manually added route takes effect because it has a longer netmask (plus it’s also earlier in the table).

The following animation depicts `host1` pinging `host7` and `host0` pinging `host6`. Routes to `host7` are visualized.

Note that only routes towards `host7` are diverted at `router0`. The ping reply packet uses the original route between `router0` and `router2`. Ping packets to `host6` (and back) also use the original route.

2.6.3 Part B - Overriding routes to a set of hosts

In this part, we will override routes going from the subnet of hosts 0–2 to the subnet of hosts 6–8. These routes will go through `router1`, just as in Part A.

Configuration

The configuration in `omnetpp.ini`:

```
[Config Step5B]
sim-time-limit = 500s
extends = Step4
description = "Manually overriding individual routes - route to a set of hosts"

*.configurator.config = xmldoc("step5b.xml")
*.configurator.optimizeRoutes = false           #! TODO: this shouldn't be here, it's
↪ here because of an error in the optimizer

*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[*].destAddr = "host6"
*.host0.app[*].startTime = 0.6s
```

As in Part A, the routing table of `router0` has to be altered, so that packets to hosts 6–8 go towards `router1`. The XML configuration in `step5b.xml` is as follows:

```
<config>
  <interface hosts="*" address="10.x.x.x" netmask="255.x.x.x"/>
  <route hosts="router0" destination="10.0.0.32" netmask="255.255.255.248"
    ↪ gateway="10.0.0.18" interface="eth1"/>
</config>
```

The `<route>` element specifies a routing table entry for `router0`. The destination is 10.0.0.32 with netmask 255.255.255.248, which designates the addresses of hosts 6–8. The gateway is `router1`'s address, the interface is the one connected towards `router1` (`eth1`). This rule is added to `router0`'s routing table *in addition* to the rule added automatically by the configurator. They match the same packets, but the parameters are different (see at the result section below.) The manually added routes come before the automatic ones in routing tables, their prefix length and metrics are the same; thus, the manual ones take precedence.

Results

Here is the routing table of `router0` (the manually added route highlighted):

Node ConfiguratorB.router0				
-- Routing table --				
Destination	Netmask	Gateway	Iface	Metric
10.0.0.10	255.255.255.255	10.0.0.18	eth1 (10.0.0.17)	0
10.0.0.18	255.255.255.255	*	eth1 (10.0.0.17)	0
10.0.0.22	255.255.255.255	*	eth2 (10.0.0.21)	0
10.0.0.25	255.255.255.255	10.0.0.22	eth2 (10.0.0.21)	0
10.0.0.26	255.255.255.255	10.0.0.18	eth1 (10.0.0.17)	0
10.0.0.33	255.255.255.255	10.0.0.22	eth2 (10.0.0.21)	0
10.0.0.0	255.255.255.248	*	eth0 (10.0.0.4)	0
10.0.0.8	255.255.255.248	10.0.0.18	eth1 (10.0.0.17)	0
10.0.0.32	255.255.255.248	10.0.0.18	eth1 (10.0.0.17)	0
10.0.0.32	255.255.255.248	10.0.0.22	eth2 (10.0.0.21)	0

The following is the animation of `host1` pinging `host7` and `host0` pinging `host6`, similarly to Part A. Routes to `host7` are visualized.

This time both packets destined to hosts 6 and 7 take the diverted route and the replies come back on the original route.

Sources: `omnetpp.ini`, `ConfiguratorA.ned`

2.6.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.7 Step 6. Setting different metric for automatic routing table configuration

2.7.1 Goals

When setting up routes, the configurator uses the shortest path algorithm. By default, paths are optimized for hop count. However, there are other cost functions available, like data rate, error rate, etc. This step consists of two parts:

- **Part A** demonstrates using the data rate metric for automatically setting up routes.
- **Part B** demonstrates instructing the configurator not to use a link when setting up routes, by manually specifying a high link cost.

2.7.2 Part A: Using the data rate metric

When setting up routes, the configurator first builds a graph representing the network topology. A vertex in the graph represents a network node along with all of its interfaces. An edge represents a wired or wireless connection between two network interfaces. When building the network topology, wireless nodes are considered to be connected to all other wireless nodes in the same wireless network.

After the graph is built, the configurator assigns weights to vertices and edges according to the configured metric. Vertices that represent network nodes with IP forwarding turned off have infinite weight; all others have 0. Finally, the shortest path algorithm is used to determine the routes based on the assigned weights.

The available metrics are the following:

- **hopCount**: routes are optimized for hop count. All edges have a cost of 1. This is the default metric.
- **dataRate**: routes prefer connections with higher bandwidth. Edge costs are inversely proportional to the data rate of the connection.
- **delay**: routes are optimized for lower delay. Edge costs are proportional to the delay of the connection.
- **errorRate**: routes are optimized for smaller error rate. Edge costs are proportional to the error rate of the connection. This is mostly useful for wireless networks because the error rate of wired connections is usually negligible.

Configuration

The configuration for this step extends Step 4, thus, it uses the [ConfiguratorA](#) network. The configuration in `omnetpp.ini` is the following:

```
[Config Step6A]
sim-time-limit = 500s
extends = Step4
description = "Setting different metric for automatic routing table configuration -_
↳using dataRate metric"

*.configurator.config = xmldoc("step6a.xml")

*.visualizer.routingTableVisualizer.destinationFilter = "host1"
```

Below is the XML configuration in `step6a.xml`:

```
<config>
  <interface hosts='*' address='10.x.x.x' netmask='255.x.x.x' />
  <autoroute sourceHosts='*' metric='dataRate' />
</config>
```

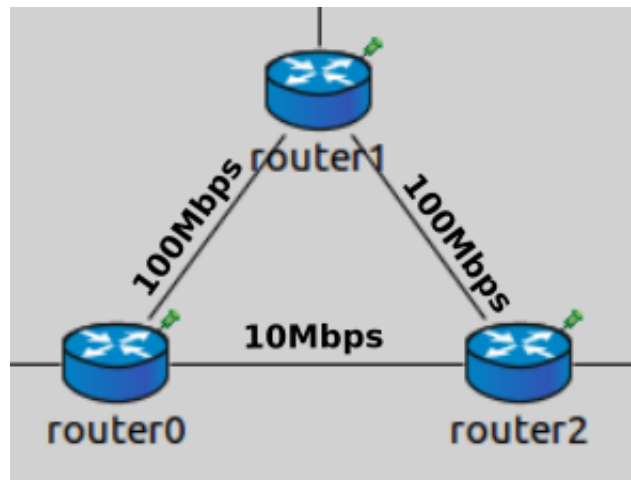
The XML configuration contains the default rule for IP address assignment, and an `<autoroute>` element that configures the metric to be used. The `<autoroute>` element specifies parameters for automatic static routing table configuration. If no `<autoroute>` element is specified, the configurator assumes a default that affects all routing tables in the network, and computes shortest paths to all interfaces according to the hop count metric. The `<autoroute>` element can contain the following attributes:

- **sourceHosts**: Selector attribute that selects which hosts' routing tables should be modified. The default value is `"**"`.
- **destinationInterfaces**: Parameter attribute that selects destination interfaces for which the shortest paths will be calculated. The default value is `"**"`.
- **metric**: Parameter attribute that sets the metric to be used when calculating shortest paths. The default value is `"hopCount"`.

There are sub-elements available in `<autoroute>`, which will be discussed in Part B.

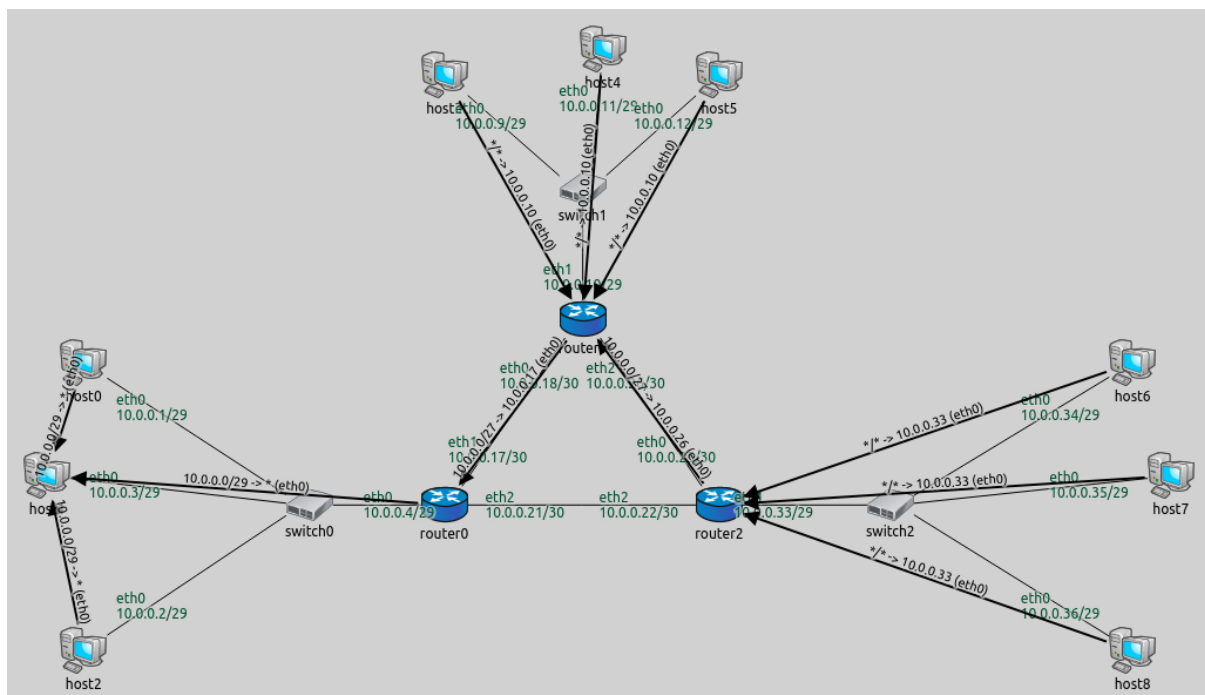
Here the `<autoroute>` element specifies that routes should be added to the routing table of each host and the metric should be `dataRate`. The configurator assigns weights to the graph's edges that are inversely proportional to the data rate of the network links. This way route generation will favor routes with higher data rates.

Note that `router0` and `router2` are connected with a 10 Mbit/s ethernet cable, while `router1` connects to the other routers with 100 Mbit/s ethernet cables. Since routes are optimized for data rate, packets from `router0` to `router2` will go via `router1` because this path has higher bandwidth.



Results

The following image shows the backward routes towards host1. The resulting routes are similar to the ones in Step 5B. The difference is that routes going backward, from hosts 6–8 to hosts 0–2, go through router1. No traffic is routed between router0 and router2 at all (as opposed to Step 4 and 5.)



The routing table of router0 is as follows:

```
Node ConfiguratorA.router0
```

```
-- Routing table --
```

Destination	Netmask	Gateway	Iface	Metric
10.0.0.18	255.255.255.255	*	eth1 (10.0.0.17)	0
10.0.0.0	255.255.255.248	*	eth0 (10.0.0.4)	0
10.0.0.0	255.255.255.192	10.0.0.18	eth1 (10.0.0.17)	0

The first two rules describe reaching router1 and hosts 0–2 directly. The last rule specifies that traffic to any other destination should be routed towards router1.

The routing table of router2 is similar:

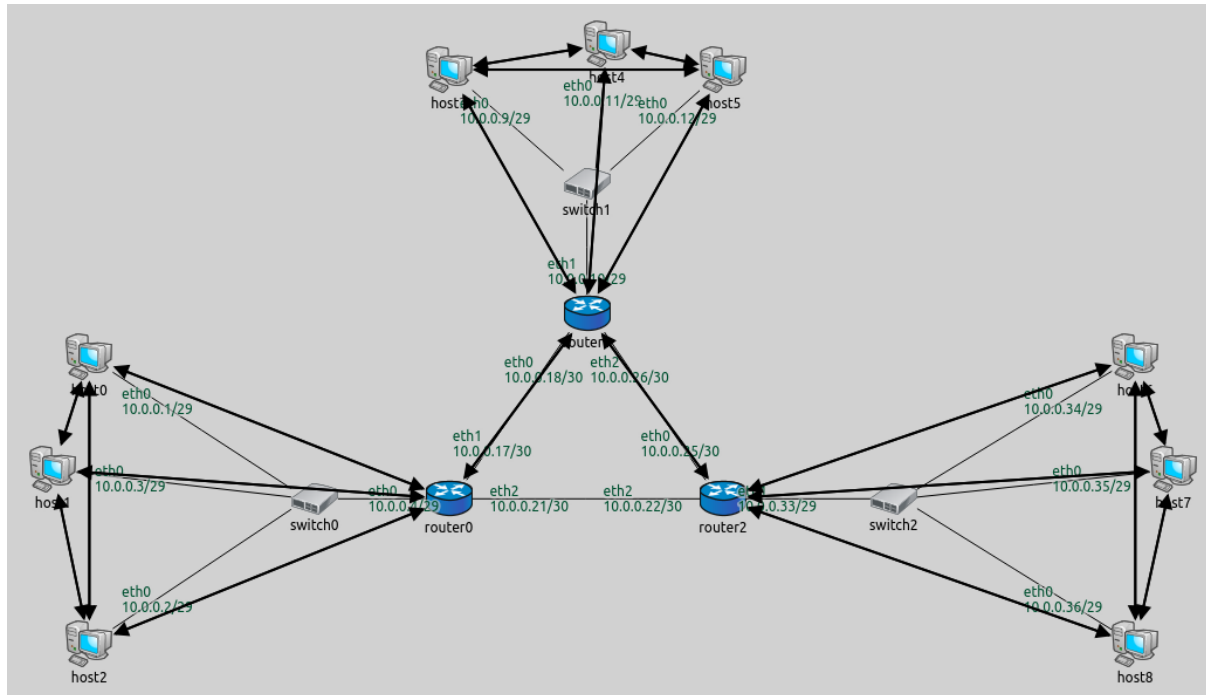
Node ConfiguratorA.router2

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.26	255.255.255.255	*	eth0 (10.0.0.25)	0
10.0.0.32	255.255.255.248	*	eth1 (10.0.0.33)	0
10.0.0.0	255.255.255.224	10.0.0.26	eth0 (10.0.0.25)	0

The following video shows host1 pinging host7 and host0 pinging host6. Routes towards host1 are visualized. The packets don't use the link between router0 and router2.

One can easily check that no routes go through the link between router0 and router2 by setting the destination filter to "*" in the visualizer. This indicates all routes in the network:



2.7.3 Part B - Manually specifying link cost

This part configures the same routes as Part A, where routes between router0 and router2 lead through router1.

The configurator is instructed not to use the link between router0 and router2 when setting up routes, by specifying the cost of the link to be infinite.

Configuration

The configuration for this step in omnetpp.ini is the following:

```
[Config Step6B]
sim-time-limit = 500s
extends = Step4
description = "Setting different metric for automatic routing table configuration ->
->manually specifying link cost"

*.configurator.config = xmldoc("step6b.xml")

*.visualizer.routingTableVisualizer.destinationFilter = "host1"
```

The XML configuration in step6b.xml is as follows:

```

<config>
  <interface hosts='*' address='10.x.x.x' netmask='255.x.x.x' />
  <autoroute metric='hopCount'>
    <link interfaces='*.router0.eth2' cost='infinite' />
  </autoroute>
</config>

```

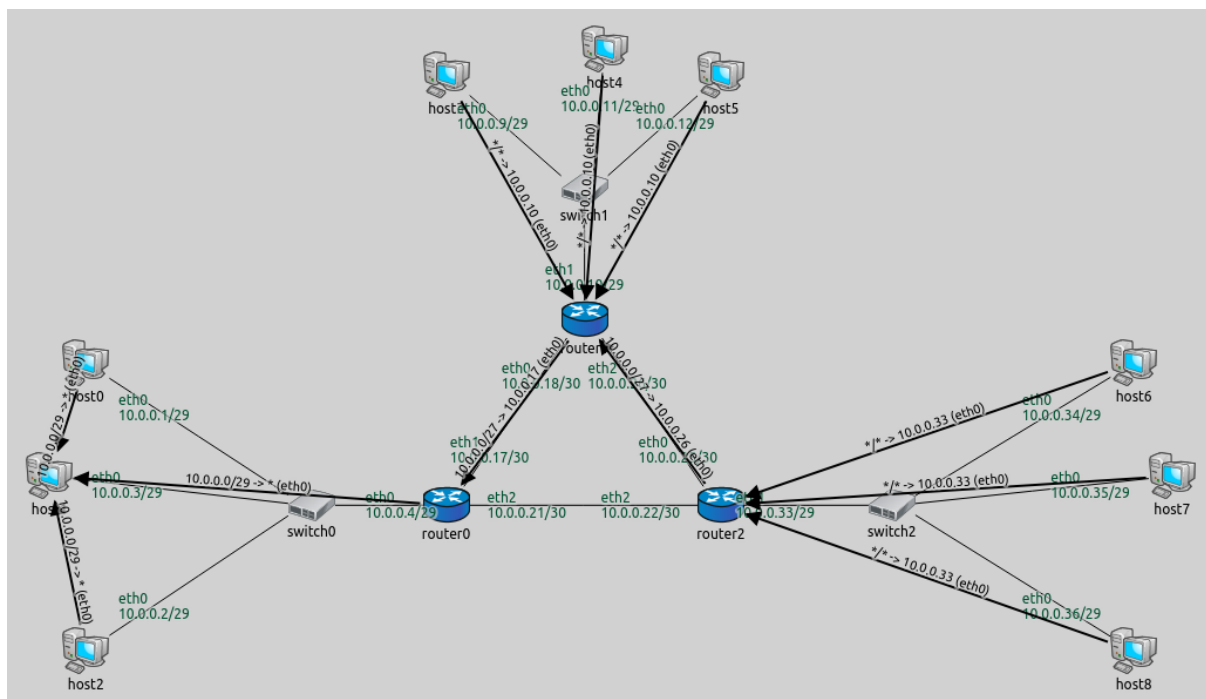
The `<autoroute>` elements can also contain the following optional sub-elements, which can be used to specify costs in the graph:

- `<node>`: Specifies cost parameters to network nodes. The `hosts` selector attribute selects which hosts are affected, and the `cost` parameter sets their costs. Both attributes are mandatory.
- `<link>`: Specifies cost parameters to network links. The `interfaces` selector attribute selects which links are affected, by specifying an interface they belong to. The `cost` parameter sets the cost. Both attributes are mandatory.

This XML configuration specifies the metric to be hop count and sets the cost of `router0`'s `eth2` interface to infinite. This affects the link between `router0` and `router2` - no routes should go through it.

Results

The routes towards `host1` are visualized on the following image:



The routes are the same as in Part A, where the data rate metric was used, and routes didn't use the 10Mbps link between `router0` and `router2`. In this part, the link between `router0` and `router2` is "turned off" by specifying an infinite cost for it. The following video shows `host1` pinging `host7`. Routes towards `host1` are visualized. The ping packets take the diverted route in both directions:

Sources: `omnetpp.ini`, `ConfiguratorA.ned`

2.7.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.8 Step 7. Configuring a hierarchical network

2.8.1 Goals

In complex hierarchical networks, routing tables can grow very large. This step demonstrates ways the configurator can reduce the size of routing tables by optimization and the use of hierarchically assigned addresses. The step contains three parts:

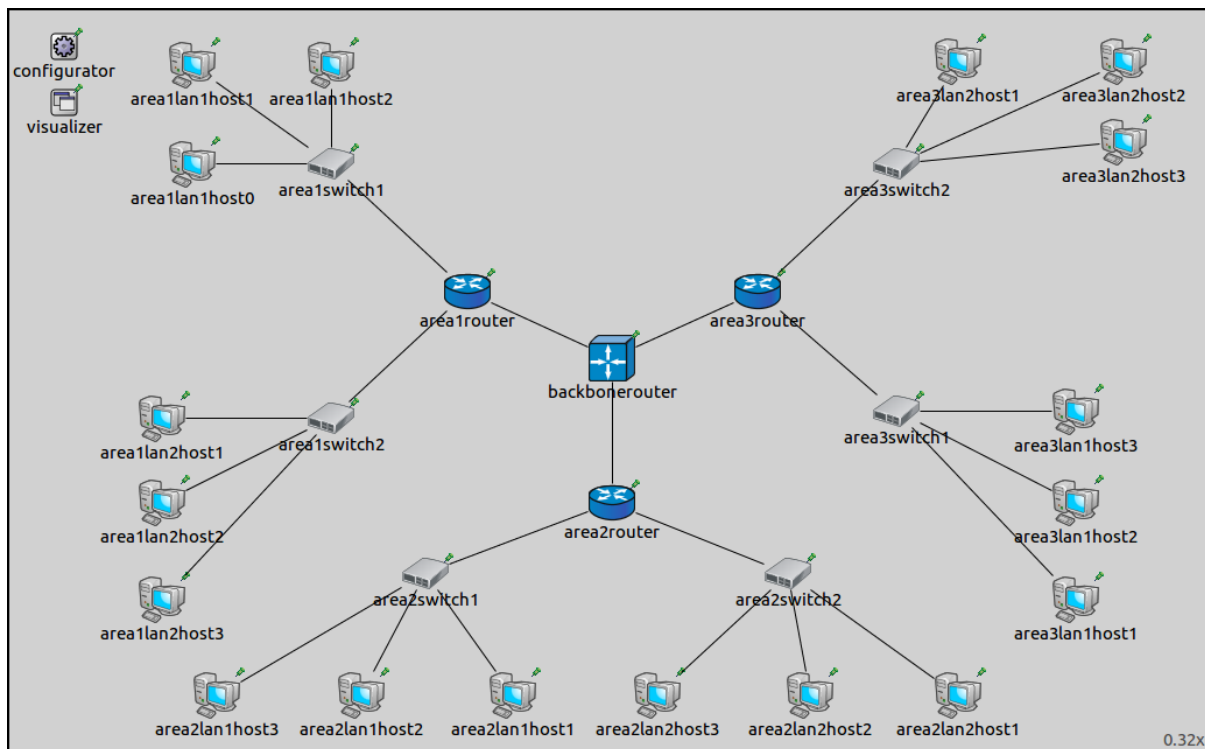
- **Part A:** Automatically assigned addresses, no optimization
- **Part B:** Automatically assigned addresses, using optimization
- **Part C:** Hierarchically assigned addresses, using optimization

2.8.2 Part A - Automatically assigned addresses

Assigning addresses hierarchically in a network with hierarchical topology can reduce the size of routing tables. However, the configurator's automatic address assignment with its default settings doesn't assign addresses hierarchically. This part uses automatic address assignment, and the configurator's routing table optimization features are turned off. The size of routing tables in this part can serve as a baseline to compare with.

Configuration

All three parts in this step use the [ConfiguratorB](#) network defined in `ConfiguratorB.ned`. The network looks like this:



The network is comprised of three areas, each containing two local area networks (LANs). Each LAN contains three hosts. The hosts in the LAN connect to an area router through switches. The three area routers connect to a central backbone router. The network contains three hierarchical levels, which correspond to the hosts in the LANs, the area routers, and the backbone router.

The configuration for this part in `omnetpp.ini` is the following:

```
[Config Step7A]
sim-time-limit = 500s
network = ConfiguratorB
```

(continues on next page)

(continued from previous page)

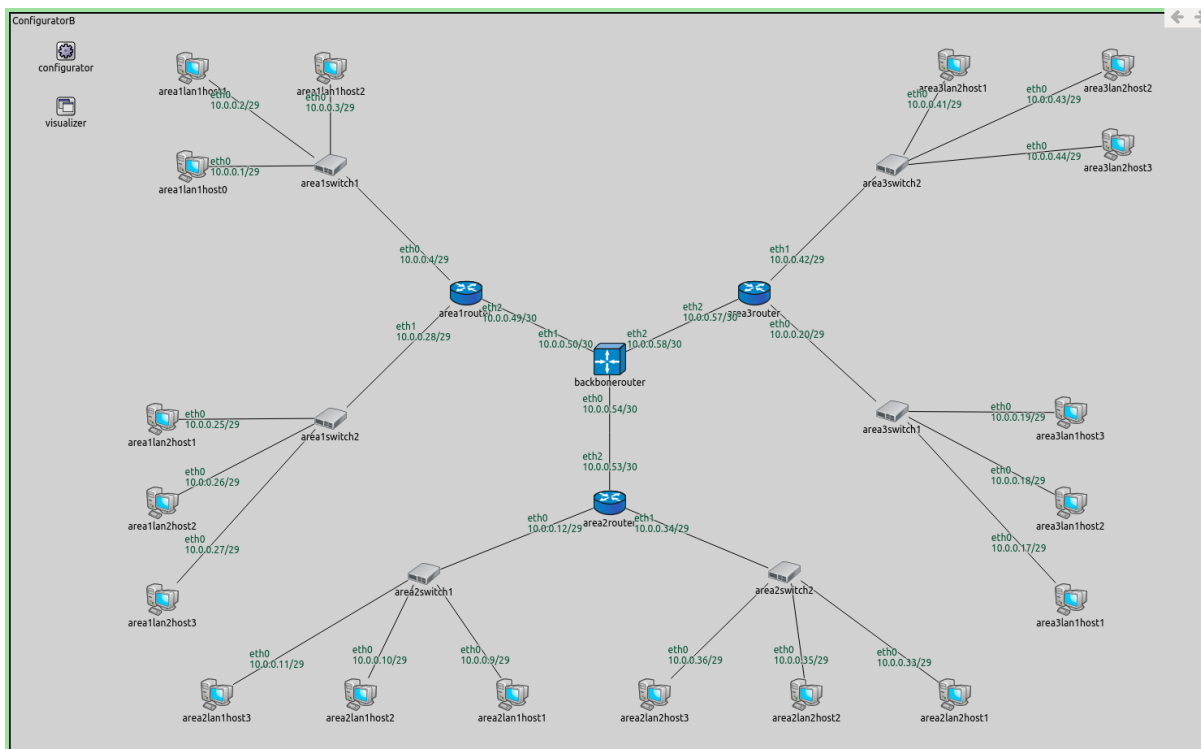
```
description = "Configuring a hierarchical network - A: non-optimized, fully automatic,
↳IP address assignment and static routes"
```

```
*.configurator.assignDisjunctSubnetAddresses = false
*.configurator.addDefaultRoutes = false
*.configurator.addSubnetRoutes = false
*.configurator.optimizeRoutes = false
```

This configuration turns off every kind of optimization relating to address assignment and route generation. This means that nodes will have an individual routing table entry to every destination interface.

Results

The assigned addresses are shown in the image below:



The size of some of the routing tables are the following:

The routing tables of a host (area1lan2host2) and a router (area1router) are shown below. The backbonerouter's routing table is similar to area1router's.

Node ConfiguratorB.area1lan2host2

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.0.0.1	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.2	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.3	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.4	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.9	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.10	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.11	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.12	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.17	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.18	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0

(continues on next page)

Name	
ConfiguratorB.area1lan1host0.routingTable.routes	size=29
ConfiguratorB.area1lan1host1.routingTable.routes	size=29
ConfiguratorB.area1lan1host2.routingTable.routes	size=29
ConfiguratorB.area1lan2host1.routingTable.routes	size=29
ConfiguratorB.area1lan2host2.routingTable.routes	size=29
ConfiguratorB.area1lan2host3.routingTable.routes	size=29
ConfiguratorB.area1router.routingTable.routes	size=27
ConfiguratorB.area2lan1host1.routingTable.routes	size=29
ConfiguratorB.area2lan1host2.routingTable.routes	size=29
ConfiguratorB.area2lan1host3.routingTable.routes	size=29
ConfiguratorB.area2lan2host1.routingTable.routes	size=29
ConfiguratorB.area2lan2host2.routingTable.routes	size=29
ConfiguratorB.area2lan2host3.routingTable.routes	size=29
ConfiguratorB.area2router.routingTable.routes	size=27
ConfiguratorB.area3lan1host1.routingTable.routes	size=29
ConfiguratorB.area3lan1host2.routingTable.routes	size=29
ConfiguratorB.area3lan1host3.routingTable.routes	size=29
ConfiguratorB.area3lan2host1.routingTable.routes	size=29
ConfiguratorB.area3lan2host2.routingTable.routes	size=29
ConfiguratorB.area3lan2host3.routingTable.routes	size=29
ConfiguratorB.area3router.routingTable.routes	size=27
ConfiguratorB.backbonerouter.routingTable.routes	size=27

(continued from previous page)

10.0.0.19	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.20	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.25	255.255.255.255	*	eth0 (10.0.0.26)	0
10.0.0.27	255.255.255.255	*	eth0 (10.0.0.26)	0
10.0.0.28	255.255.255.255	*	eth0 (10.0.0.26)	0
10.0.0.33	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.34	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.35	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.36	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.41	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.42	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.43	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.44	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.49	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.50	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.53	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.54	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.57	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
10.0.0.58	255.255.255.255	10.0.0.28	eth0 (10.0.0.26)	0
Node ConfiguratorB.arealrouter				
-- Routing table --				
Destination	Netmask	Gateway	Iface	Metric
10.0.0.1	255.255.255.255	*	eth0 (10.0.0.4)	0
10.0.0.2	255.255.255.255	*	eth0 (10.0.0.4)	0
10.0.0.3	255.255.255.255	*	eth0 (10.0.0.4)	0
10.0.0.9	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.10	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.11	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.12	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.17	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.18	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.19	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.20	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.25	255.255.255.255	*	eth1 (10.0.0.28)	0
10.0.0.26	255.255.255.255	*	eth1 (10.0.0.28)	0
10.0.0.27	255.255.255.255	*	eth1 (10.0.0.28)	0
10.0.0.33	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.34	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.35	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.36	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.41	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.42	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.43	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.44	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.50	255.255.255.255	*	eth2 (10.0.0.49)	0
10.0.0.53	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.54	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.57	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0
10.0.0.58	255.255.255.255	10.0.0.50	eth2 (10.0.0.49)	0

There are 30 interfaces in the network (18 hosts * 1 interface + 4 routers * 3 interfaces). All routing table entries have 255.255.255.255 netmasks, i.e. separate routes to all destination interfaces. Thus, hosts have 29 entries in their routing tables, for the 29 other interfaces. Similarly, routers have 27 entries.

2.8.3 Part B - Automatically assigned addresses, using optimization

In this part, we turn on the optimization features of the configurator that were turned off in Part A. This should optimize routing tables and decrease table size.

Configuration

The configuration for this part in omnetpp.ini is the following:

```
[Config Step7B]
sim-time-limit = 500s
network = ConfiguratorB
description = "Configuring a hierarchical network - B: optimized, fully automatic IP,
↳address assignment and static routes"
```

The configuration is empty, the default NED parameter values take effect. That means that the following optimization features are turned on: assignDisjunctSubnetAddresses, addDefaultRoutes, addSubnetRoutes, optimizeRoutes.

Results

The addresses are the same, but the routing table sizes have gone down:

The routing tables of a host, an area router and the backbone router are the following:

```
Node ConfiguratorB.area1lan1host0
-- Routing table --
Destination      Netmask      Gateway      Iface      Metric
10.0.0.0         255.255.255.248 *            eth0 (10.0.0.1) 0
*                *            10.0.0.4     eth0 (10.0.0.1) 0

Node ConfiguratorB.area1router
-- Routing table --
Destination      Netmask      Gateway      Iface      Metric
10.0.0.50        255.255.255.255 *            eth2 (10.0.0.49) 0
10.0.0.0         255.255.255.248 *            eth0 (10.0.0.4) 0
10.0.0.24        255.255.255.248 *            eth1 (10.0.0.28) 0
10.0.0.0         255.255.255.192 10.0.0.50    eth2 (10.0.0.49) 0

Node ConfiguratorB.backbonerouter
-- Routing table --
Destination      Netmask      Gateway      Iface      Metric
10.0.0.49        255.255.255.255 *            eth0 (10.0.0.50) 0
10.0.0.53        255.255.255.255 *            eth2 (10.0.0.54) 0
10.0.0.57        255.255.255.255 *            eth1 (10.0.0.58) 0
10.0.0.8         255.255.255.248 10.0.0.53    eth2 (10.0.0.54) 0
10.0.0.16        255.255.255.248 10.0.0.57    eth1 (10.0.0.58) 0
10.0.0.32        255.255.255.248 10.0.0.53    eth2 (10.0.0.54) 0
10.0.0.40        255.255.255.248 10.0.0.57    eth1 (10.0.0.58) 0
10.0.0.0         255.255.255.224 10.0.0.49    eth0 (10.0.0.50) 0
10.0.0.0         255.255.255.192 10.0.0.53    eth2 (10.0.0.54) 0
10.0.0.0         255.255.255.192 10.0.0.57    eth1 (10.0.0.58) 0
```

We can make the following observations:

- Hosts have just two routing table entries. One for reaching other hosts in their LANs, and a default route.
- The area routers have a rule for reaching the backbone router, two rules for reaching the two LANs they're connected to, and a default rule for reaching the rest of the network through the backbone router.
- Similarly, the backbone router has three rules for reaching the three area routers, and six rules for reaching the six LANs in the network.

Name	
ConfiguratorB.area1lan1host0.routingTable.routes	size=2
ConfiguratorB.area1lan1host1.routingTable.routes	size=2
ConfiguratorB.area1lan1host2.routingTable.routes	size=2
ConfiguratorB.area1lan2host1.routingTable.routes	size=2
ConfiguratorB.area1lan2host2.routingTable.routes	size=2
ConfiguratorB.area1lan2host3.routingTable.routes	size=2
ConfiguratorB.area1router.routingTable.routes	size=4
ConfiguratorB.area2lan1host1.routingTable.routes	size=2
ConfiguratorB.area2lan1host2.routingTable.routes	size=2
ConfiguratorB.area2lan1host3.routingTable.routes	size=2
ConfiguratorB.area2lan2host1.routingTable.routes	size=2
ConfiguratorB.area2lan2host2.routingTable.routes	size=2
ConfiguratorB.area2lan2host3.routingTable.routes	size=2
ConfiguratorB.area2router.routingTable.routes	size=4
ConfiguratorB.area3lan1host1.routingTable.routes	size=2
ConfiguratorB.area3lan1host2.routingTable.routes	size=2
ConfiguratorB.area3lan1host3.routingTable.routes	size=2
ConfiguratorB.area3lan2host1.routingTable.routes	size=2
ConfiguratorB.area3lan2host2.routingTable.routes	size=2
ConfiguratorB.area3lan2host3.routingTable.routes	size=2
ConfiguratorB.area3router.routingTable.routes	size=4
ConfiguratorB.backbonerouter.routingTable.routes	size=10

- The backbone router has separate rules for the two LANs connected to an area router because the addresses are not contiguously assigned to the two LANs (e.g. area2lan1 has address 10.0.0.8/29, area2lan2 has 10.0.0.32/29. But area3lan1 has 10.0.0.16/29, which is between the two former address ranges). Thus, area 2 cannot be covered by a single rule.

2.8.4 Part C - Hierarchically assigned addresses, optimized routing tables

Having hierarchically assigned addresses in a network results in smaller routing table sizes, because a large distant network can be covered with just one rule in a core router's routing table.

Configuration

The configuration for this part in omnetpp.ini is the following:

```
[Config Step7C]
sim-time-limit = 500s
network = ConfiguratorB
description = "Configuring a hierarchical network - C: optimized, hierarchically_
↳assigned IP addresses and static routes"

*.configurator.config = xmldoc("step7c.xml")
```

As in the previous part, all of the configurator's routing table optimization features are enabled. The XML configuration for this part in step7c.xml is the following:

```
<config>
  <interface hosts="area1lan1*" address="10.1.1.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan2*" address="10.1.2.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan1*" address="10.2.1.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan2*" address="10.2.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan1*" address="10.3.1.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan2*" address="10.3.2.x" netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area1*" address="10.1.3.x" netmask=
↳"255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area2*" address="10.2.3.x" netmask=
↳"255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area3*" address="10.3.3.x" netmask=
↳"255.255.255.x"/>
  <interface hosts="*" address="10.x.x.x" netmask="255.255.255.x"/>
</config>
```

This XML configuration assigns addresses hierarchically in the following way, when looking down the hierarchy from the backbone router towards the hosts: - The first octet of the address for all nodes is 10, i.e. 10.x.x.x - The second octet denotes the area, e.g. 10.2.x.x corresponds to area 2 - The third octet denotes the LAN within the area, e.g. 10.2.1.x corresponds to lan1 in area 2 - The forth octet is the host identifier within a LAN, e.g. 10.2.1.4 corresponds to host4 in lan1 in area 2

With this setup, it is possible to cover an area with just one rule in the routing table of the backbone router. Similarly, the area routers need two rules for each LAN that they are connected to.

Results

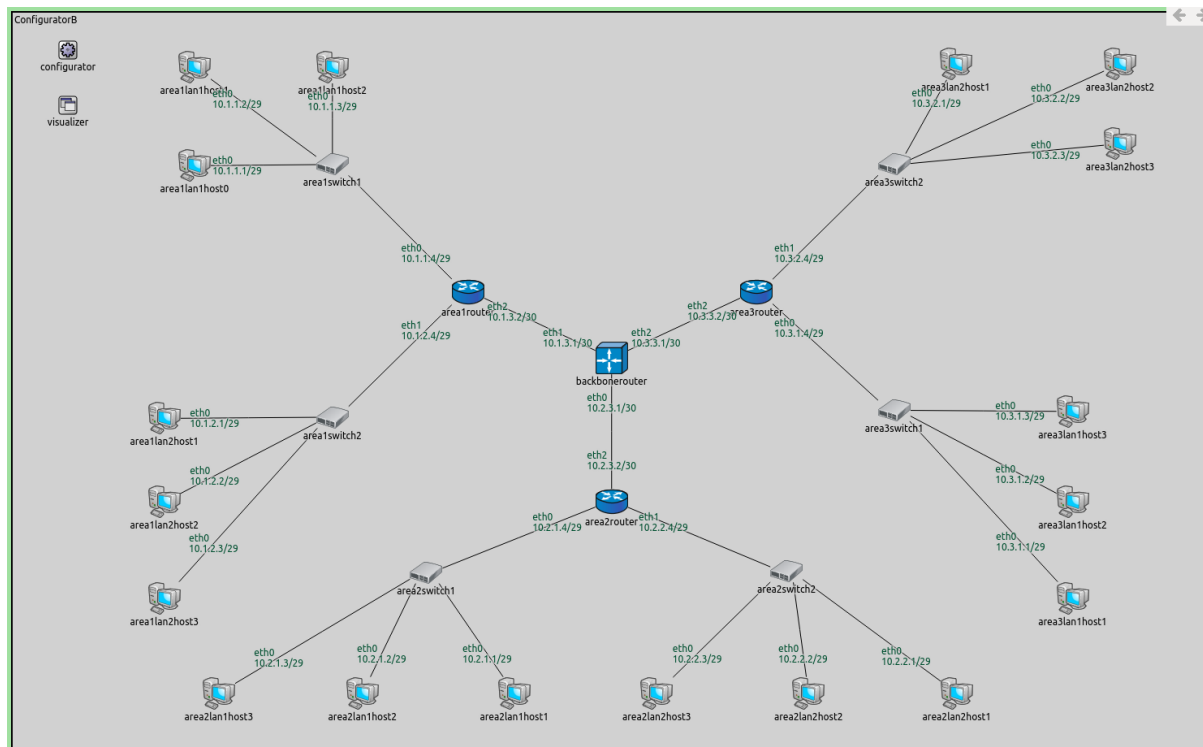
The image below shows the assigned addresses.

The sizes of some of the routing tables are displayed on the following image.

The routing tables are the following:

```
Node ConfiguratorB.area1lan1host0
-- Routing table --
```

(continues on next page)



(continued from previous page)

Destination	Netmask	Gateway	Iface	Metric
10.1.1.0	255.255.255.248	*	eth0 (10.1.1.1)	0
*	*	10.1.1.4	eth0 (10.1.1.1)	0

Node ConfiguratorB.area1router

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.1.3.1	255.255.255.255	*	eth2 (10.1.3.2)	0
10.1.1.0	255.255.255.248	*	eth0 (10.1.1.4)	0
10.1.2.0	255.255.255.248	*	eth1 (10.1.2.4)	0
10.2.0.0	255.254.0.0	10.1.3.1	eth2 (10.1.3.2)	0

Node ConfiguratorB.backbonerouter

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.1.3.2	255.255.255.255	*	eth0 (10.1.3.1)	0
10.2.3.2	255.255.255.255	*	eth2 (10.2.3.1)	0
10.3.3.2	255.255.255.255	*	eth1 (10.3.3.1)	0
10.1.0.0	255.255.252.0	10.1.3.2	eth0 (10.1.3.1)	0
10.2.0.0	255.255.252.0	10.2.3.2	eth2 (10.2.3.1)	0
10.3.0.0	255.255.252.0	10.3.3.2	eth1 (10.3.3.1)	0

Note the following:

- Hosts' routing tables contain just two rules, as in the previous part. One is for reaching the other members of the host's LAN, and a default rule for reaching everything else through the area's router.
- The area routers' routing tables contain a specific rule for reaching the backbone router, two rules for reaching the two LANs that belong to the router's area, and a default rule for reaching everything else through the backbone router.
- The backbone router's routing table contains three specific rules for reaching the three area routers, and three rules to reach the three areas.

Name	
ConfiguratorB.area1lan1host0.routingTable.routes	size=2
ConfiguratorB.area1lan1host1.routingTable.routes	size=2
ConfiguratorB.area1lan1host2.routingTable.routes	size=2
ConfiguratorB.area1lan2host1.routingTable.routes	size=2
ConfiguratorB.area1lan2host2.routingTable.routes	size=2
ConfiguratorB.area1lan2host3.routingTable.routes	size=2
ConfiguratorB.area1router.routingTable.routes	size=4
ConfiguratorB.area2lan1host1.routingTable.routes	size=2
ConfiguratorB.area2lan1host2.routingTable.routes	size=2
ConfiguratorB.area2lan1host3.routingTable.routes	size=2
ConfiguratorB.area2lan2host1.routingTable.routes	size=2
ConfiguratorB.area2lan2host2.routingTable.routes	size=2
ConfiguratorB.area2lan2host3.routingTable.routes	size=2
ConfiguratorB.area2router.routingTable.routes	size=4
ConfiguratorB.area3lan1host1.routingTable.routes	size=2
ConfiguratorB.area3lan1host2.routingTable.routes	size=2
ConfiguratorB.area3lan1host3.routingTable.routes	size=2
ConfiguratorB.area3lan2host1.routingTable.routes	size=2
ConfiguratorB.area3lan2host2.routingTable.routes	size=2
ConfiguratorB.area3lan2host3.routingTable.routes	size=2
ConfiguratorB.area3router.routingTable.routes	size=4
ConfiguratorB.backbonerouter.routingTable.routes	size=6

The difference between the configuration for this part and the previous one is that addresses are assigned hierarchically in this part. The routing table of the backbone router contains six entries instead of 10 in the previous part. The other nodes' routing tables remained the same. The difference is not drastic because the network is small. However, using hierarchical address assignment in a larger network would make a significant difference in routing table size.

Sources: `omnetpp.ini`, `ConfiguratorB.ned`

2.8.5 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.9 Step 8. Configuring a mixed wired/wireless network

2.9.1 Goals

This step demonstrates routing table configuration in a mixed wired/wireless network. This step consists of two parts:

- **Part A:** Determining members of wireless networks with the `<wireless>` element
- **Part B:** Determining members of wireless networks by SSID

2.9.2 Part A: Using the `<wireless>` element

The configurator assumes that interfaces of wireless nodes in the same wireless network can reach each other directly. It can determine which nodes belong to a wireless network in two ways:

- It looks at the `defaultSsid` parameter in nodes' agent submodule. Nodes with the same SSID are assumed to be in the same wireless network.
- Members of wireless networks can be specified by the `<wireless>` element in the XML configuration. In the `<wireless>` element, the `hosts` and `interfaces` selector attributes can be used to specify members.

Note that nodes need to use the same radio medium module to be in the same wireless network.

Configuration

This step uses the `ConfiguratorC` network defined in `ConfiguratorC.ned`. The network is displayed on the following image.

The `ConfiguratorC` network extends `ConfiguratorB` by adding two wireless LANs, `area1lan3` and `area3lan3`. The additional LANs consist of an `AccessPoint` and three `WirelessHosts`.

The default SSID setting of all wireless nodes is "SSID". By default, the configurator would put them in the same wireless network, and assume that they can all reach each other directly. This would be reflected in the routes, hosts in `area1lan3` would reach hosts in `area3lan3` directly. This is obviously wrong because they are out of range of each other. The two LANs need to be in two different wireless networks.

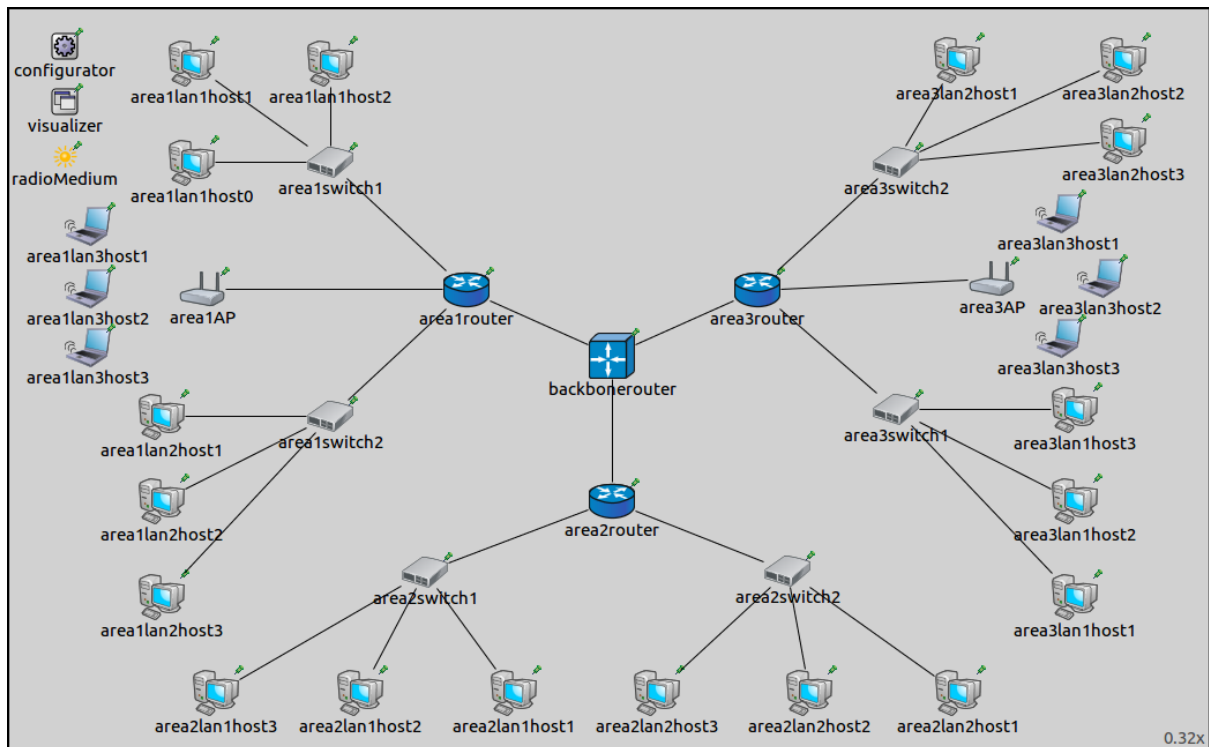
Here is the configuration for this step in `omnetpp.ini`:

```
[Config Step8A]
sim-time-limit = 100s
network = ConfiguratorC
description = "Mixed wired/wireless network configuration - using <wireless> attribute
↪"

*.configurator.config = xmldoc("step8a.xml")

*.area1lan3host2.numApps = 1
*.area1lan3host2.app[0].typename = "PingApp"
*.area1lan3host2.app[*].destAddr = "area3lan3host2"
```

(continues on next page)



(continued from previous page)

```
# Wireless settings
*.wlan[*].bitrate = 54Mbps

# visualizer settings
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "area3lan3*"

*.visualizer.routingTableVisualizer.lineShift = 0
```

A wireless host in area 1 is configured to ping a wireless host in area 3, which helps in visualizing routes.

There is a setting in the General configuration pertaining to wireless networks: the control bit rate of all wireless nodes is set to 54 Mbps for faster ACK transmissions.

```
[General]
description = "(abstract)"

# Configurator settings
*.configurator.dumpAddresses = true
*.configurator.dumpTopology = true
*.configurator.dumpLinks = true
*.configurator.dumpRoutes = true

# Routing settings
*.ipv4.arp.typename = "GlobalArp"
*.ipv4.routingTable.netmaskRoutes = ""

# Visualizer settings
*.visualizer.interfaceTableVisualizer.displayInterfaceTables = true
*.visualizer.interfaceTableVisualizer.nodeFilter = "not (*switch* or *Switch* or *AP*)"
↪ "
```

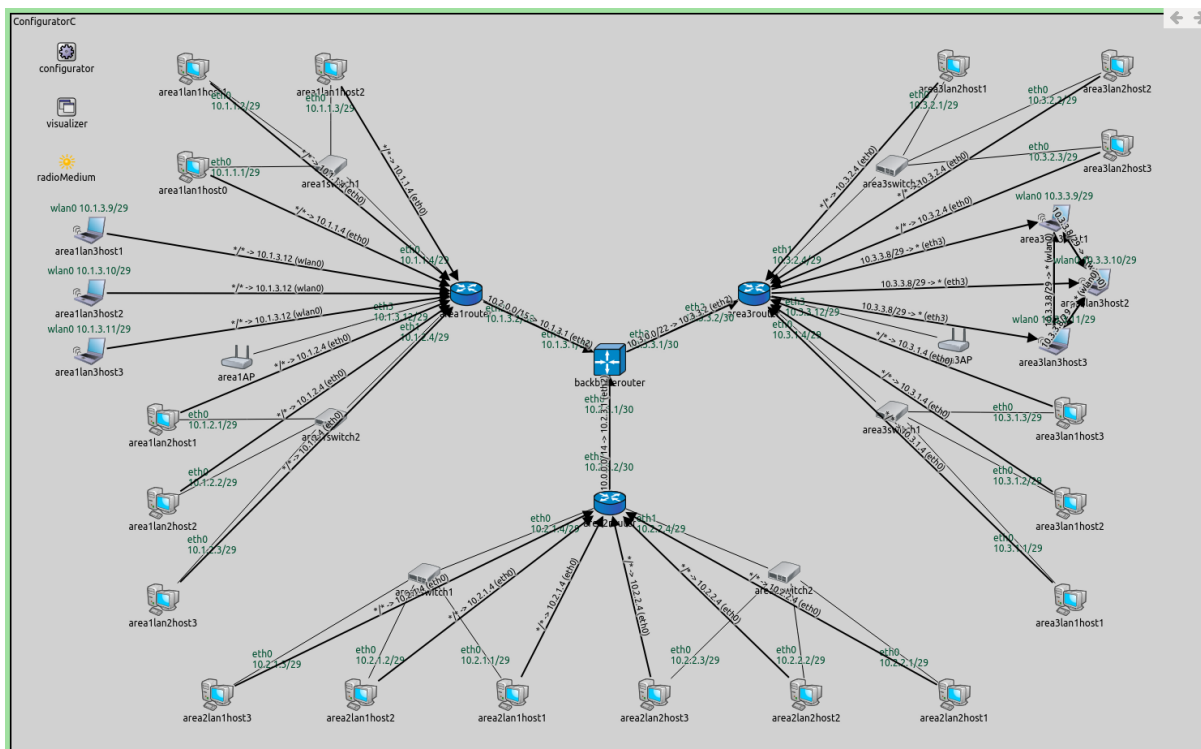
The XML configuration in step8a.xml is the following:

```
<config>
  <interface hosts="area1lan1*" address="10.1.1.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan2*" address="10.1.2.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan3*" address="10.1.3.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan1*" address="10.2.1.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan2*" address="10.2.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan1*" address="10.3.1.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan2*" address="10.3.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan3*" address="10.3.3.x" netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area1router" address="10.1.3.x"
↳netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area2router" address="10.2.3.x"
↳netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area3router" address="10.3.3.x"
↳netmask="255.255.255.x"/>
  <interface hosts="*" address="10.x.x.x" netmask="255.255.255.x"/>
  <wireless hosts="area1lan3* area1AP"/>
  <wireless hosts="area3lan3* area3AP"/>
</config>
```

The XML configuration uses the same hierarchical addressing scheme as in Step 7. The two wireless LANs are specified to be in separate wireless networks. Note that the wireless interfaces of the access points also belong to the wireless networks.

Results

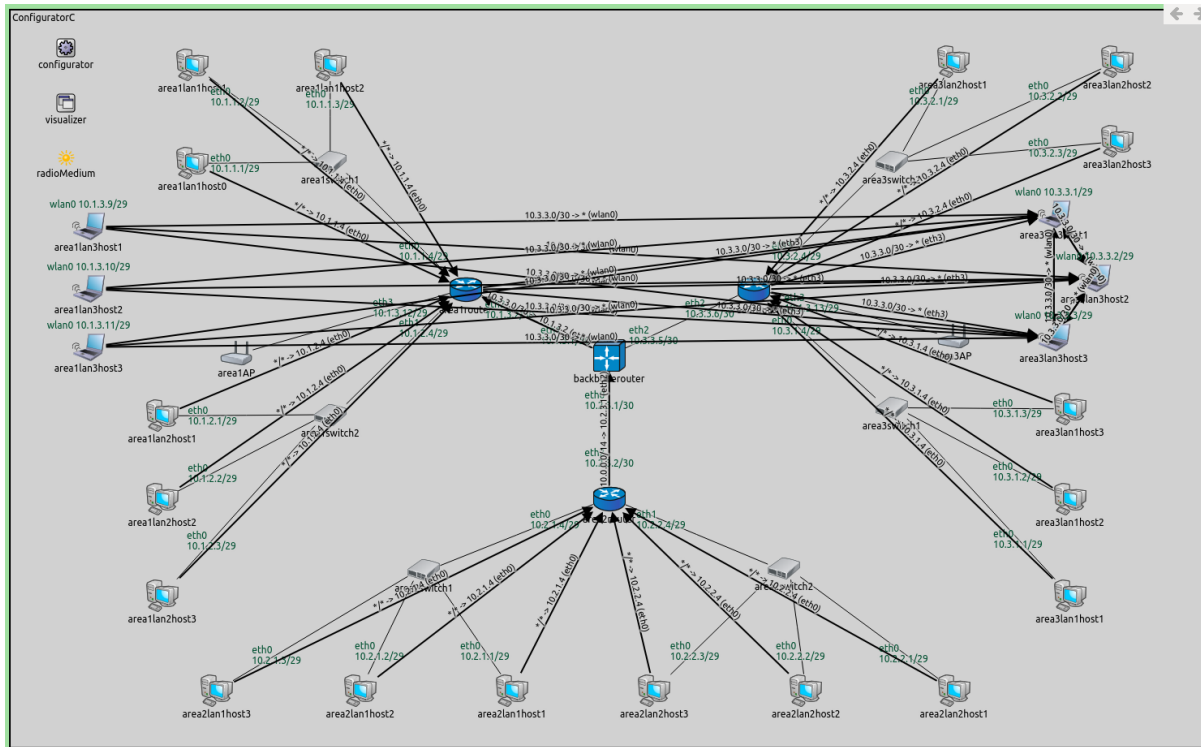
The addresses and routes are indicated in the following image. Routes leading towards hosts area3lan3 are visualized.



Wireless hosts connect to the router through the access points. The access points are L2 devices, similar to switches, so they are transparent for the routing table visualizer arrows. Wireless hosts get associated with their corresponding access points before they can communicate with the rest of the network.

On the following video, area1lan3host2 pings area3lan3host2. Transmissions are sent to all wireless nodes, but only those in communication range can receive them correctly (i.e. nodes in the same LAN).

The following image shows how the routes would look like if the XML configuration didn't contain the <wireless> elements:



There are no routes visualized between the backbonerouter and area3router, because routes towards area3lan3 go via arearouter. Area3lan3 can be reached from the backbonerouter in two ways:

- arearouter → area1AP → area3lan3 (area1AP reaches area3lan3 directly since they are assumed to be in the same wireless network)
- area3router → area3AP → area3lan3

In both ways, area3lan3 is two hops away from the backbonerouter. Routes are configured according to hop count, and the configurator chose the first way.

2.9.3 Part B - Using SSID

In this part, the SSID is used to put the two wireless LANs in two different wireless networks.

Configuration

The configuration for this part extends Part A. The configuration in omnetpp.ini is the following:

```
[Config Step8B]
sim-time-limit = 100s
extends = Step8A
description = "Mixed wired/wireless network configuration - using SSID"

*.configurator.config = xmldoc("step8b.xml")

*.area1AP.wlan[*].mgmt.ssid = "area1"
*.area3AP.wlan[*].mgmt.ssid = "area3"
```

(continues on next page)

(continued from previous page)

```
*.area1lan3*.wlan[*].agent.defaultSsid = "area1"
*.area3lan3*.wlan[*].agent.defaultSsid = "area3"
```

The XML configuration in step8b.xml (displayed below) is the same as the XML configuration for Part A, except there are no <wireless> elements that defined wireless networks. They are not needed because different SSIDs are configured for the members of the two wireless LANs.

```
<config>
  <interface hosts="area1lan1*" address="10.1.1.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan2*" address="10.1.2.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan3*" address="10.1.3.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan1*" address="10.2.1.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan2*" address="10.2.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan1*" address="10.3.1.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan2*" address="10.3.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan3*" address="10.3.3.x" netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="arearouter" address="10.1.3.x"
↪netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area2router" address="10.2.3.x"
↪netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area3router" address="10.3.3.x"
↪netmask="255.255.255.x"/>
  <interface hosts="*" address="10.x.x.x" netmask="255.255.255.x"/>
</config>
```

Results

The results are the same as in the previous part. The two wireless LANs are considered to be different wireless networks.

Sources: omnetpp.ini, ConfiguratorC.ned

2.9.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.10 Step 9. Leaving some part of the network unconfigured

2.10.1 Goals

Configuring the whole network is not always desirable, because some parts of the network should rather be configured dynamically. In this step, some wired and wireless LANs' addresses are left unspecified by the configurator, and they get addresses with DHCP.

2.10.2 The model

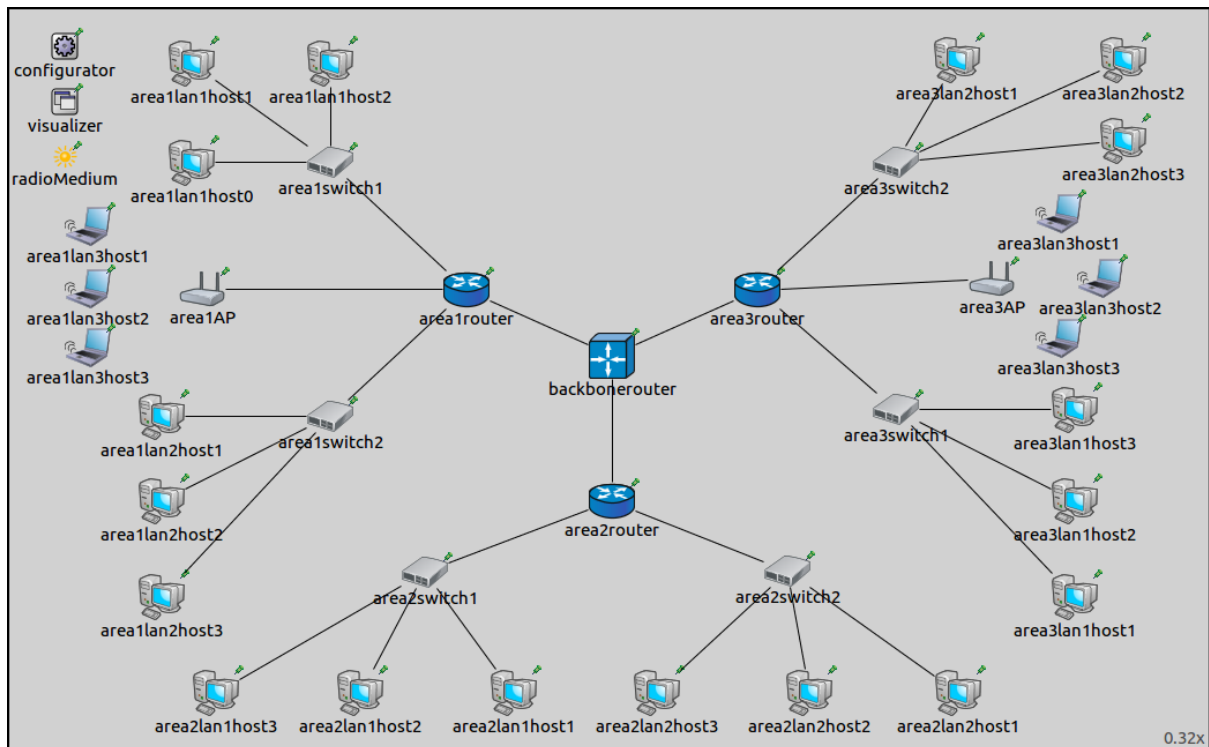
This step uses the [ConfiguratorC](#) network, defined in ConfiguratorC.ned.

The configuration for this step in omnetpp.ini is the following:

```
[Config Step9]
sim-time-limit = 100s
network = ConfiguratorC
description = "Leaving some part of the network unconfigured"

# Configurator settings
*.configurator.config = xmldoc("step9.xml")
```

(continues on next page)



(continued from previous page)

```
# SSID settings
*.area1AP.wlan[*].mgmt.ssid = "area1lan3"
*.area3AP.wlan[*].mgmt.ssid = "area3lan3"

*.area1lan3host*.wlan[*].agent.defaultSsid = "area1lan3"
*.area3lan3host*.wlan[*].agent.defaultSsid = "area3lan3"

# DHCP server in routers
*.area1router.hasDhcp = true
*.area1router.dhcp.numReservedAddresses = 2
*.area1router.dhcp.leaseTime = 100s
*.area1router.dhcp.maxNumClients = 3
*.area1router.dhcp.interface = "eth3"

*.area2router.hasDhcp = true
*.area2router.dhcp.numReservedAddresses = 2
*.area2router.dhcp.leaseTime = 100s
*.area2router.dhcp.maxNumClients = 3
*.area2router.dhcp.interface = "eth0"

*.area3router.hasDhcp = true
*.area3router.dhcp.numReservedAddresses = 2
*.area3router.dhcp.leaseTime = 100s
*.area3router.dhcp.maxNumClients = 3
*.area3router.dhcp.interface = "eth3"

# DHCP in hosts
*.area1lan3host1.numApps = 1
*.area1lan3host2.numApps = 2
*.area1lan3host3.numApps = 1
```

(continues on next page)

(continued from previous page)

```

*.area1lan3*.app[0].typename = "DhcpClient"

*.area2lan1*.numApps = 1
*.area2lan1*.app[0].typename = "DhcpClient"

*.area3lan3*.numApps = 1
*.area3lan3*.app[*].typename = "DhcpClient"

# PingApp in host
*.area1lan3host2.app[1].typename = "PingApp"
*.area1lan3host2.app[1].destAddr = "area3lan3host2"
*.area1lan3host2.app[1].startTime = 3s
# TODO: should it be [0] or [*] ?
# Visualizer settings
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "area3lan3host2"

**.forwarding = true

# Wireless settings
*.*.wlan[*].bitrate = 54Mbps

```

It boils down to the following:

- Similarly to Step 8B, members of the two wireless LANs are specified by SSID.
- Area1lan3host2 is configured to ping area3lan3host3. The ping application is delayed, so it starts sending pings after the hosts associated with the access points and got their addresses from the DHCP servers.
- DhcpServer submodules are added to the area routers. The DHCP server is configured to listen on the interface connecting to the unspecified LAN. The interface's netmask is the DHCP server's address range.
- DhcpClient submodules are added to the LANs which are unspecified by the configurator. There is one such LAN in each area; they are area1lan3, area2lan1 and area3lan3. Hosts in these LANs get the addresses from the DHCP server in the corresponding area router.
- Routes to area3lan3host3 are visualized.

The XML configuration in step9.xml is the following:

```

<config>
  <interface hosts="area1lan1*" address="10.1.1.x" netmask="255.255.255.x"/>
  <interface hosts="area1lan2*" address="10.1.2.x" netmask="255.255.255.x"/>
  <interface hosts="area2lan2*" address="10.2.2.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan1*" address="10.3.1.x" netmask="255.255.255.x"/>
  <interface hosts="area3lan2*" address="10.3.2.x" netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area1router" address="10.1.0.x" ↵
↵netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area2router" address="10.2.0.x" ↵
↵netmask="255.255.255.x"/>
  <interface hosts="backbonerouter" towards="area3router" address="10.3.0.x" ↵
↵netmask="255.255.255.x"/>
  <interface hosts="area1router" names="eth3" address="10.1.3.1" netmask="255.
↵255.255.x"/>
  <interface hosts="area2router" names="eth0" address="10.2.1.1" netmask="255.
↵255.255.x"/>
  <interface hosts="area3router" names="eth3" address="10.3.3.1" netmask="255.
↵255.255.x"/>

```

(continues on next page)

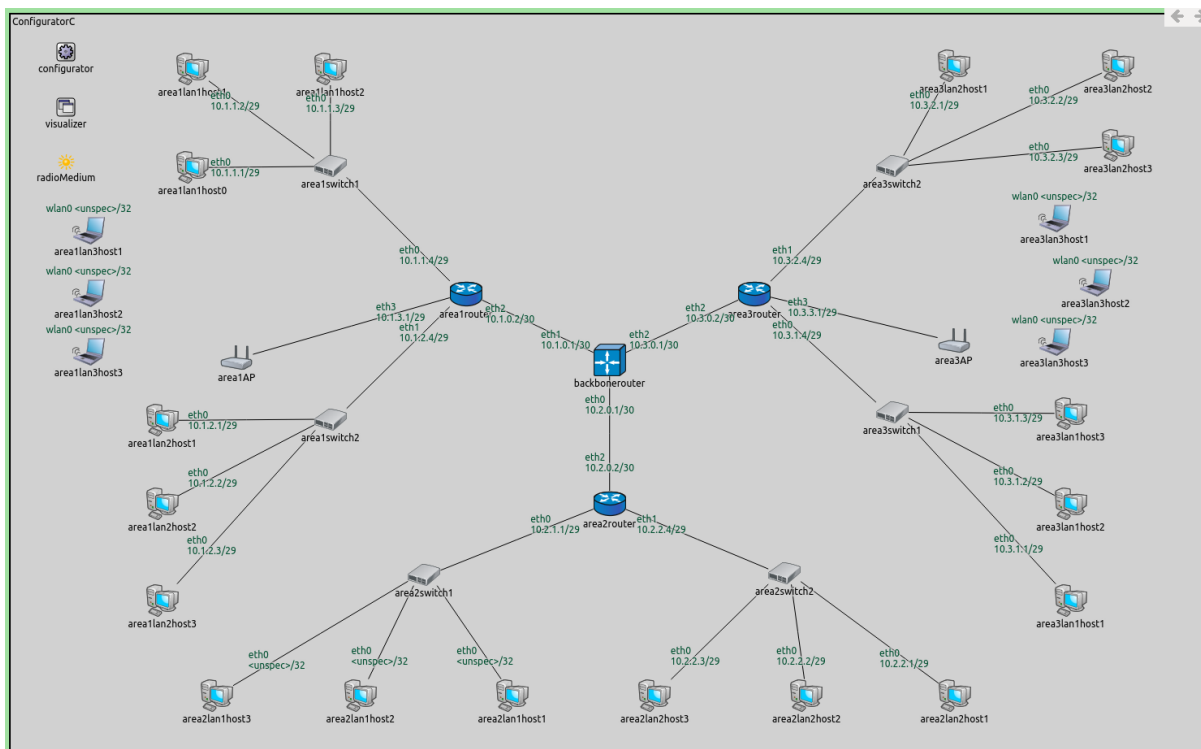
(continued from previous page)

```
<interface hosts="*router*" address="10.x.x.x" netmask="255.255.255.x"/>
</config>
```

Addresses are assigned hierarchically. Five LANs in the network have addresses assigned by the configurator. Three LANs get their addresses from DHCP servers, their interfaces are left unspecified by the configurator. This is accomplished by the lack of address assignment rules for these hosts in the XML configuration. The area routers' interfaces connecting to the latter LANs need to be specified in order to have correct routes to these LANs. Additionally, the addresses for these interfaces need to be assigned specifically, and they have to fall in the configured DHCP server address ranges.

2.10.3 Results

The addresses and routes are visualized below. The state of the network at the start of the simulation is shown on the following image:



The hosts of area1lan3, area2lan1, and area3lan3 have unspecified addresses. The routing tables of all hosts contain subnet routes to these three LANs. Since these hosts don't have addresses at the start of the simulation, there are no routes leading to area3lan3host2 that can be visualized.

Though the hosts in the three LANs have unspecified addresses, subnet routes leading to these LANs are added to the routing tables of all hosts. The addresses for the interfaces connecting to these LANs have a netmask assigned, so there are addresses allocated for the unspecified hosts. For example, area1router's eth3 interface has the address 10.1.4.1/29 and has four addresses allocated (10.1.4.2..5).

The routing tables of area1lan3host2, area1router and backbonerouter are the following (routes for reaching the unspecified hosts are highlighted):

Node ConfiguratorC.area1lan3host2

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.1.3.0	255.255.255.248	*	wlan0 (unspec)	0
*	*	10.1.3.1	wlan0 (unspec)	0

(continues on next page)

(continued from previous page)

Node ConfiguratorC.area1router

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.1.0.1	255.255.255.255	*	eth2 (10.1.0.2)	0
10.1.1.0	255.255.255.248	*	eth0 (10.1.1.4)	0
10.1.2.0	255.255.255.248	*	eth1 (10.1.2.4)	0
10.1.3.0	255.255.255.248	*	eth3 (10.1.3.1)	0
10.2.0.0	255.254.0.0	10.1.0.1	eth2 (10.1.0.2)	0

Node ConfiguratorC.backbonerouter

-- Routing table --

Destination	Netmask	Gateway	Iface	Metric
10.1.0.2	255.255.255.255	*	eth1 (10.1.0.1)	0
10.2.0.2	255.255.255.255	*	eth0 (10.2.0.1)	0
10.3.0.2	255.255.255.255	*	eth2 (10.3.0.1)	0
10.1.0.0	255.255.252.0	10.1.0.2	eth1 (10.1.0.1)	0
10.2.0.0	255.255.252.0	10.2.0.2	eth0 (10.2.0.1)	0
10.3.0.0	255.255.252.0	10.3.0.2	eth2 (10.3.0.1)	0

Note

- area1lan3host2 has a default route for reaching the other hosts in the LAN.
- area1Router has a route for reaching hosts in area1lan3, and a default route for reaching area 2 and area 3.
- backbonerouter has subnet routes to each area.

In the following video, area1lan3host2 sends a ping packet to area3lan3host2:

No routes are visualized initially because area3lan3host2 (the destination of route visualization) has an unspecified IP address. When it gets an address from the DHCP server, the routes leading towards area3lan3host2 appear.

Sources: omnetpp.ini, ConfiguratorC.ned

2.10.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.11 Step 10. Configuring a completely wireless network

2.11.1 Goals

This step demonstrates using the error rate metric for configuring static routes. It also demonstrates leaving the routing tables unconfigured, so that a dynamic routing protocol can configure them. The step consists of three parts:

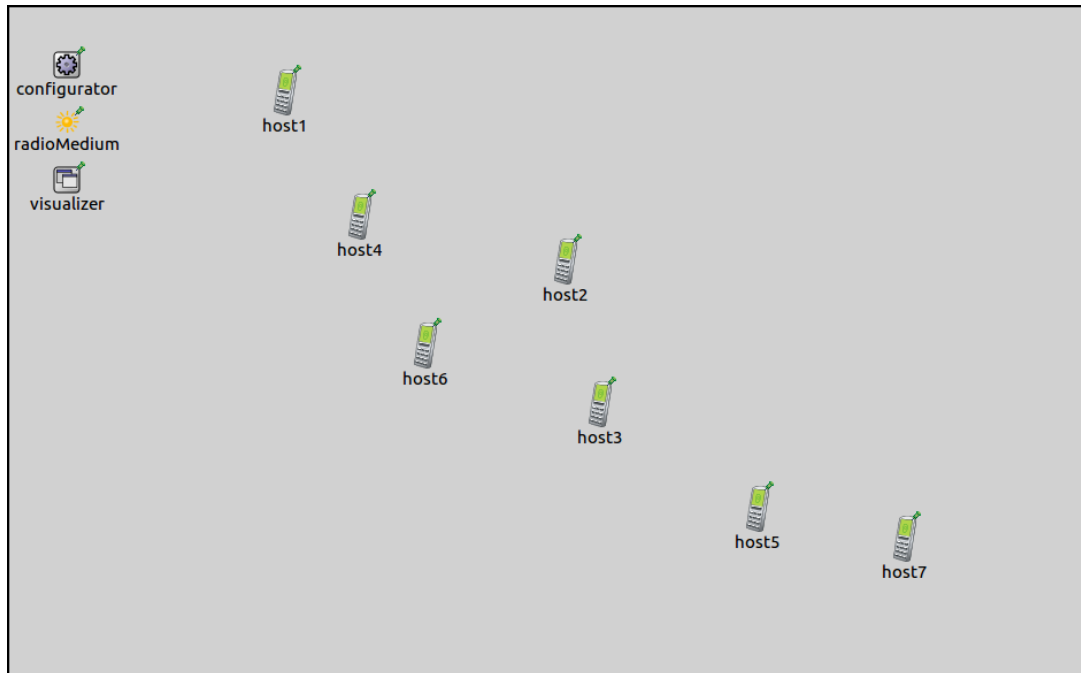
- **Part A:** Static routing based on the error rate metric
- **Part B:** Unconfigured routing tables, prepared for MANET routing
- **Part C:** Routing tables configured using AODV protocol

2.11.2 Part A: Static routing based on error rate metric

The topology of completely wireless networks is unclear in a static analysis. By default, the configurator assumes all nodes can directly talk to each other. When they can't, the error rate metric can be used for automatic route configuration instead of the default hop count.

Configuration

This step uses the [ConfiguratorD](#) network, defined in `ConfiguratorD.ned`. The network looks like this:



It contains seven AODVRouters laid out in a chain.

The configuration for this part in `omnetpp.ini` is the following:

```
[Config Step10A]
sim-time-limit = 100s
network = ConfiguratorD
description = "Completely wireless network, static routing based on error rate metric"

*.configurator.config = xmldoc("step10a.xml")

# Wireless settings
*.wlan[*].bitrate = 54Mbps
*.wlan[*].radio.transmitter.power = 1mW

*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.labelFormat = ""
*.visualizer.routingTableVisualizer.lineShiftMode = "none"
*.visualizer.routingTableVisualizer.destinationFilter = "*"
*.visualizer.mediumVisualizer.displayCommunicationRanges = true
```

The transmitter power of radios determines their communication range. The range is set up so that hosts are only in the range of the adjacent hosts in the chain. `RoutingTableCanvasVisualizer` is set to visualize routes to all destinations. The routing table visualization is simplified by turning off arrow labels and setting the arrow line shift to 0. The latter setting causes the visualizer to draw only one arrow between any nodes even if there would be multiple arrows, e.g. one for both directions (bi-directional routes will be displayed as bi-directional arrows now.) Communication ranges of all hosts will be displayed.

The transmission power outside the communication range is below the sensitivity of the receiving node, thus, the error rate is infinite. However, the fact that the receiving host is within the communication range circle doesn't mean that it can receive the transmission correctly.

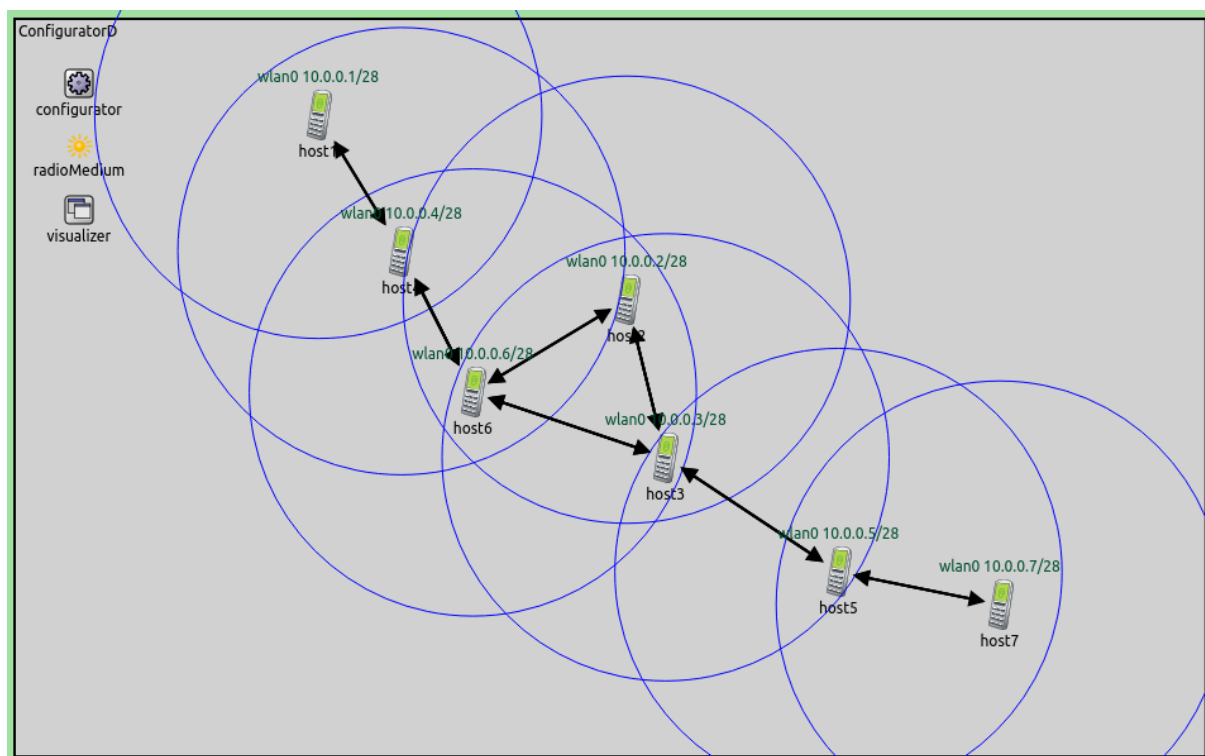
The XML configuration in step10a.xml is as follows:

```
<config>
  <interface hosts='**' address='10.x.x.x' netmask='255.x.x.x' />
  <autoroute metric="errorRate" />
</config>
```

It contains a copy of the default address configurations, and an <autoroute> element using the error rate metric. The configurator calculates the packet error rate for a Maximum Transfer Unit (MTU) sized packet. Edge costs in the connectivity graph are assigned accordingly.

Results

Configured routes and communication ranges are displayed on the following image. Error rate outside the communication range is infinite, thus, all arrows are within the circles. Routes lead through adjacent hosts in the chain. In each segment of the path, correct reception is possible.



2.11.3 Part B: Unconfigured routing tables, prepared for MANET routing

Static routing is often not adequate in wireless networks, as the nodes might move and the topology can change. Dynamic routing protocols can react to these changes. When using dynamic protocols, the configurator is only used to configure the addresses. It leaves the routing table configuration to the dynamic protocol.

The configuration for this part in omnetpp.ini extends the one for Part A:

```
[Config Step10B]
sim-time-limit = 100s
extends = Step10A
description = "Completely wireless network, routing tables unconfigured, prepared for_
↳MANET routing"
```

(continues on next page)

(continued from previous page)

```
*.configurator.addStaticRoutes = false
```

The configurator is instructed to leave the routing tables empty, by setting `addStaticRoutes` to false. The configurator just assigns the addresses according to the default XML configuration.

The visualizer is still set to visualize all routes towards all destinations.

Results



As instructed, the configurator didn't add any routes, as indicated by the lack of arrows. The routing tables are empty.

Name	Info
ConfiguratorE.host1.routingTable.routes	empty
ConfiguratorE.host2.routingTable.routes	empty
ConfiguratorE.host3.routingTable.routes	empty
ConfiguratorE.host4.routingTable.routes	empty
ConfiguratorE.host5.routingTable.routes	empty
ConfiguratorE.host6.routingTable.routes	empty
ConfiguratorE.host7.routingTable.routes	empty

2.11.4 Part C: Routing tables configured using AODV protocol

In this part, routing tables are set up by the Ad-hoc On-demand Distance Vector (AODV) dynamic routing protocol. The configuration for this part extends Part B. The configuration in `omnetpp.ini` is the following:

```
[Config Step10C]
sim-time-limit = 100s
extends = Step10B
description = "Completely wireless network, routing tables unconfigured, using AODV_
↳routing"

*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[*].destAddr = "host7"
```

As specified in the previous part, the configurator is still instructed not to add any routes. Also, the visualizer is still set to visualize all routes. Additionally, host1 is set to ping host2. Since AODV is a reactive routing protocol, the ping is required to trigger the AODV protocol to set up routes.

Results

The routing tables are initially empty. The first ping packet triggers AODV's route discovery process, which eventually configures the routes. AODV is a reactive protocol, so unused routes expire after a while. This happens to the routes to host2, as it's not in the path between host1 and host7. This is displayed in the following video.

To record the video, the simulation was run in fast mode, thus, routes appear instantly. It takes a few seconds of simulation time for the unused routes to expire.

Sources: `omnetpp.ini`, `ConfiguratorD.ned`

2.11.5 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.12 Step 11. Manually modifying an automatically created configuration

2.12.1 Goals

Sometimes the configurator's configuration is just almost right. In such a case, it's possible to dump the configuration into a file, edit it and use the file in place of the original configuration. This step consists of two parts:

- **Part A** - Dumping the full configuration
- **Part B** - Using the modified configuration

2.12.2 Part A - Dumping the full configuration

The configurator can be instructed to dump its configuration into a config file in the XML configuration format. This file contains all the assigned addresses, routing table entries, and members of wireless links, so they can be easily modified. The modified config file can be used as the XML configuration for subsequent simulation runs.

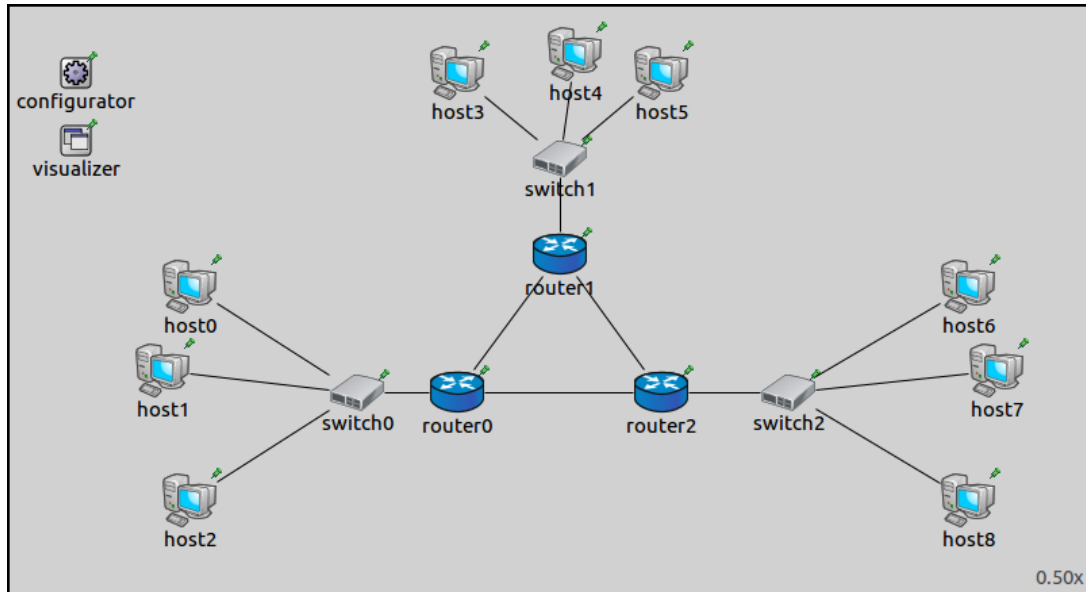
Configuration

Both parts in this step use the `ConfiguratorA` network defined in `ConfiguratorA.ned`:

The configuration for this part in `omnetpp.ini` is the following:

```
[Config Step11A]
sim-time-limit = 500s
network = ConfiguratorA
description = "Manually modifying an automatically created configuration - dumping_
↳the full configuration"
```

(continues on next page)



(continued from previous page)

```

*.configurator.dumpConfig = "step11a_dump.xml"

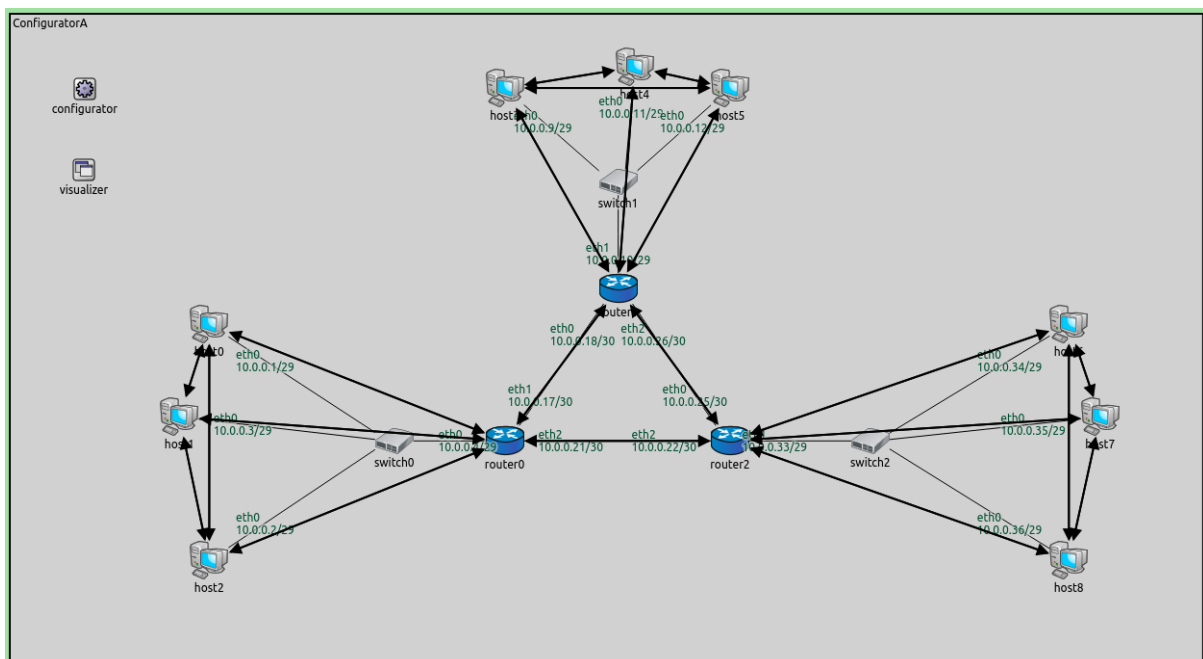
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "*"
*.visualizer.routingTableVisualizer.displayRoutesIndividually = false
*.visualizer.routingTableVisualizer.displayLabels = false
*.visualizer.routingTableVisualizer.lineShift = 0

```

The configurator's `dumpConfig` parameter can be used to dump the configuration into a file. The parameter's value is the name of the config file.

Results

Routes to all nodes are visualized in the following image.



The configuration is dumped into `step11a_dump.xml`. We will modify this config file in the next part.

2.12.3 Part B - Using the modified configuration

In this part, we edit the config file and use it as the XML configuration. The goal is that packets should travel counter-clockwise in the triangle of the three routers, i.e., each router should forward packets in the triangle through its interface on the right.

Configuration

The configuration for this part in `omnetpp.ini` is the following:

```
[Config Step11B]
sim-time-limit = 500s
network = ConfiguratorA
description = "Manually modifying an automatically created configuration - using the_
↳modified configuration"

# Configurator settings
*.configurator.config = xmldoc("step11b.xml")
*.configurator.addStaticRoutes = false

# Ping settings
*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[*].destAddr = "host4"

*.host4.numApps = 1
*.host4.app[0].typename = "PingApp"
*.host4.app[*].destAddr = "host7"

*.host7.numApps = 1
*.host7.app[0].typename = "PingApp"
*.host7.app[*].destAddr = "host1"

# Visualizer settings
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "*"
*.visualizer.routingTableVisualizer.displayRoutesIndividually = false
*.visualizer.routingTableVisualizer.displayLabels = false
*.visualizer.routingTableVisualizer.lineShift = 0
```

The modified config is used as the XML configuration. Since the configuration specifies all routes, `addStaticRoutes` needs to be set to `false`, so the configurator doesn't add automatic static routes. A host in each LAN pings another host in the adjacent LAN in the counter-clockwise direction.

The routes in all three routers' routing tables are modified. Routes that would send packets the wrong way (i.e., not counter-clockwise in the triangle) are redirected to the other interface. In essence, all routers send out packets through their interface to the right (except for packets destined for the connecting LAN.)

The modified XML configuration is in `step11b.xml` (see `step11a_dump.xml` for the original). The differences between the original and the modified config files are displayed below (the original is shown in red).

Results

Routes to all nodes are visualized in the following image. Note that arrows point only counter-clockwise in the triangle.

The ping exchanges highlight the modified routes in the following video:

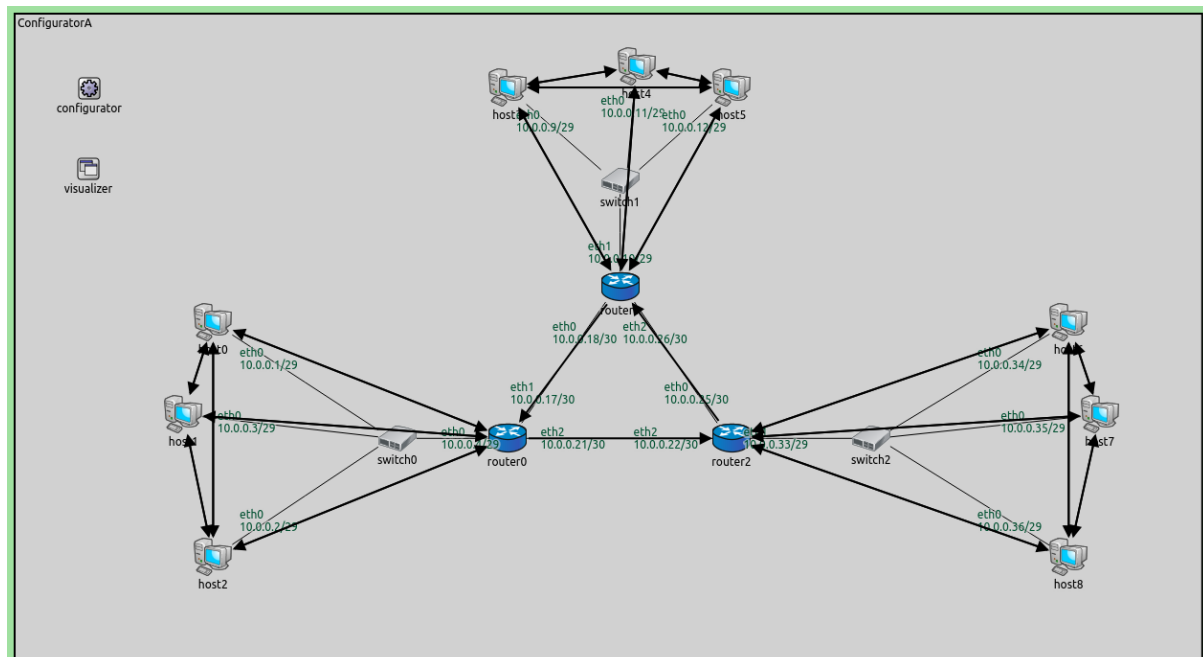
The packets travel only counter-clockwise in the triangle of the three routers.

Sources: `omnetpp.ini`, `ConfiguratorA.ned`


```

<route hosts="ConfiguratorB.host2" destination="" netmask="" gateway="10.0.0.4" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.host3" destination="10.0.0.8" netmask="255.255.255.248" gateway="" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.host3" destination="" netmask="" gateway="10.0.0.10" interface="eth0" metric="0"/>
- <route hosts="ConfiguratorB.router0" destination="10.0.0.18" netmask="255.255.255.255" gateway="" interface="eth1" metric="0"/>
+ <route hosts="ConfiguratorB.router0" destination="10.0.0.18" netmask="255.255.255.255" gateway="10.0.0.22" interface="eth2" metric="0"/>
<route hosts="ConfiguratorB.router0" destination="10.0.0.22" netmask="255.255.255.255" gateway="" interface="eth2" metric="0"/>
<route hosts="ConfiguratorB.router0" destination="10.0.0.25" netmask="255.255.255.255" gateway="10.0.0.22" interface="eth2" metric="0"/>
<route hosts="ConfiguratorB.router0" destination="10.0.0.0" netmask="255.255.255.248" gateway="" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router0" destination="10.0.0.32" netmask="255.255.255.248" gateway="10.0.0.22" interface="eth2" metric="0"/>
- <route hosts="ConfiguratorB.router0" destination="10.0.0.0" netmask="255.255.255.224" gateway="10.0.0.18" interface="eth1" metric="0"/>
+ <route hosts="ConfiguratorB.router0" destination="10.0.0.0" netmask="255.255.255.224" gateway="10.0.0.22" interface="eth2" metric="0"/>
<route hosts="ConfiguratorB.router2" destination="10.0.0.18" netmask="255.255.255.255" gateway="10.0.0.26" interface="eth0" metric="0"/>
- <route hosts="ConfiguratorB.router2" destination="10.0.0.21" netmask="255.255.255.255" gateway="" interface="eth2" metric="0"/>
+ <route hosts="ConfiguratorB.router2" destination="10.0.0.21" netmask="255.255.255.255" gateway="10.0.0.26" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router2" destination="10.0.0.26" netmask="255.255.255.255" gateway="" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router2" destination="10.0.0.26" netmask="255.255.255.248" gateway="10.0.0.26" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router2" destination="10.0.0.32" netmask="255.255.255.248" gateway="" interface="eth1" metric="0"/>
- <route hosts="ConfiguratorB.router2" destination="10.0.0.0" netmask="255.255.255.224" gateway="10.0.0.21" interface="eth2" metric="0"/>
+ <route hosts="ConfiguratorB.router2" destination="10.0.0.0" netmask="255.255.255.224" gateway="10.0.0.26" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router1" destination="10.0.0.17" netmask="255.255.255.255" gateway="" interface="eth0" metric="0"/>
- <route hosts="ConfiguratorB.router1" destination="10.0.0.22" netmask="255.255.255.255" gateway="10.0.0.25" interface="eth2" metric="0"/>
- <route hosts="ConfiguratorB.router1" destination="10.0.0.25" netmask="255.255.255.255" gateway="" interface="eth2" metric="0"/>
+ <route hosts="ConfiguratorB.router1" destination="10.0.0.22" netmask="255.255.255.255" gateway="10.0.0.17" interface="eth0" metric="0"/>
+ <route hosts="ConfiguratorB.router1" destination="10.0.0.25" netmask="255.255.255.255" gateway="10.0.0.17" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router1" destination="10.0.0.8" netmask="255.255.255.248" gateway="" interface="eth1" metric="0"/>
- <route hosts="ConfiguratorB.router1" destination="10.0.0.32" netmask="255.255.255.248" gateway="10.0.0.25" interface="eth2" metric="0"/>
+ <route hosts="ConfiguratorB.router1" destination="10.0.0.32" netmask="255.255.255.248" gateway="10.0.0.17" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.router1" destination="10.0.0.0" netmask="255.255.255.224" gateway="10.0.0.17" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.host4" destination="10.0.0.8" netmask="255.255.255.248" gateway="" interface="eth0" metric="0"/>
<route hosts="ConfiguratorB.host4" destination="" netmask="" gateway="10.0.0.10" interface="eth0" metric="0"/>

```



2.12.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

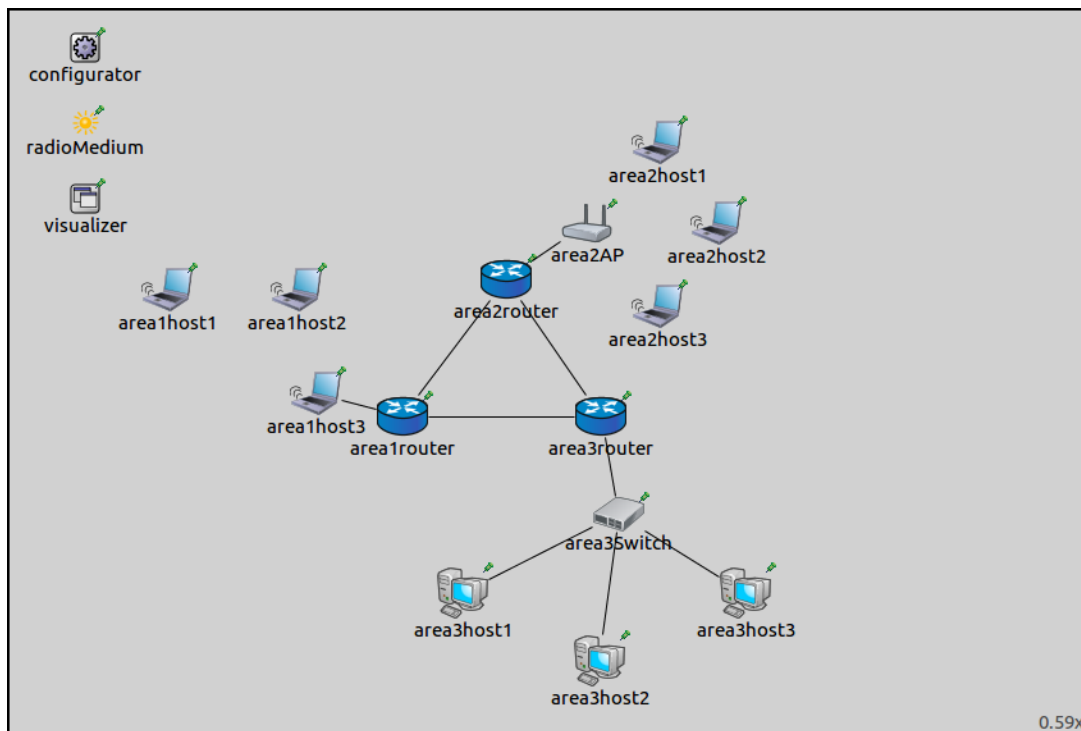
2.13 Step 12. Mixing different kinds of autorouting

2.13.1 Goals

Sometimes it is best to configure different parts of a network according to different metrics. This step demonstrates using the hop count and error rate metrics in a mixed wired/wireless network.

2.13.2 The model

This step uses the [ConfiguratorE](#) network, defined in `ConfiguratorE.ned`. The network looks like this:



The core of the network is composed of three routers connected to each other, each belonging to an area. There are three areas, each containing a number of hosts, connected to the area router.

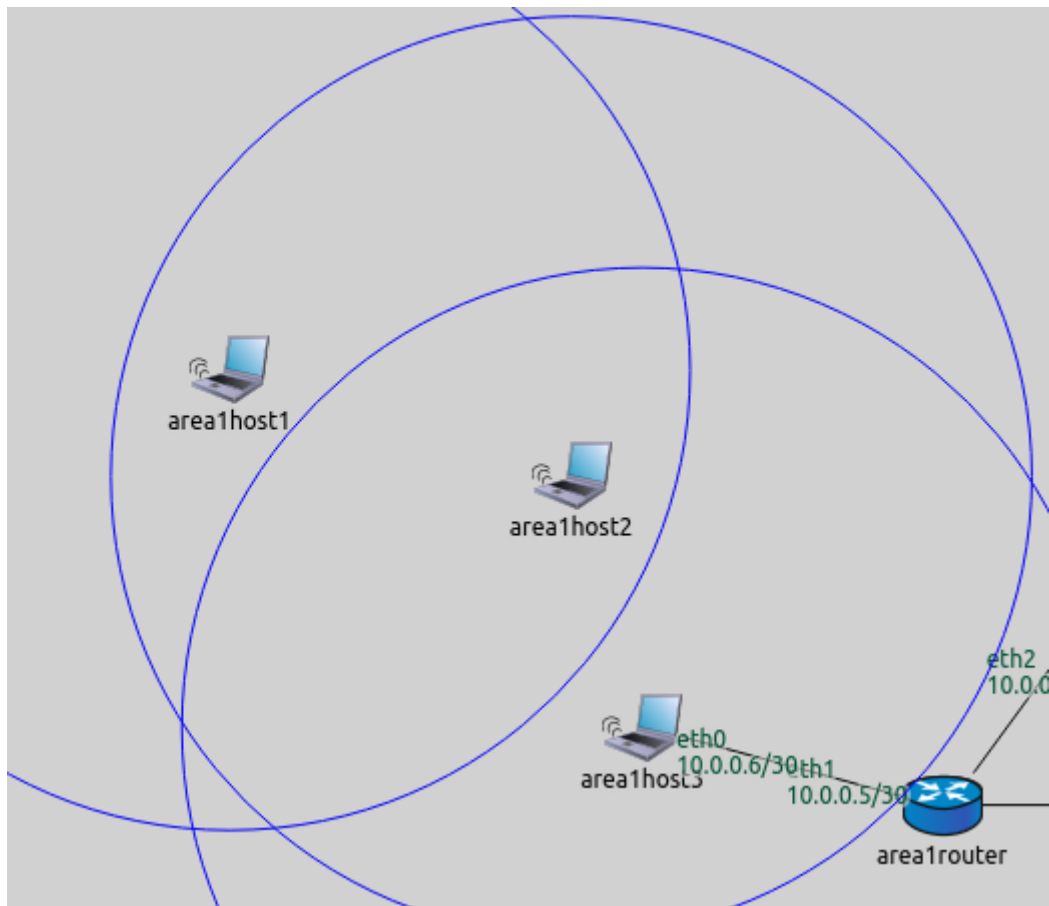
- Area1 is composed of three `WirelessHosts`, one of which is connected to the router with a wired connection.
- Area2 has an `AccessPoint` and three `WirelessHosts`.
- Area3 has three `StandardHosts` connected to the router via a switch.

There is no access point in area 1; the hosts form an ad-hoc wireless network. They connect to the rest of the network through `area1host3`, which has a wired connection to the router. However, `area1host3` is not in the communication range of `area1host1` (illustrated on the image below.) Thus, `area1host2` needs to be configured to forward `area1host1`'s packets to `area1host3`. The error rate metric, rather than the hop count, is best suited to configure routes in this LAN. Routes in the rest of the network can be configured properly based on the hop count metric.

The configuration for this step in `omnetpp.ini` is the following:

```
[Config Step12]
sim-time-limit = 100s
```

(continues on next page)



(continued from previous page)

```

network = ConfiguratorE
description = "Mixing different kinds of autorouting"

# Configurator settings
*.configurator.config = xmldoc("step12.xml")
*.configurator.optimizeRoutes = false           #! TODO! különben nem működik -> ezért kell

# Wireless node settings
*.*.wlan[*].bitrate = 54Mbps
*.*.wlan[*].radio.transmitter.power = 1mW
*.area1host*.forwarding = true
*.area1*.wlan[*].mgmt.typename = "Ieee80211MgmtAdhoc"
*.area1*.wlan[*].agent.typename = ""

# pingApp settings
*.area1host1.numApps = 1
*.area1host1.app[0].typename = "PingApp"
*.area1host1.app[*].destAddr = "area2host1"

# Visualizer settings
*.visualizer.mediumVisualizer.displayCommunicationRanges = true
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.displayRoutesIndividually = false
*.visualizer.routingTableVisualizer.displayLabels = false
*.visualizer.routingTableVisualizer.lineShift = 0

```

(continues on next page)

(continued from previous page)

```
*.visualizer.routingTableVisualizer.destinationFilter = ""
```

Explanation:

- For hosts in area 1 to operate in ad-hoc mode, IP forwarding is turned on, and their management modules are set to ad-hoc management.
- area1host1 is configured to ping area2host1, which is on the other side of the network.
- Routes to all hosts and communication ranges are visualized.

The XML configuration in step12.xml is the following:

```
<config>
  <interface hosts="area1*" address="10.x.x.x" netmask="255.x.x.x" add-default-
    route="false"/>
  <interface hosts="*" address="10.x.x.x" netmask="255.x.x.x"/>
  <autoroute metric="hopCount" sourceHosts="*.area2host* *router area3host*"/>
  <autoroute metric="errorRate" sourceHosts="*.area1host*"/>
</config>
```

To have routes from every node to every other node, all nodes must be covered by an autoroute element. The XML configuration contains two autoroute elements. Routing tables of hosts in area 1 are configured according to the error rate metric, while all others according to hop count.

The global `addStaticRoutes`, `addDefaultRoutes` and `addSubnetRoutes` parameters can also be specified per interface, with attributes of the `<interface>` element. The attribute names are `add-static-route`, `add-default-route` and `add-subnet-route`, and they are all booleans with `true` as the default value. The global and per-interface settings are in a logical AND relationship, so both have to be true to take effect.

The default route assumes there is one gateway, and all nodes on the link can reach it directly. This is not the case for area 1 because area1host1 is out of the range of the gateway host. The `add-default-route` parameter is set to `false` for the area 1 hosts.

2.13.3 Results

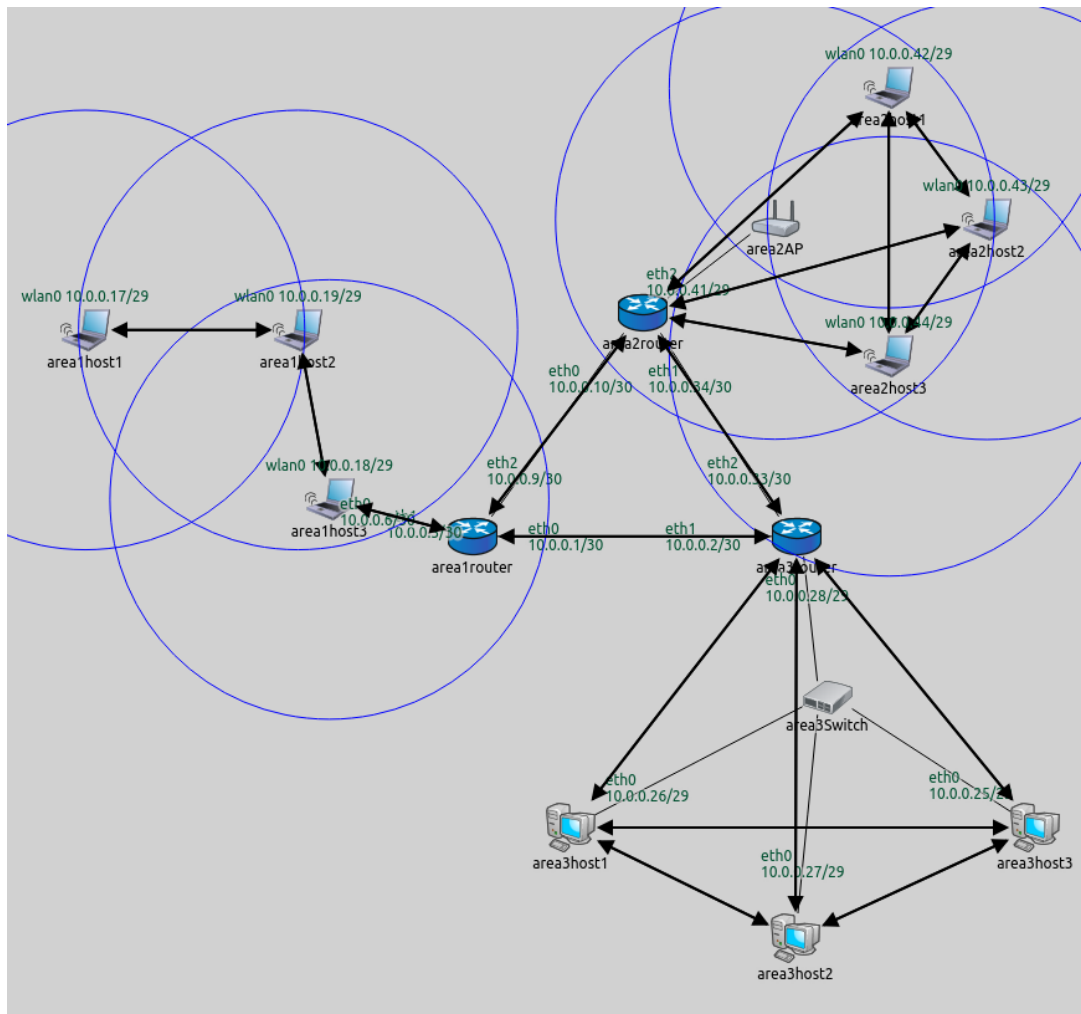
The routes are visualized on the following image.

As intended, area1host1 connects to the network via area1host2.

The routing table of area1host1 is as follows:

```
Node ConfiguratorF.area1host1
-- Routing table --
Destination      Netmask          Gateway          Iface             Metric
10.0.0.1          255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.2          255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.5          255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.6          255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.9          255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.10         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.18         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.28         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.33         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.34         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.41         255.255.255.255  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.16         255.255.255.248  *                wlan0 (10.0.0.17) 0
10.0.0.24         255.255.255.248  10.0.0.19        wlan0 (10.0.0.17) 0
10.0.0.40         255.255.255.248  10.0.0.19        wlan0 (10.0.0.17) 0
```

The gateway is 10.0.0.19 (area1host2) in all rules, except in the one where it is *. That rule is for reaching the other hosts in the LAN directly. This doesn't seem to be according to the error rate metric, but the * rule matches



destinations 10.0.0.18 and 10.0.0.19 only. Since 10.0.0.18 is covered by a previous rule, this one is actually for reaching 10.0.0.19 directly.

The following video shows `area1host1` pinging `area2host1`:

Sources: `omnetpp.ini`, `ConfiguratorE.ned`

2.13.4 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

2.14 Conclusion

Congratulations, you've made it! You should now be able to set up IPv4 static autoconfiguration for networks in the INET Framework. Use the link below if you have any questions or if you'd like to help us improve this tutorial.

2.14.1 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

QUEUEING TUTORIAL

Explore the use of queueing components in INET, including traffic generators, queues, and traffic conditioners. These components can be utilized to build diverse functionalities at various layers of the protocol stack.

Introduction

3.1 Getting Started

INET contains a queueing library which provides various components such as traffic generators, queues, and traffic conditioners. Elements of the library can be used to assemble queueing functionality that can be used at layer 2 and layer 3 of the protocol stack. With extra elements, the queueing library can also be used to define custom application behavior without C++ programming.

Each step in this tutorial demonstrates one of the available queueing elements, with a few more complex examples at the end. Note that most of the available elements are demonstrated here, but not all (for example, elements specific to DiffServ are omitted from here). See the INET Reference for the complete list of elements.

This is an advanced tutorial, and it assumes that you are familiar with creating and running simulations in OMNeT++ and INET. If you aren't, you can check out the TicToc Tutorial to get started with using OMNeT++.

3.1.1 Try It Yourself

If you already have INET and OMNeT++ installed, start the IDE by typing `omnetpp`, import the INET project into the IDE, then navigate to the `inet/tutorials/queueing` folder in the *Project Explorer*. There, you can view and edit the tutorial files, run simulations, and analyze results.

Otherwise, there is an easy way to install INET and OMNeT++ using `opp_env`, and run the simulation interactively. Ensure that `opp_env` is installed on your system, then execute:

```
$ opp_env run inet-4.6 --init -w inet-workspace --install --build-modes=release --  
↪chdir \  
-c 'cd inet-4.6.*/tutorials/queueing && inet'
```

This command creates an `inet-workspace` directory, installs the appropriate versions of INET and OMNeT++ within it, and launches the `inet` command in the showcase directory for interactive simulation.

Alternatively, for a more hands-on experience, you can first set up the workspace and then open an interactive shell:

```
$ opp_env install --init -w inet-workspace --build-modes=release inet-4.6  
$ cd inet-workspace  
$ opp_env shell
```

Inside the shell, start the IDE by typing `omnetpp`, import the INET project, then start exploring.

3.1.2 Discussion

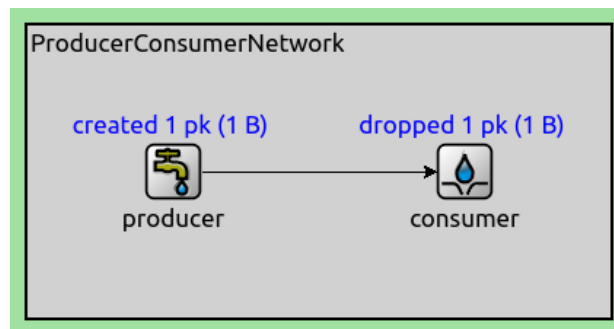
Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

Sources and Sinks

3.2 Active Source - Passive Sink

This step demonstrates the `ActivePacketSource` and `PassivePacketSink` modules. The active packet source has a configurable packet production interval; the passive packet sink has a configurable consumption interval, which limits when packets can be pushed into the module.

In this example network, packets are produced periodically by the active packet source (`ActivePacketSource`) and pushed into the passive packet sink (`PassivePacketSink`).



```
network ProducerConsumerTutorialStep
{
    @display("bgb=400,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        consumer: PassivePacketSink {
            @display("p=300,100");
        }
    connections:
        producer.out --> consumer.in;
}
```

```
[Config ActiveSourcePassiveSink]
network = ProducerConsumerTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
```

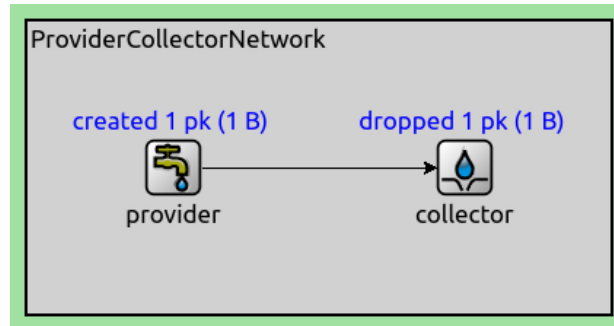
3.3 Passive Source - Active Sink

This step demonstrates the `ActivePacketSink` and `PassivePacketSource` modules. The active packet sink (`ActivePacketSink`) periodically pops packets from the passive packet source (`PassivePacketSource`).

The active packet sink module has a configurable collection interval. The passive packet source also has a configurable providing interval, which limits the times at which packets can be popped from the module.

```
network ProviderCollectorTutorialStep
{
```

(continues on next page)



(continued from previous page)

```
@display("bgb=400,200");
submodules:
  provider: PassivePacketSource {
    @display("p=100,100");
  }
  collector: ActivePacketSink {
    @display("p=300,100");
  }
connections:
  provider.out --> collector.in;
}
```

```
[Config PassiveSourceActiveSink]
network = ProviderCollectorTutorialStep
sim-time-limit = 10s

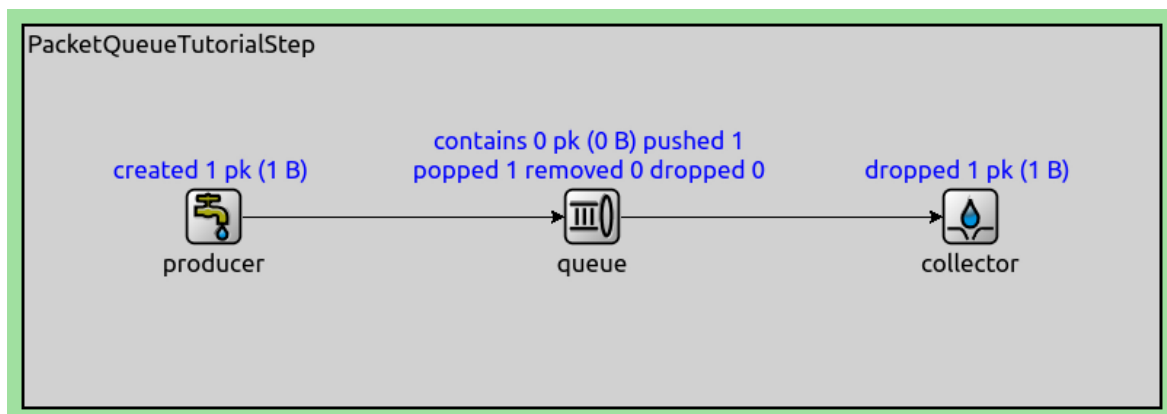
*.provider.packetLength = 1B
*.collector.collectionInterval = 1s
```

Queues and Buffers

3.4 Enqueueing Packets

This step demonstrates the `PacketQueue` module, which can store a configurable number of packets. By default, it works as a FIFO queue and has unlimited capacity. Optionally, the queue can be limited (both by the number of packets and byte count), and ordering can be specified.

In the following example network, packets are produced at random intervals by an active packet source (`ActivePacketSource`). The packets are pushed into an unlimited FIFO queue (`PacketQueue`), where they are stored temporarily. An active packet sink (`ActivePacketSink`) module pops packets from the queue at random intervals.



```

network PacketQueueTutorialStep
{
  @display("bgb=600,200");
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
    queue: PacketQueue {
      @display("p=300,100");
    }
    collector: ActivePacketSink {
      @display("p=500,100");
    }
  connections allowunconnected:
    producer.out --> queue.in;
    queue.out --> collector.in;
}

```

```

[Config PacketQueue]
network = PacketQueueTutorialStep
sim-time-limit = 10s

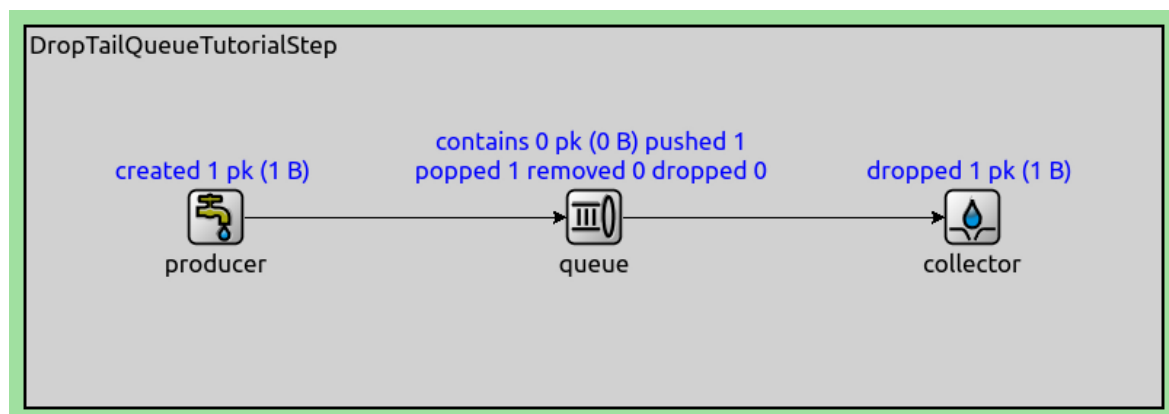
*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 1s)
*.collector.collectionInterval = uniform(0s, 2s)

```

3.5 Dropping Packets from a Finite Queue

The `DropTailQueue` module extends `PacketQueue` by specifying a packet capacity, and setting a packet dropper function. By default, the packet capacity is set to 100, and packets are dropped from the end of the queue.

In this example network, packets are created at random intervals by an active packet source (`ActivePacketSource`). The packets are pushed into a drop-tail queue (`DropTailQueue`) with a capacity of 4 packets. The packets are popped from the queue at random intervals by an active packet sink (`ActivePacketSink`).



```

network DropTailQueueTutorialStep
{
  @display("bgb=700,200");
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
}

```

(continues on next page)

(continued from previous page)

```

queue: DropTailQueue {
    @display("p=325,100");
}
collector: ActivePacketSink {
    @display("p=550,100");
}
connections allowunconnected:
    producer.out --> queue.in;
    queue.out --> collector.in;
}

```

```

[Config DropTailQueue]
network = DropTailQueueTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 1s)
*.queue.packetCapacity = 4
*.collector.collectionInterval = uniform(0s, 2s)

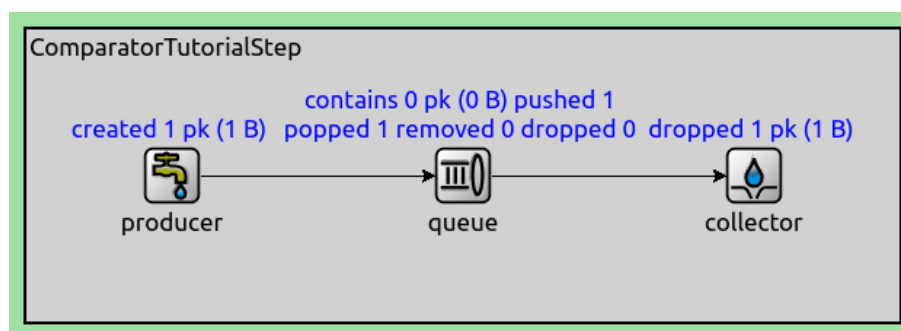
```

3.6 Ordering the Packets in the Queue

The `PacketQueue` module can order the packets it contains using a packet comparator function, which makes it suitable for implementing priority queuing. (Other ways of implementing priority queueing, for example using `PriorityScheduler`, will be covered in later steps.)

In this example network, an active packet source (`ActivePacketSource`) creates 1-byte packets every second with a random byte as content. The packets are pushed into a queue (`PacketQueue`). The queue is configured to use the `PacketDataComparator` function, which orders packets by data. The packets are popped from the queue by an active packet sink (`ActivePacketSink`) every 2 seconds.

During simulation, the producer creates packets more frequently than the collector pops them from the queue, so packets accumulate in the queue. When this happens, packets will be ordered in the queue, so the collector will receive a series of ordered sequences.



```

network ComparatorTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        queue: PacketQueue {
            @display("p=300,100");
        }
}

```

(continues on next page)

(continued from previous page)

```

    }
    collector: ActivePacketSink {
        @display("p=500,100");
    }
    connections allowunconnected:
        producer.out --> queue.in;
        queue.out --> collector.in;
}

```

[Config Comparator]

```

network = ComparatorTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.packetData = intuniform(0,255)
*.producer.productionInterval = 1s
*.collector.collectionInterval = 2s
*.queue.comparatorClass = "inet::queueing::PacketDataComparator"

```

The following screenshot demonstrates the queue's contents at the end of the simulation without the comparator function:

```

▼ storage (inet::queueing::cPacketQueue) len=5, 40 bits (5 bytes)
  ► ● producer-6 (inet::Packet) (inet::Packet)producer-6 (1 B) [ByteCountChunk, length = 1 B, data = 195]
  ► ● producer-7 (inet::Packet) (inet::Packet)producer-7 (1 B) [ByteCountChunk, length = 1 B, data = 103]
  ► ● producer-8 (inet::Packet) (inet::Packet)producer-8 (1 B) [ByteCountChunk, length = 1 B, data = 9]
  ► ● producer-9 (inet::Packet) (inet::Packet)producer-9 (1 B) [ByteCountChunk, length = 1 B, data = 211]
  ► ● producer-10 (inet::Packet) (inet::Packet)producer-10 (1 B) [ByteCountChunk, length = 1 B, data = 21]

```

And this one using the comparator function (head at the top, tail at the bottom):

```

  ► ● producer-2 (inet::Packet) (inet::Packet)producer-2 (1 B) [ByteCountChunk, length = 1 B, data = 117]
  ► ● producer-7 (inet::Packet) (inet::Packet)producer-7 (1 B) [ByteCountChunk, length = 1 B, data = 103]
  ► ● producer-4 (inet::Packet) (inet::Packet)producer-4 (1 B) [ByteCountChunk, length = 1 B, data = 67]
  ► ● producer-10 (inet::Packet) (inet::Packet)producer-10 (1 B) [ByteCountChunk, length = 1 B, data = 21]
  ► ● producer-8 (inet::Packet) (inet::Packet)producer-8 (1 B) [ByteCountChunk, length = 1 B, data = 9]

```

3.7 Storing Packets on Behalf of Multiple Queues

The `PacketBuffer` module stores packets on behalf of multiple queues, acting as a shared buffer for them. Instead of each individual queue having a separate capacity, the whole group of queues has a global, shared capacity, represented by the buffer. The buffer keeps a record of which packets belong to which queue, and also maintains the order of packets.

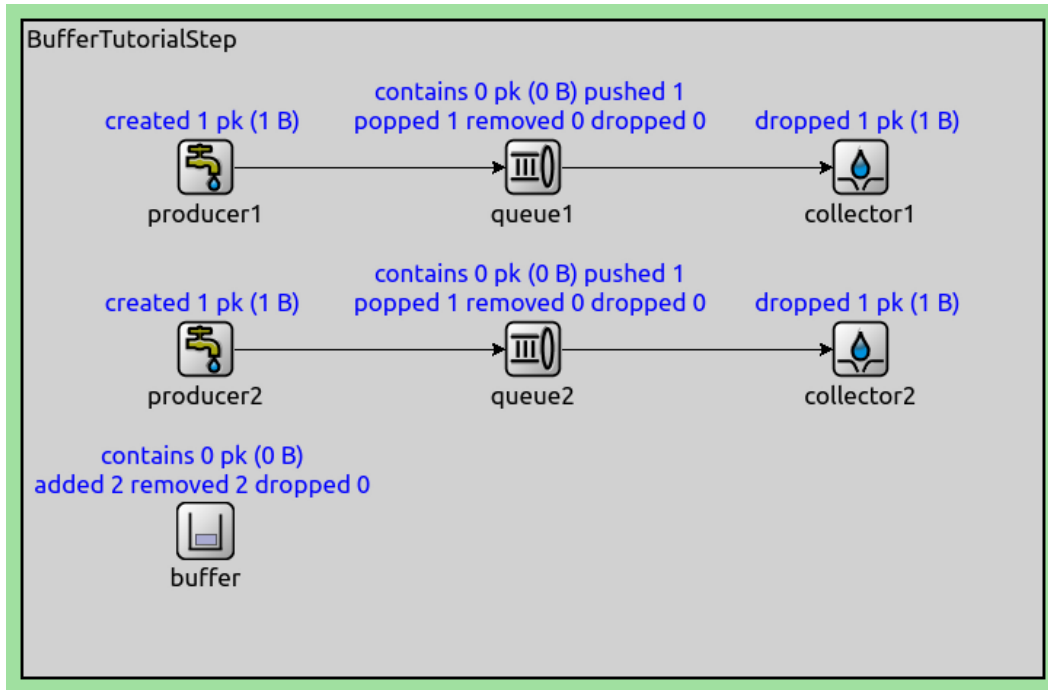
In this example network, packets are produced at random intervals by two active packet sources (`ActivePacketSource`). Each packet source pushes packets into its associated queue (`PacketQueue`). The two queues share a packet buffer module (`PacketBuffer`). The buffer is configured to have a capacity of 2 packets. Packets are popped from the queues by two active packet sinks (`ActivePacketSink`). Since the buffer has a limited packet capacity, and the `PacketAtCollectionBeginDropper` function is used as the dropper function, the buffer drops packets from the beginning of the buffer when it gets overloaded.

```

network BufferTutorialStep
{
    submodules:
        buffer: PacketBuffer {
            @display("p=125,350");
        }
}

```

(continues on next page)



(continued from previous page)

```

producer1: ActivePacketSource {
    @display("p=125,100");
}
producer2: ActivePacketSource {
    @display("p=125,225");
}
queue1: PacketQueue {
    @display("p=350,100");
    bufferModule = "^..buffer";
}
queue2: PacketQueue {
    @display("p=350,225");
    bufferModule = "^..buffer";
}
collector1: ActivePacketSink {
    @display("p=575,100");
}
collector2: ActivePacketSink {
    @display("p=575,225");
}
connections:
    producer1.out --> queue1.in;
    queue1.out --> collector1.in;
    producer2.out --> queue2.in;
    queue2.out --> collector2.in;
}

```

[Config Buffer]

```

network = BufferTutorialStep
sim-time-limit = 10s

*.buffer.dropperClass = "inet::queueing::PacketAtCollectionBeginDropper"
*.producer*.packetLength = 1B

```

(continues on next page)

(continued from previous page)

```

*.producer*.productionInterval = uniform(0s, 1s)
*.collector*.collectionInterval = uniform(0s, 2s)
*.buffer.packetCapacity = 2

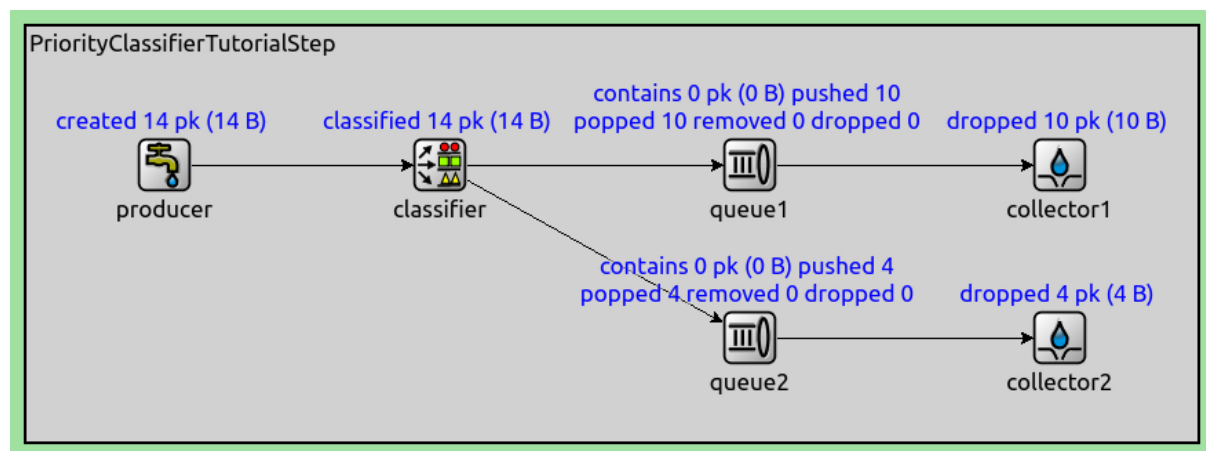
```

Classifying Packets from One Input to Multiple Outputs

3.8 Priority Classifier

The `PriorityClassifier` module pushes packets to the first non-full connected queue.

In this step, packets are produced periodically by an active packet source (`ActivePacketSource`). The packet source is connected to a priority classifier (`PriorityClassifier`), which connects to two queues (`PacketQueue`). The queues are configured to have a capacity for storing one packet. The queues are connected to two active packet sinks (`ActivePacketSink`). The classifier sends packets to the queues, favoring queue1. It will only consider queue2 when queue1 is full. The packets are popped from the queues by two active packet sinks (`ActivePacketSink`).



```

network PriorityClassifierTutorialStep
{
    @display("bgb=850,300");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        classifier: PriorityClassifier {
            @display("p=300,100");
        }
        queue1: PacketQueue {
            @display("p=525,100");
        }
        queue2: PacketQueue {
            @display("p=525,225");
        }
        collector1: ActivePacketSink {
            @display("p=750,100");
        }
        collector2: ActivePacketSink {
            @display("p=750,225");
        }
    connections allowunconnected:
        producer.out --> classifier.in;
        classifier.out++ --> queue1.in;

```

(continues on next page)

(continued from previous page)

```

classifier.out++ --> queue2.in;
queue1.out --> collector1.in;
queue2.out --> collector2.in;
}

```

```

[Config PriorityClassifier]
network = PriorityClassifierTutorialStep
sim-time-limit = 10s

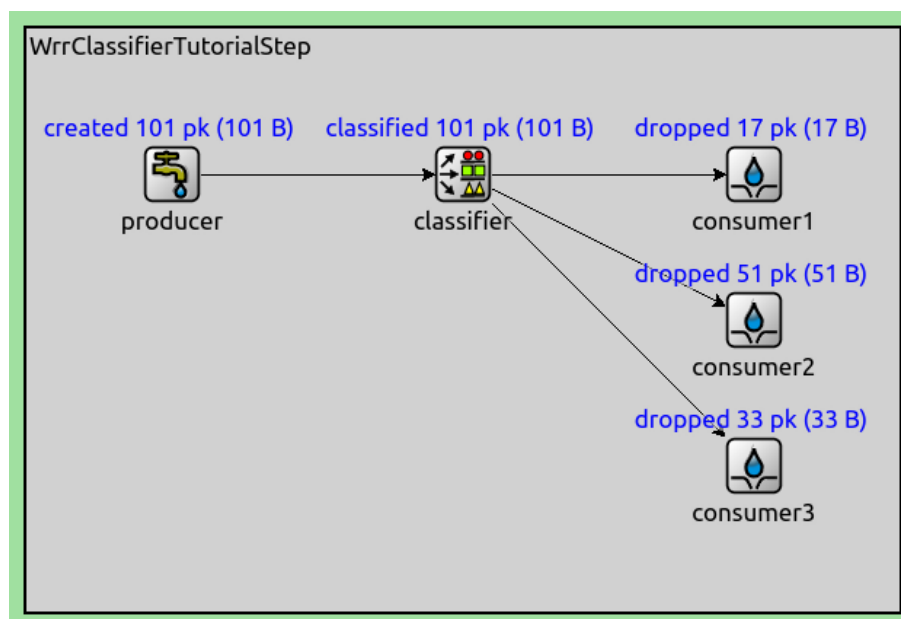
*.queue*.packetCapacity = 1
*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 1s)
*.collector*.collectionInterval = uniform(0s, 2s)

```

3.9 Weighted Round-Robin Classifier

The Weighted Round-Robin Classifier ([WrrClassifier](#)) module classifies packets to its outputs in a round-robin fashion, based on the configured weights of the outputs. For example, if the configured weights are [2,3], then the first two packets are sent to output 0, and the next three to output 1. If a component connected to an output is busy (the output is not *pushable*), the scheduler waits until it becomes available.

In this step, packets are produced periodically by an active packet source ([ActivePacketSource](#)). The single source is connected to a classifier ([WrrClassifier](#)), which is connected to three passive packet sinks ([PassivePacketSink](#)). The outputs of the classifier are configured to have the weights [1,3,2], thus the classifier forwards 1, 3, and 2 packet(s) to the three sinks in one round.



```

network WrrClassifierTutorialStep
{
    @display("bgb=600,400");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        classifier: WrrClassifier {
            @display("p=300,100");
        }
}

```

(continues on next page)

(continued from previous page)

```

    }
    consumer1: PassivePacketSink {
        @display("p=500,100");
    }
    consumer2: PassivePacketSink {
        @display("p=500,200");
    }
    consumer3: PassivePacketSink {
        @display("p=500,300");
    }

    connections allowunconnected:
    producer.out --> classifier.in;
    classifier.out++ --> consumer1.in;
    classifier.out++ --> consumer2.in;
    classifier.out++ --> consumer3.in;
}

```

```

[Config WrrClassifier]
network = WrrClassifierTutorialStep
sim-time-limit = 100s

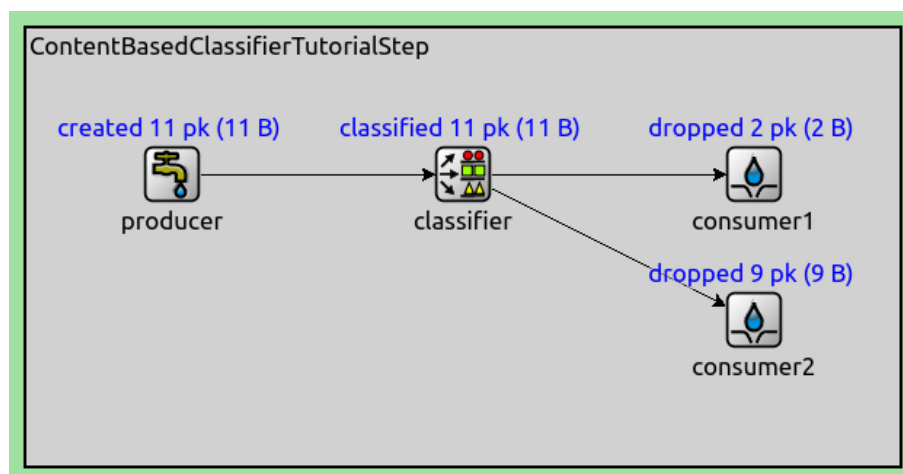
*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.classifier.weights = "1 3 2"

```

3.10 Content-Based Classifier

The `ContentBasedClassifier` module classifies packets according to the configured packet filter and packet data filter expressions.

In this example network, packets are generated by an active packet source (`ActivePacketSource`). The packets are pushed into a `ContentBasedClassifier`, which is connected to two passive packet sinks (`PassivePacketSink`). The packet source generates 1-byte packets with either 0 or 1 as data randomly. The classifier's packet data filter is configured to send packets which contain 0 to output 0, and those that contain 1 to output 1.



```

network ContentBasedClassifierTutorialStep
{
    @display("bgb=600,300");
}

```

(continues on next page)

(continued from previous page)

```

submodules:
  producer: ActivePacketSource {
    @display("p=100,100");
  }
  classifier: ContentBasedClassifier {
    @display("p=300,100");
  }
  consumer1: PassivePacketSink {
    @display("p=500,100");
  }
  consumer2: PassivePacketSink {
    @display("p=500,200");
  }
connections allowunconnected:
  producer.out --> classifier.in;
  classifier.out++ --> consumer1.in;
  classifier.out++ --> consumer2.in;
}

```

[Config ContentBasedClassifier]

```

network = ContentBasedClassifierTutorialStep
sim-time-limit = 10s

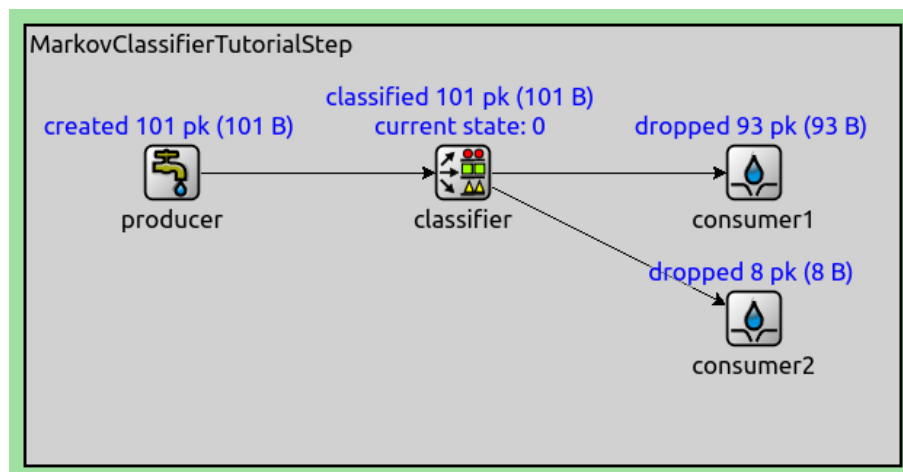
*.producer.packetLength = 1B
*.producer.packetData = intuniform(0, 1)
*.producer.productionInterval = 1s
*.classifier.packetFilters = [expr(ByteCountChunk.data == 0), expr(ByteCountChunk.
↳data == 1)]

```

3.11 Markov-Chain-Based Classifier

The [MarkovClassifier](#) module uses a Markov process to classify packets. The current state of the Markov process selects the output gate, a configurable transition matrix determines the probabilities of state change, and the configured wait intervals determine the time between state changes.

In this example network, packets are generated by an active packet source ([ActivePacketSource](#)). The packet source pushes packets into a classifier ([MarkovClassifier](#)). The classifier is connected to two passive packet sinks ([PassivePacketSink](#)). The classifier pushes packets into one of the packet sinks based on the current state of the Markov process.



```

network MarkovClassifierTutorialStep
{
    @display("bgb=600,300");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        classifier: MarkovClassifier {
            @display("p=300,100");
        }
        consumer1: PassivePacketSink {
            @display("p=500,100");
        }
        consumer2: PassivePacketSink {
            @display("p=500,200");
        }
    connections allowunconnected:
        producer.out --> classifier.in;
        classifier.out++ --> consumer1.in;
        classifier.out++ --> consumer2.in;
}

```

```

[Config MarkovClassifier]
network = MarkovClassifierTutorialStep
sim-time-limit = 100s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.classifier.transitionProbabilities = "0 1 1 0"
*.classifier.waitIntervals = "40 4"

```

3.12 Generic Classifier

The `PacketClassifier` module classifies packets based on the configured packet classifier function.

In this example network, packets are generated by an active packet source (`ActivePacketSource`). The packets are pushed into a `PacketClassifier` module, which is connected to two passive packet sinks (`PassivePacketSink`).

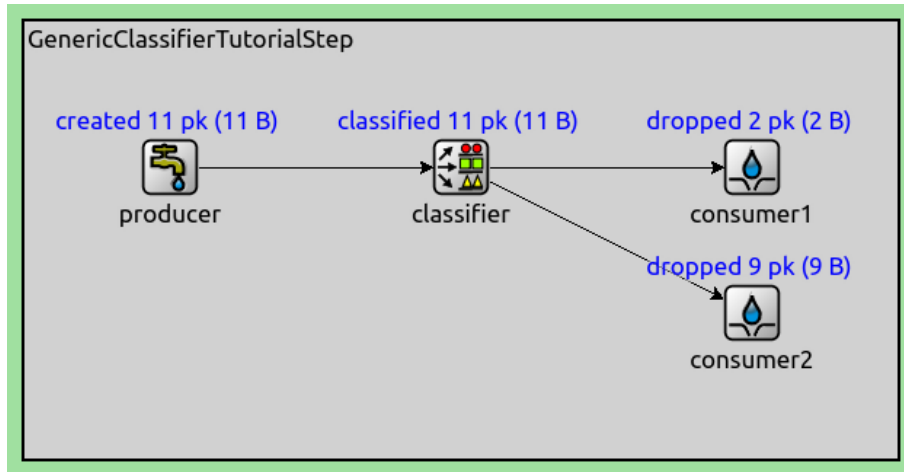
The source generates 1-byte packets that randomly contain either 0 or 1 as data. The classifier is configured to use the `PacketDataClassifier` function to classify packets. The function selects the classifier's output where the output index matches the data contained in the packet. Thus, packets with data 0 are pushed to `consumer1`, and those with data 1 are pushed to `consumer2`.

```

network GenericClassifierTutorialStep
{
    @display("bgb=600,300");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        classifier: PacketClassifier {
            @display("p=300,100");
        }
        consumer1: PassivePacketSink {
            @display("p=500,100");
        }
        consumer2: PassivePacketSink {

```

(continues on next page)



(continued from previous page)

```

        @display("p=500,200");
    }
    connections allowunconnected:
        producer.out --> classifier.in;
        classifier.out++ --> consumer1.in;
        classifier.out++ --> consumer2.in;
}

```

```

[Config GenericClassifier]
network = GenericClassifierTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.packetData = intuniform(0, 1)
*.producer.productionInterval = 1s
*.classifier.classifierClass = "inet::queueing::PacketDataClassifier"

```

Scheduling Packets from Multiple Inputs to One Output

3.13 Priority Scheduler

This step demonstrates the [PriorityScheduler](#) module. The module pops packets from the first connected non-empty queue or packet provider.

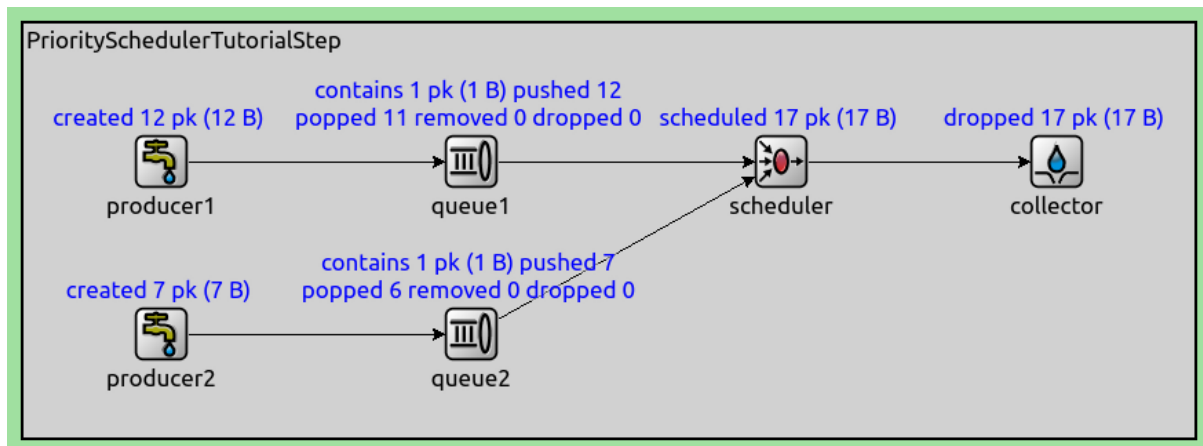
In this example network, two active packet sources ([ActivePacketSource](#)) generate packets in random periods. The packets are pushed into queues ([PacketQueue](#)), where they are stored temporarily. The queues are connected to a priority scheduler ([PriorityScheduler](#)). An active packet sink ([ActivePacketSink](#)) pops packets from the scheduler, which in turn pops packets from one of the queues in a prioritized way, favoring the first queue.

```

network PrioritySchedulerTutorialStep
{
    @display("bgb=850,300");
    submodules:
        producer1: ActivePacketSource {
            @display("p=100,100");
        }
        producer2: ActivePacketSource {
            @display("p=100,225");
        }
}

```

(continues on next page)



(continued from previous page)

```

queue1: PacketQueue {
    @display("p=325,100");
}
queue2: PacketQueue {
    @display("p=325,225");
}
scheduler: PriorityScheduler {
    @display("p=550,100");
}
collector: ActivePacketSink {
    @display("p=750,100");
}
connections allowunconnected:
producer1.out --> queue1.in;
producer2.out --> queue2.in;
queue1.out --> scheduler.in++;
queue2.out --> scheduler.in++;
scheduler.out --> collector.in;
}

```

[Config PriorityScheduler]

```

network = PrioritySchedulerTutorialStep
sim-time-limit = 10s

```

```

*.producer*.packetLength = 1B
*.producer*.productionInterval = uniform(0s, 2s)
*.collector.collectionInterval = uniform(0s, 1s)

```

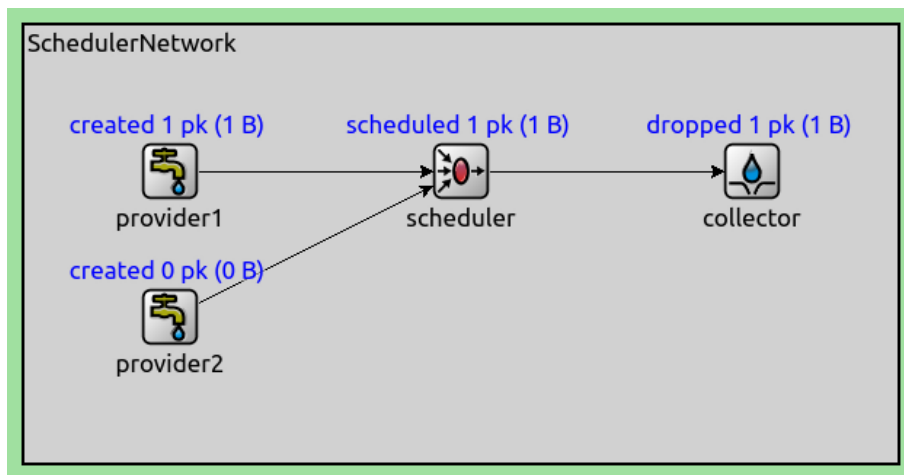
3.14 Weighted Round-Robin Scheduler

The Weighted Round-Robin Scheduler ([WrrScheduler](#)) module connects multiple passive packet sources to a single active packet sink. When the collector requests a packet, the scheduler pops a packet from one of the connected packet sources.

The [WrrScheduler](#) schedules packets from one of its connected packet sources in a round-robin fashion, based on the configured weights of the inputs. For example, if the configured weights are [2,3], then the first two packets are popped from input 0, the next three from input 1.

In this example network, packets are created by two passive packet sources ([PassivePacketSource](#)). The sources are connected to a [WrrScheduler](#). The scheduler is connected to an active packet sink ([ActivePacketSink](#)), which pops packets from the scheduler periodically. The inputs of the scheduler are configured to have the weights [1, 3,

2], thus the scheduler pops 1, 3, and 2 packet(s) from the three providers in one round.



```
network WrrSchedulerTutorialStep
{
    @display("bgb=600,400");
    submodules:
        provider1: PassivePacketSource {
            @display("p=100,100");
        }
        provider2: PassivePacketSource {
            @display("p=100,200");
        }
        provider3: PassivePacketSource {
            @display("p=100,300");
        }

        scheduler: WrrScheduler {
            @display("p=300,100");
        }
        collector: ActivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        provider1.out --> scheduler.in++;
        provider2.out --> scheduler.in++;
        provider3.out --> scheduler.in++;
        scheduler.out --> collector.in;
}
```

[Config WrrScheduler]

```
network = WrrSchedulerTutorialStep
sim-time-limit = 100s

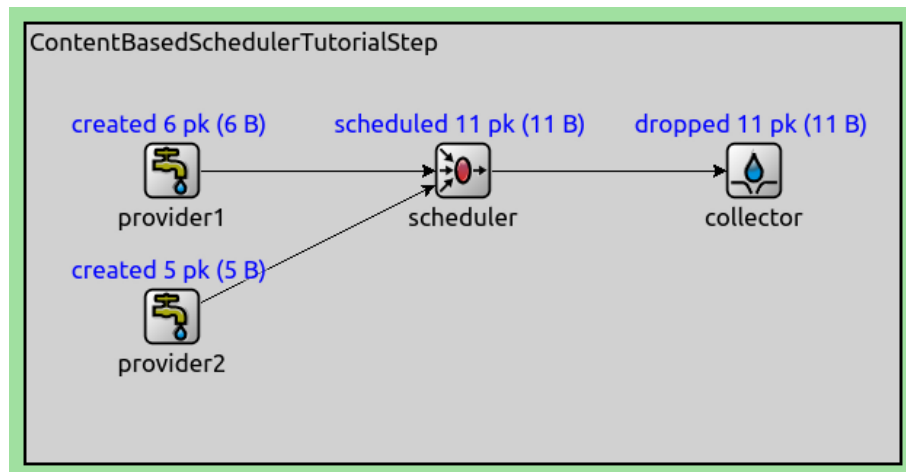
*.provider*.packetLength = 1B
*.collector.collectionInterval = 1s
*.scheduler.weights = "1 3 2"
```

3.15 Content-Based Scheduler

The `ContentBasedScheduler` pops packets from the connected passive packet sources based on the configured packet filter and packet data filter functions. There is one filter (of both kinds) per packet source. The scheduler iterates over the packet filter expressions and the packets offered by each connected packet source, and chooses the first matching packet to pop. It inherently prioritizes earlier expressions and earlier input gates.

In this example network, packets are generated by two passive packet sources (`PassivePacketSource`). An active packet sink (`ActivePacketSink`) pops packets from the scheduler, which pops packets from one of the packet sources.

The providers generate packets at random intervals between 0 and 2 seconds, randomly containing either 0 or 1 as data; the collector pops a packet from the scheduler every second. The scheduler's packet data filter is configured to first match packets containing 1, then those containing 0.



```
network ContentBasedSchedulerTutorialStep
{
    @display("bgb=600,300");
    submodules:
        provider1: PassivePacketSource {
            @display("p=100,100");
        }
        provider2: PassivePacketSource {
            @display("p=100,200");
        }
        scheduler: ContentBasedScheduler {
            @display("p=300,100");
        }
        collector: ActivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        provider1.out --> scheduler.in++;
        provider2.out --> scheduler.in++;
        scheduler.out --> collector.in;
}
```

```
[Config ContentBasedScheduler]
network = ContentBasedSchedulerTutorialStep
sim-time-limit = 10s

*.provider*.providingInterval = uniform(0s, 2s)
*.provider*.packetLength = 1B
```

(continues on next page)

(continued from previous page)

```

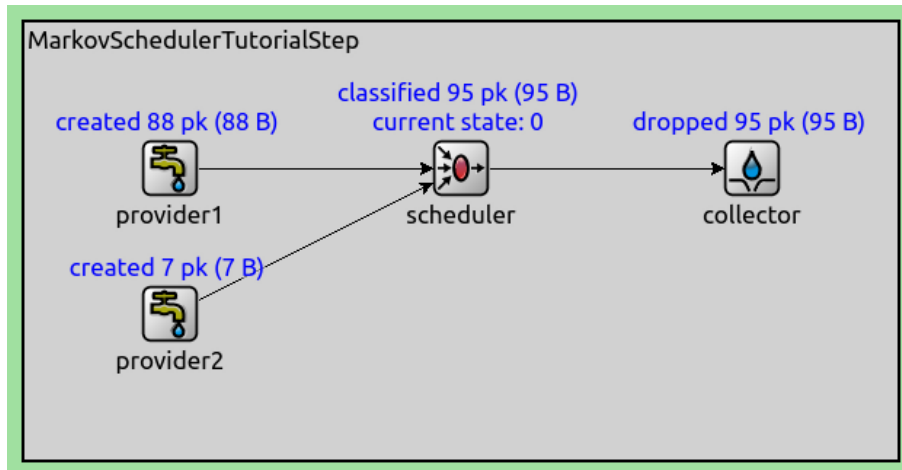
*.provider*.packetData = intuniform(0,1)
*.collector.collectionInterval = 1s
*.scheduler.packetFilters = [expr(ByteCountChunk.data == 1), expr(ByteCountChunk.data_
↳ == 0)]

```

3.16 Markov-Chain-Based Scheduler

The `MarkovScheduler` module uses a Markov process to schedule packets. The current state of the Markov process selects the input gate; a configurable transition matrix determines the probabilities of state change; the configured wait intervals determine the time between state changes. If the provider at the selected input cannot provide a packet for the duration of the state's wait interval, the process switches to the next state without pulling a packet.

In this example network, packets are generated by two passive packet sources (`PassivePacketSource`). The packet sources are connected to a scheduler (`MarkovScheduler`). Packets are popped from the scheduler by an active packet sink (`ActivePacketSink`). The scheduler in turn pops packets from one of the packet sources, based on the current state of the Markov process.



```

network MarkovSchedulerTutorialStep
{
    @display("bgb=600,300");
    submodules:
        provider1: PassivePacketSource {
            @display("p=100,100");
        }
        provider2: PassivePacketSource {
            @display("p=100,200");
        }
        scheduler: MarkovScheduler {
            @display("p=300,100");
        }
        collector: ActivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        provider1.out --> scheduler.in++;
        provider2.out --> scheduler.in++;
        scheduler.out --> collector.in;
}

```

```
[Config MarkovScheduler]
network = MarkovSchedulerTutorialStep
sim-time-limit = 100s

*.provider*.packetLength = 1B
*.collector*.collectionInterval = uniform(0s, 2s)
*.scheduler.transitionProbabilities = "0 1 1 0"
*.scheduler.waitIntervals = "40 4"
```

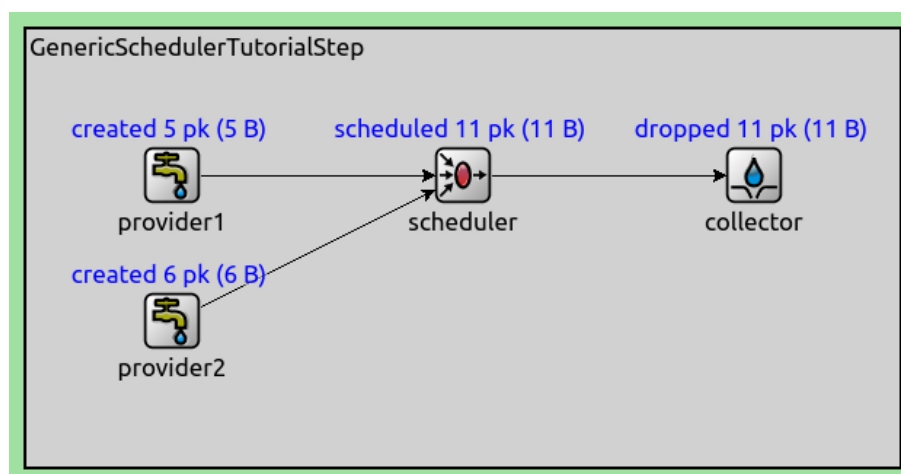
3.17 Generic Scheduler

The `PacketScheduler` connects to multiple passive packet sources on its inputs, and one active packet sink on its output. When a packet is popped from the scheduler, it pops a packet from one of its inputs, based on the configured packet scheduler function.

In this network, packets are generated by two passive packet sources (`PassivePacketSource`). The sources are connected to a `PacketScheduler`. An active packet sink (`ActivePacketSink`) pops packets from the scheduler, which pops packets from one of the packet sources.

Packets are generated by the providers at random intervals of 0 to 2 seconds; the collector pops a packet each second. The packets contain one byte of random data (0-255). The scheduler is configured to use the `PacketDataScheduler` function to schedule packets. The function selects the highest data value of the packets currently offered by the connected schedulers.

Note that when a provider generates a packet, it cannot generate another one for the configured providing interval. In this period, the provider doesn't have an available packet. In this case, the scheduler pops packets from the other provider if it has an available packet; if it doesn't, the next packet is popped as soon as one becomes available.



```
network GenericSchedulerTutorialStep
{
  @display("bgb=600,300");
  submodules:
    provider1: PassivePacketSource {
      @display("p=100,100");
    }
    provider2: PassivePacketSource {
      @display("p=100,200");
    }
    scheduler: PacketScheduler {
      @display("p=300,100");
    }
}
```

(continues on next page)

(continued from previous page)

```

    collector: ActivePacketSink {
        @display("p=500,100");
    }
    connections allowunconnected:
        provider1.out --> scheduler.in++;
        provider2.out --> scheduler.in++;
        scheduler.out --> collector.in;
}

```

```

[Config GenericScheduler]
network = GenericSchedulerTutorialStep
sim-time-limit = 10s

*.provider*.providingInterval = uniform(0s, 2s)
*.provider*.packetLength = 1B
*.provider*.packetData = intuniform(0,255)
*.collector.collectionInterval = 1s

```

Advanced Queues and Buffers

3.18 Priority Buffer

The `PriorityBuffer` module extends `PacketBuffer` by configuring a packet drop strategy. By default, when the buffer is full, packets belonging to the last queue are dropped first. This is the behavior that can be changed by the packet drop strategy.

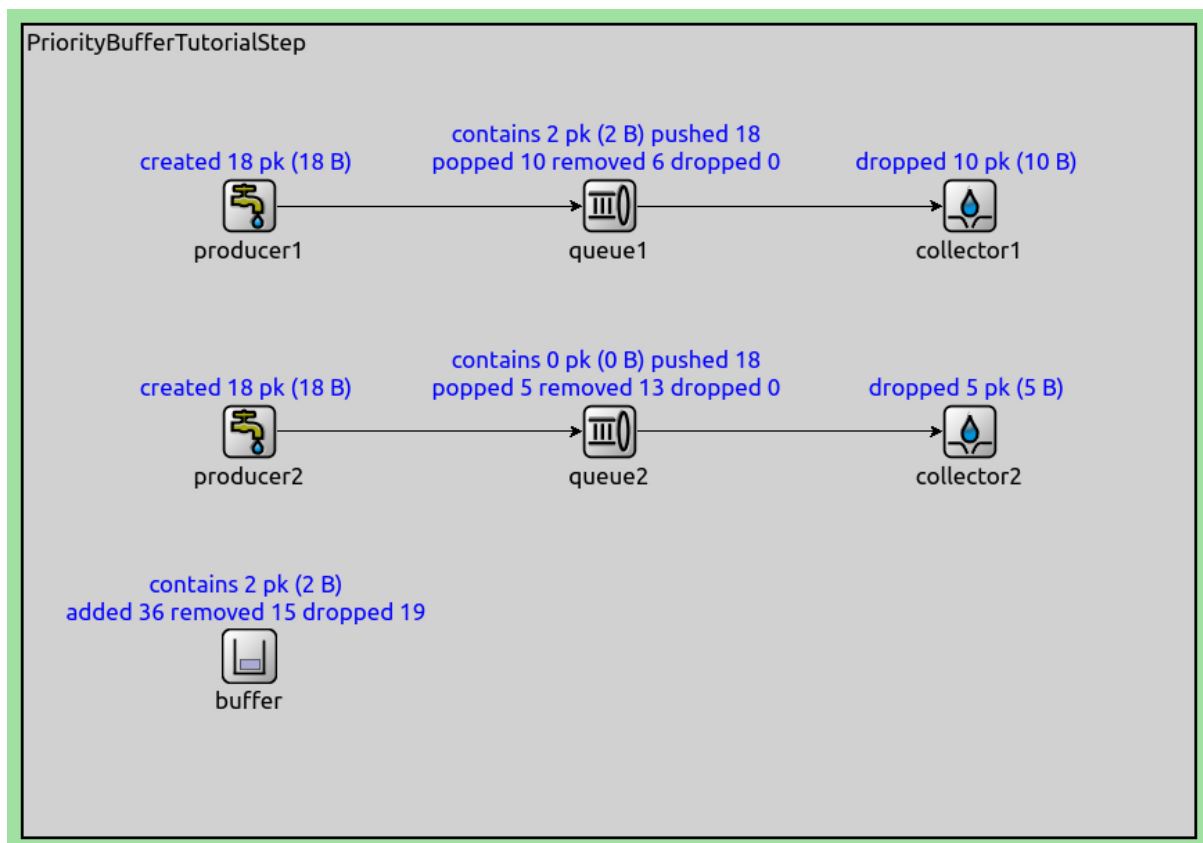
In this example network, packets are produced at random intervals by two active packet sources (`ActivePacketSource`). The packet packet sources push packets into two queues (`PacketQueue`). The queues share a `PriorityBuffer` with a packet capacity of 2. Packets are popped from the queues at random intervals by two active packet sinks (`ActivePacketSink`). When the buffer becomes full, it drops packets from the second queue first.

```

network PriorityBufferTutorialStep
{
    submodules:
        buffer: PriorityBuffer {
            @display("p=125,350");
        }
        producer1: ActivePacketSource {
            @display("p=125,100");
        }
        producer2: ActivePacketSource {
            @display("p=125,225");
        }
        queue1: PacketQueue {
            @display("p=325,100");
            bufferModule = "^..buffer";
        }
        queue2: PacketQueue {
            @display("p=325,225");
            bufferModule = "^..buffer";
        }
        collector1: ActivePacketSink {
            @display("p=525,100");
        }
        collector2: ActivePacketSink {
            @display("p=525,225");
        }
}

```

(continues on next page)



(continued from previous page)

```

}
connections:
  producer1.out --> queue1.in;
  queue1.out --> collector1.in;
  producer2.out --> queue2.in;
  queue2.out --> collector2.in;
}

```

```

[Config PriorityBuffer]
network = PriorityBufferTutorialStep
sim-time-limit = 10s

*.producer*.packetLength = 1B
*.producer*.productionInterval = uniform(0s, 1s)
*.collector*.collectionInterval = uniform(0s, 2s)
*.buffer.packetCapacity = 2

```

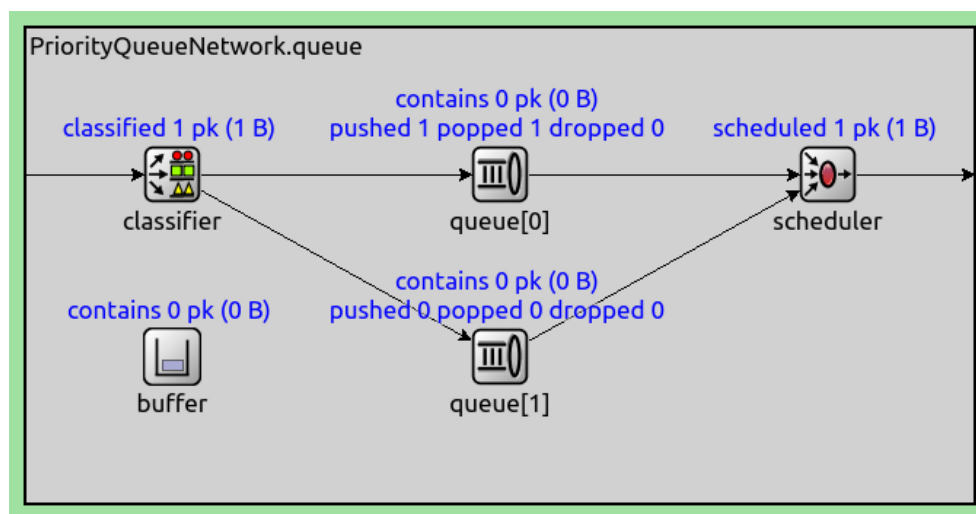
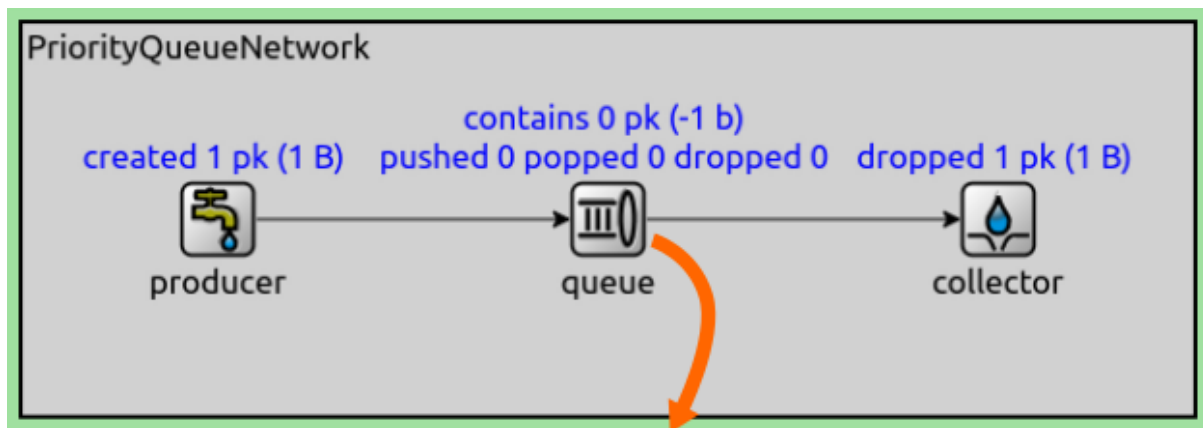
3.19 Priority Queue

The `PriorityQueue` module is a compound module that implements priority queueing with the help of a classifier submodule, multiple queues, and a scheduler.

It contains a configurable number of `PacketQueue`'s. A `PacketClassifier` module classifies packets into the queues according to the configured packet classifier function. A `PriorityScheduler` pops packets from the first non-empty queue; thus, earlier queues have a priority over the later ones.

In this example network, packets are produced at random intervals by an active packet source (`ActivePacketSource`). The source is connected to a priority queue (`PriorityQueue`) with two inner queues (`PacketQueue`). The packets

are collected at random intervals by an active packet sink (`ActivePacketSink`).



```
network PriorityQueueTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        queue: PriorityQueue {
            @display("p=300,100");
        }
        collector: ActivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        producer.out --> queue.in;
        queue.out --> collector.in;
}
```

```
[Config PriorityQueue]
network = PriorityQueueTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
```

(continues on next page)

(continued from previous page)

```

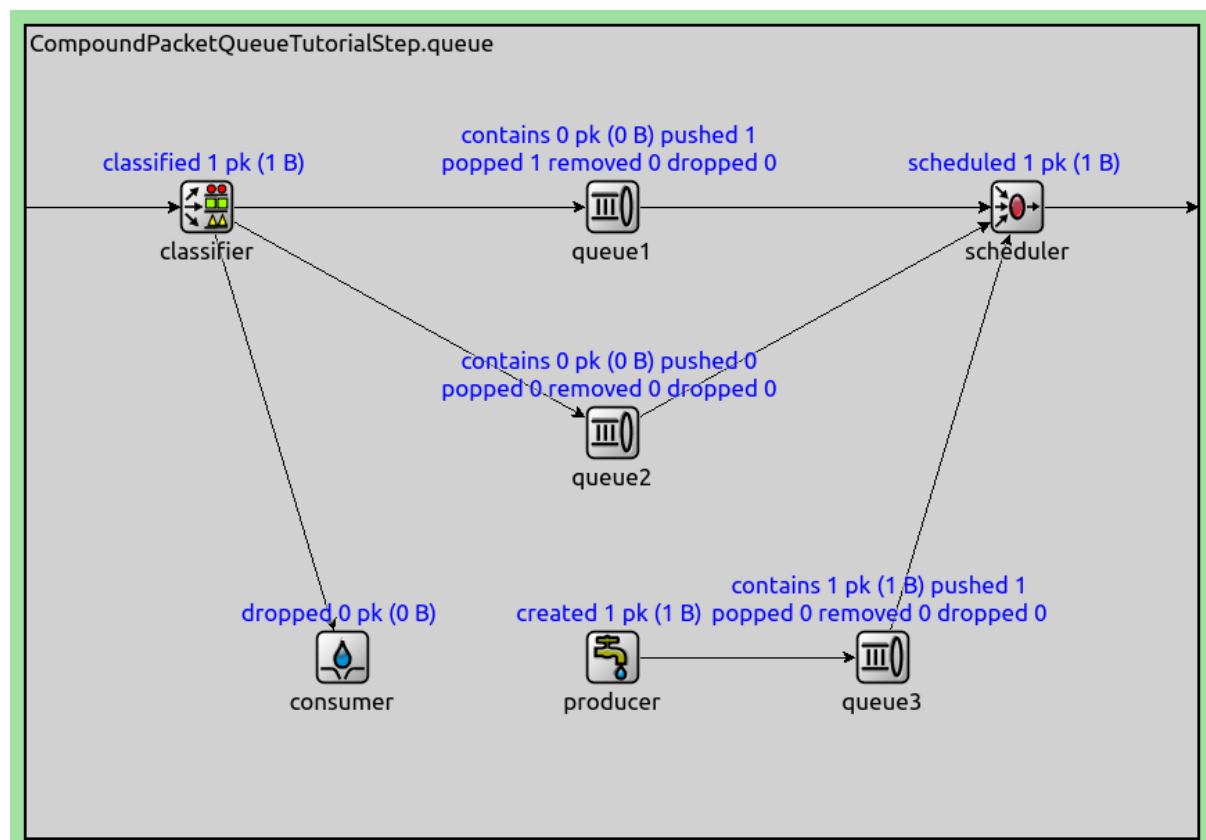
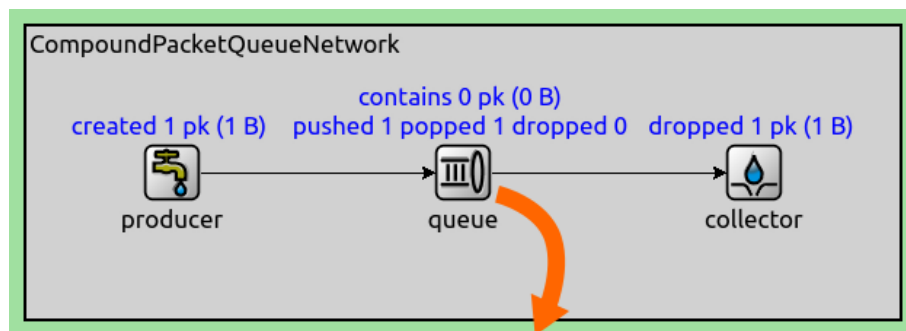
*.producer.productionInterval = uniform(0s, 2s)
*.queue.numQueues = 2
*.queue.classifier.typename = "WrrClassifier"
*.queue.classifier.weights = "1 1"
*.collector.collectionInterval = uniform(0s, 2s)

```

3.20 Building Complex Queues via Composition

This step demonstrates a compound priority queue (ExampleCompoundPriorityQueue) built from queueing components. The compound queue contains three packet queues. A classifier pushes packets to the first two queues and a passive packet sink in a round-robin fashion. An active packet source produces packets into the third queue. The three queues are connected to a priority scheduler, which pops packets from the first non-empty queue; the earlier queues have priority.

In this step, packets are produced at random intervals by an active packet source (ActivePacketSource). The source is connected to a compound priority queue (ExampleCompoundPriorityQueue) where packets are stored temporarily. Packets are collected at random intervals by an active packet sink (ActivePacketSink).



```

network CompoundPacketQueueTutorialStep
{
  @display("bgb=600,200");
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
    queue: ExampleCompoundPriorityQueue {
      @display("p=300,100");
    }
    collector: ActivePacketSink {
      @display("p=500,100");
    }
  connections allowunconnected:
    producer.out --> queue.in;
    queue.out --> collector.in;
}

```

```

module ExampleCompoundPriorityQueue extends CompoundPacketQueueBase
{
  parameters:
    @class(::inet::queueing::CompoundPacketQueueBase);
  submodules:
    classifier: WrrClassifier {
      @display("p=100,100");
    }
    queue1: PacketQueue {
      @display("p=325,100");
    }
    queue2: PacketQueue {
      @display("p=325,225");
    }
    queue3: PacketQueue {
      @display("p=475,350");
    }
    consumer: PassivePacketSink {
      @display("p=175,350");
    }
    producer: ActivePacketSource {
      @display("p=325,350");
    }
    scheduler: PriorityScheduler {
      @display("p=550,100");
    }
  connections:
    in --> { @display("m=w"); } --> classifier.in;
    classifier.out++ --> queue1.in;
    classifier.out++ --> queue2.in;
    classifier.out++ --> consumer.in;
    queue1.out --> scheduler.in++;
    queue2.out --> scheduler.in++;
    producer.out --> queue3.in;
    queue3.out --> scheduler.in++;
    scheduler.out --> { @display("m=e"); } --> out;
}

```

```
[Config CompoundQueue]
network = CompoundPacketQueueTutorialStep
sim-time-limit = 10s

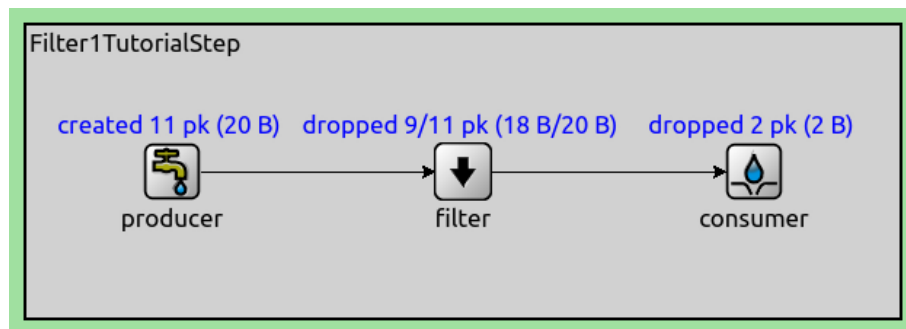
*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 2s)
*.collector.collectionInterval = uniform(0s, 2s)
*.queue.classifier.weights = "1 1 1"
```

Filtering and Dropping Packets

3.21 Content-Based Filtering (Active Source)

The `ContentBasedFilter` module filters packets according to the configured packet filter and packet data filter. Packets that match the filter expressions are pushed/popped on the output; non-matching packets are dropped.

In this example network, an active packet source (`ActivePacketSource`) generates 1-byte and 2-byte packets randomly. The source pushes packets into a filter (`ContentBasedFilter`), which pushes 1-byte packets into a passive packet sink (`PassivePacketSink`) and drops 2-byte ones.



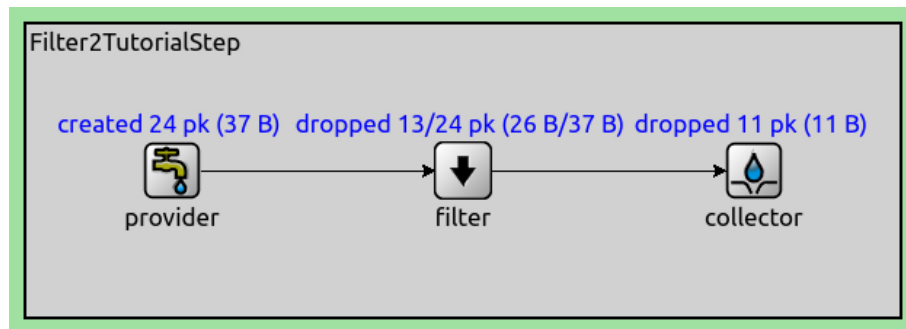
```
network Filter1TutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        filter: ContentBasedFilter {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        producer.out --> filter.in;
        filter.out --> consumer.in;
}
```

```
[Config Filter1]
network = Filter1TutorialStep
sim-time-limit = 10s

*.producer.packetLength = intuniform(1B, 2B)
*.producer.productionInterval = 1s
*.filter.packetFilter = expr(totalLength == 1B)
```

3.22 Content-Based Filtering (Active Sink)

This step is very similar to the previous one, but packets are popped from the filter by an active packet sink (`ActivePacketSink`), and provided by a passive packet source (`PassivePacketSource`).



```
network Filter2TutorialStep
{
    @display("bgb=600,200");
    submodules:
        provider: PassivePacketSource {
            @display("p=100,100");
        }
        filter: ContentBasedFilter {
            @display("p=300,100");
        }
        collector: ActivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        provider.out --> filter.in;
        filter.out --> collector.in;
}
```

```
[Config Filter2]
network = Filter2TutorialStep
sim-time-limit = 10s

*.provider.packetLength = intuniform(1B, 2B)
*.collector.collectionInterval = 1s
*.filter.packetFilter = expr(totalLength == 1B)
```

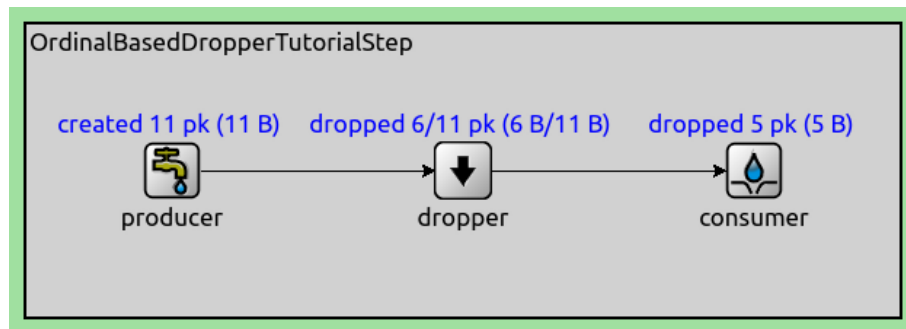
3.23 Ordinal-Based Dropper

This step demonstrates the `OrdinalBasedDropper` module. The module drops packets based on the ordinal number of incoming packets.

In this step, packets are produced periodically by an active packet source (`ActivePacketSource`). The packet source pushes packets into a dropper (`OrdinalBasedDropper`). Every second packet is dropped based on its ordinal number. Packets that are not dropped are consumed by a passive packet sink (`PassivePacketSink`).

```
network OrdinalBasedDropperTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
```

(continues on next page)



(continued from previous page)

```

    @display("p=100,100");
  }
  dropper: OrdinalBasedDropper {
    @display("p=300,100");
  }
  consumer: PassivePacketSink {
    @display("p=500,100");
  }
  connections:
    producer.out --> dropper.in;
    dropper.out --> consumer.in;
}

```

[Config OrdinalBasedDropper]

```

network = OrdinalBasedDropperTutorialStep
sim-time-limit = 10s

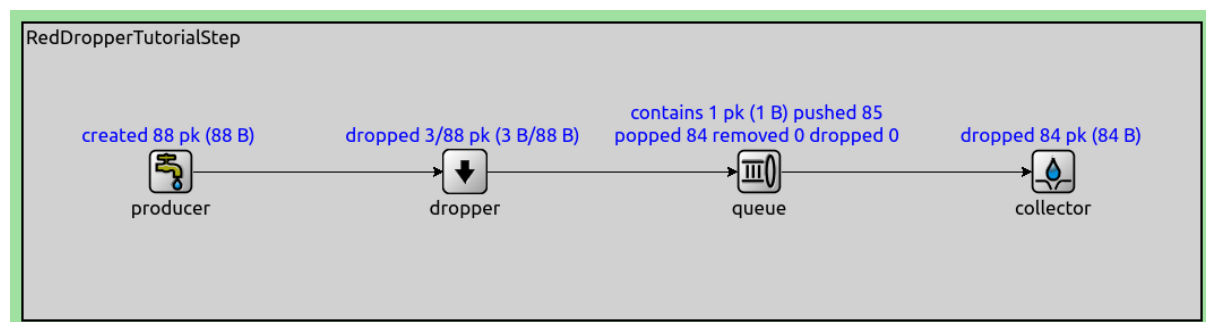
*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.dropper.dropsVector = "0; 2; 4; 6; 8; 10"

```

3.24 RED Dropper

This step demonstrates the `RedDropper` module, which implements the Random Early Detection algorithm.

Packets are created by an active packet source (`ActivePacketSource`), which pushes packets to the dropper module. The dropper module drops packets with a probability depending on the number of packets contained in the queue (`PacketQueue`) connected to its output. The packets are collected by an active packet sink (`ActivePacketSink`).



```

network RedDropperTutorialStep
{
  submodules:

```

(continues on next page)

(continued from previous page)

```

    producer: ActivePacketSource {
        @display("p=100,100");
    }
    dropper: RedDropper {
        @display("p=300,100");
    }
    queue: PacketQueue {
        @display("p=500,100");
    }
    collector: ActivePacketSink {
        @display("p=700,100");
    }
    connections:
    producer.out --> dropper.in;
    dropper.out --> queue.in;
    queue.out --> collector.in;
}

```

```

[Config RedDropper]
network = RedDropperTutorialStep
sim-time-limit = 100s

*.producer.packetLength = 1B
*.producer.productionInterval = exponential(1s)
*.dropper.minth = 1
*.dropper.maxth = 3
*.dropper.maxp = 0.1
*.dropper.wq = 0.5
*.collector.collectionInterval = exponential(1s)

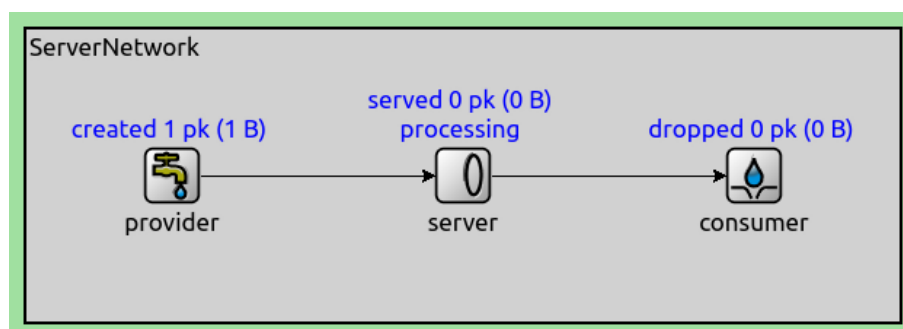
```

Actively Serving Packets from a Passive Source

3.25 Time-Based Server

The `PacketServer` module connects to passive packet sources and sinks. The server actively pops packets from the connected packet provider and after a configurable processing delay, it pushes them onto the connected packet consumer.

In this step, packets are passed through from the source to the sink periodically (randomly) by the active packet processor (`PacketServer`). The packets are generated by a passive packet source (`PassivePacketSource`) and consumed by a passive packet sink (`PassivePacketSink`).



```

network ServerTutorialStep
{

```

(continues on next page)

(continued from previous page)

```

@display("bgb=600,200");
submodules:
  provider: PassivePacketSource {
    @display("p=100,100");
  }
  server: PacketServer {
    @display("p=300,100");
  }
  consumer: PassivePacketSink {
    @display("p=500,100");
  }
connections allowunconnected:
  provider.out --> server.in;
  server.out --> consumer.in;
}

```

[Config Server]

```

network = ServerTutorialStep
sim-time-limit = 10s

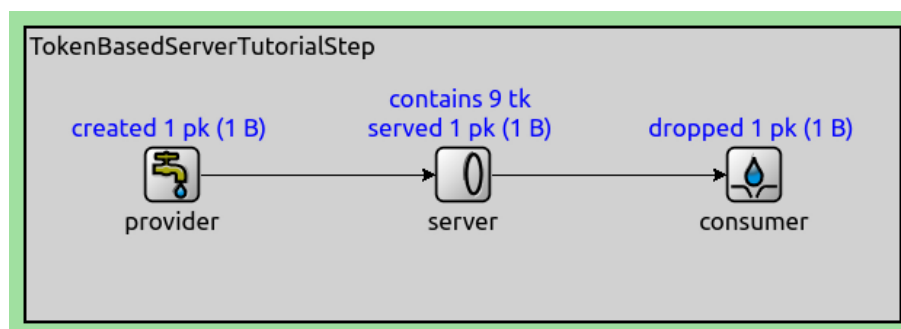
*.provider.packetLength = 1B
*.server.processingTime = uniform(0s, 2s)

```

3.26 Token-Based Server

The `TokenBasedServer` module pops packets from a passive source and processes them when it has enough tokens. A configured amount of tokens is lost when a packet is processed. The module processes packets in zero simulation time, and pushes them onto the packet consumer connected to its output. The module acquires tokens from token generator modules.

In this step, packets are generated by a passive packet source (`PassivePacketSource`). The packets are popped by the `TokenBasedServer` module. There are no token generators in this network, but the server has 10 initial tokens to process packets; the tokens eventually run out. Packets are consumed by a passive packet sink (`PassivePacketSink`).



```

network TokenBasedServerTutorialStep
{
  @display("bgb=600,200");
  submodules:
    provider: PassivePacketSource {
      @display("p=100,100");
    }
    server: TokenBasedServer {
      @display("p=300,100");
    }
}

```

(continues on next page)

(continued from previous page)

```

    consumer: PassivePacketSink {
        @display("p=500,100");
    }
    connections allowunconnected:
        provider.out --> server.in;
        server.out --> consumer.in;
}

```

```

[Config TokenBasedServer]
network = TokenBasedServerTutorialStep
sim-time-limit = 10s

*.provider.packetLength = 1B
*.server.initialNumTokens = 10

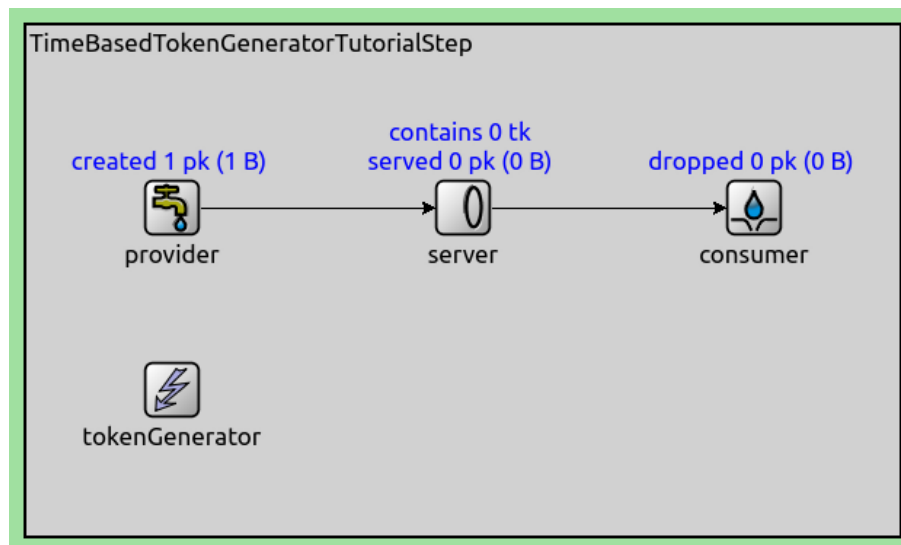
```

Generating Tokens for a Token-based Server

3.27 Generating Tokens Periodically

The `TimeBasedTokenGenerator` generates tokens periodically into a token-based server, with a configurable generation interval.

In this step, packets are generated by a passive packet source (`PassivePacketSource`). A `TokenBasedServer` pops packets from the connected packet source when it has enough tokens, and pushes them onto a passive packet sink (`PassivePacketSink`). Tokens are generated at random intervals by a `TimeBasedTokenGenerator`.



```

network TimeBasedTokenGeneratorTutorialStep
{
    @display("bgb=600,350");
    submodules:
        provider: PassivePacketSource {
            @display("p=100,125");
        }
        server: TokenBasedServer {
            @display("p=300,125");
        }
        consumer: PassivePacketSink {

```

(continues on next page)

(continued from previous page)

```

        @display("p=500,125");
    }
    tokenGenerator: TimeBasedTokenGenerator {
        @display("p=100,250");
        storageModule = "^server";
    }
    connections allowunconnected:
        provider.out --> server.in;
        server.out --> consumer.in;
}

```

```

[Config TimeBasedTokenGenerator]
network = TimeBasedTokenGeneratorTutorialStep
sim-time-limit = 10s

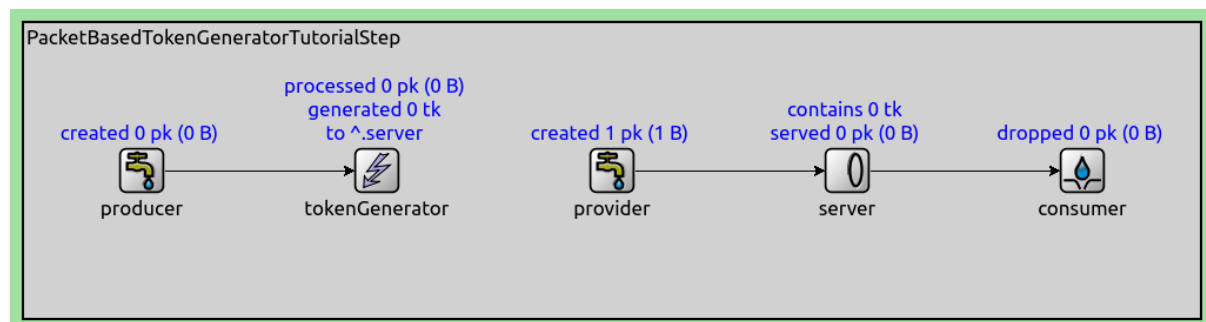
*.provider.packetLength = 1B
*.tokenGenerator.generationInterval = intuniform(1s,2s)
*.tokenGenerator.numTokens = intuniform(1,2)
*.server.tokenConsumptionPerPacket = intuniform(1,2)

```

3.28 Generating Tokens after Received Packets

The `PacketBasedTokenGenerator` is a passive packet sink, and generates a configurable number of tokens into a token-based server when receiving a packet.

Packets are generated by an active packet source (`ActivePacketSource`), and pushed into the token generator (`PacketBasedTokenGenerator`). The generator generates tokens into a token-based server (`TokenBasedServer`). When the server has tokens, it pops packets from the connected packet provider (`PassivePacketSource`), and pushes them into a packet consumer (`PassivePacketSink`).



```

network PacketBasedTokenGeneratorTutorialStep
{
    @display("bgb=1000,250");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,125");
        }
        provider: PassivePacketSource {
            @display("p=500,125");
        }
        server: TokenBasedServer {
            @display("p=700,125");
        }
        consumer: PassivePacketSink {

```

(continues on next page)

(continued from previous page)

```

    @display("p=900,125");
  }
  tokenGenerator: PacketBasedTokenGenerator {
    @display("p=300,125");
    storageModule = "^server";
  }
  connections allowunconnected:
    producer.out --> tokenGenerator.in;
    provider.out --> server.in;
    server.out --> consumer.in;
}

```

```

[Config PacketBasedTokenGenerator]
network = PacketBasedTokenGeneratorTutorialStep
sim-time-limit = 10s

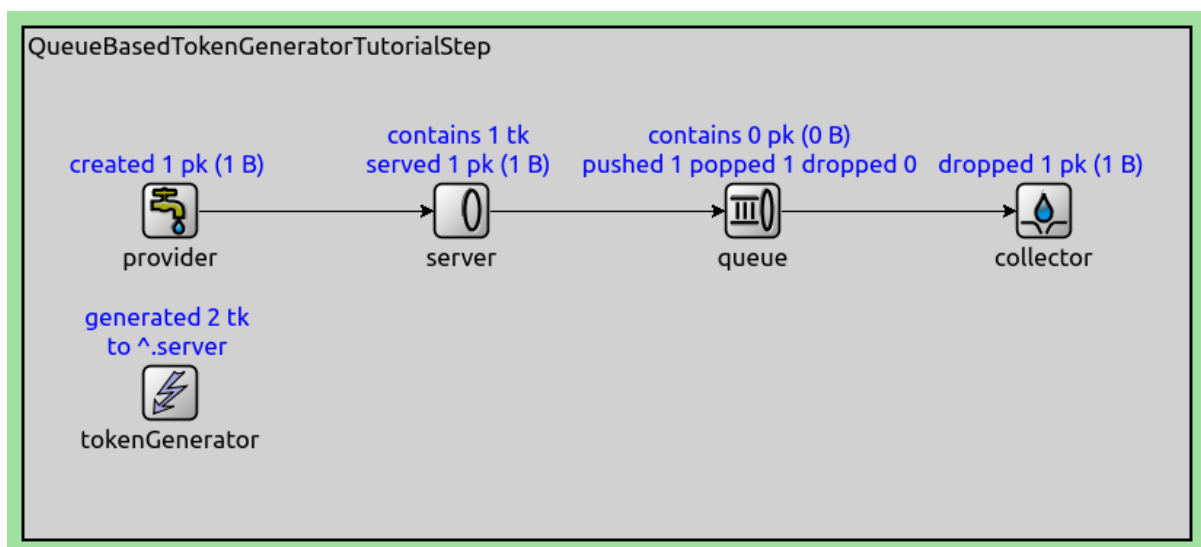
*.provider.packetLength = 1B
*.producer.packetLength = 1B
*.producer.productionInterval = 1s

```

3.29 Generating Tokens When a Queue Becomes Empty

The `QueueBasedTokenGenerator` module generates tokens into a token-based server when an observed queue becomes empty. The module can be used by applications to create traffic that completely utilizes a network interface, for example.

In this example network, packets are generated by a passive packet source (`PassivePacketSource`). A token-based server (`TokenBasedServer`) pops packets from the source when it has enough tokens. Then it pushes packets into the connected queue (`PacketQueue`). Packets are collected from the queue by an active packet sink (`ActivePacketSink`). Tokens are generated by a `QueueBasedTokenGenerator`, which observes the queue and generates a token whenever the queue is empty.



```

network QueueBasedTokenGeneratorTutorialStep
{
  @display("bgb=875,350");
  submodules:
    provider: PassivePacketSource {

```

(continues on next page)

(continued from previous page)

```

        @display("p=100,125");
    }
    server: TokenBasedServer {
        @display("p=325,125");
    }
    queue: PacketQueue {
        @display("p=550,125");
    }
    collector: ActivePacketSink {
        @display("p=775,125");
    }
    tokenGenerator: QueueBasedTokenGenerator {
        @display("p=100,250");
        storageModule = "^server";
        queueModule = "^queue";
    }
    connections allowunconnected:
        provider.out --> server.in;
        server.out --> queue.in;
        queue.out --> collector.in;
}

```

```

[Config QueueBasedTokenGenerator]
network = QueueBasedTokenGeneratorTutorialStep
sim-time-limit = 10s

*.provider.packetLength = 1B
*.collector.collectionInterval = 1s

```

3.30 Generating Tokens Based on Received Signals

The `SignalBasedTokenGenerator` module generates tokens in a token-based server when receiving signals.

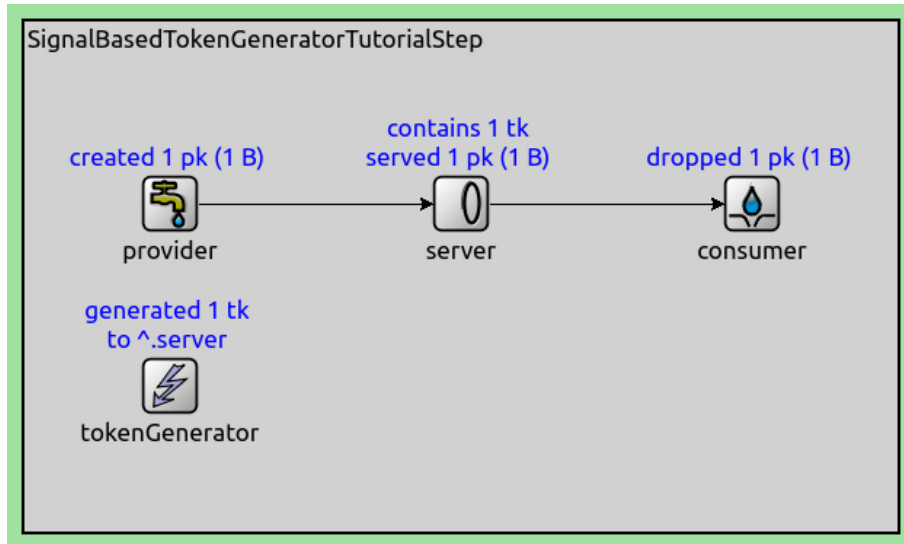
In this example network, packets are generated by a passive packet source (`PassivePacketSource`). A token-based server (`TokenBasedServer`) pops packets from a passive packet source (`PassivePacketSource`) when it has tokens, and pushes them into a passive packet sink (`PassivePacketSink`). The token generator is subscribed to the `packetDropped` signal of the consumer, and generates a token whenever the signal is emitted. The provider has a configured providing interval of 1s. Setting a providing interval is necessary because otherwise (if the provider could generate packets at any time), an infinite amount of packets would be generated and dropped in zero simulation time, resulting in an error.

```

network SignalBasedTokenGeneratorTutorialStep
{
    @display("bgb=600,350");
    submodules:
        provider: PassivePacketSource {
            @display("p=100,125");
        }
        server: TokenBasedServer {
            @display("p=300,125");
        }
        consumer: PassivePacketSink {
            @display("p=500,125");
        }
        tokenGenerator: SignalBasedTokenGenerator {

```

(continues on next page)



(continued from previous page)

```

@display("p=100,250");
signals = "packetDropped";
subscriptionModule = "^.consumer";
storageModule = "^.server";
}
connections allowunconnected:
  provider.out --> server.in;
  server.out --> consumer.in;
}

```

```

[Config SignalBasedTokenGenerator]
network = SignalBasedTokenGeneratorTutorialStep
sim-time-limit = 10s

*.provider.packetLength = 1B
*.provider.providingInterval = 1s
*.server.initialNumTokens = 1

```

Markers and Meters

3.31 Limiting the Data Rate of a Packet Stream

Rate meter modules (`ExponentialRateMeter` and `SlidingWindowRateMeter`) and rate limiter modules (`StatisticalRateLimiter`) can be used to limit the data rate of a stream of packets. The meter measures the data rate of the incoming stream of packets, and attaches a rate tag to each packet. The limiter module, based on the rate tag, limits the outgoing data rate to a configurable value.

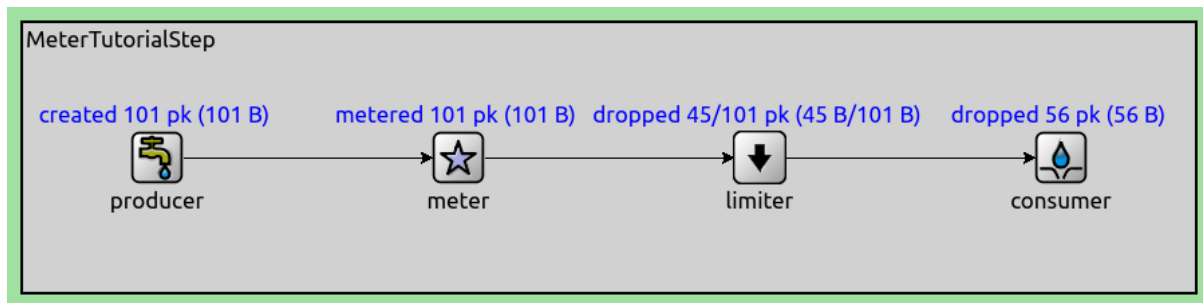
In this step, packets are produced periodically by an active packet source (`ActivePacketSource`). The packets are consumed by a passive packet sink (`PassivePacketSink`). The packet rate is measured by an `ExponentialRateMeter`, and if the rate of packets is higher than a predefined threshold, then packets are dropped by the `StatisticalRateLimiter`.

```

network MeterTutorialStep
{
  @display("bgb=875,200");
  submodules:
    producer: ActivePacketSource {

```

(continues on next page)



(continued from previous page)

```

        @display("p=100,100");
    }
    meter: ExponentialRateMeter {
        @display("p=325,100");
    }
    limiter: StatisticalRateLimiter {
        @display("p=550,100");
    }
    consumer: PassivePacketSink {
        @display("p=775,100");
    }
    connections allowunconnected:
        producer.out --> meter.in;
        meter.out --> limiter.in;
        limiter.out --> consumer.in;
}

```

```

[Config Meter]
network = MeterTutorialStep
sim-time-limit = 100s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.meter.alpha = 0.9
*.limiter.maxPacketrate = 0.5

```

3.32 Requesting Protocol-Specific Behavior for Packets

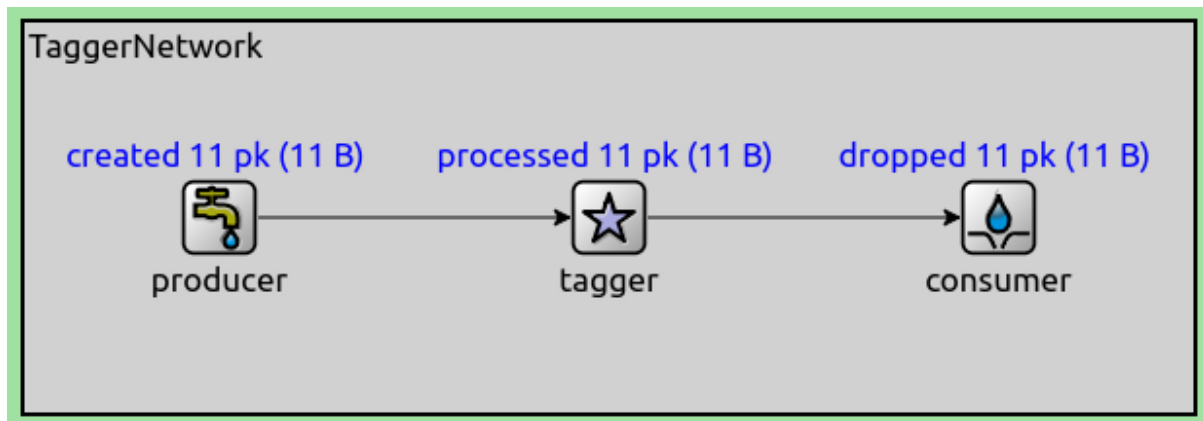
This step demonstrates the [PacketTagger](#) module, which attaches various protocol-specific request tags to packets. Several of the request tags available in INET can be attached, such as the following:

- Request to send the packet via a particular interface
- Request to use a specific transmission power when transmitting the packet
- etc.

For the list of tags that can be attached, see the parameters of [PacketTagger](#).

The packet tagger module has a filter class to filter which packets to attach a tag onto. By default, packets are not filtered.

In this step, packets are produced periodically by an active packet source ([ActivePacketSource](#)). The packets pass through a packet tagger which attaches a `HopLimitReq` tag. The packets are consumed by a passive packet sink ([PassivePacketSink](#)).



```

network TaggerTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        tagger: PacketTagger {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections:
        producer.out --> tagger.in;
        tagger.out --> consumer.in;
}

```

```

[Config Tagger]
network = TaggerTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.tagger.hopLimit = 1

```

3.33 Requesting Protocol-Specific Behavior Based on Packet Data

The `ContentBasedTagger` module, similarly to `PacketTagger`, attaches protocol-specific request tags to packets based on their content, according to the configured packet filter and packet data filter.

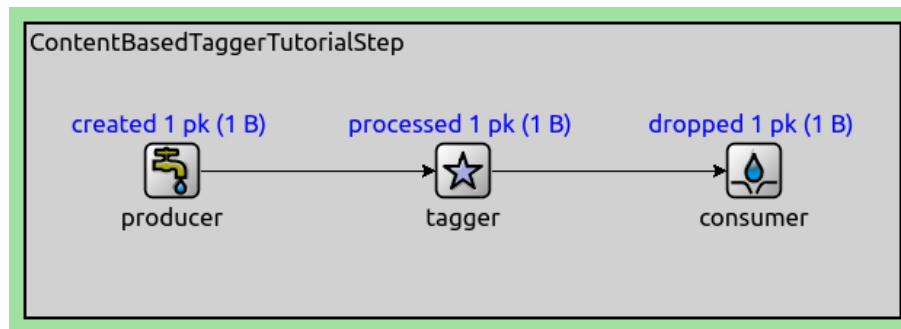
In this example network, an active packet source (`ActivePacketSource`) generates 1-byte and 2-byte packets randomly. The packet source pushes packets into the tagger (`ContentBasedTagger`), which is configured to attach a hop limit tag to 1-byte packets. Packets are consumed by a passive packet sink (`PassivePacketSink`).

```

network ContentBasedTaggerTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }

```

(continues on next page)



(continued from previous page)

```

    }
    tagger: ContentBasedTagger {
        @display("p=300,100");
    }
    consumer: PassivePacketSink {
        @display("p=500,100");
    }
    connections:
        producer.out --> tagger.in;
        tagger.out --> consumer.in;
}

```

```

[Config ContentBasedTagger]
network = ContentBasedTaggerTutorialStep
sim-time-limit = 10s

*.producer.packetLength = intuniform(1B,2B)
*.producer.productionInterval = 1s
*.tagger.packetFilter = expr(totalLength == 1B)
*.tagger.hopLimit = 1

```

3.34 Labeling Packets with Textual Tags

The `PacketLabeler` module attaches a textual label to incoming packets based on a configured packet classifier function. Other modules such as `LabelScheduler` and `LabelClassifier`, can use these labels as input for the treatment of the packet. For example, `LabelClassifier` classifies packets to one of its outputs, based on the presence of certain labels. Labels allow the place of identifying the packet as belonging to some category and the place of acting upon that classification information to be physically separate.

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The single source is connected to a classifier (`LabelClassifier`). The packets are consumed by two passive packet sinks (`PassivePacketSink`). The classifier forwards packets alternately to one or the other sink based on the packet's label. The label is attached by a `PacketLabeler` based on the packet length.

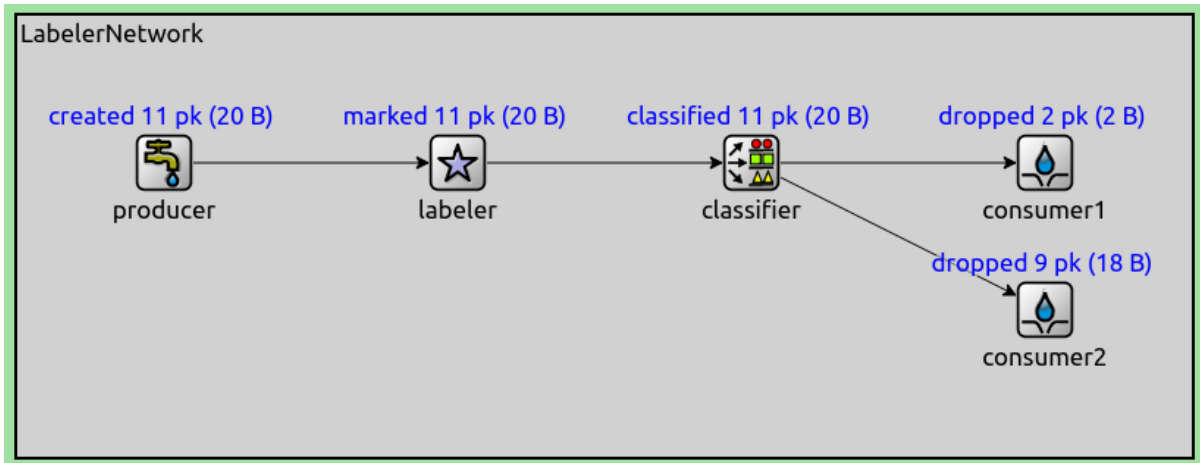
In this example network, an active packet source (`ActivePacketSource`) generates randomly either 1-byte or 2-byte packets. The packets are pushed into a `PacketLabeler`, which attaches the small text label to 1-byte packets, and the large text label to 2-byte packets. The labeler pushes packets into a `LabelClassifier`, which classifies packets labeled small to `consumer1`, and packets labeled large to `consumer2`.

```

network LabelerTutorialStep
{
    @display("bgb=800,300");
    submodules:
        producer: ActivePacketSource {

```

(continues on next page)



(continued from previous page)

```

        @display("p=100,100");
    }
    labeler: ContentBasedLabeler {
        @display("p=300,100");
    }
    classifier: LabelClassifier {
        @display("p=500,100");
    }
    consumer1: PassivePacketSink {
        @display("p=700,100");
    }
    consumer2: PassivePacketSink {
        @display("p=700,200");
    }
    connections allowunconnected:
        producer.out --> labeler.in;
        labeler.out --> classifier.in;
        classifier.out++ --> consumer1.in;
        classifier.out++ --> consumer2.in;
}

```

[Config Labeler]

```

network = LabelerTutorialStep
sim-time-limit = 10s

*.producer.packetLength = intuniform(1B, 2B)
*.producer.productionInterval = 1s
*.labeler.packetFilters = [expr(totalLength == 1B), expr(totalLength == 2B)]
*.labeler.labels = "small large"
*.classifier.labelsToGateIndices = "small 0 large 1"

```

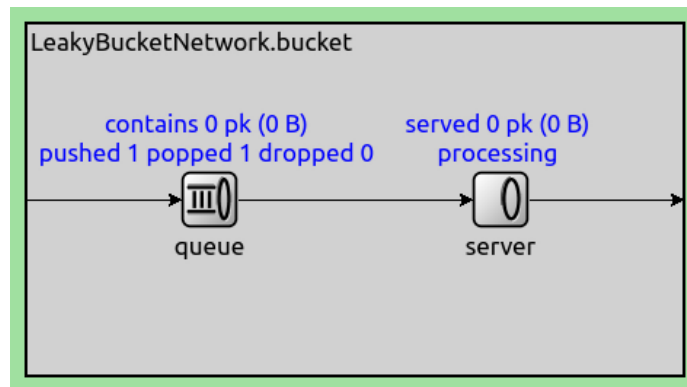
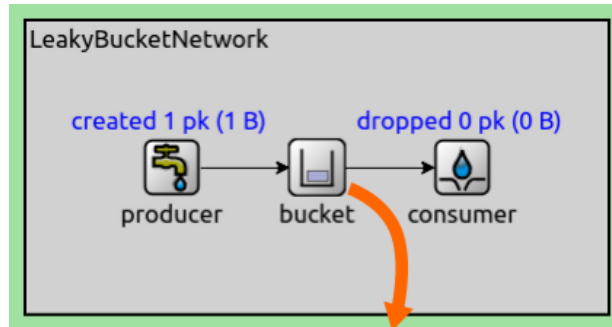
Traffic Conditioning

3.35 Leaky Bucket

The `LeakyBucket` compound module implements the leaky bucket algorithm. By default, the module contains a `DropTailQueue`, and a `PacketServer`. The queue capacity and the processing time of the server can be used to parameterize the leaky bucket algorithm.

In this example network, packets are produced by an active packet source (`ActivePacketSource`). The packet source

pushes packets into a `LeakyBucket` module, which pushes them into a passive packet sink (`PassivePacketSink`). The leaky bucket is configured with a processing time of 1s, and a packet capacity of 1.



```
network LeakyBucketTutorialStep
{
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
    bucket: LeakyBucket {
      @display("p=200,100");
    }
    consumer: PassivePacketSink {
      @display("p=300,100");
    }
  connections allowunconnected:
    producer.out --> bucket.in;
    bucket.out --> consumer.in;
}
```

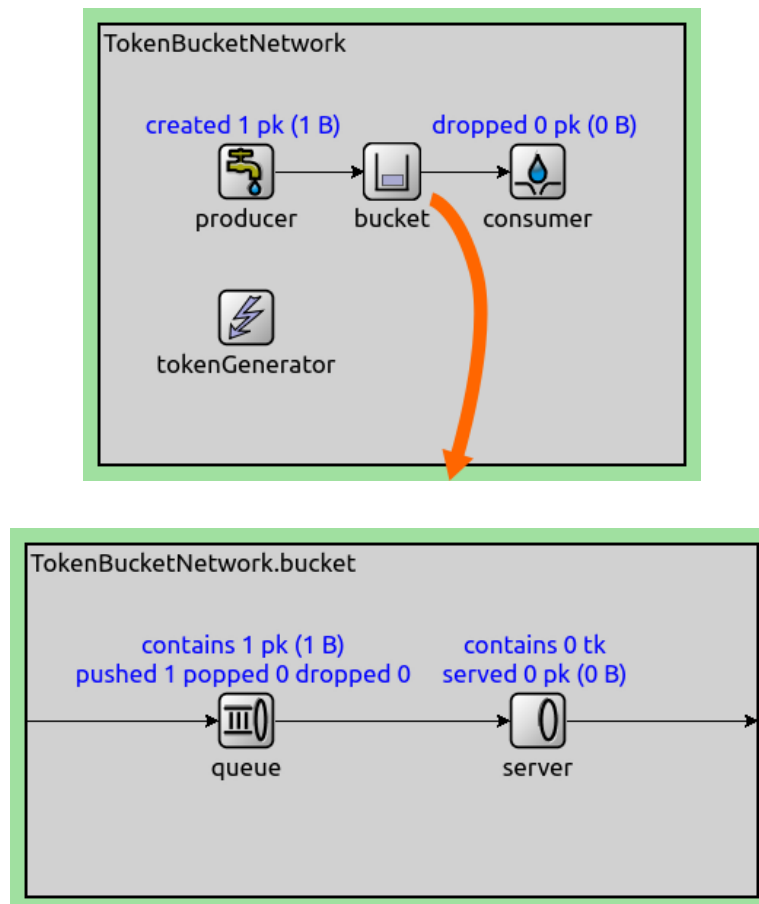
```
[Config LeakyBucket]
network = LeakyBucketTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 0.5s)
*.bucket.server.processingTime = 1s
```

3.36 Token Bucket

The `TokenBucket` compound module implements the token bucket algorithm. By default, the module contains a `DropTailQueue` and a `TokenBasedServer`. The queue capacity and the token consumption of the token-based server can be used to parameterize the token bucket algorithm. The module uses an external token generator for the token-based server.

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The packets are pushed into a token bucket (`TokenBucket`), which pushes them into a passive packet sink (`PassivePacketSink`). A token generator (`TimeBasedTokenGenerator`) generates tokens periodically into the token bucket module.



```
network TokenBucketTutorialStep
{
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
    bucket: TokenBucket {
      @display("p=200,100");
    }
    consumer: PassivePacketSink {
      @display("p=300,100");
    }
    tokenGenerator: TimeBasedTokenGenerator {
      @display("p=100,200");
      storageModule = ".bucket.server";
    }
}
```

(continues on next page)

(continued from previous page)

```

connections allowunconnected:
    producer.out --> bucket.in;
    bucket.out --> consumer.in;
}

```

```

[Config TokenBucket]
network = TokenBucketTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = uniform(0s, 2s)
*.bucket.server.maxNumTokens = 10
*.tokenGenerator.generationInterval = uniform(0s, 2s)

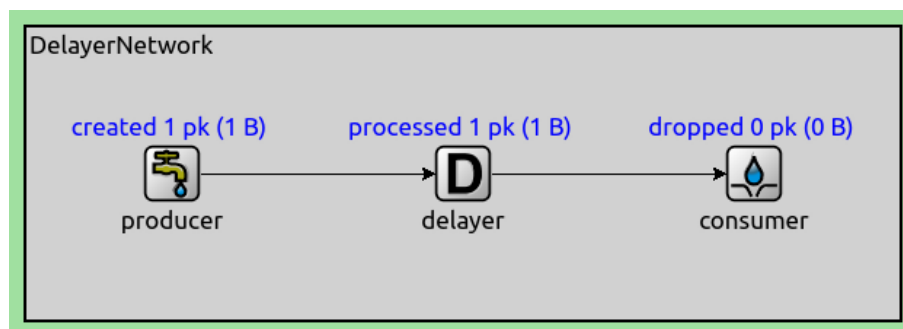
```

Other Generic Elements

3.37 Delaying Packets

The `PacketDelayer` module delays packets for a configured amount of time.

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The produced packets are delayed (`PacketDelayer`) for a random amount of time. Finally, the packets are pushed into a passive packet sink (`PassivePacketSink`).



```

network DelayerTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        delayer: PacketDelayer {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        producer.out --> delayer.in;
        delayer.out --> consumer.in;
}

```

```

[Config Delayer]
network = DelayerTutorialStep

```

(continues on next page)

(continued from previous page)

```

sim-time-limit = 10s

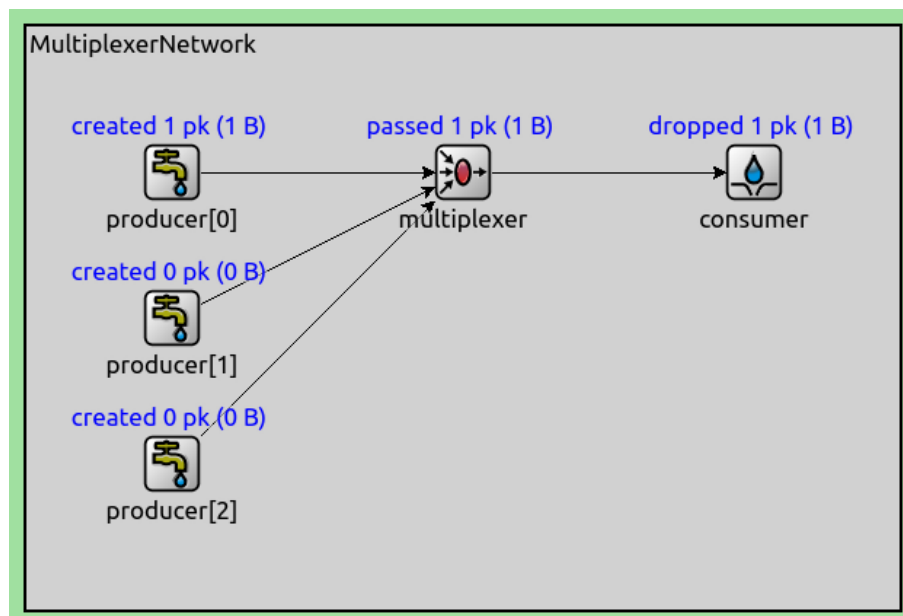
*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.delayer.delay = uniform(0s, 2s)

```

3.38 Connecting Multiple Active Sources to a Passive Sink

The `PacketMultiplexer` module connects to multiple active packet sources on its inputs, and pushes all incoming packets onto a passive packet sink on its single output.

In this example network, packets are produced at random intervals by several active packet sources (`ActivePacketSource`). The packets are consumed by a single passive packet sink (`PassivePacketSink`) upon arrival. The single sink is connected to multiple sources using an intermediary component (`PacketMultiplexer`) which simply forwards packets.



```

network MultiplexerTutorialStep
{
    parameters:
        int numProducers;
    submodules:
        producer[numProducers]: ActivePacketSource {
            @display("p=100,100,c,100");
        }
        multiplexer: PacketMultiplexer {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        for i=0..numProducers-1 {
            producer[i].out --> multiplexer.in++;
        }
        multiplexer.out --> consumer.in;
}

```

(continues on next page)

(continued from previous page)

}

```

[Config Multiplexer]
network = MultiplexerTutorialStep
sim-time-limit = 10s

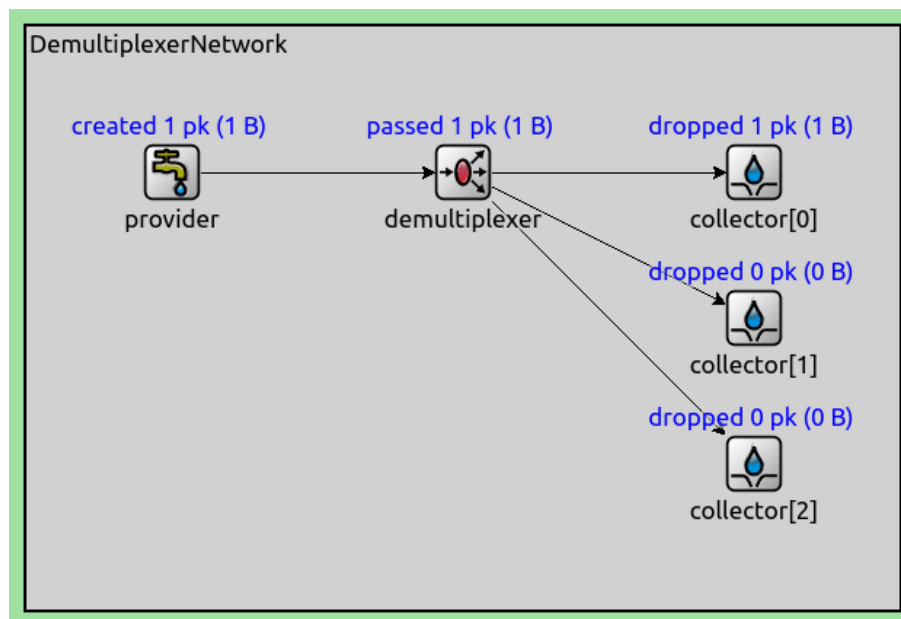
*.numProducers = 3
*.producer[*].packetLength = 1B
*.producer[*].productionInterval = uniform(0s, 2s)

```

3.39 Connecting a Passive Source to Multiple Active Sinks

The `PacketDemultiplexer` module connects to a passive packet source on its input and multiple active packet sinks on its outputs. When one of the collectors requests a packet from the demultiplexer, it pops a packet from the provider and forwards it to the collector.

In this example network, packets are collected at random intervals by several active packet sinks (`ActivePacketSink`). The packets are provided by a single passive packet source (`PassivePacketSource`). The source is connected to the sinks using a demultiplexer, which simply forwards packets.



```

network DemultiplexerTutorialStep
{
  parameters:
    int numCollectors;
  submodules:
    provider: PassivePacketSource {
      @display("p=100,100");
    }
    demultiplexer: PacketDemultiplexer {
      @display("p=300,100");
    }
    collector[numCollectors]: ActivePacketSink {
      @display("p=500,100,c,100");
    }
  connections allowunconnected:

```

(continues on next page)

(continued from previous page)

```

provider.out --> demultiplexer.in;
for i=0..numCollectors-1 {
    demultiplexer.out++ --> collector[i].in;
}
}

```

```

[Config Demultiplexer]
network = DemultiplexerTutorialStep
sim-time-limit = 10s

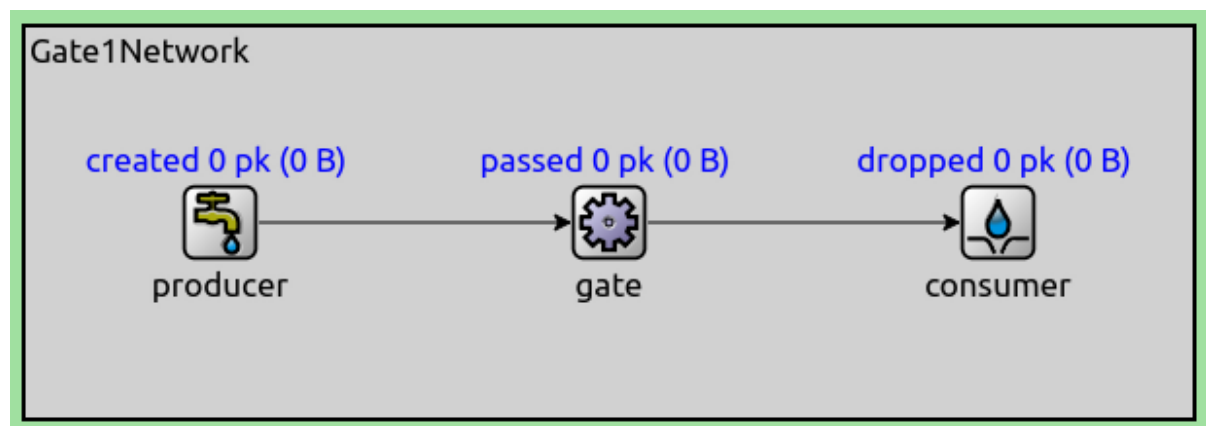
*.provider.packetLength = 1B
*.numCollectors = 3
*.collector[*].collectionInterval = uniform(0s, 2s)

```

3.40 Blocking/Unblocking Packet Flow (Active Source)

The `PacketGate` module allows packets to pass through when it is open, and blocks them when it is closed. The gate opens and closes according to the configured schedule. There are multiple ways to specify the schedule: simple open/close time parameters, a script parameter, or programmatically (via C++).

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The packets pass through a packet gate if it is open, otherwise packets are not generated. The packets are consumed by a passive packet sink (`PassivePacketSink`). The time of opening and closing the gate is configured via parameters.



```

network Gate1TutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        gate: PacketGate {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections:
        producer.out --> gate.in;
        gate.out --> consumer.in;
}

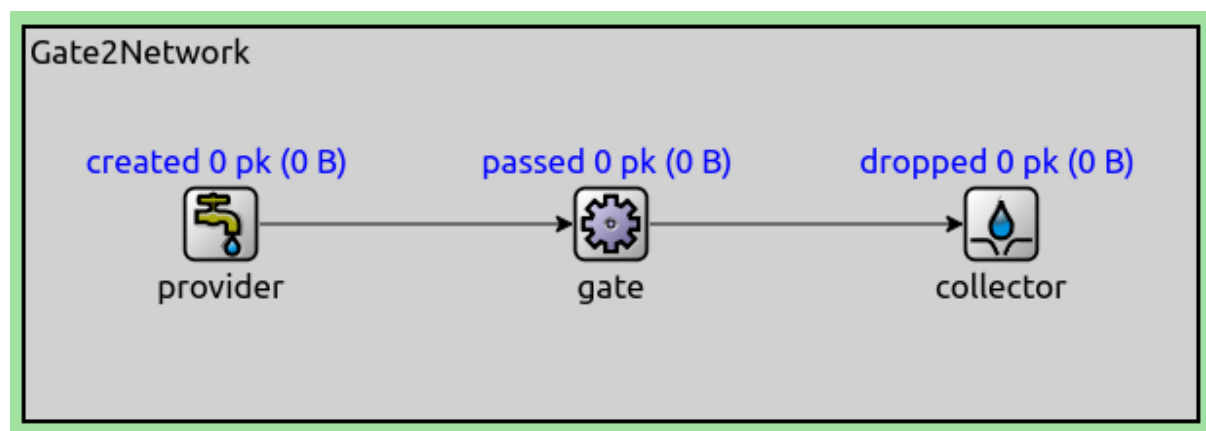
```

```
[Config Gate1]
network = Gate1TutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.gate.openTime = 3s
*.gate.closeTime = 7s
```

3.41 Blocking/Unblocking Packet Flow (Active Sink)

This step is similar to the previous one, but with an active packet sink and passive packet source. Packets are collected periodically by an active packet sink ([ActivePacketSink](#)). The packets pass through a packet gate if it is open, otherwise packets are not generated. The packets are provided by a passive packet source ([PassivePacketSource](#)).



```
network Gate2TutorialStep
{
  @display("bgb=600,200");
  submodules:
    provider: PassivePacketSource {
      @display("p=100,100");
    }
    gate: PacketGate {
      @display("p=300,100");
    }
    collector: ActivePacketSink {
      @display("p=500,100");
    }
  connections:
    provider.out --> gate.in;
    gate.out --> collector.in;
}
```

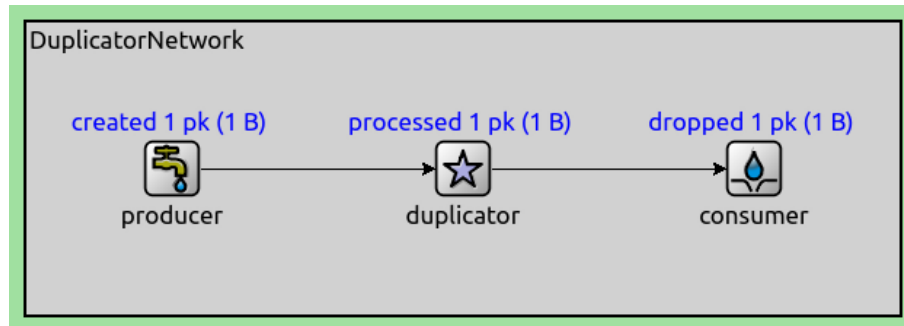
```
[Config Gate2]
network = Gate2TutorialStep
sim-time-limit = 10s

*.provider.packetLength = 1B
*.collector.collectionInterval = 1s
*.gate.openTime = 3s
*.gate.closeTime = 7s
```

3.42 Duplicating Packets from One Input to One Output

The `PacketDuplicator` module creates a configured number of duplicates of incoming packets.

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The packets are pushed into a (`PacketDuplicator`), which randomly creates 0 or 1 copy of them. Finally, the packets are all sent into a passive packet sink (`PassivePacketSink`).



```
network DuplicatorTutorialStep
{
    @display("bgb=600,200");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        duplicator: PacketDuplicator {
            @display("p=300,100");
        }
        consumer: PassivePacketSink {
            @display("p=500,100");
        }
    connections allowunconnected:
        producer.out --> duplicator.in;
        duplicator.out --> consumer.in;
}
```

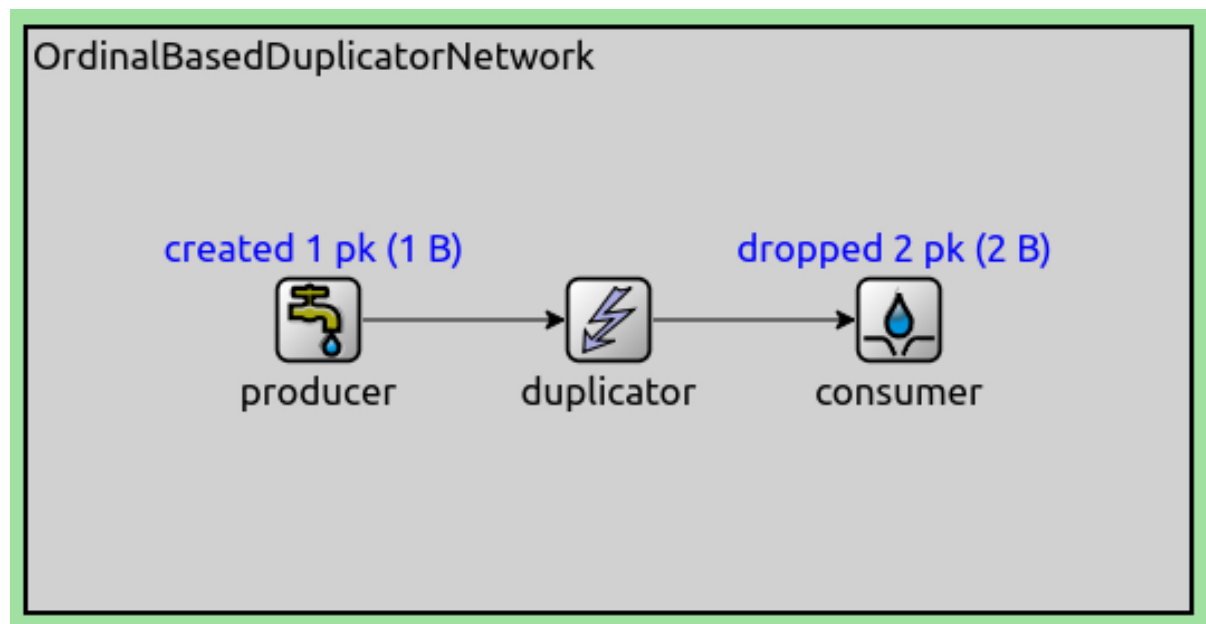
```
[Config Duplicator]
network = DuplicatorTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.duplicator.numDuplicates = intuniform(0, 1)
```

3.43 Duplicating Packets Based On Their Ordinal Number

The `OrdinalBasedDuplicator` module duplicates packets based on the ordinal number of incoming packets. The packets to be duplicated can be configured by listing their ordinal numbers in a parameter. When the packets to be duplicated need to be selected using more complex criteria, a duplicator module can be combined with a classifier and a multiplexer to achieve that.

In this example network, packets are produced periodically by an active packet source (`ActivePacketSource`). The packet source pushes packets into a duplicator (`OrdinalBasedDuplicator`). The duplicator is configured to duplicate every second packet based on its ordinal number. The packets are then consumed by a passive packet sink (`PassivePacketSink`).



```
network OrdinalBasedDuplicatorTutorialStep
{
  submodules:
    producer: ActivePacketSource {
      @display("p=100,100");
    }
    duplicator: OrdinalBasedDuplicator {
      @display("p=200,100");
    }
    consumer: PassivePacketSink {
      @display("p=300,100");
    }
  connections:
    producer.out --> duplicator.in;
    duplicator.out --> consumer.in;
}
```

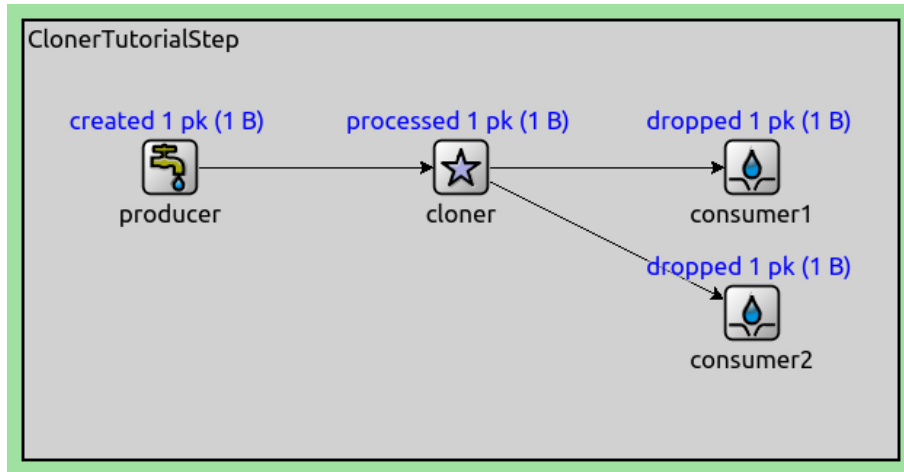
```
[Config OrdinalBasedDuplicator]
network = OrdinalBasedDuplicatorTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s
*.duplicator.duplicatesVector = "0; 2; 4; 6; 8; 10"
```

3.44 Cloning Packets from One Input To Multiple Outputs

The [PacketCloner](#) module has one input gate and multiple output gates. Packets pushed to the input are copied and pushed out on all outputs.

In this example network, packets are created by an active packet source ([ActivePacketSource](#)). The packet source pushes packets into the cloner ([PacketCloner](#)), which pushes a copy of the packet into the two passive packet sources ([PassivePacketSource](#)) it is connected to.



```

network ClonerTutorialStep
{
    @display("bgb=600,300");
    submodules:
        producer: ActivePacketSource {
            @display("p=100,100");
        }
        cloner: PacketCloner {
            @display("p=300,100");
        }
        consumer1: PassivePacketSink {
            @display("p=500,100");
        }
        consumer2: PassivePacketSink {
            @display("p=500,200");
        }
    connections allowunconnected:
        producer.out --> cloner.in;
        cloner.out++ --> consumer1.in;
        cloner.out++ --> consumer2.in;
}

```

[Config Cloner]

```

network = ClonerTutorialStep
sim-time-limit = 10s

*.producer.packetLength = 1B
*.producer.productionInterval = 1s

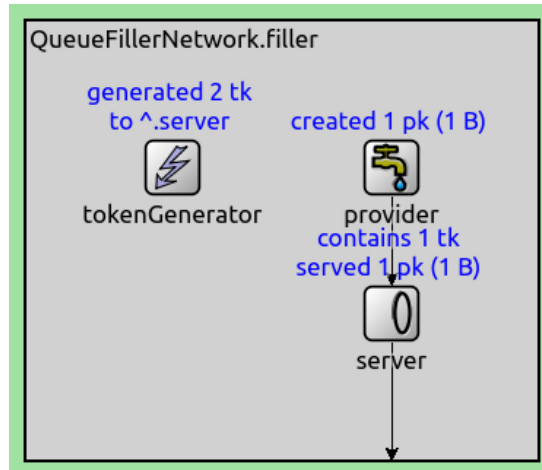
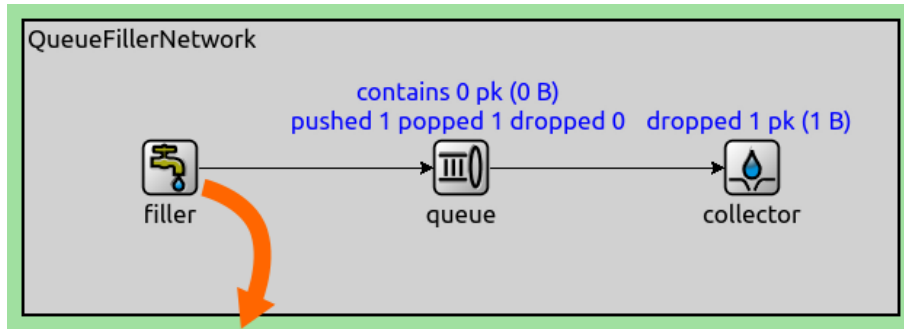
```

Advanced Sources and Sinks

3.45 Preventing a Queue from Becoming Empty

The `QueueFiller` module creates packets whenever a connected queue becomes empty. One use of this module is to generate continuous traffic on a network interface.

In this example network, an active packet sink (`ActivePacketSink`) periodically pops packets from a queue (`PacketQueue`). Whenever the queue becomes empty, a queue filler module (`QueueFiller`) pushes a packet into it.



```

network QueueFillerTutorialStep
{
  @display("bgb=600,200");
  submodules:
    filler: QueueFiller {
      @display("p=100,100");
      tokenGenerator.queueModule = "queue";
    }
    queue: PacketQueue {
      @display("p=300,100");
    }
    collector: ActivePacketSink {
      @display("p=500,100");
    }
  connections allowunconnected:
    filler.out --> queue.in;
    queue.out --> collector.in;
}

```

```

[Config QueueFiller]
network = QueueFillerTutorialStep
sim-time-limit = 10s

*.filler.provider.packetLength = 1B
*.collector.collectionInterval = 1s

```

3.46 Example: Request/Response-Based Communication

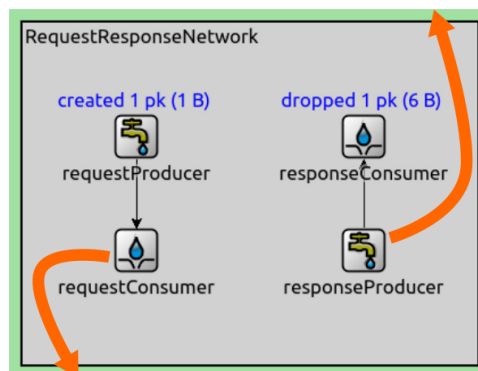
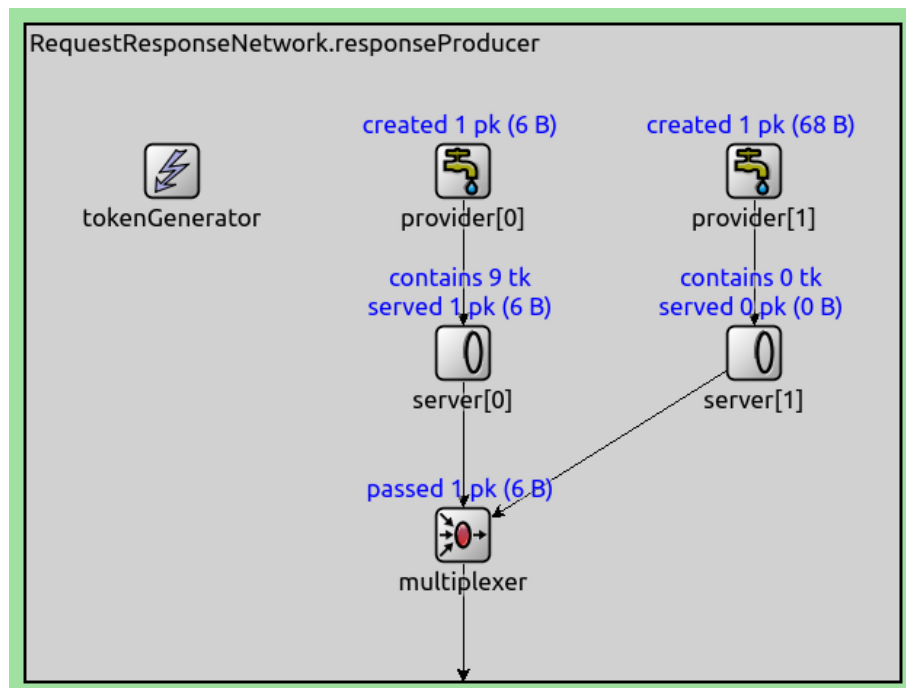
This step contains a simplified version of a client/server and a request/response-based communication. The client initiates the communication by sending a request to the server. The server processes requests in the order of arrival and generates the response traffic.

Request packets are produced periodically and randomly by an active packet source ([ActivePacketSource](#)) in the client. The generated requests fall into one of two categories based on the data they contain.

The server processes requests one by one in order, using a compound consumer ([RequestConsumer](#)). Each request is first classified based on the data it contains, and then a certain number of tokens are generated as the request is consumed.

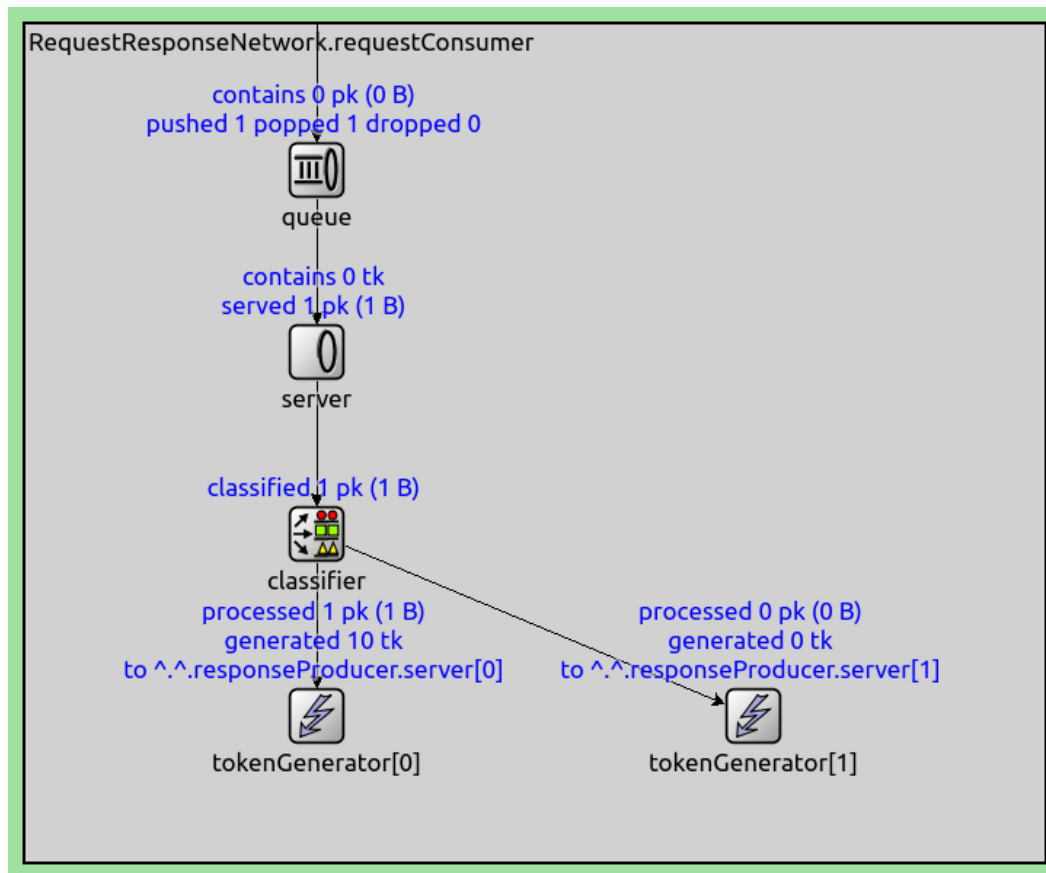
The tokens are added to a response server in a compound producer ([ResponseProducer](#)). The response producer generates different traffic randomly over a period of time for each kind of request.

The client consumes the response packets by a passive packet sink ([PassivePacketSink](#)).



```
network RequestResponseTutorialStep
{
    @display("bgb=400,300");
    submodules:
        requestProducer: ActivePacketSource {
            @display("p=100,100");
```

(continues on next page)



(continued from previous page)

```

}
responseConsumer: PassivePacketSink {
  @display("p=300,100");
}
requestConsumer: RequestConsumer {
  @display("p=100,200");
  responseProducerModule = "^.responseProducer";
}
responseProducer: ResponseProducer {
  @display("p=300,200");
  tokenGenerator.storageModule = "^.^.requestConsumer.server";
}
connections allowunconnected:
  requestProducer.out --> requestConsumer.in;
  responseProducer.out --> responseConsumer.in;
}

```

[Config RequestResponse]

```

network = RequestResponseTutorialStep
sim-time-limit = 100s

*.requestProducer.packetLength = 1B
*.requestProducer.packetData = intuniform(0, 1)
*.requestProducer.productionInterval = uniform(0s, 2s)
*.requestConsumer.numKind = 2
*.requestConsumer.classifier.typeName = "ContentBasedClassifier"
*.requestConsumer.classifier.packetFilters = [expr(ByteCountChunk.data == 0),_]

```

(continues on next page)

(continued from previous page)

```

↳expr(ByteCountChunk.data == 1)]
*.requestConsumer.tokenGenerator[0].numTokensPerPacket = 10
*.requestConsumer.tokenGenerator[1].numTokensPerPacket = 10
*.responseProducer.numKind = 2
*.responseProducer.provider[0].packetLength = intuniform(1B, 10B)
*.responseProducer.provider[0].providingInterval = uniform(0s, 0.2s)
*.responseProducer.provider[1].packetLength = intuniform(1B, 100B)
*.responseProducer.provider[1].providingInterval = uniform(0s, 0.2s)

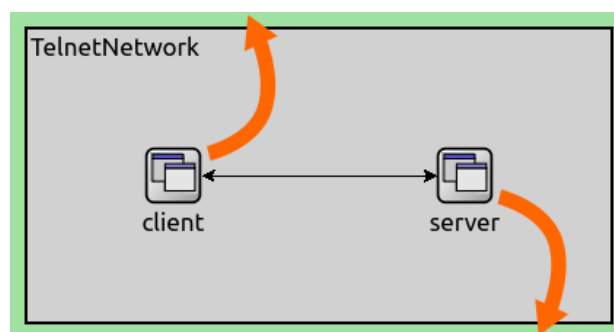
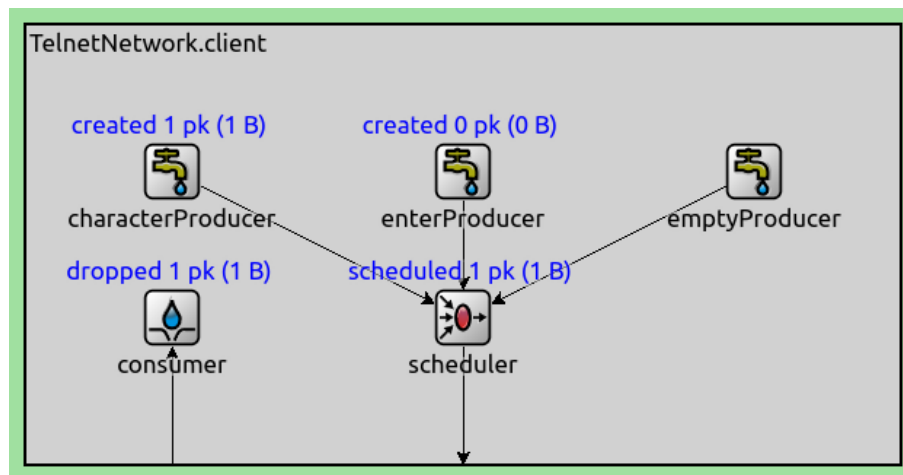
```

Some Complex Examples

3.47 Example: Generating Telnet Traffic

This example demonstrates a telnet client (TelnetClientTraffic) and a telnet server (TelnetServerTraffic) module built using queueing components.

These two modules are created by copying the `TelnetClientApp` and `TelnetServerApp` modules available in INET, and omitting the `IApp` interface. As such, the telnet traffic apps can be connected to each other directly, without any sockets or protocols.

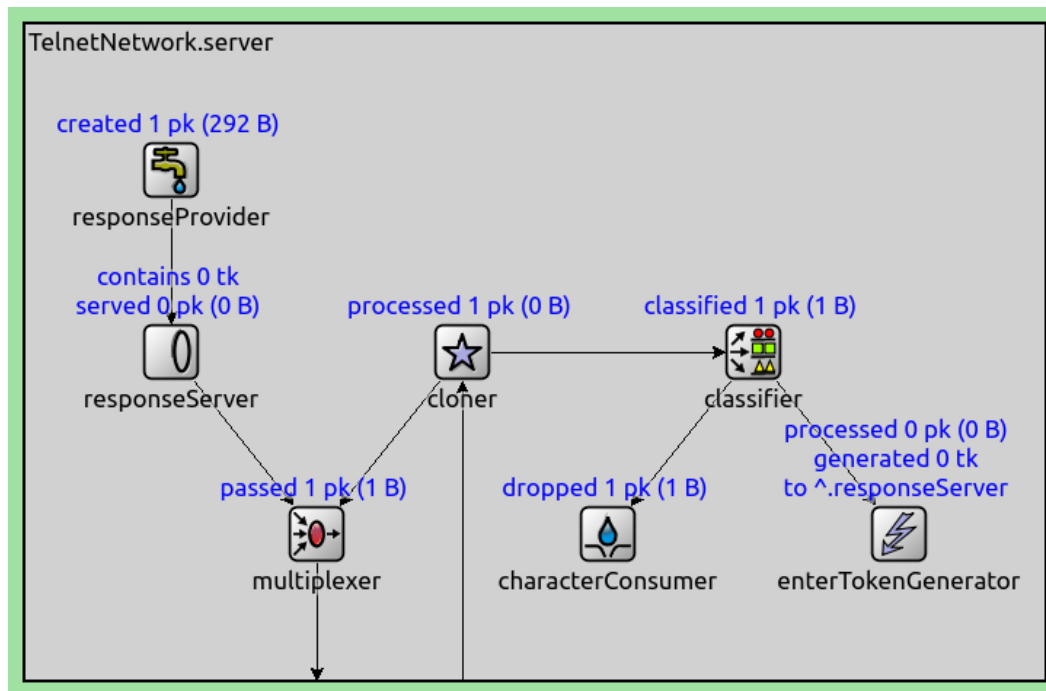


```

network TelnetTutorialStep
{
    @display("bgb=400,200");
    submodules:
        client: TelnetClientTraffic {
            @display("p=100,100");
        }
        server: TelnetServerTraffic {

```

(continues on next page)



(continued from previous page)

```

        @display("p=300,100");
    }
    connections:
        client.out --> server.in;
        client.in <-- server.out;
}

```

```

module TelnetClientTraffic
{
    parameters:
        @display("i=block/app");
    gates:
        input in;
        output out;
    submodules:
        characterProducer: <default("ActivePacketSource")> like IActivePacketSource {
            parameters:
                packetLength = 1B;
                packetData = intuniform(97, 122); // lower case ASCII characters
                productionInterval = uniform(0.1s, 0.2s); // typing speed between 5
→and 10 characters per second
                @display("p=100,100");
        }
        enterProducer: <default("ActivePacketSource")> like IActivePacketSource {
            parameters:
                packetLength = 1B;
                packetData = 13; // enter character
                productionInterval = 0.1s;
                @display("p=300,100");
        }
        emptyProducer: <default("EmptyPacketSource")> like IActivePacketSource {
            parameters:
                @display("p=500,100");
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    scheduler: <default("MarkovScheduler")> like IPacketScheduler {
        parameters:
            transitionProbabilities = "0 1 0 0 0 1 1 0 0"; // character -> enter -
↪> wait -> character
            waitIntervals = "uniform(0,3) 0 uniform(10,30)";
            @display("p=300,200");
    }
    consumer: <default("PassivePacketSink")> like IPassivePacketSink {
        parameters:
            @display("p=100,200");
    }
    connections:
        characterProducer.out --> scheduler.in++;
        enterProducer.out --> scheduler.in++;
        emptyProducer.out --> scheduler.in++;
        scheduler.out --> { @display("m=s"); } --> out;
        in --> { @display("m=s"); } --> consumer.in;
}

```

```

module TelnetServerTraffic
{
    parameters:
        @display("i=block/app");
    gates:
        input in;
        output out;
    submodules:
        cloner: PacketCloner {
            parameters:
                @display("p=300,225");
        }
        responseProvider: <default("PassivePacketSource")> like IPassivePacketSource {
            parameters:
                @display("p=100,100");
        }
        responseServer: <default("TokenBasedServer")> like IPacketServer {
            parameters:
                @display("p=100,225");
        }
        multiplexer: PacketMultiplexer {
            parameters:
                @display("p=200,350");
        }
        classifier: <default("PacketClassifier")> like IPacketClassifier {
            parameters:
                classifierClass = default(
↪ "inet::queueing::PacketCharacterOrEnterClassifier");
                @display("p=500,225");
        }
        characterConsumer: <default("PassivePacketSink")> like IPassivePacketSink {
            parameters:
                @display("p=400,350");
        }
        enterTokenGenerator: PacketBasedTokenGenerator {
            parameters:

```

(continues on next page)

(continued from previous page)

```

        storageModule = default("^responseServer");
        @display("p=600,350");
    }
    connections:
    in --> { @display("m=s"); } --> cloner.in;
    cloner.out++ --> classifier.in;
    cloner.out++ --> multiplexer.in++;
    responseProvider.out --> responseServer.in;
    responseServer.out --> multiplexer.in++;
    classifier.out++ --> characterConsumer.in;
    classifier.out++ --> enterTokenGenerator.in;
    multiplexer.out --> { @display("m=s"); } --> out;
}

```

[Config Telnet]

```

network = TelnetTutorialStep
sim-time-limit = 100s

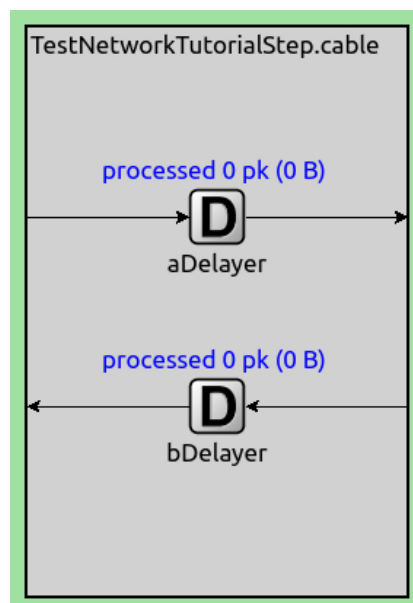
**.server.enterTokenGenerator.numTokensPerPacket = intuniform(0, 10)
**.server.responseProvider.packetLength = intuniform(100B, 1000B)
**.server.responseProvider.providingInterval = uniform(0s, 0.1s)

```

3.48 Example: Simulating a Transmission Channel

This example network demonstrates how queueing components can be combined to simulate a point-to-point link with a finite bit rate. (This is only interesting as a demonstration of the power of the queueing library, network links are never actually simulated like this in OMNeT++ or INET.)

The network features two hosts (ExampleHost) communicating. The hosts are connected by a cable module (ExampleCable) which adds delay to the connection. Each host contains a packet source and a packet sink application, connected to the network level by an interface module (ExampleInterface).

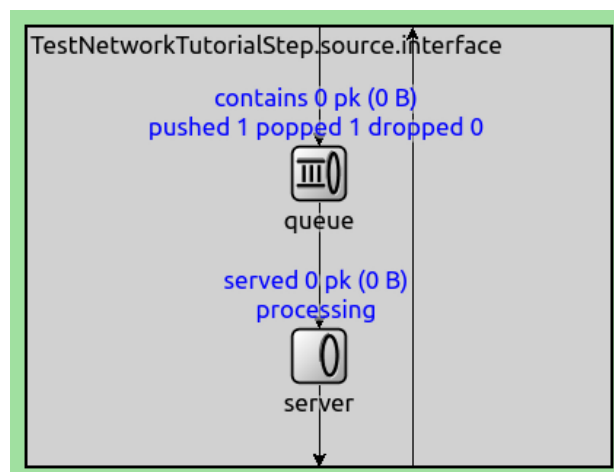
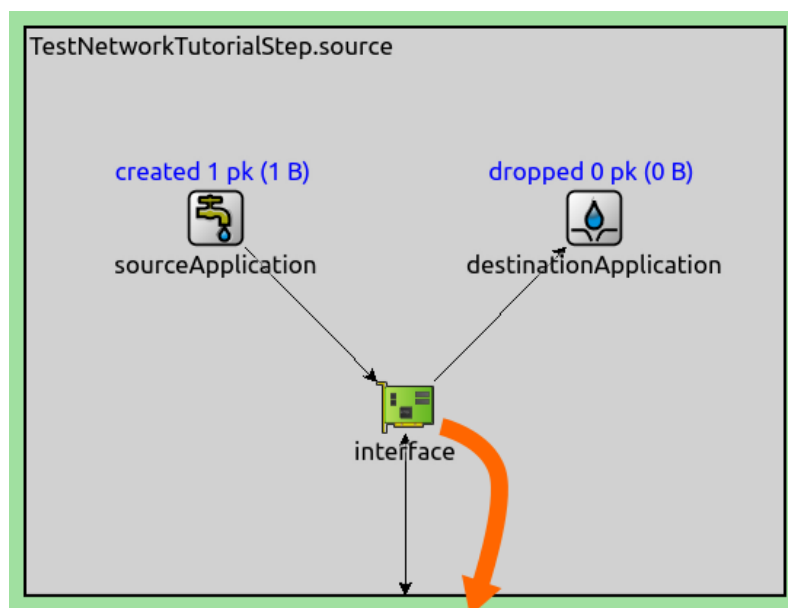
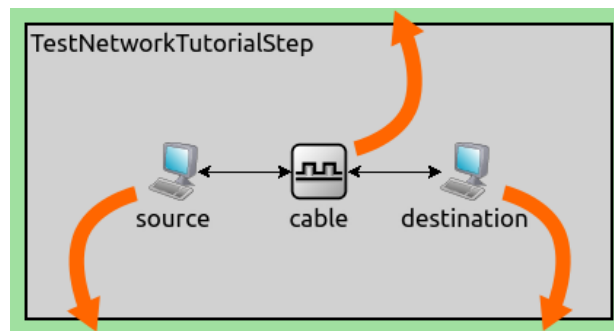


```

network ExampleNetworkTutorialStep
{
    submodules:

```

(continues on next page)



(continued from previous page)

```

    source: ExampleHost {
        @display("p=100,100");
    }
    cable: ExampleCable {
        @display("p=200,100");
    }
    destination: ExampleHost {
        @display("p=300,100");
    }
    connections:
    source.lowerOut --> cable.aIn;
    cable.aOut --> destination.lowerIn;
    destination.lowerOut --> cable.bIn;
    cable.bOut --> source.lowerIn;
}

```

```

module ExampleHost
{
    parameters:
        @display("i=device/pc");
    gates:
        input lowerIn;
        output lowerOut;
    submodules:
        sourceApplication: ActivePacketSource {
            @display("p=100,100");
        }
        destinationApplication: PassivePacketSink {
            @display("p=300,100");
        }
        interface: ExampleInterface {
            @display("p=200,200");
        }
    connections:
        sourceApplication.out --> interface.upperIn;
        interface.lowerOut --> lowerOut;
        lowerIn --> interface.lowerIn;
        interface.upperOut --> destinationApplication.in;
}

```

```

module ExampleInterface
{
    @display("bgb=400,300;i=device/card");
    gates:
        input lowerIn;
        input upperIn;
        output lowerOut;
        output upperOut;
    submodules:
        queue: PacketQueue {
            @display("p=200,100");
        }
        server: PacketServer {
            @display("p=200,225");
        }
    connections:

```

(continues on next page)

(continued from previous page)

```

upperIn --> queue.in;
queue.out --> server.in;
server.out --> { @display("m=s"); } --> lowerOut;
lowerIn --> { @display("m=m,66,100,66,0"); } --> upperOut;
}

```

```

module ExampleCable
{
  parameters:
    @display("i=block/mac");
  gates:
    input aIn;
    output aOut;
    input bIn;
    output bOut;
  submodules:
    aDelayer: PacketDelayer {
      @display("p=100,100");
    }
    bDelayer: PacketDelayer {
      @display("p=100,200");
    }
  connections:
    aIn --> { @display("m=w"); } --> aDelayer.in;
    aDelayer.out --> { @display("m=e"); } --> aOut;
    bIn --> { @display("m=e"); } --> bDelayer.in;
    bDelayer.out --> { @display("m=w"); } --> bOut;
}

```

```

[Config ExampleNetwork]
network = ExampleNetworkTutorialStep
sim-time-limit = 10s

*.sourceApplication.packetLength = 1B
*.sourceApplication.productionInterval = uniform(0s, 2s)
*.interface.server.processingTime = uniform(0s, 2s)
*.cable.*Delayer.delay = uniform(0s, 2s)

```

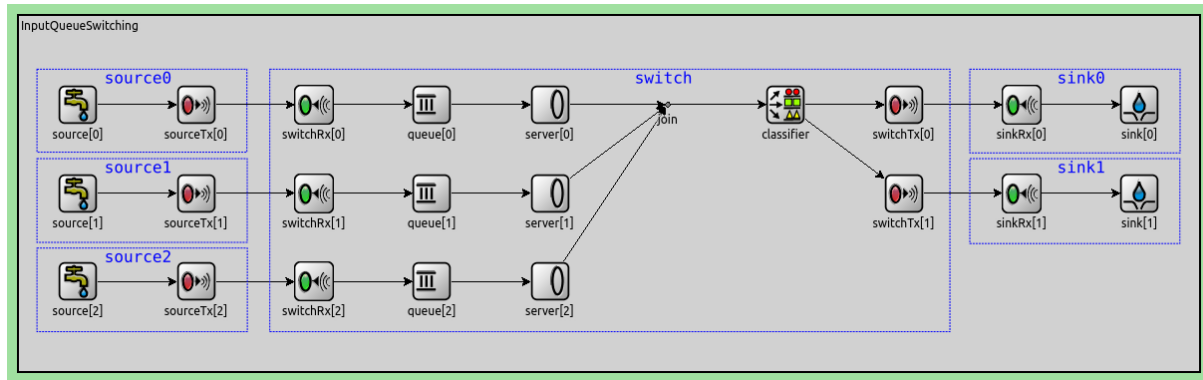
3.49 Example: Input Queue Switching

In this example, we'll build a network containing three packet sources, a switch, and two packet sinks, from queueing components. The queues in the switch are in the input "interfaces". Here is the network, with the conceptual network nodes highlighted:

The sources generate packets, which randomly contain either 0 or 1 as data. The classifier in the switch sends packets based on this data to either sink0 or sink1. Input queue switching is susceptible to head-of-line blocking, i.e. if the first packet in the queue cannot be forwarded for some reason (e.g. the target sink is busy receiving another packet), it can delay subsequent packets in the queue that would otherwise be transmittable at the current time.

Note

The next example demonstrates output queue switching, which is not susceptible to head-of-line blocking.



network InputQueueSwitching

```
{
  parameters:
    int numSources;
    int numSinks;
  submodules:
    source[numSources]: ActivePacketSource {
      @display("p=100,150,col,150");
    }
    sourceTx[numSources]: PacketTransmitter {
      @display("p=300,150,col,150");
    }
    switchRx[numSources]: PacketReceiver {
      @display("p=500,150,col,150");
    }
    queue[numSources]: PacketQueue {
      @display("p=700,150,col,150");
    }
    server[numSources]: InstantServer {
      @display("p=900,150,col,150");
    }
    join: PacketMultiplexer {
      @display("p=1100,150");
    }
    classifier: ContentBasedClassifier {
      @display("p=1300,150");
    }
    switchTx[numSinks]: PacketTransmitter {
      @display("p=1500,150,col,150");
    }
    sinkRx[numSinks]: PacketReceiver {
      @display("p=1700,150,col,150");
    }
    sink[numSinks]: PassivePacketSink {
      @display("p=1900,150,col,150");
    }
  connections:
    for i=0..numSources-1 {
      source[i].out --> sourceTx[i].in;
      sourceTx[i].out --> { datarate = 1Gbps; delay = 1ns; } --> switchRx[i].
      switchRx[i].out --> queue[i].in;
      queue[i].out --> server[i].in;
    }
}
```

(continues on next page)

(continued from previous page)

```

        server[i].out --> join.in++;
    }
    join.out --> classifier.in;
    for i=0..numSinks-1 {
        classifier.out++ --> switchTx[i].in;
        switchTx[i].out --> { datarate = 1Gbps; delay = 1ns; } --> sinkRx[i].in;
        sinkRx[i].out --> sink[i].in;
    }
}

```

```

[Config InputQueueSwitching]
network = InputQueueSwitching
sim-time-limit = 1000s

*.numSources = 3
*.numSinks = 2

*.*.datarate = 16bps

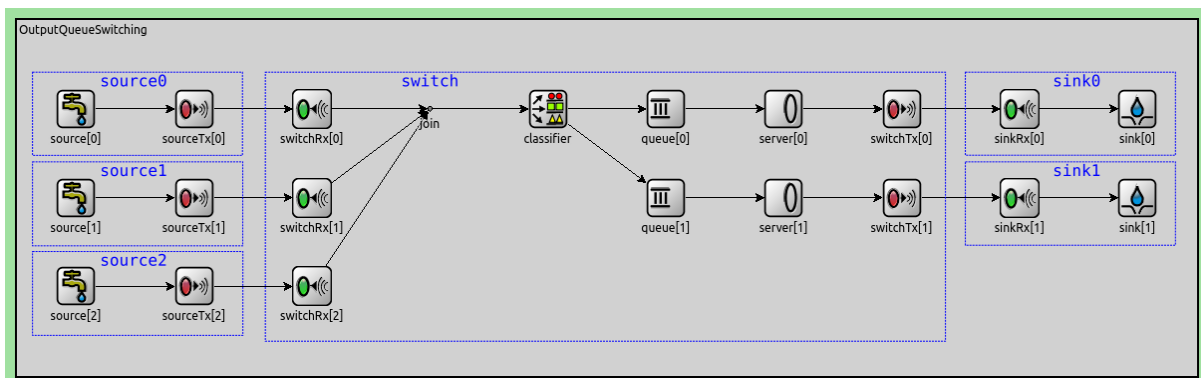
*.source[*].packetLength = 1B
*.source[*].packetData = intuniform(1, 2)
*.source[*].productionInterval = exponential(1s)

*.classifier.packetFilters = [expr(ByteCountChunk.data % 2 == 0), expr(ByteCountChunk.
↪data % 2 == 1)]

```

3.50 Example: Output Queue Switching

In this example, we'll build a network containing three packet sources, a switch, and two packet sinks, from queueing components. The queues in the switch are in the output "interfaces". Here is the network, with the conceptual network nodes highlighted:



The sources generate packets, which randomly contain either 0 or 1 as data. The classifier in the switch sends packets based on this data to either sink0 or sink1. Output queue switching isn't susceptible to head-of-line blocking, as opposed to input queue switching.

```

network OutputQueueSwitching
{
    parameters:
        int numSources;
        int numSinks;
    submodules:

```

(continues on next page)

(continued from previous page)

```

source[numSources]: ActivePacketSource {
    @display("p=100,150,col,150");
}
sourceTx[numSources]: PacketTransmitter {
    @display("p=300,150,col,150");
}
switchRx[numSources]: PacketReceiver {
    @display("p=500,150,col,150");
}
join: PacketMultiplexer {
    @display("p=700,150");
}
classifier: ContentBasedClassifier {
    @display("p=900,150");
}
queue[numSinks]: PacketQueue {
    @display("p=1100,150,col,150");
}
server[numSinks]: InstantServer {
    @display("p=1300,150,col,150");
}
switchTx[numSinks]: PacketTransmitter {
    @display("p=1500,150,col,150");
}
sinkRx[numSinks]: PacketReceiver {
    @display("p=1700,150,col,150");
}
sink[numSinks]: PassivePacketSink {
    @display("p=1900,150,col,150");
}
connections:
    for i=0..numSources-1 {
        source[i].out --> sourceTx[i].in;
        sourceTx[i].out --> DatarateChannel { delay = 1ns; } --> switchRx[i].in;
        switchRx[i].out --> join.in++;
    }
    join.out --> classifier.in;
    for i=0..numSinks-1 {
        classifier.out++ --> queue[i].in;
        queue[i].out --> server[i].in;
        server[i].out --> switchTx[i].in;
        switchTx[i].out --> DatarateChannel { delay = 1ns; } --> sinkRx[i].in;
        sinkRx[i].out --> sink[i].in;
    }
}

```

[Config OutputQueueSwitching]

```

network = OutputQueueSwitching
sim-time-limit = 1000s

*.numSources = 3
*.numSinks = 2

*.*.datarate = 16bps

*.source[*].packetLength = 1B

```

(continues on next page)

(continued from previous page)

```
*.source[*].packetData = intuniform(1, 2)
*.source[*].productionInterval = exponential(1s)

*.classifier.packetFilters = [expr(ByteCountChunk.data % 2 == 0), expr(ByteCountChunk.
↪data % 2 == 1)]
```


REGRESSION TESTING AND FINGERPRINTS TUTORIAL

Learn how to use fingerprint testing to detect regressions in existing simulation code.

Contents

4.1 Getting Started

During the development of simulation models, it is important to detect regressions, i.e. to notice that a change in a correctly working model led to incorrect behavior.

Regressions can be detected by manually examining simulations in Qtenv. However, some behavioral changes might be subtle and go unnoticed, but still would invalidate the model. Regression testing automates this process, so the model doesn't have to be manually checked as thoroughly and as often. It might also detect regressions that would have been missed during manual examination.

Fingerprint testing is a useful and low-cost tool that can be used for this purpose. A fingerprint is a hash value, which is calculated during a simulation run and is characteristic of the simulation's trajectory. After the change in the model, a change in the fingerprint can indicate that the model no longer works along the same trajectory. This may or may not mean that the model is incorrect. When the fingerprints stay the same, the model can be assumed to be still working correctly (thus protecting against regressions). Fingerprints can be calculated in different ways, using so called 'ingredients' that specify which aspects of the simulation to be taken into account when calculating a hash value.

The steps in this tutorial contain examples for typical changes in the model during development, and describe the model validation workflow using fingerprint tests.

Information about OMNeT++'s fingerprint support can be found in the [corresponding section](#) of the OMNeT++ Simulation Manual. Also, the Testing section in the INET Developer's Guide describes regression testing.

4.1.1 Try It Yourself

If you already have INET and OMNeT++ installed, start the IDE by typing `omnetpp`, import the INET project into the IDE, then navigate to the `inet/tutorials/fingerprint` folder in the *Project Explorer*. There, you can view and edit the tutorial files, run simulations, and analyze results.

Otherwise, there is an easy way to install INET and OMNeT++ using `opp_env`, and run the simulation interactively. Ensure that `opp_env` is installed on your system, then execute:

```
$ opp_env run inet-4.6 --init -w inet-workspace --install --build-modes=release --  
↪ chdir \  
-c 'cd inet-4.6.*/tutorials/fingerprint && inet'
```

This command creates an `inet-workspace` directory, installs the appropriate versions of INET and OMNeT++ within it, and launches the `inet` command in the showcase directory for interactive simulation.

Alternatively, for a more hands-on experience, you can first set up the workspace and then open an interactive shell:

```
$ opp_env install --init -w inet-workspace --build-modes=release inet-4.6
$ cd inet-workspace
$ opp_env shell
```

Inside the shell, start the IDE by typing `omnetpp`, import the INET project, then start exploring.

4.1.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

4.2 About Fingerprint Testing

A fingerprint is a hash value, which is calculated during a simulation run from specified “ingredients”, i.e. properties of the simulation such as time of events, module names, packet data, etc. The hash is updated at each event which takes part in the fingerprint calculation (as defined by the ingredients, filtering, etc) until the end of the simulation (or some defined time limit), resulting in a fingerprint value. This value is characteristic of the simulation’s trajectory; a fingerprint change indicates a change in the trajectory.

The fingerprint contains a 32-bit hexadecimal hash value and the ingredients used for calculating it, denoted by letters. Here is an example:

```
53de-64a7/tplx
```

In this example, `t` means time of event, `p` the full path of modules, `l` the message length, and `x` the extra data added programatically. Other possible ingredients include various properties of messages, modules and results. For the complete list, check out the [Fingerprint section](#) in the OMNeT++ Simulation Manual.

The fingerprints can be specified in the ini file for each configuration. Specifying any fingerprint enables fingerprint calculation for the simulation. For example:

```
[Config Ethernet]
network = Wired
fingerprint = 0000-0000/tplx
```

The fingerprint will be calculated for that simulation when run in `Qtenv` or `Cmdenv`. `Qtenv` and `Cmdenv` indicate whether the fingerprint is verified, and print the calculated fingerprint value. The original value can be replaced with the resulting one in the ini file.

INET’s fingerprint test tool automates this process, making it easier to work with many simulations and fingerprints.

4.2.1 The fingerprint tool

The fingerprint tool is a convenient way to run fingerprint tests. It is located in the `inet/bin` folder, and when the `inet` directory is added to the `PATH`, it can be run from any project directory.

The fingerprint test tool uses `.csv` files to run fingerprint tests. It uses `.csv` files to keep track of test runs. A line in the `.csv` file defines a simulation run by specifying the working directory, command line arguments, sim time limit, fingerprint+ingredients, expected result, and tags. The result of one fingerprint test can either be `PASS`, `FAILED` or `ERROR`. A test results in `FAILED` if the calculated fingerprint differs from the one in the `.csv` file; it results in `ERROR` when there was an error during the simulation.

When run without arguments, the fingerprint test tool runs all tests in all `.csv` files in the current directory. Specific `.csv` files to run can be selected with the first argument. Also, the set of tests to run can be filtered with the `-m` command line option, by providing a regex expression/regex that is matched against all information in the `.csv` files.

The fingerprint test tool runs tests concurrently by default, using all CPU cores.

For the list of all available options, run the tool with the `-h` argument.

When the tests are finished, the tool may create additional `.csv` files (appending `UPDATED`, `FAILED`, and `ERROR` to the `.csv` file’s name):

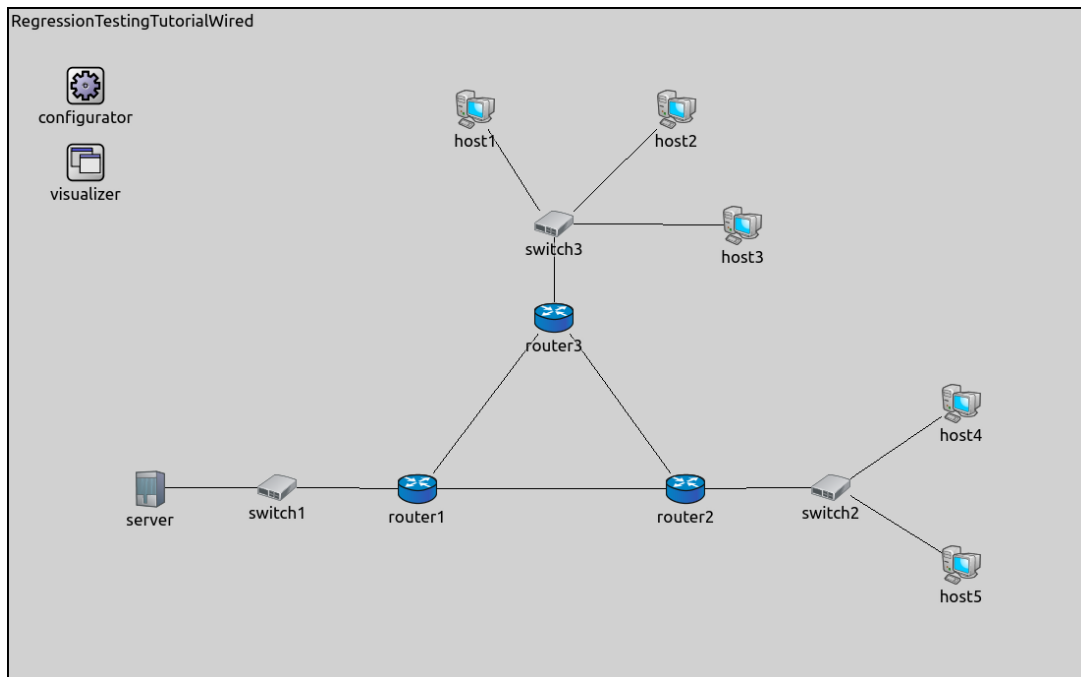
- an UPDATED file, which has the fingerprints just calculated for all lines
- a FAILED file, which contains just the lines for the failed simulations, with the calculated fingerprint. This file is useful for re-running just the simulations with failed fingerprints
- an ERROR file, which contains the lines for simulations which finished with errors, making it easier to rerun just those simulations

The updated file can be used to overwrite the original one to accept the new fingerprints.

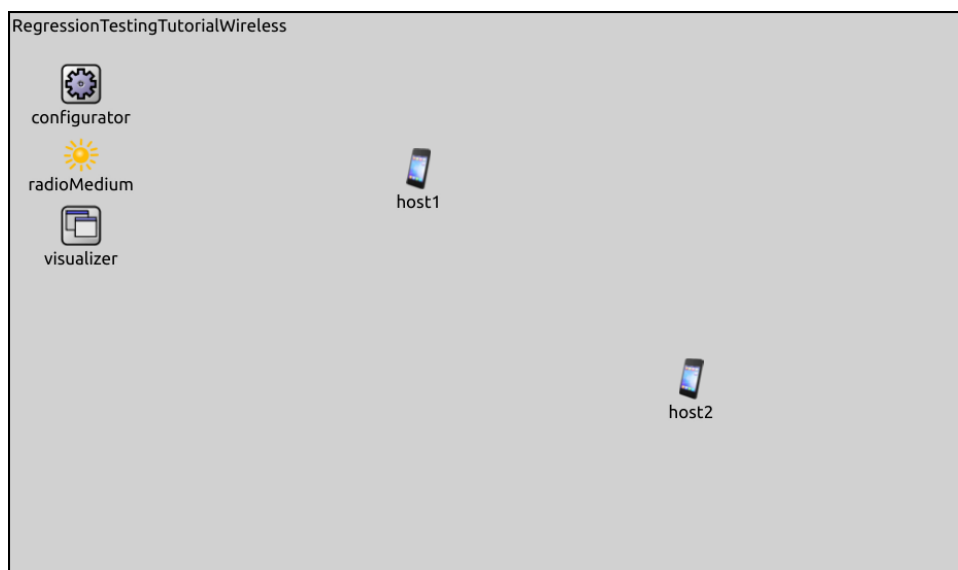
4.2.2 The configurations

The tutorial contains several simulations, which will be used to demonstrate how various changes affect fingerprints. Here is an overview of the simulations:

- **Ethernet:** Contains an Ethernet network with wired hosts and routers. The server host sends UDP packets to two of the hosts:



- **Wifi:** Contains two ad-hoc hosts; one of them sends UDP packets to the other:



- `EthernetShortPacket`: The same as the `Ethernet` configuration, but the server sends 10-Byte UDP packets.
- `WifiShortPacket`: The same as the `Wifi` configuration, but the host sends 10-Byte UDP packets.

The `baseline.csv` file in the tutorial's directory containing the fingerprints:

```
# working directory, command line arguments, simulation time limit, fingerprint,
↪expected result, tags
.,          -f omnetpp.ini -c Ethernet -r 0,          0.2s,          4500-0673/tplx,
↪PASS, EasyToHandleChanges RenamingSubmodule RenamingParameter ChangingPacketLength
↪AddingNewEvents1 AddingNewEvents2 AcceptingFPChanges
.,          -f omnetpp.ini -c EthernetShortPacket -r 0, 0.2s,          ea97-154f/tplx,
↪PASS, ChangingPacketLength
.,          -f omnetpp.ini -c Wifi -r 0,          5s,          791d-aba6/tplx,
↪PASS, EasyToHandleChanges RenamingSubmodule ChangingPacketLength ChangingTimer
↪AddingNewEvents1 AddingNewEvents2 AcceptingFPChanges
.,          -f omnetpp.ini -c WifiShortPacket -r 0,          5s,          d801-fc01/tplx,
↪PASS, ChangingPacketLength
```

The `.csv` file contains the correct fingerprints for the latest INET version, i.e. they all pass (the actual values in the `baseline.csv` might differ from the values in the tutorial text). To try the examples, the user is expected to make the changes described in the tutorial (or experiment with others) and run the fingerprint tests to see how the model changes affect the fingerprints.

Note

Before trying the example in a step, reset the model changes of the previous step.

4.2.3 Running the tests

The fingerprints can be run from the command line. Make sure to run `. setenv` in both the `omnetpp` and `inet` folders:

```
$ cd ~/workspace/omnetpp
$ . setenv
$ cd ~/workspace/inet
$ . setenv
$ cd ~/workspace/inet/tutorials/fingerprint
$ inet_fingerprinttest
```

The `inet_fingerprinttest` runs all simulations specified in all `.csv` files in the current folder:

```
$ inet_fingerprinttest
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c EthernetShortPacket -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
. -f omnetpp.ini -c WifiShortPacket -r 0 ... : PASS
```

Ran 4 tests in 12.451s

OK

Log has been saved to test.out
Test results equals to expected results

Each simulation in `baseline.csv` has tags specified indicating which step it is relevant for, so that only the relevant simulations can be run for each step. The tags can be filtered for with the `-m` command line arguments. For example, running the simulations for the *Changing a Timer* step:

```
$ inet_fingerprinttest -m ChangingTimer
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
```

```
-----
Ran 1 test in 8.013s
```

```
OK
```

```
Log has been saved to test.out
Test results equals to expected results
```

The tags to use is indicated at each step.

4.2.4 Baseline ingredients

The baseline fingerprint ingredients are chosen because they are sensitive to a broad range of changes in the model (the default ingredients in fingerprint tests in INET is often `tplx`). If the ingredients are too sensitive, even trivial changes might lead to failed tests; if the ingredients are not sensitive enough, they might fail to detect regressions.

4.2.5 The workflow

The tutorial describes the workflow of making typical changes and verifying the model. Generally, the workflow involves a change in the model, which we suspect will not invalidate it. We also know that the baseline fingerprints are sensitive to this change; we choose other ingredients and create alternative fingerprints which are not sensitive to the particular change, but sensitive to others. Then, after making the model change, the fingerprint tests should pass, and the model should be verified. When the model is verified, the baseline ingredients can be restored.

Note

Fingerprint testing depends on repeatable deterministic simulations, random number generation, etc. If the tests are not repeatable, i.e. they don't result in the same fingerprints when run multiple times (e.g. emulation), fingerprint tests are useless.

4.3 Easy to Handle Changes

Some changes in the model don't change fingerprints at all, and are unlikely to cause regressions. These changes include renaming C++ classes, functions and variables, and extracting methods or classes, refactoring algorithms.

For example, we extracted a part of the `handleUpperCommand()` function in the `Udp` module to a new function:

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp.cc.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp.cc.extract
@@ -66,8 +66,8 @@
    OperationalBase::initialize(stage);

    if (stage == INITSTAGE_LOCAL) {
-       const char *crcModeString = par("crcMode");
-       crcMode = parseCrcMode(crcModeString, true);
+       const char *checksumModeString = par("checksumMode");
+       checksumMode = parseChecksumMode(checksumModeString, true);

        lastEphemeralPort = EPHEMERAL_PORTRANGE_START;
        ift = getModuleFromPar<IInterfaceTable>(par("interfaceTableModule"), this);
```

(continues on next page)

(continued from previous page)

```

@@ -88,7 +88,7 @@
    WATCH_MAP(socketsByPortMap);
}
else if (stage == INITSTAGE_TRANSPORT_LAYER) {
-   if (crcMode == CRC_COMPUTED) {
+   if (checksumMode == CHECKSUM_COMPUTED) {
        //TODO:
        // Unlike IPv4, when UDP packets are originated by an IPv6 node,
        // the UDP checksum is not optional. That is, whenever
@@ -101,12 +101,12 @@
#ifdef WITH_IPv4
        auto ipv4 = dynamic_cast<INetfilter *>(getModuleByPath("^ipv4.ip"));
        if (ipv4 != nullptr)
-       ipv4->registerHook(0, &crcInsertion);
+       ipv4->registerHook(0, &checksumInsertion);
#endif
#ifdef WITH_IPv6
        auto ipv6 = dynamic_cast<INetfilter *>(getModuleByPath("^ipv6.ipv6"));
        if (ipv6 != nullptr)
-       ipv6->registerHook(0, &crcInsertion);
+       ipv6->registerHook(0, &checksumInsertion);
#endif
    }
    registerService(Protocol::udp, gate("appIn"), gate("ipIn"));
@@ -130,6 +130,12 @@
    }
    else
        throw cRuntimeError("Unknown protocol: %s(%d)", protocol->getName(),
-        protocol->getId());
+        protocol->getId());
+}
+
+void Udp::udpConnect(cMessage *msg) {
+    int socketId = check_and_cast<Request*>(msg)->getTag<SocketReq>()->getSocketId();
+    UdpConnectCommand *ctrl = check_and_cast<UdpConnectCommand*>(msg->
+        getControlInfo());
+    connect(socketId, msg->getArrivalGate()->getIndex(), ctrl->getRemoteAddr(), ctrl->
+        getRemotePort());
+}

void Udp::handleUpperCommand(cMessage *msg)
@@ -143,9 +149,7 @@
    }

    case UDP_C_CONNECT: {
-        int socketId = check_and_cast<Request *>(msg)->getTag<SocketReq>()->
-        getSocketId();
-        UdpConnectCommand *ctrl = check_and_cast<UdpConnectCommand *>(msg->
-        getControlInfo());
-        connect(socketId, msg->getArrivalGate()->getIndex(), ctrl->
-        getRemoteAddr(), ctrl->getRemotePort());
+        udpConnect(msg);
+        break;
    }
}

@@ -783,13 +787,13 @@
    throw cRuntimeError("send: total UDP message size exceeds %u", UDP_MAX_

```

(continues on next page)

(continued from previous page)

```

    MESSAGE_SIZE);

    udpHeader->setTotalLengthField(totalLength);
-   if (crcMode == CRC_COMPUTED) {
-       udpHeader->setCrcMode(CRC_COMPUTED);
-       udpHeader->setCrc(0x0000);    // crcMode == CRC_COMPUTED is done in an
    INetfilter hook
+   if (checksumMode == CHECKSUM_COMPUTED) {
+       udpHeader->setChecksumMode(CHECKSUM_COMPUTED);
+       udpHeader->setChecksum(0x0000);    // checksumMode == CHECKSUM_COMPUTED is
    done in an INetfilter hook
    }
    else {
-       udpHeader->setCrcMode(crcMode);
-       insertCrc(l3Protocol, srcAddr, destAddr, udpHeader, packet);
+       udpHeader->setChecksumMode(checksumMode);
+       insertChecksum(l3Protocol, srcAddr, destAddr, udpHeader, packet);
    }

    insertTransportProtocolHeader(packet, Protocol::udp, udpHeader);
@@ -803,39 +807,39 @@
    numSent++;
}

-void Udp::insertCrc(const Protocol *networkProtocol, const L3Address& srcAddress,
    const L3Address& destAddress, const Ptr<UdpHeader>& udpHeader, Packet *packet)
-{
-    CrcMode crcMode = udpHeader->getCrcMode();
-    switch (crcMode) {
-        case CRC_DISABLED:
-            // if the CRC mode is disabled, then the CRC is 0
-            udpHeader->setCrc(0x0000);
+void Udp::insertChecksum(const Protocol *networkProtocol, const L3Address&
    srcAddress, const L3Address& destAddress, const Ptr<UdpHeader>& udpHeader, Packet
    *packet)
+{
+    ChecksumMode checksumMode = udpHeader->getChecksumMode();
+    switch (checksumMode) {
+        case CHECKSUM_DISABLED:
+            // if the checksum mode is disabled, then the checksum is 0
+            udpHeader->setChecksum(0x0000);
            break;
-        case CRC_DECLARED_CORRECT:
-            // if the CRC mode is declared to be correct, then set the CRC to an
    easily recognizable value
-            udpHeader->setCrc(0xC00D);
+        case CHECKSUM_DECLARED_CORRECT:
+            // if the checksum mode is declared to be correct, then set the checksum
    to an easily recognizable value
+            udpHeader->setChecksum(0xC00D);
            break;
-        case CRC_DECLARED_INCORRECT:
-            // if the CRC mode is declared to be incorrect, then set the CRC to an
    easily recognizable value
-            udpHeader->setCrc(0xBAAD);
+        case CHECKSUM_DECLARED_INCORRECT:

```

(continues on next page)

(continued from previous page)

```

+         // if the checksum mode is declared to be incorrect, then set the
+         ↪checksum to an easily recognizable value
+         udpHeader->setChecksum(0xBAAD);
+         break;
-         case CRC_COMPUTED: {
-         // if the CRC mode is computed, then compute the CRC and set it
+         case CHECKSUM_COMPUTED: {
+         // if the checksum mode is computed, then compute the checksum and set it
+         // this computation is delayed after the routing decision, see
+         ↪INetfilter hook
-         udpHeader->setCrc(0x0000); // make sure that the CRC is 0 in the Udp
+         ↪header before computing the CRC
-         udpHeader->setCrcMode(CRC_DISABLED); // for serializer/deserializer
+         ↪checks only: deserializer sets the crcMode to disabled when crc is 0
+         udpHeader->setChecksum(0x0000); // make sure that the checksum is 0 in
+         ↪the Udp header before computing the checksum
+         udpHeader->setChecksumMode(CHECKSUM_DISABLED); // for serializer/
+         ↪deserializer checks only: deserializer sets the checksumMode to disabled when
+         ↪checksum is 0
+         auto udpData = packet->peekData(Chunk::PF_ALLOW_EMPTY);
-         auto crc = computeCrc(networkProtocol, srcAddress, destAddress,
+         ↪udpHeader, udpData);
-         udpHeader->setCrc(crc);
-         udpHeader->setCrcMode(CRC_COMPUTED);
+         auto checksum = computeChecksum(networkProtocol, srcAddress, destAddress,
+         ↪ udpHeader, udpData);
+         udpHeader->setChecksum(checksum);
+         udpHeader->setChecksumMode(CHECKSUM_COMPUTED);
+         break;
+     }
-     default:
-         throw cRuntimeError("Unknown CRC mode: %d", (int)crcMode);
-     }
- }
- }
-
- uint16_t Udp::computeCrc(const Protocol *networkProtocol, const L3Address&
+ ↪srcAddress, const L3Address& destAddress, const Ptr<const UdpHeader>& udpHeader,
+ ↪const Ptr<const Chunk>& udpData)
+     throw cRuntimeError("Unknown checksum mode: %d", (int)checksumMode);
+ }
+ }
+
+ uint16_t Udp::computeChecksum(const Protocol *networkProtocol, const L3Address&
+ ↪srcAddress, const L3Address& destAddress, const Ptr<const UdpHeader>& udpHeader,
+ ↪const Ptr<const Chunk>& udpData)
+ {
+     auto pseudoHeader = makeShared<TransportPseudoHeader>();
+     pseudoHeader->setSrcAddress(srcAddress);
@@ -855,14 +859,14 @@
+     Chunk::serialize(stream, pseudoHeader);
+     Chunk::serialize(stream, udpHeader);
+     Chunk::serialize(stream, udpData);
-     uint16_t crc = internetChecksum(stream.getData());
+     uint16_t checksum = internetChecksum(stream.getData());

    // Excerpt from RFC 768:

```

(continues on next page)

(continued from previous page)

```

// If the computed checksum is zero, it is transmitted as all ones (the
// equivalent in one's complement arithmetic). An all zero transmitted
// checksum value means that the transmitter generated no checksum (for
// debugging or for higher level protocols that don't care).
- return crc == 0x0000 ? 0xFFFF : crc;
+ return checksum == 0x0000 ? 0xFFFF : checksum;
}

void Udp::close(int sockId)
@@ -933,7 +937,7 @@
    auto hasIncorrectLength = totalLength < udpHeader->getChunkLength() ||
    ↪totalLength > udpHeader->getChunkLength() + udpPacket->getDataLength();
    auto networkProtocol = udpPacket->getTag<NetworkProtocolInd>()->getProtocol();

- if (hasIncorrectLength || !verifyCrc(networkProtocol, udpHeader, udpPacket)) {
+ if (hasIncorrectLength || !verifyChecksum(networkProtocol, udpHeader,
    ↪udpPacket)) {
    EV_WARN << "Packet has bit error, discarding\n";
    PacketDropDetails details;
    details.setReason(INCORRECTLY_RECEIVED);
@@ -981,39 +985,39 @@
    }
}

-bool Udp::verifyCrc(const Protocol *networkProtocol, const Ptr<const UdpHeader>&
    ↪udpHeader, Packet *packet)
-{
- switch (udpHeader->getCrcMode()) {
- case CRC_DISABLED:
-     // if the CRC mode is disabled, then the check passes if the CRC is 0
-     return udpHeader->getCrc() == 0x0000;
- case CRC_DECLARED_CORRECT: {
-     // if the CRC mode is declared to be correct, then the check passes if
    ↪and only if the chunks are correct
+bool Udp::verifyChecksum(const Protocol *networkProtocol, const Ptr<const UdpHeader>&
    ↪udpHeader, Packet *packet)
+{
+ switch (udpHeader->getChecksumMode()) {
+ case CHECKSUM_DISABLED:
+     // if the checksum mode is disabled, then the check passes if the
    ↪checksum is 0
+     return udpHeader->getChecksum() == 0x0000;
+ case CHECKSUM_DECLARED_CORRECT: {
+     // if the checksum mode is declared to be correct, then the check passes
    ↪if and only if the chunks are correct
        auto totalLength = udpHeader->getTotalLengthField();
        auto udpDataBytes = packet->peekDataAt(B(0), totalLength - udpHeader->
    ↪getChunkLength(), Chunk::PF_ALLOW_INCORRECT);
        return udpHeader->isCorrect() && udpDataBytes->isCorrect();
    }
- case CRC_DECLARED_INCORRECT:
-     // if the CRC mode is declared to be incorrect, then the check fails
+ case CHECKSUM_DECLARED_INCORRECT:
+     // if the checksum mode is declared to be incorrect, then the check fails
    return false;
- case CRC_COMPUTED: {

```

(continues on next page)

(continued from previous page)

```

-         if (udpHeader->getCrc() == 0x0000)
-             // if the CRC mode is computed and the CRC is 0 (disabled), then the
- check passes
+         case CHECKSUM_COMPUTED: {
+             if (udpHeader->getChecksum() == 0x0000)
+                 // if the checksum mode is computed and the checksum is 0 (disabled),
- then the check passes
+                 return true;
+             else {
-                 // otherwise compute the CRC, the check passes if the result is
- 0xFFFF (includes the received CRC) and the chunks are correct
+                 // otherwise compute the checksum, the check passes if the result is
- 0xFFFF (includes the received checksum) and the chunks are correct
+                 auto l3AddressInd = packet->getTag<L3AddressInd>();
+                 auto srcAddress = l3AddressInd->getSrcAddress();
+                 auto destAddress = l3AddressInd->getDestAddress();
+                 auto totalLength = udpHeader->getTotalLengthField();
+                 auto udpData = packet->peekDataAt<BytesChunk>(B(0), totalLength -
- udpHeader->getChunkLength(), Chunk::PF_ALLOW_INCORRECT);
-                 auto computedCrc = computeCrc(networkProtocol, srcAddress,
- destAddress, udpHeader, udpData);
-                 // TODO: delete these isCorrect calls, rely on CRC only
-                 return computedCrc == 0xFFFF && udpHeader->isCorrect() && udpData->
- isCorrect();
+                 auto computedChecksum = computeChecksum(networkProtocol, srcAddress,
- destAddress, udpHeader, udpData);
+                 // TODO: delete these isCorrect calls, rely on checksum only
+                 return computedChecksum == 0xFFFF && udpHeader->isCorrect() &&
- udpData->isCorrect();
+             }
+         }
+     default:
-         throw cRuntimeError("Unknown CRC mode");
+         throw cRuntimeError("Unknown checksum mode");
+     }
+ }

@@ -1154,8 +1158,8 @@
    if (!icmp)
        // TODO: move to initialize?
        icmp = getModuleFromPar<Icmp>(par("icmpModule"), this);
-     if (!icmp->verifyCrc(packet)) {
-         EV_WARN << "incoming ICMP packet has wrong CRC, dropped\n";
+     if (!icmp->verifyChecksum(packet)) {
+         EV_WARN << "incoming ICMP packet has wrong checksum, dropped\n";
        PacketDropDetails details;
        details.setReason(INCORRECTLY_RECEIVED);
        emit(packetDroppedSignal, packet, &details);
@@ -1209,8 +1213,8 @@
    if (!icmpv6)
        // TODO: move to initialize?
        icmpv6 = getModuleFromPar<Icmpv6>(par("icmpv6Module"), this);
-     if (!icmpv6->verifyCrc(packet)) {
-         EV_WARN << "incoming ICMPv6 packet has wrong CRC, dropped\n";
+     if (!icmpv6->verifyChecksum(packet)) {
+         EV_WARN << "incoming ICMPv6 packet has wrong checksum, dropped\n";

```

(continues on next page)

(continued from previous page)

```

        PacketDropDetails details;
        details.setReason(INCORRECTLY_RECEIVED);
        emit(packetDroppedSignal, packet, &details);
@@ -1347,9 +1351,9 @@
        auto l3AddressInd = packet->findTag<L3AddressInd>();
        auto networkProtocolInd = packet->findTag<NetworkProtocolInd>();
        if (l3AddressInd != nullptr && networkProtocolInd != nullptr)
-            return verifyCrc(networkProtocolInd->getProtocol(), udpHeader, packet);
+            return verifyChecksum(networkProtocolInd->getProtocol(), udpHeader,
-            packet);
        else
-            return udpHeader->getCrcMode() != CrcMode::CRC_DECLARED_INCORRECT;
+            return udpHeader->getChecksumMode() != ChecksumMode::CHECKSUM_DECLARED_
-            INCORRECT;
        }
    }

@@ -1451,7 +1455,7 @@
    // other methods
    // #####

-Inetfilter::IHook::Result Udp::CrcInsertion::datagramPostRoutingHook(Packet *packet)
+Inetfilter::IHook::Result Udp::ChecksumInsertion::datagramPostRoutingHook(Packet
-    *packet)
    {
        if (packet->findTag<InterfaceInd>())
            return ACCEPT; // FORWARD
@@ -1462,10 +1466,10 @@
        ASSERT(!networkHeader->isFragment());
        packet->eraseAtFront(networkHeader->getChunkLength());
        auto udpHeader = packet->removeAtFront<UdpHeader>();
-        ASSERT(udpHeader->getCrcMode() == CRC_COMPUTED);
+        ASSERT(udpHeader->getChecksumMode() == CHECKSUM_COMPUTED);
        const L3Address& srcAddress = networkHeader->getSourceAddress();
        const L3Address& destAddress = networkHeader->getDestinationAddress();
-        Udp::insertCrc(networkProtocol, srcAddress, destAddress, udpHeader, packet);
+        Udp::insertChecksum(networkProtocol, srcAddress, destAddress, udpHeader,
-        packet);
        packet->insertAtFront(udpHeader);
        packet->insertAtFront(networkHeader);
    }

```

The refactoring doesn't change the fingerprint because the code is functionally the same. It doesn't create any new events or data packets, and it doesn't change timing or anything that the fingerprint calculation takes into account:

```

$ inet_fingerprinttest -m EasyToHandleChanges
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS

```

```
-----
Ran 2 tests in 6.714s
```

```
OK
```

```
Log has been saved to test.out
Test results equals to expected results
```

4.4 Renaming a Submodule

Renaming submodules can cause the fingerprints to change, because the default ingredients (tplx) contain the full module path, thus the submodule name as well. Renaming submodules can cause regression in some cases, e.g. when functionality depends on submodule names (e.g. submodules referring to each other).

To go around the problem of fingerprints failing due to renaming, the fingerprints need to be calculated without the full path:

- Before making the change, rerun fingerprint tests without the full path ingredient
- Update fingerprints; the new values can be accepted because the model didn't change
- Perform renaming
- Run fingerprint tests again; the new fingerprints are not sensitive to changes in submodule names

Note

To run fingerprints with tlx, delete the p from the ingredient in the .csv file. When the fingerprint test is run again, the fingerprints will be calculated with the new ingredients. The tests will fail, as the values are still the ones calculated with tplx, but it is safe to overwrite them with the updated values, as the model didn't change.

Here is the workflow demonstrated using a simplistic example:

We set the fingerprint ingredients to tlx in the .csv file:

```
.,      -f omnetpp.ini -c Ethernet -r 0,      5s,      a92f-8bfe/tlx, PASS,
.,      -f omnetpp.ini -c Wifi -r 0,      5s,      5e6e-3064/tlx, PASS,
```

Then we run the fingerprint tests:

```
$ inet_fingerprinttest -m RenamingSubmodule
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED (should be PASS)
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED (should be PASS)
```

As expected, they fail because the values are still the ones calculated with tplx. We can update the .csv file with the new values:

```
$ mv baseline.csv.UPDATED baseline.csv
```

When we rerun the fingerprint tests, now they PASS:

```
$ inet_fingerprinttest -m RenamingSubmodule
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
```

As a simplistic example, we rename the eth module vector to ethernet in `LinkLayerNodeBase` and `NetworkLayerNodeBase`. This change affects all host-types such as `StandardHost` and `AdhocHost` since they are derived modules:

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/LinkLayerNodeBase.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/LinkLayerNodeBase.ned.modified
@@ -59,7 +59,7 @@
     parameters:
         @display("p=300,526,row,150;q=txQueue");
     }
-    eth[sizeof(ethg)]: <default("EthernetInterface")> like IEthernetInterface {
+    ethernet[sizeof(ethg)]: <default("EthernetInterface")> like_
->IEthernetInterface {
```

(continues on next page)

(continued from previous page)

```

        parameters:
            @display("p=900,526,row,150;q=txQueue");
    }
@@ -78,7 +78,7 @@
    }

    for i=0..sizeof(ethg)-1 {
-       ethg[i] <--> { @display("m=s"); } <--> eth[i].phys;
+       ethg[i] <--> { @display("m=s"); } <--> ethernet[i].phys;
    }

    for i=0..sizeof(pppg)-1 {

```

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/NetworkLayerNodeBase.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/NetworkLayerNodeBase.ned.
  ↪modified
@@ -61,8 +61,8 @@
    }

    for i=0..sizeof(ethg)-1 {
-       eth[i].upperLayerOut --> nl.in++;
-       eth[i].upperLayerIn <-- nl.out++;
+       ethernet[i].upperLayerOut --> nl.in++;
+       ethernet[i].upperLayerIn <-- nl.out++;
    }

    for i=0..sizeof(pppg)-1 {

```

We run the fingerprint tests again:

```

$ inet_fingerprinttest -m RenamingSubmodule
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS

```

As expected, the fingerprints didn't change, so we can assume the model is correct.

Now, we can change the ingredients back to `tplx`, rerun the tests, and accept the new values (described in more detail in the [Accepting Fingerprint Changes](#) step).

However/In other cases, renaming submodules can lead to ERROR in the fingerprint tests, e.g. when modules cross-reference each other and look for other modules by name. Also, renaming can lead to FAILED fingerprint tests, because there might be no check in the model on cross-referencing module names.

Note

Before trying the following example, reset the model changes of the previous example.

Here is an example change which results in ERROR in the tests when using `tlx` fingerprints:

In `Ipv4NetworkLayer`, we rename the `routingTable` submodule to `ipv4RoutingTable`:

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/Ipv4NetworkLayer.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/Ipv4NetworkLayer.ned.
  ↪routingtable.modified
@@ -43,7 +43,7 @@
    parameters:
        @display("p=100,100;is=s");

```

(continues on next page)

(continued from previous page)

```

    }
-   routingTable: Ipv4RoutingTable {
+   ipv4RoutingTable: Ipv4RoutingTable {
        parameters:
            @display("p=100,200;is=s");
    }

```

Here are the results of the fingerprint tests:

```

$ inet_fingerprinttest -m RenamingSubmodule
. -f omnetpp.ini -c Wifi -r 0 ... : ERROR (should be PASS)
. -f omnetpp.ini -c Ethernet -r 0 ... : ERROR (should be PASS)

```

The simulations finished with an error, because the `Ipv4NetworkConfigurator` module was looking for the routing table module by the original name, and couldn't find it.

Note that we omitted the error messages from the above output to make it more concise. The simulations give the following error message:

```

Error: Module not found on path 'RegressionTestingTutorialWireless.host1.ipv4.
↳routingTable'
defined by par 'RegressionTestingTutorialWireless.host1.ipv4.configurator.
↳routingTableModule'
-- in module (inet::Ipv4NodeConfigurator) RegressionTestingTutorialWireless.host1.
↳ipv4.configurator
(id=83), during network initialization

```

4.5 Renaming a Module Parameter

The renaming of NED parameters can also cause regression; the parameter might be used by derived modules; a parameter setting in a derived module might not have the effect it had before; forgetting to update ini keys can also cause problems.

The following is a simplistic example for module parameter renaming causing a regression. The `Router` module sets the `forwarding` parameter to `true` which it inherits from `NetworkLayerNodeBase`. The latter uses the parameter to enable forwarding in its various submodules, such as `Ipv4` and `Ipv6`:

```

module Router extends ApplicationLayerNodeBase
{
    parameters:
        forwarding = true;
}

```

```

module NetworkLayerNodeBase extends LinkLayerNodeBase
{
    parameters:
        bool forwarding = default(false);
        bool multicastForwarding = default(false);
        *.forwarding = forwarding;
        *.multicastForwarding = multicastForwarding;
}

```

In `Ipv4NetworkLayer`, we rename the `forwarding` parameter to `unicastForwarding` to make it similar to `multicastForwarding`:

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/Ipv4NetworkLayer.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/Ipv4NetworkLayer.ned.
↳forwarding.modified

```

(continues on next page)

(continued from previous page)

```

@@ -19,11 +19,11 @@
module Ipv4NetworkLayer like INetworkLayer
{
    parameters:
-    bool forwarding = default(false);
+    bool unicastForwarding = default(false);
    bool multicastForwarding = default(false);
    string interfaceTableModule;
    string displayStringTextFormat = default("%i");
-    *.forwarding = forwarding;
+    *.forwarding = unicastForwarding;
    *.multicastForwarding = multicastForwarding;
    *.interfaceTableModule = default(absPath(interfaceTableModule));
    *.routingTableModule = default(absPath(".routingTable"));

```

The `Ipv4NetworkLayer` module sets the forwarding parameters in its `Ipv4` submodule to the value of `unicastForwarding`. Now, the `forwarding = true` key in `Router` doesn't take effect in the router's submodules, and the router doesn't forward packets. This change causes the fingerprint tests/tplx fingerprint tests to fail:

```

$ inet_fingerprinttest -m RenamingParameter
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED

```

To correct the model, the renaming needs to be followed everywhere. When we change the deep parameter assignment of the forwarding parameter in `NetworkLayerNodeBase` to `unicastForwarding`, the fingerprint tests pass:

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/NetworkLayerNodeBase.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/NetworkLayerNodeBase.ned.
↪ forwarding
@@ -17,7 +17,7 @@
    bool hasGn = default(false);
    bool forwarding = default(false);
    bool multicastForwarding = default(false);
-    *.forwarding = forwarding;
+    *.unicastForwarding = forwarding;
    *.multicastForwarding = multicastForwarding;
    @figure[networkLayer](type=rectangle; pos=250,306; size=1000,130; fillColor=
↪ #00ff00; lineColor=#808080; cornerRadius=5; fillOpacity=0.1);
    @figure[networkLayer.title](type=text; pos=1245,311; anchor=ne; text=
↪ "network layer");

```

```

$ inet_fingerprinttest -m RenamingParameter
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS

```

This time we don't have to accept the fingerprint changes, because they didn't change.

Note

Before trying the following example, reset the model changes of the previous example.

Renaming module parameters can also lead to ERROR in the fingerprint tests. For example, we rename the queue submodule to `txQueue` in `EtherMacFullDuplex.ned`:

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/EtherMacFullDuplex.ned.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/EtherMacFullDuplex.ned.mod

```

(continues on next page)

(continued from previous page)

```

@@ -149,7 +149,7 @@
    output upperLayerOut @labels(EtherFrame); // to ~EtherEncap or ~
↪ IMacRelayUnit
    inout phys @labels(EtherFrame);           // to physical layer or the_
↪ network
    submodules:
-     queue: <default("EtherQueue")> like IPacketQueue {
+     txQueue: <default("EtherQueue")> like IPacketQueue {
        parameters:
            packetCapacity = default(1000);
            @display("p=100,100");

```

When run the `tplx` fingerprint tests, they result in ERROR:

```

$ inet_fingerprinttest -m RenamingParameter
. -f omnetpp.ini -c Ethernet -r 0 ... : ERROR

```

This error message was omitted from the above output for simplicity:

```

Error: check_and_cast(): Cannot cast nullptr to type 'inet::queueing::IPacketQueue *'
-- in module (inet::EtherMacFullDuplex) RegressionTestingTutorialWired.router1.eth[0].
↪ mac (id=58),
during network initialization

```

4.6 Changing Packet Length

Some changes in the model, e.g. adding fields to a protocol header, can cause the packet lengths to change. This in turn leads to changes in the `tplx` fingerprints. The `l` stands for packet length, and the length of packets anywhere in the network (including host submodules) is taken into account at every event. If we drop the packet length from the fingerprint ingredients, the fingerprints might still change due to the different timings of the packets.

However, small packets are padded to fit into a minimum-size Ethernet frame of 64 bytes. In this case, the changed protocol header size wouldn't affect `tpx` fingerprints, as the Ethernet frame sizes, and thus the timings, are the same. A similar padding effect happens in 802.11 frames. The data contained in the frame is a multiple of the bits/symbol times the number of subcarriers for the given modulation. For QAM-64, this is around 30 bytes. Thus we expect that for small packets the fingerprints wouldn't change; for larger ones, they would (assuming small changes in the header size).

In the following example, we'll increase the `Udp` header size from 8 bytes to 10 bytes.

Before making the change, we drop the packet length from the fingerprints and run the tests:

```

.,      -f omnetpp.ini -c Ethernet -r 0,      0.2s,      4500-0673/tpx,↪
↪ PASS,
.,      -f omnetpp.ini -c EthernetShortPacket -r 0,      0.2s,      ea97-154f/tpx,↪
↪ PASS,
.,      -f omnetpp.ini -c Wifi -r 0,      5s,      791d-aba6/tpx,↪
↪ PASS,
.,      -f omnetpp.ini -c WifiShortPacket -r 0,      5s,      d801-fc01/tpx,↪
↪ PASS,

```

```

$ inet_fingerprinttest -m ChangingPacketLength
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED
. -f omnetpp.ini -c WifiShortPacket -r 0 ... : FAILED
. -f omnetpp.ini -c EthernetShortPacket -r 0 ... : FAILED

```

The tests failed because the values in the .csv file were calculated with the default ingredients. We can update the .csv with the new fingerprints:

```
$ mv baseline.csv.UPDATED baseline.csv
```

Then we increase the Udp header size:

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/UdpHeader.msg.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/UdpHeader.msg.mod
@@ -7,14 +7,14 @@

import inet.common.INETDefs;
-import inet.transportlayer.common.CrcMode;
+import inet.common.checksum.ChecksumMode;
import inet.transportlayer.contract.TransportHeaderBase;

namespace inet;

cplusplus {{
-const B UDP_HEADER_LENGTH = B(8);
+const B UDP_HEADER_LENGTH = B(10);
}}

//
@@ -26,8 +26,8 @@
    unsigned short destPort;
    chunkLength = UDP_HEADER_LENGTH;
    B totalLengthField = B(-1);    // UDP header + payload in bytes
-    uint16_t crc = 0;
-    CrcMode crcMode = CRC_MODE_UNDEFINED;
+    uint16_t checksum = 0;
+    ChecksumMode checksumMode = CHECKSUM_MODE_UNDEFINED;
}

cplusplus(UdpHeader) {{
```

The change needs to be followed in UdpHeaderSerializer.cc as well: (otherwise the packets couldn't be serialized, leading to dropped packets)

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/UdpHeaderSerializer.cc.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/UdpHeaderSerializer.cc.modified
@@ -16,10 +16,11 @@
    stream.writeUint16Be(udpHeader->getSourcePort());
    stream.writeUint16Be(udpHeader->getDestinationPort());
    stream.writeUint16Be(B(udpHeader->getTotalLengthField()).get());
-    auto crcMode = udpHeader->getCrcMode();
-    if (crcMode != CRC_DISABLED && crcMode != CRC_COMPUTED)
-        throw cRuntimeError("Cannot serialize UDP header without turned off or
+properly computed CRC, try changing the value of crcMode parameter for Udp");
-    stream.writeUint16Be(udpHeader->getCrc());
+    auto checksumMode = udpHeader->getChecksumMode();
+    if (checksumMode != CHECKSUM_DISABLED && checksumMode != CHECKSUM_COMPUTED)
+        throw cRuntimeError("Cannot serialize UDP header without turned off or
+properly computed checksum, try changing the value of checksumMode parameter for Udp
+");
+    stream.writeUint16Be(udpHeader->getChecksum());
```

(continues on next page)

(continued from previous page)

```

+   stream.writeUInt16Be(0);
}

const Ptr<Chunk> UdpHeaderSerializer::deserialize(MemoryInputStream& stream) const
@@ -28,9 +29,10 @@
    udpHeader->setSourcePort(stream.readUInt16Be());
    udpHeader->setDestinationPort(stream.readUInt16Be());
    udpHeader->setTotalLengthField(B(stream.readUInt16Be()));
-   auto crc = stream.readUInt16Be();
-   udpHeader->setCrc(crc);
-   udpHeader->setCrcMode(crc == 0 ? CRC_DISABLED : CRC_COMPUTED);
+   auto checksum = stream.readUInt16Be();
+   udpHeader->setChecksum(checksum);
+   udpHeader->setChecksumMode(checksum == 0 ? CHECKSUM_DISABLED : CHECKSUM_
  ↪COMPUTED);
+   stream.readUInt16Be();
    return udpHeader;
}

```

We run the fingerprint tests again:

```

$ inet_fingerprinttest
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED
. -f omnetpp.ini -c WifiShortPacket -r 0 ... : PASS
. -f omnetpp.ini -c EthernetShortPacket -r 0 ... : PASS

```

As expected, the tests pass when the packet size is small.

4.7 Changing a Timer

Some changes in a model's timers might break fingerprints, but otherwise wouldn't invalidate the model, such as timers which don't affect the model's behavior. However, the change in the timers might break the default ingredient fingerprint tests. In such case, the events or modules that contain the timer need to be filtered:

- Before making the change, filter the events or modules which involve the affected timer, and calculate the fingerprints
- Accept the new fingerprint values
- Make the change
- Rerun the fingerprint tests with the same filtering

The tests should pass.

As a simplistic example, we'll change how the `removeNonInterferingTransmissionsTimer` is scheduled in `RadioMedium.cc`. The timer deletes transmissions that no longer cause any interference (e.g. they have already left the vicinity of network nodes).

We run the fingerprint tests and filter the events in the `RadioMedium` module:

```

$ inet_fingerprinttest -m ChangingTimer -a --fingerprint-events='"not name=~
  ↪removeNonInterferingTransmissions"'
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED

```

The tests fail because the set of events used to calculate them changed. Since there was no change in the model, just in how the fingerprints are calculated, the `.csv` file can be updated with the new values:

```
$ mv baseline.csv.UPDATED baseline.csv
```

Now, we change the timer:

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/RadioMedium.cc.orig
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/RadioMedium.cc.mod
@@ -233,7 +233,7 @@
     communicationCache->mapTransmissions([&] (const ITransmission *transmission) {
         auto interferenceEndTime = communicationCache->
->getCacheInterferenceEndTime(transmission);
         if (!removeNonInterferingTransmissionsTimer->isScheduled() &&
->interferenceEndTime > simTime())
-            scheduleAt(interferenceEndTime, removeNonInterferingTransmissionsTimer);
+            scheduleAt(interferenceEndTime + 1E-6,
->removeNonInterferingTransmissionsTimer);
     });
 }
```

We re-run the fingerprint tests:

```
$ inet_fingerprinttest -m ChangingTimer -a --fingerprint-events='"not name=~
->removeNonInterferingTransmissions"'
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
```

The model is verified. The fingerprints can be calculated without filtering, and the .csv file can be updated with the new values. This process is described in more detail in the *Accepting Fingerprint Changes* step.

4.8 Adding New Events - Part 1

Some changes in the model can add new events to the simulation. These events inevitably change the default fingerprints, but they don't necessarily invalidate the model. One option is to filter out the modules in which the new events take place, so they are not included in the fingerprint calculation. Taking into account the rest of the modules might show that the simulation trajectory, in fact, stayed the same.

To filter out modules, run the fingerprint test with the `--fingerprint-modules` command-line option, e.g.:

```
$ inet_fingerprinttest -a --fingerprint-modules='"not fullPath=~**.wlan[*].mac.**"'
```

As a simplistic example, we will make changes to the Udp module's code; the changes add new events but don't alter the module's behavior. However, the fingerprints will change; we will filter out the Udp module from the fingerprint calculation to verify the model. Here is the workflow:

We run the fingerprint tests without taking the Udp module into account (the order of the quotes is important):

```
$ inet_fingerprinttest -m AddingNewEvents1 -a --fingerprint-modules='"not fullPath=~
->**.Udp"'
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED
```

We can accept the fingerprints because there was no change in the model

```
$ mv baseline.csv.UPDATED baseline.csv
```

Then we add a line to one of the functions in Udp.cc which schedules a self-message; we also add a `handleSelfMessage()` function which just deletes the message when it's received:

```

--- /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp_orig.cc
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp_mod.cc
@@ -114,8 +114,16 @@
    }
}

+void Udp::handleSelfMessage(cMessage *message)
+{
+    if (message->getKind() == 42) {
+        delete message;
+    }
+}
+
void Udp::handleLowerPacket(Packet *packet)
{
+    scheduleAt(simTime() + 1E-6, new cMessage(nullptr, 42));
    // received from IP layer
    ASSERT(packet->getControlInfo() == nullptr);
    auto protocol = packet->getTag<PacketProtocolTag>()->getProtocol();
@@ -1496,4 +1504,3 @@
}

} // namespace inet
-

```

We run the fingerprint tests again without the Udp module:

```

$ inet_fingerprinttest -m AddingNewEvents1 -a --fingerprint-modules="not fullPath=~
->**.Udp"
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS

```

The fingerprint tests PASS; the model is verified.

Note

The `-fingerprint-module=""` command-line argument can be added to the .csv file as well.

We can change the ingredients back to `tplx` (or some other default), re-run the tests, and accept the new values. This process is described in more detail in the [Accepting Fingerprint Changes](#) step.

Another option for dealing with the effect of new events is to filter out only the new event from the fingerprint calculation. This approach is beneficial because alternate fingerprints with different ingredients are not needed. If only the new event is filtered out, the fingerprints should stay the same.

As a simplistic example, we'll make the same change to the Udp module as before. We'll run the tests with the baseline ingredients and filter out the new self-message by its id:

```

$ inet_fingerprinttest -m AddingNewEvents1 -a --fingerprint-events="not kind=~42"
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS

```

The tests PASS, and the model is verified.

Note

As a rule of thumb, it is best to filter out specifically those aspects of the simulation which were affected by

the change. The more specific the filtering, the better; if you filter specifically, the fingerprints will retain their sensitivity to a broad range of changes, but won't be sensitive to that specific change.

4.9 Adding New Events - Part 2

When a change introduces new events to the simulation and breaks fingerprints, one option is to use an alternative fingerprint calculator instead of the default one. INET's fingerprint calculator (`inet::FingerprintCalculator`) extends the default calculator and adds new ingredients that can be used alongside the default ones. This calculator can be used to take into account only the communication between the network nodes when calculating fingerprints. Thus, protocol implementation details and events inside network nodes don't affect the fingerprints, only the data of the packets sent between network nodes.

The fingerprint calculator has four new ingredients available:

- N: Network node path in the module hierarchy
- I: Network interface path in the module hierarchy; the superset of N
- D: Packet data
- ~: Filter events to network communication

Note

Both d and D are packet data ingredients; d includes the C++ packet data representation and meta-infos such as annotations, tags, and flags; D includes only the packet data in network byte order, as seen on the network.

To use the new ingredients, add the following line to the `General` configuration:

[General]

```
fingerprintcalculator-class = "inet::FingerprintCalculator"
```

Now, the new and the default ingredients can be mixed.

The ~ ingredient toggles filtering of events to those that are messages between two different network nodes, effectively limiting the set of events taking part in fingerprint calculation to network communication. Note that this behavior affects the default ingredients as well.

Note

If the ingredients contain only D (and not t), the fingerprints are defined/affected only by the order of the packets (and the data they contain), the timings are irrelevant.

To filter out the effects of newly added events, run fingerprints with ingredients that only take into account network communication (e.g. ~tID).

Note

When deciding which fingerprint ingredients to use, a general rule of thumb is that you should use the most sensitive fingerprint that you think will not change because of the updated model, i.e. it is not sensitive to the model change but sensitive to everything else. In this step, we chose ~tID: this only takes into account messages between different network nodes; it uses the time, the interface full path, and the data in network byte order to calculate the fingerprints.

Here is the workflow:

- Before making the change in the model, run the fingerprint with ~tID ingredients only

- Make the changes in the model
- Run the fingerprint tests again

If fingerprint tests pass, there was no change in the communication between network nodes, and the model can be assumed to be correct with respect to the data of the exchanged packets staying the same.

As a simplistic example, we will make the same change to the [Udp](#) module as in the previous step. We will use only network communication ingredients to calculate fingerprints and verify the model.

We replace the default ingredients with `~tID` in the `.csv` file:

```
.,      -f omnetpp.ini -c Ethernet -r 0,      5s,      aeb0-6fd3/~tID, PASS,
.,      -f omnetpp.ini -c Wifi -r 0,      5s,      c1f4-8059/~tID, PASS,
```

Then we run the fingerprint tests:

```
$ inet_fingerprinttest -m AddingNewEvents2
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED
```

As there was no change in the model, the new fingerprints can be accepted:

```
$ mv baseline.csv.UPDATED baseline.csv
```

We make the change:

```
--- /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp_orig.cc
+++ /home/rhornig/w/inet/tutorials/fingerprint/sources/Udp_mod.cc
@@ -114,8 +114,16 @@
     }
 }

+void Udp::handleSelfMessage(cMessage *message)
+{
+    if (message->getKind() == 42) {
+        delete message;
+    }
+}
+
void Udp::handleLowerPacket(Packet *packet)
{
+    scheduleAt(simTime() + 1E-6, new cMessage(nullptr, 42));
    // received from IP layer
    ASSERT(packet->getControlInfo() == nullptr);
    auto protocol = packet->getTag<PacketProtocolTag>()->getProtocol();
@@ -1496,4 +1504,3 @@
 }

 } // namespace inet
-
```

We run the fingerprint tests again:

```
$ inet_fingerprinttest -m AddingNewEvents2
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
```

After making the change, the fingerprint tests pass, thus the model can be assumed correct.

It might turn out that the selected fingerprint ingredients are too sensitive. For example, the `~tID` ingredients take into account the interfaces between which the packets pass. If due to the model change, a packet uses another

interface of a host with multiple interfaces, but the data and the source and destination hosts stay the same, the model might be valid, but the fingerprint tests don't pass. Using `~tND` instead would not be sensitive to the interfaces, and the tests would pass.

When the model is verified, we can change the ingredients back to `tplx` (or some other default), re-run the tests, and accept the new values. This process is described in more detail in the [Accepting Fingerprint Changes](#) step.

4.10 Removing Events

Some changes in the model remove events from the simulation, resulting in changed fingerprints, but don't necessarily invalidate the model. The methods described in the previous steps involving new events are applicable in this case as well. The workflow is the following:

- Create fingerprints which are not sensitive to the removal of the event/doesn't contain the event (such as filter the module the event takes place in, use ingredients which only take into account network communication, or filter only the event itself)
- Make the change in the model
- Run the fingerprint tests again

As a simplistic example, we'll delete the `removeNonInterferingTransmissions` self message from the `RadioMedium` module. This message schedules a timer to remove those transmissions from the radio medium which can no longer cause any interference. This change doesn't alter the simulation, but deletes events from it, thus it would change the baseline fingerprints.

First, we create alternate fingerprints by filtering out the `removeNonInterferingTransmissions` events:

```
$ inet_fingerprinttest -m RemovingEvent -a --fingerprint-events="not fullName=~
→removeNonInterferingTransmissions"
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED
```

Now, we can update the `.csv` file with the new fingerprints:

```
$ mv baseline.csv.UPDATED baseline.csv
```

We remove the `nonInterferingTransmissions` message from the code of `RadioMedium.cc`; we delete the declaration, and all instances of its scheduling. We build INET and run the fingerprint tests again with the filter expression:

```
$ inet_fingerprinttest -m RemoveEvent -a --fingerprint-events="not fullName=~
→removeNonInterferingTransmissions"
. -f omnetpp.ini -c Wifi -r 0 ... : PASS
```

We can restore the baseline fingerprints by re-running the tests without filtering and accept the new values. This process is described in more detail in the [Accepting Fingerprint Changes](#) step.

4.11 Accepting Fingerprint Changes

After successfully validating a set of changes using the methods demonstrated in this tutorial, the baseline fingerprints should be updated. The new baseline will be used in the future to validate new changes by comparing against.

For example, if the fingerprint ingredients have been changed, then it's advisable to revert the changes because the default ingredients were selected to have the right amount of sensitivity to a broad range of changes, thus may be more suitable for detecting regressions.

To do that, replace the ingredients with `tplx` (or another chosen set of baseline ingredients):

```
. ,      -f omnetpp.ini -c Ethernet -r 0,      5s,      a92f-8bfe/tplx,↵  
↪PASS,  
. ,      -f omnetpp.ini -c EthernetShortPacket -r 0,      5s,      879f-5956/tplx,↵  
↪PASS,  
. ,      -f omnetpp.ini -c Wifi -r 0,      5s,      5e6e-3064/tplx,↵  
↪PASS,  
. ,      -f omnetpp.ini -c WifiShortPacket -r 0,      5s,      7678-3e16/tplx,↵  
↪PASS,
```

Then run the fingerprint tests:

```
$ inet_fingerprinttest  
. -f omnetpp.ini -c Ethernet -r 0 ... : FAILED  
. -f omnetpp.ini -c Wifi -r 0 ... : FAILED  
. -f omnetpp.ini -c WifiShortPacket -r 0 ... : FAILED  
. -f omnetpp.ini -c EthernetShortPacket -r 0 ... : FAILED
```

The tests fail because the fingerprint values in the .csv file correspond to the previous set of ingredients or filtering. Since there was no change in the model since validation, it is safe to accept the new values/the new values can be accepted/it is safe to overwrite the .csv file with the new values:

```
$ mv baseline.csv.UPDATED baseline.csv
```

It is advisable to re-run the tests to check whether the fingerprints are stable; i.e., it might happen that each run gives different values, e.g., when the simulation trajectory depends on memory layout (caused by iteration on std::map of object pointers, for example).

When we re-run the tests, they pass:

```
$ inet_fingerprinttest fingerprintshowcase.csv  
. -f omnetpp.ini -c Ethernet -r 0 ... : PASS  
. -f omnetpp.ini -c Wifi -r 0 ... : PASS  
. -f omnetpp.ini -c WifiShortPacket -r 0 ... : PASS  
. -f omnetpp.ini -c EthernetShortPacket -r 0 ... : PASS
```

If the fingerprints are not stable, it indicates that something's wrong with the model.

RIP ROUTING TUTORIAL

This tutorial demonstrates the operation of the RIP routing protocol model in INET.

Contents:

5.1 Step 1. Static Routing

5.1.1 Introduction

Welcome to the RIP (Routing Information Protocol) tutorial for the INET Framework. This tutorial series will guide you through understanding and experimenting with the RIP routing protocol implementation in OMNeT++. RIP is one of the oldest distance-vector routing protocols, and despite its limitations, it remains useful for understanding fundamental routing concepts.

Before diving into the dynamic routing capabilities of RIP, we'll start with static routing to establish a baseline understanding of our network topology and routing concepts. This approach allows us to compare static and dynamic routing behaviors in later steps.

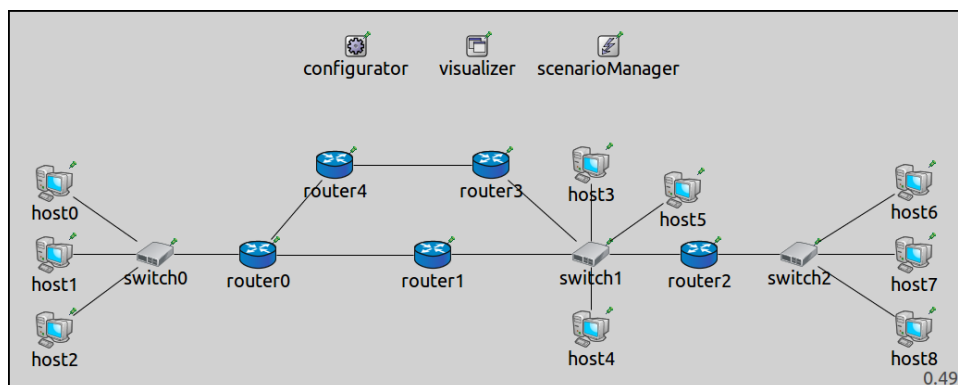
5.1.2 Goals

In this first step, our goals are to:

1. Familiarize ourselves with the experimental network topology that we'll use throughout the tutorial
2. Understand how static routing works using the `Ipv4NetworkConfigurator` module
3. Observe how packets travel through a network with pre-configured routing tables
4. Establish a baseline for comparison with dynamic RIP routing in subsequent steps

5.1.3 The Network Model

Our network consists of three switched LANs connected by routers to form an internetwork. Each LAN contains multiple hosts connected to an Ethernet switch. The LANs are interconnected through a network of routers, creating multiple possible paths between endpoints.



Key components of the network:

- **Three LANs:** Each with a switch and multiple hosts
- **Five routers:** Connecting the LANs and providing multiple routing paths
- **Connection types:** A mix of 10Mbps and 100Mbps Ethernet links
- **Host0 and Host6:** Special hosts (highlighted in red and blue) that we'll use for testing connectivity

The complete network topology is defined in the `RipNetworkA.ned` file. The network is designed to demonstrate various routing scenarios in the following steps, including route discovery, convergence, and handling of link failures.

5.1.4 Static Routing Configuration

In this step, we use static routing rather than RIP. Static routing means that the routing tables are calculated and configured at the beginning of the simulation and remain unchanged throughout. This is handled by the `Ipv4NetworkConfigurator` module, which performs two essential tasks:

1. **Address Assignment:** Automatically assigns IP and MAC addresses to all network interfaces, saving us from the tedious process of manual configuration
2. **Route Calculation:** Computes optimal routes between all network nodes and populates the routing tables accordingly

The configurator uses graph algorithms to determine the shortest paths between nodes, ensuring efficient routing. It's important to note that in this step, once the routing tables are set up, they remain static - there's no dynamic adaptation to network changes as would happen with RIP.

The configuration in `omnetpp.ini` for Step 1 includes:

```
[Config Step1]
description = "Static routing"
network = RipNetworkA
sim-time-limit = 100s

# do not add direct routes
*.configurator.addDirectRoutes = false

# Application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"

*.*.ipv4.arp.typename = "GlobalArp"

# Visualizer settings
*.visualizer.interfaceTableVisualizer[0].displayInterfaceTables = true
```

Note the `addDirectRoutes = false` setting, which instructs the configurator not to add routes to directly connected networks automatically. This makes the routing configuration more explicit and easier to understand for our learning purposes.

5.1.5 Experiment: Testing Connectivity

To test the routing configuration, we set up a simple experiment where `host0` sends ping packets to `host6`. This allows us to observe how packets traverse the network using the pre-configured static routes.

When the simulation starts:

1. The configurator module assigns addresses to all interfaces
2. The configurator calculates optimal routes and populates routing tables

3. `host0` begins sending ICMP echo (ping) requests to `host6`
4. The packets travel through the network following the routes in the routing tables
5. `host6` responds with ICMP echo replies that travel back to `host0`

The visualization settings in the general section of `omnetpp.ini` help us observe the routing paths:

```
*.visualizer.routingTableVisualizer[0].destinationFilter = "host0"
*.visualizer.routingTableVisualizer[0].lineColor = "red"
*.visualizer.routingTableVisualizer[1].destinationFilter = "host6"
*.visualizer.routingTableVisualizer[1].lineColor = "blue"
```

These settings display:

- Red arrows: Routes towards `host0`
- Blue arrows: Routes towards `host6`

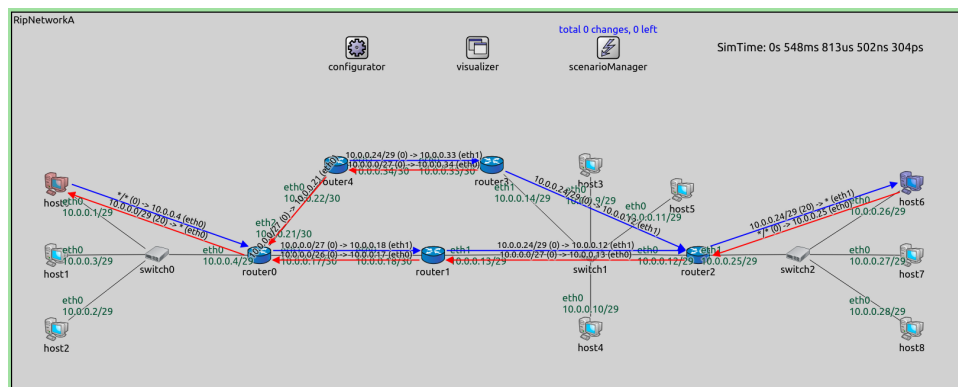
The following video demonstrates the simulation:

5.1.6 Observing the Routing Paths

In the simulation, you can observe:

1. **Complete routing paths:** The network has full connectivity with pre-configured optimal routes
2. **Bidirectional communication:** Packets can flow in both directions between `host0` and `host6`
3. **Multiple routing paths:** The network has redundant paths, though only the optimal ones are used with static routing

This screenshot shows the routing paths (red towards `host0`, blue towards `host6`):



Notice how the routing tables are complete from the start of the simulation. This is the key difference from dynamic routing protocols like RIP, where routes are discovered gradually through router advertisements.

5.1.7 Conclusion and Next Steps

In this step, we've established our network topology and observed how static routing works with pre-configured routing tables. The `Ipv4NetworkConfigurator` has done all the hard work of calculating optimal routes and setting up the routing tables.

While static routing works well in this stable network, it has limitations:

- It doesn't adapt to network changes (link failures, new routers, etc.)
- It requires manual reconfiguration when the network topology changes
- It doesn't scale well to large, dynamic networks

In the next step, we'll introduce RIP and observe how routers dynamically build their routing tables through the exchange of routing information. This will demonstrate how RIP discovers routes and converges to a stable routing state without pre-configuration.

Sources: `omnetpp.ini`, `RipNetworkA.ned`

5.1.8 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.2 Step 2. Pinging after RIP Convergence

5.2.1 Introduction

In Step 1, we used static routing with pre-configured routing tables. Now, we'll introduce the Routing Information Protocol (RIP) to demonstrate how routers can dynamically discover routes and build their routing tables through the exchange of routing information.

RIP is a distance-vector routing protocol that uses hop count as its metric. Each router maintains a routing table and periodically advertises it to its neighbors. Through these advertisements, routers learn about available networks and the “distance” (number of hops) to reach them. Over time, all routers in the network discover optimal paths to all destinations - a process known as convergence.

5.2.2 Goals

In this step, our goals are to:

1. Introduce RIP and observe how it dynamically builds routing tables
2. Understand the process of route discovery and convergence in RIP
3. Compare the dynamic routing approach with the static routing from Step 1
4. Observe how routers exchange routing information to learn about the network topology

In order to keep the discussion easy to digest, we do not introduce any changes in the network while the simulation is running, and we do not use advanced RIP features like split horizon or triggered updates yet. These will be explored in subsequent steps.

If you would like to brush up your RIP knowledge, you can find a good discussion of the distance-vector algorithm and RIP in [this section](#) of L. Peterson and B. Davie's superb book *Computer Networks: A Systems Approach*, now freely available [online](#).

5.2.3 Understanding Distance-Vector Routing

Before diving into the simulation, let's briefly review how distance-vector routing works:

1. **Initial state:** Each router knows only about its directly connected networks
2. **Information sharing:** Routers periodically send their entire routing table to all neighbors
3. **Route learning:** When a router receives routing information from a neighbor, it: * Adds the neighbor's hop count to each route * Compares these routes with its existing routing table * Updates its table if it finds a better route (fewer hops) or a new destination
4. **Convergence:** After several rounds of updates, all routers eventually learn the best routes to all destinations

RIP specifically uses a maximum hop count of 16, with 16 representing “infinity” (unreachable). Routes with a metric of 16 or more are considered invalid. RIP also implements various timers to manage route aging and removal, which we'll explore in later steps.

5.2.4 Network Configuration

We use the same network topology as in Step 1, but with a crucial difference: instead of using the configurator to pre-calculate all routes, we start with minimal routing tables and let RIP build them dynamically.

The key changes in the configuration are:

1. We enable RIP on all routers
2. We configure the hosts with default routes to their local routers
3. We start with empty routing tables on the routers

The experiment is set up in `omnetpp.ini`, in the Step2 section:

```
[Config Step2]
description = "Pinging after RIP convergence"
network = RipNetworkA
sim-time-limit = 100s

# adding default routes in all hosts
*.configurator.config = xml("<config> \
                                <interface hosts='*' address='10.x.x.x' netmask='255.
↪x.x.x' /> \
                                <autoroute sourceHosts='host*' /> \
                                </config>")

*.configurator.addStaticRoutes = true

# RIP parameters on routers
*.router*.hasRip = true
*.router*.rip.startupTime = uniform(0s,1s)
# disable advanced features for now!
*.router*.rip.ripConfig = xml("<config> \
                                <interface hosts='router*' mode='NoSplitHorizon' />
↪\
                                </config>")
*.router*.rip.triggeredUpdate = false

# Application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 50s

# turning on global ARP to focus on the RIP messages
*.*.ipv4.arp.typename = "GlobalArp"

# Visualizer settings
*.visualizer.interfaceTableVisualizer[0].displayInterfaceTables = true
```

Let's examine the key RIP configuration parameters:

- `*.router*.hasRip = true`: Enables RIP on all routers
- `*.router*.rip.startupTime = uniform(0s,1s)`: Sets a random startup time for RIP on each router (between 0 and 1 second)
- `*.router*.rip.ripConfig`: XML configuration for RIP, setting the mode to “NoSplitHorizon” (we'll explore split horizon in later steps)
- `*.router*.rip.triggeredUpdate = false`: Disables triggered updates (we'll explore this feature in Step 5)

The configurator is now only responsible for:

1. Assigning IP addresses to interfaces
2. Setting up default routes for hosts (`autoroute sourceHosts='host*'`)

5.2.5 Experiment: Observing RIP Convergence

In this experiment, we observe how RIP progressively builds the routing tables of all routers through the exchange of routing information. Unlike in Step 1, where all routes were available immediately, here we'll see routes being discovered gradually.

When the simulation starts:

1. Each router knows only about its directly connected networks
2. Routers begin exchanging RIP messages containing their routing tables
3. As routers receive these messages, they update their own routing tables
4. Over several iterations, routers discover paths to all networks in the topology
5. Eventually, the network reaches convergence - a stable state where all routers have optimal routes to all destinations
6. After convergence (at $t=50s$), `host0` begins sending ping packets to `host6` to test connectivity

The following video demonstrates this process. Watch how the blue and red arrows (representing routes to `host6` and `host0` respectively) gradually appear as routers learn about these destinations:

5.2.6 Understanding RIP Messages

During the simulation, you can observe RIP messages being exchanged between routers. Each RIP message contains:

1. **RIP header:** Protocol version and other control information
2. **Route entries:** A list of network destinations with their associated metrics (hop counts)

When a router receives a RIP message, it processes each route entry:

- If the route is new, it adds it to its routing table (with the metric increased by 1)
- If the route exists but the new path has a lower metric, it updates the route
- If the route exists but the new path has a higher metric, it ignores the update (unless it came from the same router that originally provided the route)

5.2.7 Analyzing Convergence

By the end of the simulation, you'll notice that the network has converged to the same routing state as in Step 1. This demonstrates an important principle: given the same network topology and link costs, both static and dynamic routing should eventually produce the same optimal routes.

However, there are key differences:

1. **Time to convergence:** Static routing is immediate, while RIP takes time to converge
2. **Adaptation to changes:** RIP can adapt to network changes (which we'll explore in subsequent steps)
3. **Overhead:** RIP generates ongoing traffic as routers exchange routing information
4. **Scalability:** RIP has limitations in larger networks due to its maximum hop count of 16

5.2.8 Conclusion and Next Steps

In this step, we've observed how RIP dynamically builds routing tables through the exchange of routing information. We've seen the process of convergence, where routers gradually discover optimal paths to all destinations in the network.

While RIP works well in this stable network, it faces challenges when the network topology changes. In the next steps, we'll explore:

- How RIP responds to link failures (Step 3)
- The role of various RIP timers in managing route aging and removal (Step 4)
- How triggered updates can speed up convergence (Step 5)
- The “count to infinity” problem and solutions like split horizon (Steps 6-8)

These scenarios will help us understand both the strengths and limitations of distance-vector routing protocols like RIP.

Sources: `omnetpp.ini`, `RipNetworkA.ned`

5.2.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.3 Step 3. Link Breakage and Routing Table Updates

5.3.1 Introduction

In the previous steps, we observed how RIP builds routing tables in a stable network. However, one of the key advantages of dynamic routing protocols like RIP is their ability to adapt to network changes. In this step, we'll explore how RIP responds when a link in the network fails.

When a link fails, routers need to update their routing tables to reflect the new network topology. This process involves detecting the failure, propagating this information to other routers, and recalculating routes to affected destinations. RIP handles this through its regular update mechanism, with some additional features to improve convergence time.

5.3.2 Goals

In this step, our goals are to:

1. Demonstrate how RIP updates routing tables when a link failure occurs
2. Observe the process of route invalidation and removal
3. Understand how routing information propagates through the network after a topology change
4. Introduce the concept of split horizon and its importance in preventing routing loops

5.3.3 Network Configuration

We use the same network topology as in the previous step, but with two important changes:

1. We schedule a link break using the `ScenarioManager` module
2. We enable split horizon in the RIP configuration

The `ScenarioManager` allows us to simulate network events, such as link failures, at specific times during the simulation. In this case, we'll break the link connecting `router2` and `switch2` at `t=50` seconds.

Split horizon is a technique used to prevent routing loops in distance-vector protocols like RIP. With split horizon enabled, a router will not advertise routes back to the neighbor from which it learned them. This helps prevent the “count to infinity” problem, which we'll explore in more detail in Step 6.

The configuration in `omnetpp.ini` is the following:

```
[Config Step3]
description = "Link breakage"
extends = Step2
sim-time-limit = 200s

# Enable split horizon in order for the scenario to work properly
*.router*.rip.ripConfig = xml("<config> \
                                <interface hosts='router*' mode='SplitHorizon' /> \
                                </config>")

# Disable ping application
*.host0.numApps = 0

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='50' src-module='router2' dest-
↵module='switch2' /> \
                                </scenario>")
```

Note the key changes from Step 2:

- `*.router*.rip.ripConfig`: Changed from “NoSplitHorizon” to “SplitHorizon” mode
- `*.scenarioManager.script`: XML script that disconnects the link between `router2` and `switch2` at `t=50s`
- `*.host0.numApps = 0`: Disabled the ping application to focus on routing updates

5.3.4 Understanding Link Failure Detection in RIP

When a link fails, there are two ways RIP can detect and respond to the failure:

1. **Physical layer detection**: If the router can detect the physical link failure (as in our simulation), it can immediately invalidate routes that use that link.
2. **Timeout-based detection**: If a router stops receiving updates for a route from a neighbor, the timeout timer for that route will eventually expire (default 180 seconds), and the route will be marked as unreachable.

In our simulation, we’re using physical layer detection, which allows for faster convergence after the link failure.

5.3.5 Experiment: Observing Route Updates After Link Failure

In the video below, observe what happens when the link connecting `router2` and `switch2` breaks at `t=50` seconds. The video starts at approximately `t=30` seconds, after the routing tables have stabilized from the initial RIP convergence.

When the link fails, the following sequence of events occurs:

1. `router2` detects the link failure and immediately invalidates routes that use that link
2. `router2` sends RIP updates to its neighbors, informing them of the route changes
3. The neighbors update their routing tables and propagate the information to their neighbors
4. Eventually, all routers update their routing tables to reflect the new network topology

Notice that some blue arrows (routes to `host6`) disappear after the link failure, as these routes are no longer valid. However, the network still maintains connectivity to `host6` through alternative paths.

5.3.6 Understanding Route Persistence

You may notice that the blue arrow from `host6` remains even after the link failure. This is because it represents a static route, not a RIP-managed route. RIP only controls routes between routers, while host routes are statically configured by the `Ipv4NetworkConfigurator`.

The configurator sets up default routes for hosts to reach their local routers, and these routes remain unchanged regardless of RIP updates. This is a common practice in real networks as well, where end hosts typically have a single default route to their gateway router, and it's the routers' responsibility to find the best path to destinations.

5.3.7 The Role of Split Horizon

In this step, we enabled split horizon to ensure reliable convergence after the link failure. Split horizon prevents a router from advertising routes back to the neighbor from which it learned them. This is crucial when dealing with network changes, as it helps prevent routing loops that could otherwise occur during convergence.

For example, after the link between `router2` and `switch2` breaks:

1. Without split horizon, `router1` might advertise a route to `host6` back to `router2`
2. `router2` might accept this route and start using it, creating a potential loop
3. With split horizon, `router1` will not advertise routes learned from `router2` back to `router2`

We'll explore the importance of split horizon in more detail in Step 6 when we discuss the "count to infinity" problem.

5.3.8 Conclusion and Next Steps

In this step, we've observed how RIP responds to a link failure by updating routing tables and propagating this information through the network. We've seen that RIP can adapt to topology changes, though the convergence process takes some time as routing updates propagate through the network.

In the next step, we'll explore the role of RIP timers in managing route aging and removal, which is another important aspect of how RIP handles network changes.

Sources: `omnetpp.ini`, `RipNetworkA.ned`

5.3.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.4 Step 4. RIP Timers: Timeout and Garbage Collection

5.4.1 Introduction

In the previous step, we observed how RIP responds to link failures through immediate detection. However, in real networks, routers may not always be able to detect link failures immediately. Instead, RIP relies on a system of timers to manage route aging, invalidation, and removal.

RIP uses several timers to maintain its routing tables and ensure proper operation. In this step, we'll focus on two critical timers:

1. **Timeout Timer** (also called the route aging timer): Controls how long a route remains valid without receiving updates
2. **Garbage Collection Timer** (also called the flush timer): Controls how long an invalid route remains in the routing table before being removed

These timers play a crucial role in RIP's ability to adapt to network changes while maintaining stability and preventing routing loops.

5.4.2 Goals

In this step, our goals are to:

1. Understand the role of RIP timers in managing route aging and removal
2. Observe how the timeout timer and garbage collection timer operate
3. Explore different scenarios of link recovery and their impact on routing tables

4. Understand how these timers contribute to network stability during topology changes

5.4.3 Understanding RIP Timers

RIP routers exchange routing updates periodically (typically every 30 seconds). When a router receives an update for a route from a neighbor, it resets the timeout timer associated with that route. If updates for a particular route stop arriving, the timeout timer will eventually expire.

Here's how the timer system works:

1. **Regular Updates:** Routers send routing updates every 30 seconds (by default)
2. **Timeout Timer:** When a router stops receiving updates for a route:
 - The timeout timer counts down from 180 seconds (default value)
 - If no update is received before the timer expires, the route is marked as unreachable (metric set to 16)
 - The route remains in the routing table, but is no longer used for forwarding
3. **Garbage Collection Timer:** Once a route is marked as unreachable:
 - The garbage collection timer starts counting down from 120 seconds (default value)
 - The router continues to advertise the route with a metric of 16 (infinity)
 - When this timer expires, the route is completely removed from the routing table

This two-phase approach to route removal helps ensure that all routers in the network have time to learn about unreachable routes before they are completely removed from routing tables.

5.4.4 Network Configuration

We use the same network topology as in the previous steps. The configuration in `omnetpp.ini` is:

```
[Config Step4A]
description = "RIP timers: Link comes back online before the timeout timer expires"
extends = Step2
sim-time-limit = 1000s

# Enable split horizon in order for the scenario to work properly
*.router*.rip.ripConfig = xml("<config> \
                                <interface hosts='router*' mode='SplitHorizon' /> \
                                </config>")

# disable ping application
*.host0.numApps = 0

# Break the router2 <--> switch1 link at t=50 s. and reconnect at t=150 s.
*.scenarioManager.script = xmldoc("scenario5.xml")
```

5.4.5 Experiments: Three Link Recovery Scenarios

To understand how RIP timers work, we'll simulate a link failure followed by recovery at different times. We'll disconnect the link between `router2` and `switch1` at `t=50s`, and then reconnect it at different times to observe how the timers affect routing behavior.

We'll examine three scenarios:

- **Scenario A:** Link recovers before the timeout timer expires (at `t=150s`)
- **Scenario B:** Link recovers after the timeout timer expires but before the garbage collection timer expires (at `t=300s`)
- **Scenario C:** Link recovers after both timers expire and the route is purged (at `t=400s`)

These scenarios are configured in `omnetpp.ini` as Step4A, Step4B, and Step4C respectively, each using a different scenario script.

Scenario A: Recovery Before Timeout Expiration

In this scenario, we break the link at $t=50s$ and reconnect it at $t=150s$, before the 180-second timeout timer expires. The scenario is defined in `scenario5.xml`:

```
<!-- scenario5.xml -->
<scenario>
  <at t="50">
    <disconnect src-module="router2" dest-module="switch1" />
  </at>
  <at t="150">
    <connect src-module="router2" src-gate="ethg$o[0]" dest-module="switch1" dest-
    ↪gate="ethg$i[3]" channel-type="inet.node.ethernet.Eth10M" />
    <connect src-module="switch1" src-gate="ethg$o[3]" dest-module="router2" dest-
    ↪gate="ethg$i[0]" channel-type="inet.node.ethernet.Eth10M" />
  </at>
</scenario>
```

When the link recovers before the timeout timer expires:

1. The router immediately resumes receiving RIP updates for the previously affected routes
2. The timeout timer is reset to its initial value
3. Normal routing operation continues without any disruption to the routing table
4. No routes are marked as unreachable or removed

This represents the best-case scenario for network recovery, as it minimizes disruption to routing operations.

Scenario B: Recovery After Timeout, Before Garbage Collection

In this scenario, we break the link at $t=50s$ and reconnect it at $t=300s$. This is after the 180-second timeout timer expires (at $t=230s$) but before the 120-second garbage collection timer expires (which would happen at $t=350s$). The scenario is defined in `scenario6.xml`:

```
<!-- scenario6.xml -->
<scenario>
  <at t="50">
    <disconnect src-module="router2" dest-module="switch1" />
  </at>
  <at t="300">
    <connect src-module="router2" src-gate="ethg$o[0]" dest-module="switch1" dest-
    ↪gate="ethg$i[3]" channel-type="inet.node.ethernet.Eth10M" />
    <connect src-module="switch1" src-gate="ethg$o[3]" dest-module="router2" dest-
    ↪gate="ethg$i[0]" channel-type="inet.node.ethernet.Eth10M" />
  </at>
</scenario>
```

When the link recovers after the timeout timer expires but before the garbage collection timer expires:

1. The affected routes have already been marked as unreachable (metric=16)
2. When the link recovers, the router begins receiving RIP updates again
3. Since the routes are still in the routing table (though marked as unreachable), they can be “reactivated” with their new metrics
4. The garbage collection timer is canceled, and normal routing operation resumes

This scenario demonstrates RIP's ability to recover from longer outages, though with a period where the routes are considered unreachable.

Scenario C: Recovery After Complete Route Removal

In this scenario, we break the link at $t=50s$ and reconnect it at $t=400s$. This is after both the timeout timer (expires at $t=230s$) and the garbage collection timer (expires at $t=350s$) have expired. The scenario is defined in `scenario7.xml`:

```
<!-- scenario7.xml -->
<scenario>
  <at t="50">
    <disconnect src-module="router2" dest-module="switch1" />
  </at>
  <at t="602">
    <connect src-module="router2" src-gate="ethg$o[0]" dest-module="switch1" dest-
    ↪gate="ethg$i[3]" channel-type="inet.node.ethernet.Eth10M" />
    <connect src-module="switch1" src-gate="ethg$o[3]" dest-module="router2" dest-
    ↪gate="ethg$i[0]" channel-type="inet.node.ethernet.Eth10M" />
  </at>
</scenario>
```

Let's examine what happens in detail:

1. The link breaks at $t=50s$, and `router1` stops receiving updates about the `10.0.0.24/29` network from `router2`
2. The last update was received at approximately $t=30.5929s$, as shown in this routing table entry:

```
▼ [2] (std::vector<inet::RipRoute *>) ripRoutes: size=6
  ▼ elements[6] (inet::RipRoute *)
    [0] dest:10.0.0.16 gw:* prefix:30 metric:1 if:eth0 from:<none> lastUpdate:0s changed:0 tag:0 lastInvalid:0.715189364972s INTERFACE
    [1] dest:10.0.0.8 gw:* prefix:29 metric:1 if:eth1 from:<none> lastUpdate:0s changed:0 tag:0 lastInvalid:0.715189364972s INTERFACE
    [2] dest:10.0.0.24 gw:10.0.0.12 prefix:29 metric:2 if:eth1 from:10.0.0.12 lastUpdate:30.592913356389s changed:0 tag:0 lastInvalid:0s RTE
    [3] dest:10.0.0.20 gw:10.0.0.17 prefix:30 metric:2 if:eth0 from:10.0.0.17 lastUpdate:150.548907952304s changed:0 tag:0 lastInvalid:0s RTE
    [4] dest:10.0.0.0 gw:10.0.0.17 prefix:29 metric:2 if:eth0 from:10.0.0.17 lastUpdate:150.548907952304s changed:0 tag:0 lastInvalid:0s RTE
    [5] dest:10.0.0.32 gw:10.0.0.14 prefix:30 metric:2 if:eth1 from:10.0.0.14 lastUpdate:150.84436968409s changed:0 tag:0 lastInvalid:0s RTE
```

3. At $t=210.5929s$ (180 seconds after the last update), the timeout timer expires, and the route is marked as unreachable with a metric of 16:

```
▼ [2] (std::vector<inet::RipRoute *>) ripRoutes: size=6
  ▼ elements[6] (inet::RipRoute *)
    [0] dest:10.0.0.16 gw:* prefix:30 metric:1 if:eth0 from:<none> lastUpdate:0s changed:0 tag:0 lastInvalid:0.715189364972s INTERFACE
    [1] dest:10.0.0.8 gw:* prefix:29 metric:1 if:eth1 from:<none> lastUpdate:0s changed:0 tag:0 lastInvalid:0.715189364972s INTERFACE
    [2] dest:10.0.0.24 gw:10.0.0.12 prefix:29 metric:16 if:eth1 from:10.0.0.12 lastUpdate:30.592913356389s changed:0 tag:0 lastInvalid:210.715189364972s RTE
    [3] dest:10.0.0.20 gw:10.0.0.17 prefix:30 metric:2 if:eth0 from:10.0.0.17 lastUpdate:210.548907952304s changed:0 tag:0 lastInvalid:0s RTE
    [4] dest:10.0.0.0 gw:10.0.0.17 prefix:29 metric:2 if:eth0 from:10.0.0.17 lastUpdate:210.548907952304s changed:0 tag:0 lastInvalid:0s RTE
    [5] dest:10.0.0.32 gw:10.0.0.14 prefix:30 metric:2 if:eth1 from:10.0.0.14 lastUpdate:180.84436968409s changed:0 tag:0 lastInvalid:0s RTE
```

4. At $t=330.5929s$ (120 seconds after being marked unreachable), the garbage collection timer expires, and the route is completely removed from the routing table
5. When the link recovers at $t=400s$, the router must learn about the route from scratch through new RIP updates, as if it were discovering the route for the first time

This scenario represents the most disruptive case, requiring complete rediscovery of routes after the link recovery.

5.4.6 The Importance of RIP Timers

RIP timers serve several important purposes in the operation of the protocol:

1. **Detecting Failures:** The timeout timer provides a mechanism to detect when a neighbor or route is no longer available, even without explicit link-down notifications
2. **Preventing Loops:** The garbage collection timer helps prevent routing loops by ensuring that invalid routes are advertised as unreachable before being removed
3. **Stability:** The timers add stability to the network by preventing rapid flapping of routes due to transient failures

4. **Controlled Convergence:** The timer values are chosen to balance between quick adaptation to network changes and network stability

Understanding these timers is crucial for troubleshooting RIP networks and predicting how the network will respond to failures and recoveries.

5.4.7 Conclusion and Next Steps

In this step, we've explored how RIP uses timers to manage route aging, invalidation, and removal. We've seen how the timeout timer and garbage collection timer work together to handle network changes in a controlled manner.

While these timers help maintain network stability, they can also lead to slow convergence after failures. In the next step, we'll explore how triggered updates can speed up the convergence process by allowing routers to send updates immediately when their routing tables change, rather than waiting for the next scheduled update.

Sources: `omnetpp.ini`, `scenario5.xml`, `scenario6.xml`, `scenario7.xml`, `RipNetworkA.ned`

5.4.8 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.5 Step 5. Triggered Updates: Speeding Up Convergence

5.5.1 Introduction

In the previous steps, we've observed how RIP handles network changes through periodic updates and timers. While this approach ensures eventual convergence, it can be slow - potentially taking minutes for routing information to propagate through a large network. This delay can significantly impact network performance during topology changes.

To address this limitation, RIP includes a feature called *triggered updates* that can dramatically speed up convergence after network changes. In this step, we'll explore how triggered updates work and observe their impact on convergence time.

5.5.2 Goals

In this step, our goals are to:

1. Understand the concept of triggered updates and how they differ from periodic updates
2. Observe how triggered updates accelerate the propagation of routing information after a topology change
3. Compare convergence time with and without triggered updates
4. Understand the benefits and potential drawbacks of triggered updates

5.5.3 Understanding Triggered Updates

In standard RIP operation, routers exchange routing updates on a fixed schedule

- typically every 30 seconds. This means that when a topology change occurs, there can be a significant delay before other routers learn about it:
1. On average, neighbors will wait 15 seconds (half the update interval) before receiving the next scheduled update
 2. Each subsequent "hop" of routers will add another update interval to the convergence time
 3. In a network with many hops, full convergence could take several minutes

Triggered updates provide a solution to this problem:

- When a router detects a link failure or receives an update that causes it to change its routing table
- It immediately sends an update to its neighbors, without waiting for the next scheduled update

- When neighbors receive this update and change their routing tables, they also send immediate updates
- This creates a cascade of updates that propagates through the network much faster than periodic updates

This mechanism significantly reduces convergence time, especially in larger networks with many hops between routers.

5.5.4 Network Configuration

We use the same network topology as in the previous steps, but with one critical change: we enable triggered updates in the RIP configuration.

The configuration in `omnetpp.ini` is shown below:

```
[Config Step5]
description = "RIP triggered updates"
extends = Step3
sim-time-limit = 200s

*.router*.rip.triggeredUpdate = true
```

Note the key change from previous steps:

- `*.router*.rip.triggeredUpdate = true`: Enables triggered updates on all routers

The configuration extends Step 3, which means it inherits the split horizon configuration and the scenario that breaks the link between `router2` and `switch2` at `t=50s`.

5.5.5 Experiment: Observing Accelerated Convergence

In this experiment, we'll observe how triggered updates affect the speed of convergence after a link failure. We'll break the link between `router2` and `switch2` at `t=50s` and observe how quickly routing information propagates through the network.

When the link breaks at `t=50s`, the following sequence of events occurs:

1. `router2` immediately detects the link failure and updates its routing table
2. Because triggered updates are enabled, `router2` immediately sends updates to its neighbors (around `t=52s`)
3. When neighbors receive these updates and change their routing tables, they also send immediate updates
4. This cascade of updates propagates through the network much faster than with periodic updates

Without triggered updates (as in Step 3), routers would have to wait until their next scheduled update (around `t=60s`) to inform their neighbors about the change. With triggered updates, the convergence process begins immediately after the failure is detected.

5.5.6 Benefits and Considerations of Triggered Updates

Triggered updates offer several benefits:

1. **Faster Convergence:** Network adapts to changes much more quickly
2. **Reduced Downtime:** Less time with incorrect or incomplete routing information
3. **Improved Reliability:** Critical routing changes propagate immediately

However, there are also some considerations:

1. **Update Storms:** In unstable networks with frequent changes, triggered updates could lead to a flood of updates
2. **Processing Overhead:** More frequent updates require more router CPU resources
3. **Bandwidth Usage:** Additional updates consume more network bandwidth

To mitigate these concerns, RIP implementations typically include rate-limiting mechanisms that prevent a router from sending triggered updates too frequently. This balances the benefits of fast convergence with the need to prevent excessive update traffic.

5.5.7 Triggered Updates and the Count-to-Infinity Problem

While triggered updates speed up convergence, they don't solve all of RIP's limitations. In particular, they don't address the "count-to-infinity" problem that can occur in certain network topologies with routing loops.

In fact, triggered updates can sometimes make the count-to-infinity problem worse by accelerating the rate at which incorrect routing information propagates. We'll explore this issue in more detail in the next steps, where we'll examine the count-to-infinity problem and solutions like split horizon and hold-down timers.

5.5.8 Conclusion and Next Steps

In this step, we've seen how triggered updates can significantly speed up convergence after network changes. By sending immediate updates when routing tables change, rather than waiting for the next scheduled update, RIP can adapt to topology changes much more quickly.

While triggered updates improve RIP's responsiveness, the protocol still has limitations when dealing with certain network topologies. In the next step, we'll explore one of RIP's most significant challenges: the count-to-infinity problem that can occur in networks with routing loops.

Sources: `omnetpp.ini`, `RipNetworkA.ned`

5.5.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.6 Step 6. Count to Infinity Problem (Two-node Loop Instability)

5.6.1 Introduction

In the previous steps, we've explored how RIP builds and maintains routing tables in both stable networks and during topology changes. Now, we'll examine one of the most significant challenges of distance-vector routing protocols like RIP: the count-to-infinity problem.

The count-to-infinity problem is a fundamental issue in distance-vector routing that can occur when network topology changes, particularly when links fail. It can lead to routing loops, slow convergence, and temporary network unreachability. Understanding this problem and its solutions is crucial for anyone working with RIP or similar protocols.

5.6.2 Goals

In this step, our goals are to:

1. Understand the count-to-infinity problem and why it occurs in distance-vector routing
2. Observe how the problem manifests in a simple two-router network
3. Explore different solutions to the problem, including split horizon and poison reverse
4. Understand the limitations of these solutions

5.6.3 Understanding the Count-to-Infinity Problem

The count-to-infinity problem occurs when routers continue to increment metrics for routes that are no longer valid, potentially counting up to "infinity" (which is 16 in RIP) before the routes are removed.

Here's how it typically happens:

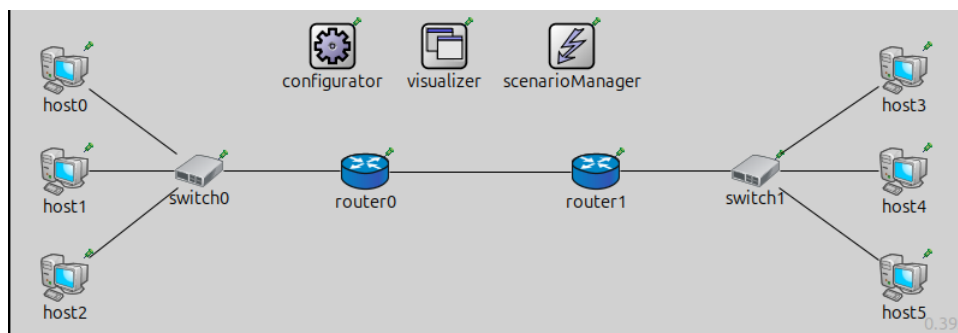
1. Router A and Router B both have routes to Network X
2. Router A's route to Network X fails

3. Router B still has its route to Network X (possibly through Router A)
4. Router B advertises this route to Router A
5. Router A accepts this route, increments the metric, and adds it to its routing table
6. Router A then advertises this route back to Router B
7. Router B sees a higher metric but still updates its route
8. This cycle continues, with metrics incrementing until they reach “infinity”

During this process, packets may loop between routers, and the network takes a long time to converge to a stable state where the invalid routes are removed.

5.6.4 Network Configuration

For this step, we use a simpler network topology defined in `RipNetworkB.ned`. This network consists of two routers connected directly to each other, with each router also connected to a LAN switch with several hosts:



This simplified topology allows us to clearly observe the count-to-infinity problem between just two routers.

The configuration in `omnetpp.ini` is the following:

```
[Config Step6]
description = "Count to infinity (two-node loop instability)"
network = RipNetworkB
sim-time-limit = 500s

# adding default routes in all hosts
*.configurator.config = xml("<config> \
                                <interface hosts='**' address='10.x.x.x' netmask='255.\
->x.x.x' /> \
                                <autoroute sourceHosts='host*' /> \
                                </config>")

*.configurator.addStaticRoutes = true

# RIP parameters on routers
*.router*.hasRip = true
*.router*.rip.startupTime = uniform(0s,1s)
# disable advanced features for now!
*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode=\
->'NoSplitHorizon' /> </config>")
*.router*.rip.triggeredUpdate = false

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='50' src-module='router1' dest-\
->module='switch1' /> \
                                </scenario>")
```

(continues on next page)

(continued from previous page)

```

# Visualizer settings
*.visualizer.interfaceTableVisualizer[0].displayInterfaceTables = true

*.visualizer.routingTableVisualizer[*].nodeFilter = "router* or host0 or host3"
*.visualizer.routingTableVisualizer[0].destinationFilter = "host0"
*.visualizer.routingTableVisualizer[1].destinationFilter = "host3"

# enable ping app to see ping packets bouncing back and forth between the two routers

## Application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host3"
*.host0.app[0].startTime = 35s
*.host0.app[0].sendInterval = 15s
*.host0.app[0].hopLimit = 8
#
*.visualizer.packetDropVisualizer[0].displayPacketDrops = true

**.arp.typename = "GlobalArp"

```

Key aspects of this configuration:

- Split horizon is disabled (mode='NoSplitHorizon')
- Triggered updates are disabled
- A scenario manager script disconnects the link between router1 and switch1 at t=50s

5.6.5 Experiment 1: Demonstrating the Count-to-Infinity Problem

In our first experiment ([Config Step6]), we demonstrate the basic count-to-infinity problem. When the link between router1 and switch1 breaks at t=50s, the following sequence of events occurs:

1. router1 loses direct connectivity to hosts 3, 4, and 5
2. router0 still has a route to these hosts (through router1)
3. router0 advertises this route to router1
4. router1 accepts this route, increments the hop count, and adds it to its routing table
5. router1 then advertises this route back to router0
6. This cycle continues, with hop counts incrementing with each exchange

The result is that convergence takes approximately 220 seconds (from the link break at t=50s to the removal of invalid routes at t=270s). During this time, the hop count for routes to hosts 3, 4, and 5 gradually increases until it reaches 16, at which point the routes are considered unreachable.

You can also observe how ping packets from host0 to host3 bounce back and forth between the two routers, indicating the presence of a routing loop. The ping packets eventually time out after 8 hops and are dropped.

5.6.6 Experiment 2: Count-to-Infinity as a Race Condition

Interestingly, the count-to-infinity problem doesn't always occur, even with the same network topology. In [Config Step6DifferentTimings], we demonstrate that the problem can be avoided depending on the timing of router updates:

```

[Config Step6DifferentTimings]
extends = Step6

```

(continues on next page)

(continued from previous page)

```
description = "Count to infinity: Using different timers, the problem does not surface
↪"
sim-time-limit = 200s

*.router1.rip.startupTime = 0.5s
*.router0.rip.startupTime = 1s
```

In this configuration, we set different startup times for the routers:

- router1 starts RIP at t=0.5s
- router0 starts RIP at t=1s

With this timing, router1 sends its update first after the link failure, informing router0 that the routes are no longer available before router0 has a chance to advertise its outdated routes back to router1. This prevents the count-to-infinity problem from occurring.

This demonstrates that the count-to-infinity problem is essentially a race condition - it depends on which router sends its updates first after a topology change.

Note

One might think that triggered updates would help solve the count-to-infinity problem by speeding up convergence. However, it can actually make the count-to-infinity problem worse by accelerating the rate at which incorrect routing information propagates. The network still experiences routing loops and slow convergence.

5.6.7 Solution 1: Split Horizon

One effective solution to the count-to-infinity problem is split horizon. In [Config Step6Solution1], we demonstrate how split horizon prevents the problem:

```
[Config Step6Solution1]
extends = Step6
description = "Solution to count to infinity problem: Split horizon"
sim-time-limit = 200s

*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode='SplitHorizon
↪' /> </config>")
```

Split horizon is a simple rule: a router will not advertise routes back to the neighbor from which it learned them. This prevents the feedback loop that causes the count-to-infinity problem.

With split horizon enabled:

1. When router1 loses its direct connection to hosts 3, 4, and 5
2. router0 still has routes to these hosts (through router1)
3. However, due to split horizon, router0 will not advertise these routes back to router1
4. This breaks the feedback loop and prevents the count-to-infinity problem

Split horizon is particularly effective in simple topologies like our two-router network, where it completely prevents the count-to-infinity problem.

5.6.8 Solution 2: Split Horizon with Poison Reverse

An even stronger solution is split horizon with poison reverse, demonstrated in [Config Step6Solution2]:

```
[Config Step6Solution2]
extends = Step6
description = "Solution to count to infinity problem: Split horizon with poison_
↳reverse"
sim-time-limit = 200s

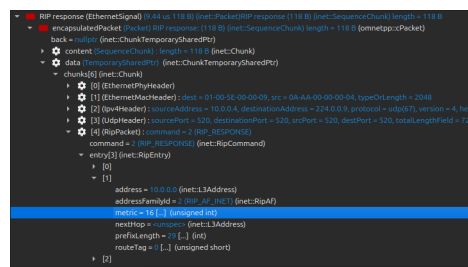
*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode=
↳'SplitHorizonPoisonedReverse' /> </config>")
```

With poison reverse, a router not only follows the split horizon rule but also actively advertises routes learned from a neighbor back to that neighbor with a metric of 16 (infinity). This explicitly tells the neighbor not to use these routes.

For example:

1. **router0** learns routes to hosts 3, 4, and 5 from **router1**
2. With poison reverse, **router0** advertises these routes back to **router1** with a metric of 16
3. This ensures that **router1** will never use **router0** as a path to these hosts

Poison reverse provides an extra layer of protection against routing loops, though it does increase the size of routing updates.



5.6.9 Limitations of These Solutions

While split horizon and poison reverse are effective in simple topologies like our two-router network, they have limitations:

1. They don't completely solve the count-to-infinity problem in networks with more than two routers (which we'll explore in Step 7)
2. They increase the size of routing updates, especially poison reverse
3. They can slow down convergence in certain scenarios

Despite these limitations, split horizon and poison reverse are widely used in RIP implementations as they significantly reduce the likelihood and impact of the count-to-infinity problem.

5.6.10 Conclusion and Next Steps

In this step, we've explored the count-to-infinity problem in a simple two-router network and examined several solutions, including split horizon and poison reverse. We've seen how these techniques can prevent routing loops and improve convergence time after topology changes.

However, these solutions have limitations, particularly in more complex network topologies. In the next step, we'll explore the count-to-infinity problem in a network with more than two routers, where split horizon and poison reverse are not completely effective.

Sources: `omnetpp.ini`, `RipNetworkB.ned`

5.6.11 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.7 Step 7. Count to Infinity in Complex Networks

5.7.1 Introduction

In the previous step, we explored the count-to-infinity problem in a simple two-router network and saw how solutions like split horizon can effectively prevent routing loops in such topologies. However, real networks often have more complex topologies with multiple routers forming potential loops.

In this step, we'll examine how the count-to-infinity problem manifests in networks with more than two routers, and why solutions like split horizon and poison reverse, while helpful, don't completely solve the problem in these more complex topologies.

5.7.2 Goals

In this step, our goals are to:

1. Understand how the count-to-infinity problem occurs in networks with more than two routers
2. Observe why split horizon alone cannot completely solve the problem in complex topologies
3. Evaluate the effectiveness of different solutions, including split horizon and triggered updates
4. Understand the implications for real-world RIP deployments

5.7.3 Understanding Count-to-Infinity in Complex Networks

In networks with more than two routers, the count-to-infinity problem becomes more challenging to solve. This is because split horizon only prevents a router from advertising routes back to the neighbor from which it learned them. It doesn't prevent routing loops that involve three or more routers.

Consider a simple scenario with three routers (A, B, and C) in a triangle topology:

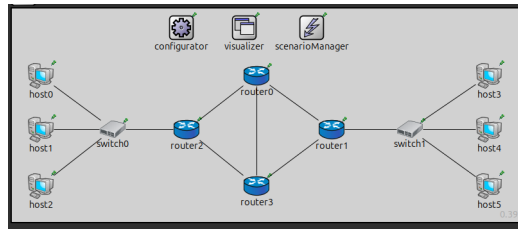
1. Router A has a direct connection to Network X
2. Routers B and C learn about Network X through Router A
3. If Router A's connection to Network X fails:
 - Router A will mark the route as unreachable
 - However, Router B still has a valid route to Network X (through Router A)
 - Router B will advertise this route to Router C
 - Router C will then advertise the route to Router A
 - Router A will accept this route, not knowing it ultimately leads nowhere
 - A routing loop is formed: $A \rightarrow C \rightarrow B \rightarrow A$

Split horizon doesn't prevent this because Router C isn't advertising the route back to Router B (from which it learned it), but to Router A. This is why more complex topologies require additional mechanisms beyond split horizon.

5.7.4 Network Model

For this step, we use a more complex network topology defined in `RipNetworkC.ned`. This network includes four routers (router0, router1, router2, and router3) connected in a mesh topology, creating multiple potential routing loops:

This step uses the following network:



```

network RipNetworkC
{
  @display("bgb=2066.82,862.47003");
  @figure[simtime](type="simTimeText"; pos=1550,60; prefix="SimTime: "; font=,14);
  submodules:
    configurator: Ipv4NetworkConfigurator {
      @display("p=683.76,77.700005");
    }
    visualizer: <default("IntegratedCanvasVisualizer")> like_
    ↪IIntegratedVisualizer {
      @display("p=936.72504,78.9375");
    }
    scenarioManager: ScenarioManager {
      @display("p=1210.375,78.9375");
    }
    host0: StandardHost {
      @display("p=116.55,271.95");
    }
    host1: StandardHost {
      @display("p=116.55,489.51");
    }
    host2: StandardHost {
      @display("p=116.55,717.43");
    }
    router0: Router {
      @display("p=986.79004,271.95");
    }
    router1: Router {
      @display("p=1289.8201,489.51");
    }
    host3: StandardHost {
      @display("p=1924.37,271.95");
    }
    host4: StandardHost {
      @display("p=1924.37,489.51");
    }
    host5: StandardHost {
      @display("p=1924.37,717.43");
    }
    switch0: EthernetSwitch {
      @display("p=406.63,486.92");
    }
    switch1: EthernetSwitch {
      @display("p=1605.8,486.92");
    }
    router2: Router {
      @display("p=704.48,489.51");
    }
}

```

(continues on next page)

(continued from previous page)

```

router3: Router {
    @display("p=986.79004,717.43");
}
connections:
host2.ethg++ <--> Eth100M <--> switch0.ethg++;
host1.ethg++ <--> Eth100M <--> switch0.ethg++;
host0.ethg++ <--> Eth100M <--> switch0.ethg++;
switch0.ethg++ <--> Eth100M <--> router2.ethg++;

switch1.ethg++ <--> Eth100M <--> host3.ethg++;
switch1.ethg++ <--> Eth100M <--> host4.ethg++;
switch1.ethg++ <--> Eth100M <--> host5.ethg++;

router0.ethg++ <--> Eth100M <--> router1.ethg++;
router1.ethg++ <--> Eth100M <--> switch1.ethg++;
router2.ethg++ <--> Eth100M <--> router3.ethg++;
router2.ethg++ <--> Eth100M <--> router0.ethg++;
router3.ethg++ <--> Eth100M <--> router0.ethg++;
router3.ethg++ <--> Eth100M <--> router1.ethg++;
}

```

The key difference from the previous step is that we now have four routers instead of two, with multiple paths between them. This creates the potential for more complex routing loops that can't be prevented by split horizon alone.

5.7.5 Experiment Configurations

We'll explore several configurations to understand the count-to-infinity problem in this more complex topology and evaluate different solutions.

Basic Configuration

The basic configuration in `omnetpp.ini` is:

```

[Config Step7]
description = "Count to infinity (loop instability with higher number of nodes)"
extends = Step6
network = RipNetworkC
sim-time-limit = 500s

*.host0.numApps = 0

```

Key aspects of this configuration:

- Split horizon is disabled (`mode='NoSplitHorizon'`)
- Triggered updates are disabled
- A scenario manager script disconnects a link at `t=50s`

Split Horizon Configuration

We also test with split horizon enabled:

```

[Config Step7SplitHorizon]
extends = Step7
sim-time-limit = 500s

```

(continues on next page)

(continued from previous page)

```
*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode='SplitHorizon  
↪' /> </config>")
```

Triggered Updates Configuration

Finally, we test with triggered updates enabled:

```
[Config Step7TriggeredUpdates]
extends = Step7
sim-time-limit = 200s

*.router*.rip.triggeredUpdate = true
*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode='SplitHorizon  
↪' /> </config>")
```

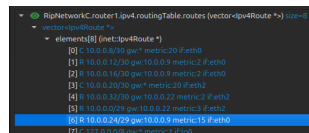
5.7.6 Experiment 1: Basic Count-to-Infinity in Complex Networks

In our first experiment, we observe the basic count-to-infinity problem in the complex network. When a link fails at $t=50s$, routing loops form among the routers, and metrics gradually increase as routing information circulates through the network.

RIP update messages are sent every 30 seconds (not visualized here). The route metrics increase incrementally (count-to-infinity):

In this scenario, we observe:

1. After the link failure, routers continue to advertise routes they learned from other routers
2. These advertisements create routing loops where packets circulate among multiple routers
3. With each update, the metrics for these routes increase
4. Eventually, after many updates, the metrics reach 16 (infinity) and the routes are removed



```
RightNetworkC.router1.ipv4.routingTable.routes (vector<ipv4Route *>) [size=8]
+ elements(8) [net:ipv4Route *]
[0] = 10.0.0.0/8 gw:1 metric:20 (f:eth0)
[1] = 10.0.0.12/30 gw:10.0.0.8 metric:2 (f:eth0)
[2] = 10.0.0.16/30 gw:10.0.0.8 metric:2 (f:eth0)
[3] = 10.0.0.20/30 gw:1 metric:20 (f:eth0)
[4] = 10.0.0.32/30 gw:10.0.0.22 metric:2 (f:eth0)
[5] = 10.0.0.24 gw:10.0.0.22 metric:3 (f:eth0)
[6] = 10.0.0.24 gw:10.0.0.8 metric:16 (f:eth0)
[7] = 127.0.0.0 gw:1 metric:1 (f:lo)
```

The key insight is that the count-to-infinity problem in complex networks can take much longer to resolve than in simple two-router networks, potentially causing extended periods of network instability.

5.7.7 Experiment 2: Split Horizon in Complex Networks

In this experiment, we enable split horizon to see if it resolves the count-to-infinity problem in our complex network.

With split horizon enabled:

1. Routers don't advertise routes back to the neighbors from which they learned them
2. This prevents some routing loops, particularly direct back-and-forth loops between two routers
3. However, loops involving three or more routers can still form

For example, if router0 learns a route from router1, split horizon prevents router0 from advertising that route back to router1. However, router0 can still advertise it to router3, which can advertise it to router1, creating a loop.

The result is that while split horizon helps reduce the severity of the count-to-infinity problem, it doesn't completely eliminate it in complex networks. Convergence is faster than without split horizon, but routing loops can still occur.

5.7.8 Experiment 3: Triggered Updates in Complex Networks

In our final experiment, we enable triggered updates to see how they affect convergence in the complex network.

With triggered updates:

1. Routing information propagates more quickly after the link failure
2. This can help the network converge faster to a stable state
3. However, it can also accelerate the rate at which incorrect routing information propagates

The result is a trade-off: triggered updates can speed up convergence in some cases, but they can also make the count-to-infinity problem more intense by accelerating the rate at which metrics increase.

When combined with split horizon, triggered updates provide a more effective solution, though still not perfect for complex networks.

5.7.9 Implications for Real-World RIP Deployments

The count-to-infinity problem in complex networks has several implications for real-world RIP deployments:

1. **Network Design:** Avoid complex topologies with multiple potential loops when using RIP
2. **Combined Solutions:** Use a combination of split horizon, poison reverse, and triggered updates
3. **Route Summarization:** Summarize routes where possible to limit the scope of routing loops
4. **Alternative Protocols:** Consider link-state protocols like OSPF for complex networks, which don't suffer from the count-to-infinity problem

In the next step, we'll explore another important mechanism for addressing the count-to-infinity problem: the hold-down timer.

5.7.10 Conclusion and Next Steps

In this step, we've seen that the count-to-infinity problem becomes more challenging in networks with more than two routers. Solutions like split horizon and triggered updates help but don't completely solve the problem in complex topologies.

In the next step, we'll explore the hold-down timer, which provides another mechanism for preventing routing loops and improving convergence in RIP networks.

Sources: `omnetpp.ini`, `RipNetworkC.ned`

5.7.11 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.8 Step 8. Hold-Down Timer: Preventing Routing Loops

5.8.1 Introduction

In the previous steps, we've explored several mechanisms for addressing the count-to-infinity problem in RIP networks, including split horizon, poison reverse, and triggered updates. While these techniques help reduce the likelihood and impact of routing loops, they don't completely solve the problem, especially in complex network topologies.

In this step, we'll explore another important mechanism for preventing routing loops: the hold-down timer. This timer provides an additional layer of protection against routing loops by preventing routers from accepting potentially incorrect routing information during network convergence.

Note

The hold-down timer is **not defined in RFC 2453** (the RIPv2 standard). It is a **vendor extension**, commonly associated with Cisco implementations, that has been widely adopted to improve RIP stability. INET also provides this feature.

5.8.2 Goals

In this step, our goals are to:

1. Understand the concept of hold-down timers and how they work
2. Observe how hold-down timers prevent routing loops in complex networks
3. Analyze the trade-offs involved in using hold-down timers
4. Understand how hold-down timers complement other mechanisms like split horizon

5.8.3 Understanding Hold-Down Timers

A hold-down timer is a mechanism that prevents a router from accepting updates for a route that has recently been marked as unreachable. When a router learns that a route has become unreachable (either through a direct link failure or from a neighbor's update), it:

1. Marks the route as unreachable (sets the metric to 16)
2. Starts a hold-down timer for that route (typically 180 seconds)
3. During the hold-down period, the router will not accept any updates for that route, even if they advertise a valid metric
4. After the hold-down timer expires, the router will accept updates for the route again

The purpose of the hold-down timer is to give the network time to converge to a stable state after a topology change. By ignoring potentially outdated or incorrect routing information during this period, routers can avoid the formation of routing loops.

5.8.4 Network Configuration

This step uses the same complex network topology as the previous step, with four routers connected in a mesh topology. The key difference is that we now enable the hold-down timer in the RIP configuration.

The configuration in `omnetpp.ini` is the following:

```
[Config Step8]
description = "Hold-down timer"
extends = Step7
sim-time-limit = 500s

*.router*.rip.holdDownTime = 30s
```

Key aspects of this configuration:

- `*.router*.rip.holdDownTime = 30s`: Enables the hold-down timer with a value of 30 seconds (reduced from the default 180 seconds to make the simulation faster)
- Split horizon is not enabled, allowing us to focus on the effect of the hold-down timer alone

5.8.5 Experiment: Hold-Down Timer in Action

In this experiment, we observe how the hold-down timer prevents routing loops after a link failure. When a link fails, the following sequence of events occurs:

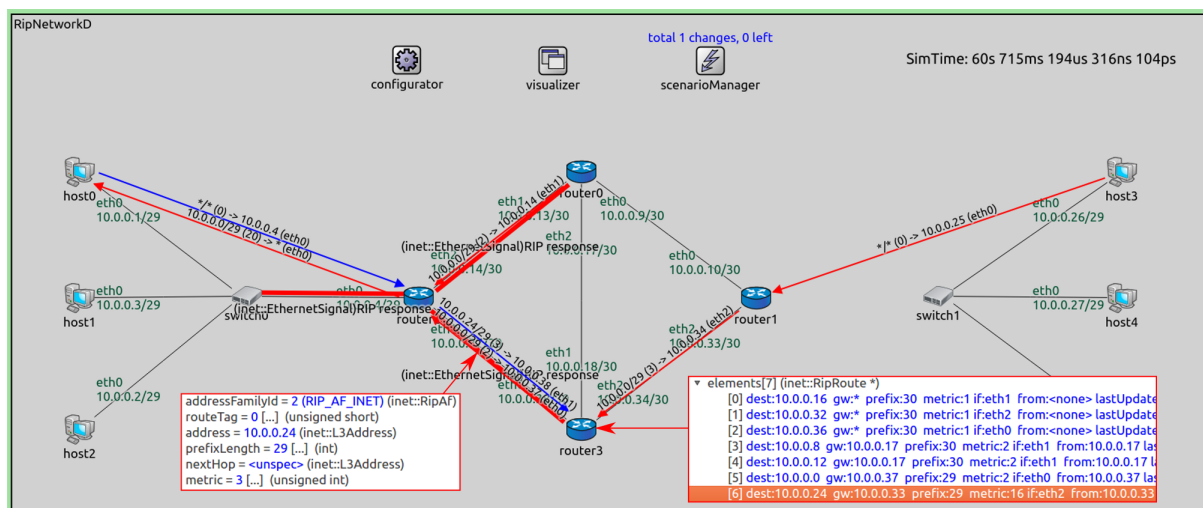
1. The router directly connected to the failed link marks routes through that link as unreachable (metric 16)

2. It starts the hold-down timer for these routes
3. It propagates this information to its neighbors
4. When neighbors receive this information, they also mark the routes as unreachable and start their own hold-down timers
5. During the hold-down period, routers ignore any updates for these routes, even if they advertise valid metrics
6. This prevents the formation of routing loops that would otherwise occur if routers accepted potentially incorrect routing information

Let's examine a specific example from the simulation:

1. The 10.0.0.24/29 network becomes unreachable, and the route in **router1** is set to metric 16
2. This information is propagated to **router3**, which also sets the route to 10.0.0.24/29 to metric 16 and starts its hold-down timer
3. Meanwhile, **router2** still has a route to 10.0.0.24/29 with a metric of 3 (which is actually incorrect)
4. **router2** sends a RIP Response message to **router3** containing this route
5. However, **router3** does not update its routing table with this information because the route is in hold-down
6. This prevents the formation of a routing loop that would otherwise occur if **router3** accepted the incorrect route from **router2**

The following image shows this scenario in action:



In this image, you can see:

- **router2** sending a RIP Response message to **router3** with a route to 10.0.0.24/29 (metric 3)
- **router3**'s routing table showing the route to 10.0.0.24/29 with a metric of 16 (unreachable)
- The route is in hold-down, preventing **router3** from accepting the update from **router2**

By ignoring the potentially incorrect routing information during the hold-down period, **router3** helps prevent the formation of routing loops.

5.8.6 Benefits and Trade-offs of Hold-Down Timers

Hold-down timers offer several benefits for RIP networks:

1. **Preventing Routing Loops:** By ignoring potentially incorrect routing information during convergence, hold-down timers help prevent the formation of routing loops
2. **Complementing Other Mechanisms:** Hold-down timers work well in conjunction with split horizon and poison reverse, providing an additional layer of protection

3. **Effective in Complex Topologies:** Unlike split horizon, hold-down timers can help prevent routing loops in complex topologies with multiple potential loops

However, there are also trade-offs to consider:

1. **Delayed Convergence:** Hold-down timers can delay network convergence, as routers must wait for the timer to expire before accepting new routes
2. **Temporary Unreachability:** During the hold-down period, some networks may be temporarily unreachable, even if valid alternate paths exist
3. **Configuration Complexity:** The optimal hold-down timer value depends on network size and topology, requiring careful configuration

Despite these trade-offs, hold-down timers are widely used in RIP implementations as they provide an effective mechanism for preventing routing loops, especially in complex networks.

5.8.7 Combining Solutions for Optimal RIP Performance

For optimal RIP performance and stability, network administrators typically combine multiple mechanisms:

1. **Split Horizon:** Prevents routes from being advertised back to the neighbor from which they were learned
2. **Poison Reverse:** Explicitly advertises unreachable routes with a metric of 16
3. **Triggered Updates:** Speeds up convergence by sending immediate updates when routing tables change
4. **Hold-Down Timers:** Prevents the acceptance of potentially incorrect routing information during convergence

By combining these mechanisms, RIP networks can achieve faster convergence while minimizing the risk of routing loops.

5.8.8 Conclusion and Next Steps

In this step, we've explored how hold-down timers help prevent routing loops in RIP networks by ignoring potentially incorrect routing information during network convergence. We've seen that while hold-down timers introduce some delay in convergence, they provide an effective mechanism for preventing routing loops, especially in complex network topologies.

In the next step, we'll measure RIP recovery time under different configurations to understand the performance implications of various RIP features and mechanisms.

Sources: `omnetpp.ini`, `RipNetworkC.ned`

5.8.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.9 Step 9. Measuring RIP Recovery Time

5.9.1 Introduction

In the previous steps, we've explored various mechanisms for preventing routing loops and improving convergence in RIP networks, including split horizon, poison reverse, triggered updates, and hold-down timers. Now, we'll focus on a critical performance metric: recovery time.

Recovery time refers to how quickly a network can adapt to topology changes and restore connectivity. In RIP networks, recovery time is influenced by various factors, including the update interval, triggered updates, and the specific mechanisms used to prevent routing loops. Understanding these factors can help network administrators optimize RIP configurations for their specific requirements.

5.9.2 Goals

In this step, our goals are to:

1. Measure RIP recovery time under different configurations
2. Understand the impact of triggered updates on recovery time
3. Analyze the trade-offs between stability and recovery speed
4. Explore how different RIP features affect network performance

5.9.3 Network Configuration

This step uses the same network topology as the previous steps, with a focus on measuring recovery time after a link failure and subsequent recovery. We'll compare different configurations to understand their impact on recovery time.

The basic configuration in `omnetpp.ini` is:

```
[Config Step9]
description = "Measure RIP recovery time"
extends = Step2
sim-time-limit = 1500s

*.host0.numApps = 1
*.host0.app[0].typename = "UdpBasicApp"
*.host0.app[0].destAddresses = "host6"
*.host0.app[0].destPort = 1234
*.host0.app[0].sendInterval = 0.5s
*.host0.app[0].messageLength = 32 bytes

*.host6.numApps = 1
*.host6.app[0].typename = "UdpSink"
*.host6.app[0].localPort = 1234

*.router*.rip.startupTime = uniform(0s,1s)
*.router*.rip.ripConfig = xml("<config> <interface hosts='router*' mode='SplitHorizon
↪' /> </config>")

*.scenarioManager.script = xmldoc("scenario7.xml")

*.router*.rip.triggeredUpdate = ${triggeredUpdate = false, true}
```

Key aspects of this configuration:

- We use UDP applications to generate continuous traffic between `host0` and `host6`
- Split horizon is enabled for stability
- We test with both triggered updates enabled and disabled
- The simulation runs for a longer period (1500 seconds) to observe complete recovery

The scenario manager script controls the link failure and recovery:

```
<!-- scenario7.xml -->
<scenario>
  <at t="50">
    <disconnect src-module="router2" dest-module="switch1" />
  </at>
  <at t="602">
    <connect src-module="router2" src-gate="ethg$0[0]" dest-module="switch1" dest-
```

(continues on next page)

(continued from previous page)

```

↪gate="ethg$i[3]" channel-type="inet.node.ethernet.Eth10M" />
    <connect src-module="switch1" src-gate="ethg$o[3]" dest-module="router2" dest-
↪gate="ethg$i[0]" channel-type="inet.node.ethernet.Eth10M" />
    </at>
</scenario>

```

This script:

1. Disconnects the link between `router2` and `switch1` at `t=50s`
2. Reconnects the link at `t=400s` (after both timeout and garbage-collection timers would have expired)

We also test an alternative configuration without netmask routes:

```

[Config Step9NoNetmaskRoutes]
extends = Step9

**.netmaskRoutes = ""
*.configurator.addDirectRoutes = true

```

5.9.4 Experiment: Measuring Recovery Time

In this experiment, we measure how long it takes for the network to recover connectivity between `host0` and `host6` after a link failure and subsequent recovery. We compare recovery times with and without triggered updates to understand their impact.

The recovery process involves several phases:

1. **Failure Detection:** How quickly routers detect the link failure
2. **Route Invalidation:** How quickly invalid routes are marked as unreachable
3. **Route Propagation:** How quickly this information propagates through the network
4. **Alternate Path Discovery:** How quickly alternate paths are discovered
5. **Route Recovery:** How quickly routes are restored after the link recovers

5.9.5 Impact of Triggered Updates on Recovery Time

Triggered updates can significantly impact recovery time in RIP networks:

With Triggered Updates Enabled:

1. Routers send immediate updates when their routing tables change
2. This accelerates the propagation of routing information after a topology change
3. The network can converge more quickly to a stable state
4. Recovery time is typically reduced, especially in larger networks

Without Triggered Updates:

1. Routers only send updates at regular intervals (typically every 30 seconds)
2. Routing information propagates more slowly through the network
3. Convergence takes longer, especially in networks with many hops
4. Recovery time is typically longer, but network overhead is reduced

5.9.6 Trade-offs Between Stability and Recovery Speed

When configuring RIP networks, administrators must balance stability and recovery speed:

Faster Recovery:

- Enables triggered updates
- Uses shorter update intervals
- Minimizes hold-down timers
- Prioritizes quick adaptation to changes

Greater Stability:

- Uses longer hold-down timers
- Implements split horizon and poison reverse
- May disable triggered updates in unstable networks
- Prioritizes preventing routing loops and oscillations

The optimal configuration depends on specific network requirements:

- Mission-critical networks may prioritize stability over recovery speed
- Real-time applications may require faster recovery at the cost of some stability
- Large networks may need longer timers to prevent excessive updates

5.9.7 Optimizing RIP Performance

Based on our experiments, we can make several recommendations for optimizing RIP performance:

1. **Enable Triggered Updates:** In most networks, triggered updates significantly improve recovery time without major drawbacks
2. **Use Split Horizon:** Always enable split horizon to prevent simple routing loops
3. **Consider Network Size:** Adjust timers based on network size and complexity
4. **Monitor Network Stability:** In unstable networks, consider longer hold-down timers and possibly disabling triggered updates
5. **Use Route Summarization:** Summarize routes where possible to reduce update size and limit the scope of routing changes

5.9.8 Conclusion and Next Steps

In this step, we've explored how different RIP configurations affect recovery time after network changes. We've seen that triggered updates can significantly improve recovery time, though there are trade-offs to consider regarding network stability and overhead.

In the final step of this tutorial, we'll explore how to configure RIP interfaces, including how to disable RIP on specific interfaces to optimize protocol operation and reduce unnecessary routing traffic.

Sources: `omnetpp.ini`, `RipNetworkC.ned`, `scenario3.xml`, `scenario7.xml`

5.9.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.10 Step 10. Configuring RIP Interfaces

5.10.1 Introduction

Throughout this tutorial, we've explored various aspects of RIP, including its basic operation, convergence behavior, and mechanisms for preventing routing loops. In this final step, we'll focus on an important aspect of RIP configuration: interface-specific settings.

In real-world networks, not all interfaces on a router need to participate in RIP. For example, interfaces connected to end-user networks typically don't need to send or receive RIP updates, as these networks don't contain other routers. By configuring interfaces appropriately, we can optimize RIP operation, reduce unnecessary routing traffic, and improve network security.

5.10.2 Goals

In this step, our goals are to:

1. Understand how to configure RIP interfaces with different modes
2. Learn when and why to disable RIP on specific interfaces
3. Observe the impact of interface configuration on RIP message flow
4. Understand best practices for RIP interface configuration in real networks

5.10.3 Understanding RIP Interface Modes

RIP supports several interface modes that control how the protocol operates on each interface:

1. **Normal Mode:** The interface sends and receives RIP updates normally
2. **Passive Mode:** The interface listens for RIP updates but doesn't send any
3. **NoRIP Mode:** The interface doesn't participate in RIP at all (no sending or receiving of updates)
4. **NoSplitHorizon Mode:** The interface operates normally but without split horizon
5. **SplitHorizon Mode:** The interface operates with split horizon enabled
6. **SplitHorizonPoisonedReverse Mode:** The interface operates with split horizon and poison reverse

By configuring these modes appropriately for each interface, network administrators can optimize RIP operation for their specific network topology.

5.10.4 Network Configuration

This step uses the same network topology as the previous steps, but with specific RIP interface configurations. We configure certain interfaces as NoRIP to demonstrate how this affects the flow of RIP messages.

The configuration in `omnetpp.ini` is the following:

```
[Config Step10]
description = "Configure an interface as NoRIP"
extends = Step2

*.router0.rip.ripConfig = xml("<config> \
    <interface names='eth[0]' mode='NoRIP' /> \
    <interface metric='1' /> \
    </config>")

*.router2.rip.ripConfig = xml("<config> \
    <interface names='eth[1]' mode='NoRIP' /> \
    <interface metric='1' /> \
    </config>")
```

Key aspects of this configuration:

- `router0.rip.ripConfig`: Configures the eth[0] interface (connected to the host network) as NoRIP
- `router2.rip.ripConfig`: Configures the eth[1] interface (connected to the host network) as NoRIP
- All other interfaces operate in normal mode with a metric of 1

This configuration reflects a common practice in real networks: disabling RIP on interfaces connected to end-user networks that don't contain other routers.

5.10.5 Experiment: Observing RIP Message Flow

In this experiment, we observe how the NoRIP interface configuration affects the flow of RIP messages in the network. With certain interfaces configured as NoRIP, we expect to see RIP messages flowing only between router interfaces that are configured to participate in RIP.

As shown in the video, RIP messages are only exchanged between router interfaces that are not configured as NoRIP. Specifically:

- RIP messages flow between router interfaces connected to other routers
- No RIP messages are sent or received on interfaces connected to host networks (configured as NoRIP)

This behavior optimizes RIP operation by:

1. Reducing unnecessary routing traffic on networks that don't need it
2. Preventing hosts from receiving routing updates they can't use
3. Focusing RIP resources on interfaces where routing information exchange is actually needed

5.10.6 Benefits of Proper Interface Configuration

Properly configuring RIP interfaces offers several benefits:

1. **Reduced Network Overhead:** By not sending RIP updates on interfaces where they're not needed, we reduce unnecessary network traffic
2. **Improved Security:** Limiting RIP updates to only necessary interfaces reduces the attack surface for potential routing protocol attacks
3. **Optimized Resource Usage:** Routers can focus their processing resources on interfaces where routing information exchange is actually needed
4. **Simplified Troubleshooting:** With clear boundaries for RIP operation, troubleshooting routing issues becomes easier

5.10.7 Best Practices for RIP Interface Configuration

Based on our experiments and industry best practices, here are some recommendations for configuring RIP interfaces:

1. **Configure End-User Interfaces as NoRIP:** Interfaces connected to networks with only end users (no other routers) should be configured as NoRIP
2. **Use Passive Mode for Backup Links:** Interfaces on backup links can be configured in passive mode to receive updates but not advertise routes
3. **Enable Split Horizon on All Active Interfaces:** To prevent routing loops, enable split horizon on all interfaces that participate in RIP
4. **Consider Interface Metrics:** Adjust interface metrics based on link speed and reliability to influence route selection
5. **Document Interface Configurations:** Maintain clear documentation of interface configurations to aid in troubleshooting and network management

5.10.8 Conclusion

In this final step of the RIP tutorial, we've explored how to configure RIP interfaces to optimize protocol operation. By selectively enabling or disabling RIP on specific interfaces, network administrators can reduce unnecessary routing traffic, improve security, and focus RIP resources where they're actually needed.

Throughout this tutorial series, we've covered the fundamental aspects of RIP operation, including:

- Basic routing table construction and convergence (Steps 1-2)
- Handling network changes and link failures (Steps 3-5)
- Addressing the count-to-infinity problem (Steps 6-7)
- Using timers and other mechanisms to prevent routing loops (Step 8)
- Measuring and optimizing recovery time (Step 9)
- Configuring interfaces for optimal operation (Step 10)

With this knowledge, you should now have a solid understanding of how RIP works, its strengths and limitations, and how to configure it for optimal performance in various network scenarios.

Sources: `omnetpp.ini`, `RipNetworkC.ned`

5.10.9 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

5.11 Conclusion

Congratulations, you've made it! You should now be able to set up RIP routing for networks in the INET Framework. Use the link below if you have questions or you'd like to help us improve this tutorial.

5.11.1 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

BGP ROUTING TUTORIAL

This tutorial demonstrates the operation of the BGP routing protocol model in INET.

Contents:

6.1 Step 1. BGP Basic Topology

6.1.1 Goals

This step introduces a basic BGP configuration. The network consists of two Autonomous Systems (AS): AS 64520 and AS 64530.

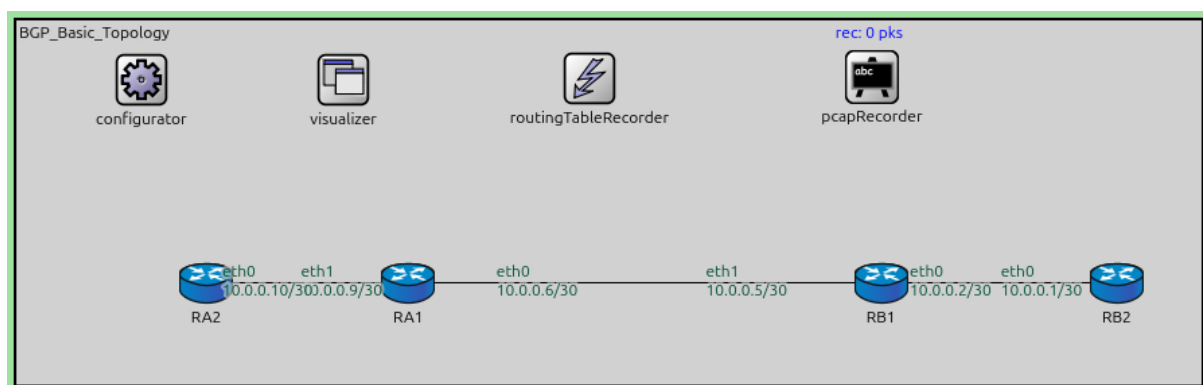
- AS 64520 contains two routers, RA1 and RA2.
- AS 64530 contains two routers, RB1 and RB2.

An External BGP (E-BGP) session is established between RA1 and RB1. The primary goal is to demonstrate how BGP peers exchange routing information. In this configuration, the `Network` attribute in the XML configuration is used to manually specify which networks should be advertised by each router:

- RA1 advertises the network `10.0.0.8/30`.
- RB1 advertises the network `10.0.0.0/30`.

Configuration

This step uses the following network:



```
network BGP_Basic_Topology
{
    @display("bgp=880,3688,269.73");

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=93,44");
```

(continues on next page)

(continued from previous page)

```

    }
    visualizer: IntegratedMultiCanvasVisualizer {
        @display("p=243.2025,43.536247");
    }
    routingTableRecorder: RoutingTableRecorder {
        @display("p=426,43");
    }
    pcapRecorder: PcapRecorder {
        @display("p=636,42");
    }
    RB2: Router {
        @display("p=817.9313,194.805");
    }
    RB1: Router {
        @display("p=642.69,194.805");
    }
    RA1: Router {
        @display("p=291.375,194.805");
    }
    RA2: Router {
        @display("p=141.10875,194.805");
    }
}

connections:
    RB1.ethg++ <--> Eth100M <--> RB2.ethg++;
    RB1.ethg++ <--> Eth100M <--> RA1.ethg++;
    RA2.ethg++ <--> Eth100M <--> RA1.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step1]
description = "BGP Basic Topology"
network = BGP_Basic_Topology
*.routingTableRecorder.logfile = "step1.rt"

*.pcapRecorder.pcapFile = "step1.pcap"

# BGP configuration
*.R*.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_Basic.xml")

*.visualizer.routingTableVisualizer[1].displayRoutingTables = false
*.visualizer.routingTableVisualizer[*].lineShift = 80
*.visualizer.routingTableVisualizer[*].destinationFilter = "*"
*.visualizer.routingTableVisualizer[*].lineColor = "black"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPCfg xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
  ↪ "BGP.xsd">

    <TimerParams>
        <connectRetryTime> 120 </connectRetryTime>
        <holdTime> 180 </holdTime>
        <keepAliveTime> 60 </keepAliveTime>
    
```

(continues on next page)

(continued from previous page)

```

    <startDelay>          5    </startDelay>
  </TimerParams>

  <AS id="64520">
    <!--router RA1-->
    <Router interAddr="10.0.0.9">
      <Network address='10.0.0.8' />
    </Router>

    <!--router RA2-->
    <Router interAddr="10.0.0.10"/>
  </AS>

  <AS id="64530">
    <!--router RB1-->
    <Router interAddr="10.0.0.2">
      <Network address='10.0.0.0' />
    </Router>

    <!--router RB2-->
    <Router interAddr="10.0.0.1"/>
  </AS>

  <!--bi-directional E-BGP session between RA1 and RB1-->
  <Session id="1">
    <Router exteAddr="10.0.0.6"/>
    <Router exteAddr="10.0.0.5"/>
  </Session>

</BGPConfig>

```

Results

Upon running the simulation, the BGP speakers RA1 and RB1 perform the following sequence:

1. They establish a TCP connection between their external interfaces.
2. They exchange BGP OPEN messages to negotiate session parameters.
3. Once the session is in the Established state, they exchange BGP UPDATE messages to advertise their configured networks.

In the routing table log (`step1.rt`), we can observe the impact of these exchanges:

- At approximately 6.5s, RB1 installs a route to `10.0.0.8/30` with the next hop set to RA1 (`10.0.0.6`).
- Simultaneously, RA1 installs a route to `10.0.0.0/30` with the next hop set to RB1 (`10.0.0.5`).

Note that in this basic step, RA2 and RB2 do not learn these routes because I-BGP (Internal BGP) is not yet configured, and BGP routes are not redistributed into any interior gateway protocol.

Sources: `BGP_Basic_Topology.ned`, `omnetpp.ini`, `BGPConfig_Basic.xml`

6.1.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.2 Step 2. BGP Scenario with E-BGP session only

6.2.1 Goals

This step demonstrates BGP operating in a more realistic scenario where an Interior Gateway Protocol (IGP) is used within each Autonomous System. Specifically, OSPF is configured within AS 64500 (RA routers) and AS 64600 (RB routers).

The key objectives are:

- Establishing an E-BGP session between the border routers RA4 and RB1.
- Redistributing OSPF routes into BGP so that internal networks of one AS can be reached from the other.

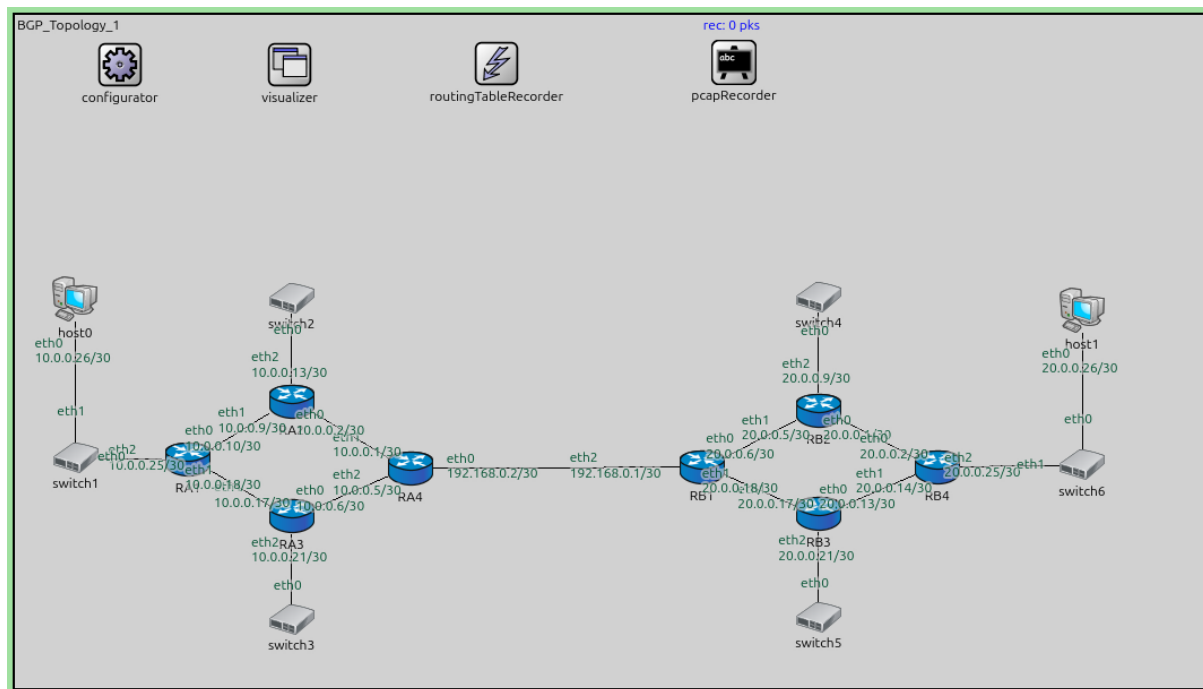
In `omnetpp.ini`, redistribution is enabled using the `redistributeOspf` parameter:

```
*.RA4.bgp.redistributeOspf = "O IA"
*.RB1.bgp.redistributeOspf = "O IA"
```

This tells the BGP module to advertise routes learned via OSPF (both intra-area 'O' and inter-area 'IA' routes) to its BGP peers.

Configuration

This step uses the following network:



```
network BGP_Topology_1
{
    @display("bgb=1050.615,594.405");

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=93,44");
        }
        visualizer: IntegratedMultiCanvasVisualizer {
            @display("p=243.2025,43.536247");
        }
}
```

(continues on next page)

(continued from previous page)

```

routingTableRecorder: RoutingTableRecorder {
    @display("p=426,43");
}
pcapRecorder: PcapRecorder {
    @display("p=636,42");
}
RB2: Router {
    @display("p=710.955,349.65");
}
RB3: Router {
    @display("p=710.955,441.225");
}
RB4: Router {
    @display("p=815.85,399.6");
}
RB1: Router {
    @display("p=607.725,399.6");
}
RA4: Router {
    @display("p=349.79123,400.83374");
}
RA2: Router {
    @display("p=244.70375,342.285");
}
RA3: Router {
    @display("p=244.70375,442.86874");
}

RA1: Router {
    @display("p=153.1275,393.32748");
}
switch1: EthernetSwitch {
    @display("p=54.045,391.82623");
}
switch2: EthernetSwitch {
    @display("p=244.70375,250.70874");
}
switch3: EthernetSwitch {
    @display("p=244.70375,534.445");
}
switch4: EthernetSwitch {
    @display("p=710.955,249.75");
}
switch5: EthernetSwitch {
    @display("p=710.955,532.8");
}
switch6: EthernetSwitch {
    @display("p=944.055,397.935");
}
host0: StandardHost {
    @display("p=53.28,251.415");
}
host1: StandardHost {
    @display("p=944.055,261.405");
}
connections:

```

(continues on next page)

(continued from previous page)

```

RB2.ethg++ <--> Eth100M <--> RB4.ethg++;
RB3.ethg++ <--> Eth100M <--> RB4.ethg++;
RB1.ethg++ <--> Eth100M <--> RB2.ethg++;
RB1.ethg++ <--> Eth100M <--> RB3.ethg++;
RB1.ethg++ <--> Eth100M <--> RA4.ethg++;
RA4.ethg++ <--> Eth100M <--> RA2.ethg++;
RA3.ethg++ <--> Eth100M <--> RA4.ethg++;
RA2.ethg++ <--> Eth100M <--> RA1.ethg++;
RA1.ethg++ <--> Eth100M <--> RA3.ethg++;
switch1.ethg++ <--> Eth100M <--> RA1.ethg++;
RA2.ethg++ <--> Eth100M <--> switch2.ethg++;
RA3.ethg++ <--> Eth100M <--> switch3.ethg++;
RB2.ethg++ <--> Eth100M <--> switch4.ethg++;
RB3.ethg++ <--> Eth100M <--> switch5.ethg++;
switch1.ethg++ <--> Eth100M <--> host0.ethg++;
switch6.ethg++ <--> Eth100M <--> host1.ethg++;
RB4.ethg++ <--> Eth100M <--> switch6.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step2]
description = "BGP Scenario with E-BGP session only"
network = BGP_Topology_1
*.routingTableRecorder.logfile = "step2.rt"

*.pcapRecorder.pcapFile = "step2.pcap"

*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface among='host0 RA*' address='10.x.x.x' _
↪netmask='255.x.x.x' /> \
    <interface hosts='RA*' address='10.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface among='host1 RB*' address='20.x.x.x' _
↪netmask='255.x.x.x' /> \
    <interface hosts='RB*' address='20.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <route hosts='RA4' destination='*' netmask='0.0.0.0' _
↪interface='eth0' /> \
    <route hosts='RB1' destination='*' netmask='0.0.0.0' _
↪interface='eth2' /> \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
    </config>")

# OSPF configuration
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig_EBGP.xml")

```

(continues on next page)

(continued from previous page)

```

*.RA4.ipv4.routingTable.routerId = "10.0.0.5"
*.RB1.ipv4.routingTable.routerId = "20.0.0.18"

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_EBGP.xml")

# enable OSPF redistribution
*.RA4.bgp.redistributeOspf = "O IA"
*.RB1.bgp.redistributeOspf = "O IA"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPCfg xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
  ↪ "BGP.xsd">

  <TimerParams>
    <connectRetryTime> 120 </connectRetryTime>
    <holdTime> 180 </holdTime>
    <keepAliveTime> 60 </keepAliveTime>
    <startDelay> 60 </startDelay>
  </TimerParams>

  <AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1" />
  </AS>

  <AS id="64600">
    <!--router RB1-->
    <Router interAddr="20.0.0.6" />
  </AS>

  <!--bi-directional E-BGP session between RA4 and RB1-->
  <Session id="1">
    <Router exteAddr="192.168.0.2"/>
    <Router exteAddr="192.168.0.1"/>
  </Session>

</BGPCfg>

```

Results

The simulation proceeds in two phases:

1. **IGP Convergence:** Initially, OSPF converges within each AS. You can see OSPF HELLO and LSU packets being exchanged, and the routing tables filling with internal routes.
2. **BGP Session Establishment:** After a configured delay (`startDelay = 60s` in `BGPCfg_EBGP.xml`), RA4 and RB1 initiate their BGP session.

Once the E-BGP session is established at approximately 61.5s:

- RA4 receives BGP UPDATE messages from RB1 containing all the internal networks of AS 64600 (the 20.0.0.0/30 subnets).
- RB1 receives BGP UPDATE messages from RA4 containing all the internal networks of AS 64500 (the

10.0.0.0/30 subnets).

Checking `step2.rt` reveals that both border routers now have complete reachability to the other AS's internal networks via the BGP peer. However, note that internal routers (like RA1 or RB2) still cannot reach the other AS because the BGP-learned routes have not been redistributed back into OSPF.

Sources: `BGP_Topology_1.ned`, `omnetpp.ini`, `OSPFConfig_EBGP.xml`, `BGPConfig_EBGP.xml`

6.2.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.3 Step 3. BGP Path Attributes

6.3.1 Goals

This step explores a more complex multi-AS topology comprising four Autonomous Systems (AS 64500 to AS 64800) with a total of 16 routers. The border routers (RA4, RB1, RC2, and RD3) are interconnected in a full mesh of E-BGP sessions.

The primary objectives are:

- Observed path selection based on the `AS_PATH` attribute.
- Verify reachability across multiple ASes using OSPF-to-BGP redistribution.

In this scenario, every border router learns paths to every other AS through multiple neighbors. BGP selects the best path primarily based on the shortest `AS_PATH` length.

Configuration

This step uses the following network:

```
network BGP_Topology_1a
{
    @display("bgb=1081.6675,985.49");

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=93,44");
        }
        visualizer: IntegratedMultiCanvasVisualizer {
            @display("p=243.2025,43.536247");
        }
        routingTableRecorder: RoutingTableRecorder {
            @display("p=426,43");
        }
        pcapRecorder: PcapRecorder {
            @display("p=636,42");
        }
        RB2: Router {
            @display("p=770.3949,436.40866");
        }
        RB3: Router {
            @display("p=770.3949,525.4717");
        }
        RB4: Router {
            @display("p=877.2705,485.3933");
        }
        RB1: Router {
            @display("p=670.19904,485.3933");
        }
}
```

(continues on next page)

(continued from previous page)

```
}
RA4: Router {
    @display("p=407.4632,487.6199");
}
RA2: Router {
    @display("p=305.04077,427.50238");
}
RA3: Router {
    @display("p=305.04077,525.4717");
}

RA1: Router {
    @display("p=209.29803,478.7136");
}
switch1: EthernetSwitch {
    @display("p=115.78189,476.48703");
}
switch2: EthernetSwitch {
    @display("p=305.04077,333.98624");
}
switch3: EthernetSwitch {
    @display("p=305.04077,618.9878");
}
switch4: EthernetSwitch {
    @display("p=770.3949,333.98624");
}
switch5: EthernetSwitch {
    @display("p=770.3949,618.9878");
}
switch6: EthernetSwitch {
    @display("p=1004.18524,483.16675");
}
host0: StandardHost {
    @display("p=113.55532,333.98624");
}
host1: StandardHost {
    @display("p=1004.18524,347.34567");
}
RC1: Router {
    @display("p=425.2758,754.8089");
}
RC3: Router {
    @display("p=527.69824,797.11383");
}
RC4: Router {
    @display("p=630.12067,754.8089");
}
switch7: EthernetSwitch {
    @display("p=761.4886,750.3557");
}
host2: StandardHost {
    @display("p=760.1975,862.96246");
}
switch8: EthernetSwitch {
    @display("p=527.69824,888.4034");
}
```

(continues on next page)

(continued from previous page)

```

RC2: Router {
    @display("p=527.69824,703.59766");
}
RD2: Router {
    @display("p=542.81,176.545");
}
RD3: Router {
    @display("p=542.81,266.135");
}
RD4: Router {
    @display("p=646.8925,225.2925");
}
switch9: EthernetSwitch {
    @display("p=778.6425,223.97499");
}
RD1: Router {
    @display("p=442.68,225.2925");
}
switch10: EthernetSwitch {
    @display("p=542.81,72.4625");
}
host3: StandardHost {
    @display("p=776.0075,115.939995");
}
switch11: EthernetSwitch {
    @display("p=304.3425,221.34");
}
switch12: EthernetSwitch {
    @display("p=304.3425,749.6575");
}
connections:
RB2.ethg++ <--> Eth100M <--> RB4.ethg++;
RB3.ethg++ <--> Eth100M <--> RB4.ethg++;
RB1.ethg++ <--> Eth100M <--> RB2.ethg++;
RB1.ethg++ <--> Eth100M <--> RB3.ethg++;
RB1.ethg++ <--> Eth100M <--> RA4.ethg++;
RA4.ethg++ <--> Eth100M <--> RA2.ethg++;
RA3.ethg++ <--> Eth100M <--> RA4.ethg++;
RA2.ethg++ <--> Eth100M <--> RA1.ethg++;
RA1.ethg++ <--> Eth100M <--> RA3.ethg++;
switch1.ethg++ <--> Eth100M <--> RA1.ethg++;
RA2.ethg++ <--> Eth100M <--> switch2.ethg++;
RA3.ethg++ <--> Eth100M <--> switch3.ethg++;
RB2.ethg++ <--> Eth100M <--> switch4.ethg++;
RB3.ethg++ <--> Eth100M <--> switch5.ethg++;
switch1.ethg++ <--> Eth100M <--> host0.ethg++;
switch6.ethg++ <--> Eth100M <--> host1.ethg++;
RB4.ethg++ <--> Eth100M <--> switch6.ethg++;
RC2.ethg++ <--> Eth100M <--> RC4.ethg++;
RC3.ethg++ <--> Eth100M <--> RC4.ethg++;
RC1.ethg++ <--> Eth100M <--> RC2.ethg++;
RC1.ethg++ <--> Eth100M <--> RC3.ethg++;
RC3.ethg++ <--> Eth100M <--> switch8.ethg++;
switch7.ethg++ <--> Eth100M <--> host2.ethg++;
RC4.ethg++ <--> Eth100M <--> switch7.ethg++;
RC2.ethg++ <--> Eth100M <--> RB1.ethg++;

```

(continues on next page)

(continued from previous page)

```

RC2.ethg++ <--> Eth100M <--> RA4.ethg++;
RD2.ethg++ <--> Eth100M <--> RD4.ethg++;
RD3.ethg++ <--> Eth100M <--> RD4.ethg++;
RD1.ethg++ <--> Eth100M <--> RD2.ethg++;
RD1.ethg++ <--> Eth100M <--> RD3.ethg++;
switch9.ethg++ <--> Eth100M <--> host3.ethg++;
RD4.ethg++ <--> Eth100M <--> switch9.ethg++;
RB1.ethg++ <--> Eth100M <--> RD3.ethg++;
RD1.ethg++ <--> Eth100M <--> switch11.ethg++;
RD2.ethg++ <--> Eth100M <--> switch10.ethg++;
RC1.ethg++ <--> Eth100M <--> switch12.ethg++;
RA4.ethg++ <--> Eth100M <--> RD3.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step3]
description = "BGP Path Attributes"
network = BGP_Topology_1a
*.routingTableRecorder.logfile = "step3.rt"

*.pcapRecorder.pcapFile = "step3.pcap"

*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface hosts='RA4' names='eth3' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC2' names='eth3' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface hosts='RA4' names='eth4' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RD3' names='eth3' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface hosts='RB1' names='eth3' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC2' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface hosts='RD3' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth4' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface among='host0 RA*' address='10.x.x.x' ␣
↪netmask='255.x.x.x' /> \
    <interface hosts='RA*' address='10.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface among='host1 RB*' address='20.x.x.x' ␣
↪netmask='255.x.x.x' /> \

```

(continues on next page)

(continued from previous page)

```

        <interface hosts='RB*' address='20.x.x.x' netmask=
↪ '255.x.x.x' /> \
        \
        <interface among='host2 RC*' address='30.x.x.x'
↪ netmask='255.x.x.x' /> \
        <interface hosts='RC*' address='30.x.x.x' netmask=
↪ '255.x.x.x' /> \
        \
        <interface among='host3 RD*' address='40.x.x.x'
↪ netmask='255.x.x.x' /> \
        <interface hosts='RD*' address='40.x.x.x' netmask=
↪ '255.x.x.x' /> \
        \
        <route hosts='host*' destination='*' netmask='0.0.0.0
↪ ' interface='eth0' /> \
        </config>")

# OSPF configuration
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig_FullAS.xml")

*.RA4.ipv4.routingTable.routerId = "10.0.0.5"
*.RB1.ipv4.routingTable.routerId = "20.0.0.18"
*.RC2.ipv4.routingTable.routerId = "30.0.0.22"
*.RD3.ipv4.routingTable.routerId = "40.0.0.17"

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RC2.hasBgp = true
*.RD3.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCConfig_FullAS.xml")

# enable OSPF redistribution
*.RA4.bgp.redistributeOspf = "O IA"
*.RB1.bgp.redistributeOspf = "O IA"
*.RC2.bgp.redistributeOspf = "O IA"
*.RD3.bgp.redistributeOspf = "O IA"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1">
<BGPCConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
↪ "BGP.xsd">

    <TimerParams>
        <connectRetryTime> 120 </connectRetryTime>
        <holdTime> 180 </holdTime>
        <keepAliveTime> 60 </keepAliveTime>
        <startDelay> 60 </startDelay>
    </TimerParams>

    <AS id="64500">
        <!--router RA4-->
        <Router interAddr="10.0.0.1" />
    </AS>

```

(continues on next page)

(continued from previous page)

```

<AS id="64600">
  <!--router RB1-->
  <Router interAddr="20.0.0.6" />
</AS>

<AS id="64700">
  <!--router RC2-->
  <Router interAddr="30.0.0.2" />
</AS>

<AS id="64800">
  <!--router RD3-->
  <Router interAddr="40.0.0.13" />
</AS>

<!--bi-directional E-BGP session between RA4 and RB1-->
<Session id="1">
  <Router exteAddr="192.168.0.2"/>
  <Router exteAddr="192.168.0.1"/>
</Session>

<!--bi-directional E-BGP session between RA4 and RC2-->
<Session id="2">
  <Router exteAddr="192.168.0.13"/>
  <Router exteAddr="192.168.0.14"/>
</Session>

<!--bi-directional E-BGP session between RA4 and RD3-->
<Session id="3">
  <Router exteAddr="192.168.0.17"/>
  <Router exteAddr="192.168.0.18"/>
</Session>

<!--bi-directional E-BGP session between RB1 and RC2-->
<Session id="4">
  <Router exteAddr="192.168.0.5"/>
  <Router exteAddr="192.168.0.6"/>
</Session>

<!--bi-directional E-BGP session between RB1 and RD3-->
<Session id="5">
  <Router exteAddr="192.168.0.9"/>
  <Router exteAddr="192.168.0.10"/>
</Session>

</BGPConfig>

```

Results

The simulation begins with OSPF convergence within each AS. At 60 seconds, the BGP sessions are initiated.

Because the border routers are fully meshed, each router has a direct path to every other AS (distance 1) and several indirect paths (distance 2 through other border routers).

Looking at the routing tables (step3.rt), we can observe:

- Border routers prefer the direct connection because it has the shortest AS_PATH.

- For example, RA4 will install RD's networks (40.0.0.0/30 subnets) with a next hop of RD3 (via the RA4-RD3 link), ignoring the longer paths offered by RB1 or RC2.
- Due to redistribution, full reachability is achieved between all subnets in the simulation. You can verify this by checking that internal routers (e.g., RA1) eventually populate their tables with routes to the other ASes via their respective border routers.

Sources: BGP_Topology_1a.ned, omnetpp.ini, OSPFConfig_FullAS.xml, BGPCongig_FullAS.xml

6.3.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.4 Step 4. BGP Scenario with I-BGP over directly-connected BGP speakers

6.4.1 Goals

This step introduces Internal BGP (I-BGP). The topology consists of three Autonomous Systems: AS 64500 (RA), AS 64600 (RB), and AS 64700 (RC).

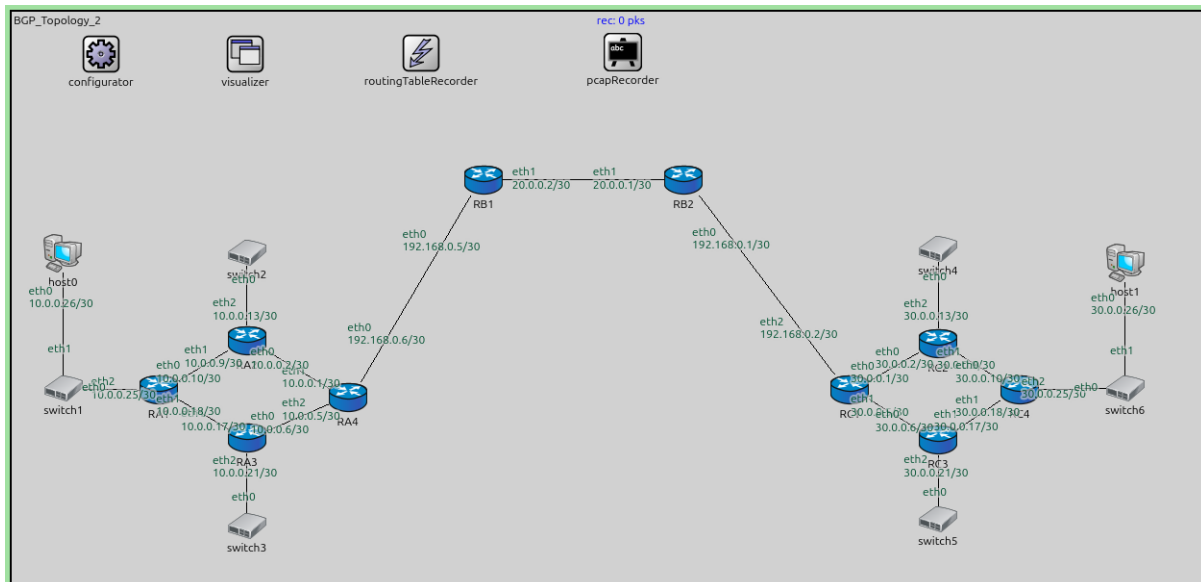
The key setup is as follows:

- **E-BGP sessions:** RA4 <-> RB1 and RB2 <-> RC1.
- **I-BGP session:** RB1 <-> RB2.
- RB1 and RB2 are directly connected via their eth1 interfaces.

The goal is to demonstrate how I-BGP is used to carry routing information across an AS. However, this step also highlights a common issue with I-BGP: the NEXT_HOP attribute behavior.

Configuration

This step uses the following network:



```
network BGP_Topology_2
{
    @display("bgb=1238.76,594.405");

    submodules:
```

(continues on next page)

(continued from previous page)

```

configurator: Ipv4NetworkConfigurator {
    @display("p=93,44");
}
visualizer: IntegratedMultiCanvasVisualizer {
    @display("p=243.2025,43.536247");
}
routingTableRecorder: RoutingTableRecorder {
    @display("p=426,43");
}
pcapRecorder: PcapRecorder {
    @display("p=636,42");
}
RB2: Router {
    @display("p=699.3,174.825");
}
RC1: Router {
    @display("p=872.46,392.94");
}
RC2: Router {
    @display("p=964.035,344.655");
}
RC3: Router {
    @display("p=964.035,444.555");
}

RB1: Router {
    @display("p=491.175,174.825");
}
RA4: Router {
    @display("p=349.79123,400.83374");
}
RA2: Router {
    @display("p=244.70375,342.285");
}
RA3: Router {
    @display("p=244.70375,442.86874");
}

RA1: Router {
    @display("p=153.1275,393.32748");
}
RC4: Router {
    @display("p=1048.95,392.94");
}
switch1: EthernetSwitch {
    @display("p=54.045,391.82623");
}
switch2: EthernetSwitch {
    @display("p=244.70375,250.70874");
}
switch3: EthernetSwitch {
    @display("p=244.70375,534.445");
}
switch4: EthernetSwitch {
    @display("p=964.035,246.42");
}

```

(continues on next page)

(continued from previous page)

```

switch5: EthernetSwitch {
    @display("p=964.035,527.805");
}
switch6: EthernetSwitch {
    @display("p=1158.84,391.275");
}
host0: StandardHost {
    @display("p=53.28,251.415");
}
host1: StandardHost {
    @display("p=1158.84,261.405");
}
connections:
RC1.ethg++ <--> Eth100M <--> RC2.ethg++;
RC1.ethg++ <--> Eth100M <--> RC3.ethg++;
RB2.ethg++ <--> Eth100M <--> RC1.ethg++;
RB1.ethg++ <--> Eth100M <--> RA4.ethg++;
RA4.ethg++ <--> Eth100M <--> RA2.ethg++;
RA3.ethg++ <--> Eth100M <--> RA4.ethg++;
RA2.ethg++ <--> Eth100M <--> RA1.ethg++;
RA1.ethg++ <--> Eth100M <--> RA3.ethg++;
RC4.ethg++ <--> Eth100M <--> RC2.ethg++;
RC3.ethg++ <--> Eth100M <--> RC4.ethg++;
switch1.ethg++ <--> Eth100M <--> RA1.ethg++;
RA2.ethg++ <--> Eth100M <--> switch2.ethg++;
RA3.ethg++ <--> Eth100M <--> switch3.ethg++;
RC2.ethg++ <--> Eth100M <--> switch4.ethg++;
RC3.ethg++ <--> Eth100M <--> switch5.ethg++;
RC4.ethg++ <--> Eth100M <--> switch6.ethg++;
switch1.ethg++ <--> Eth100M <--> host0.ethg++;
switch6.ethg++ <--> Eth100M <--> host1.ethg++;
RB2.ethg++ <--> Eth100M <--> RB1.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step4]
description = "BGP Scenario with I-BGP over directly-connected BGP speakers"
network = BGP_Topology_2
*.routingTableRecorder.logfile = "step4.rt"

*.pcapRecorder.pcapFile = "step4.pcap"

# adding default routes in RA4 and RC1 and ask OSPF to distribute it within the AS
*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RB1' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    \
    <interface hosts='RB2' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RC1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    \
    <interface among='host0 RA*' address='10.x.x.x'>

```

(continues on next page)

(continued from previous page)

```

↪netmask='255.x.x.x' /> \
                                <interface hosts='RA*' address='10.x.x.x' netmask=
↪'255.x.x.x' /> \
                                \
                                <interface among='RB1 RB2' address='20.x.x.x' netmask=
↪'255.x.x.x' /> \
                                \
                                <interface among='host1 RC*' address='30.x.x.x'
↪netmask='255.x.x.x' /> \
                                <interface hosts='RC*' address='30.x.x.x' netmask=
↪'255.x.x.x' /> \
                                \
                                <route hosts='RA4' destination='*' netmask='0.0.0.0'
↪interface='eth0' /> \
                                <route hosts='RC1' destination='*' netmask='0.0.0.0'
↪interface='eth2' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0'
↪' interface='eth0' /> \
                                </config>")

# OSPF configuration
*.RB{1,2}.hasOspf = false
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig_IBGP.xml")

*.RA4.ipv4.routingTable.routerId = "10.0.0.5"
*.RC1.ipv4.routingTable.routerId = "30.0.0.5"

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB2.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_IBGP.xml")

# enable OSPF redistribution
*.RA4.bgp.redistributeOspf = "O IA"
*.RC1.bgp.redistributeOspf = "O IA"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPCfg xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
↪"BGP.xsd">

  <TimerParams>
    <connectRetryTime> 120 </connectRetryTime>
    <holdTime> 180 </holdTime>
    <keepAliveTime> 60 </keepAliveTime>
    <startDelay> 60 </startDelay>
  </TimerParams>

  <AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1" />
  </AS>

```

(continues on next page)

(continued from previous page)

```

<AS id="64600">
  <!--router RB1-->
  <Router interAddr="20.0.0.2" >
    <Network address='20.0.0.0' />
  </Router>

  <!--router RB2-->
  <Router interAddr="20.0.0.1" >
    <Network address='20.0.0.0' />
  </Router>
</AS>

<AS id="64700">
  <!--router RC1-->
  <Router interAddr="30.0.0.1" />
</AS>

<!--bi-directional E-BGP session between RA4 and RB1-->
<Session id="1">
  <Router exteAddr="192.168.0.6"/>
  <Router exteAddr="192.168.0.5"/>
</Session>

<!--bi-directional E-BGP session between RB2 and RC1-->
<Session id="2">
  <Router exteAddr="192.168.0.1"/>
  <Router exteAddr="192.168.0.2"/>
</Session>

</BGPCfg>

```

Results

Running the simulation reveals an important BGP constraint. When an E-BGP router (like RB1) learns a route and advertises it to an I-BGP peer (like RB2), it does not change the NEXT_HOP attribute of the route by default.

Observations in `step4.rt`:

- RB1 learns RA's network (10.0.0.0/30) and correctly installs it with RA4 as the next hop.
- RB1 passes this advertisement to RB2 via their I-BGP session.
- **RB2 receives the route but does not install it.**

The reason for this is that the next hop for the RA routes remains RA4's interface address (192.168.0.6). Since RB2 belongs to AS 64600 and has no interior route to the RA4-RB1 external subnet, the next hop is considered unreachable. For a BGP route to be installed in the IP routing table, its next hop must be resolvable via the existing routing table.

Sources: `BGP_Topology_2.ned`, `omnetpp.ini`, `OSPFCfg_IBGP.xml`, `BGPCfg_IBGP.xml`

6.4.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.5 Step 4a. Enable nextHopSelf on RB1 and RB2

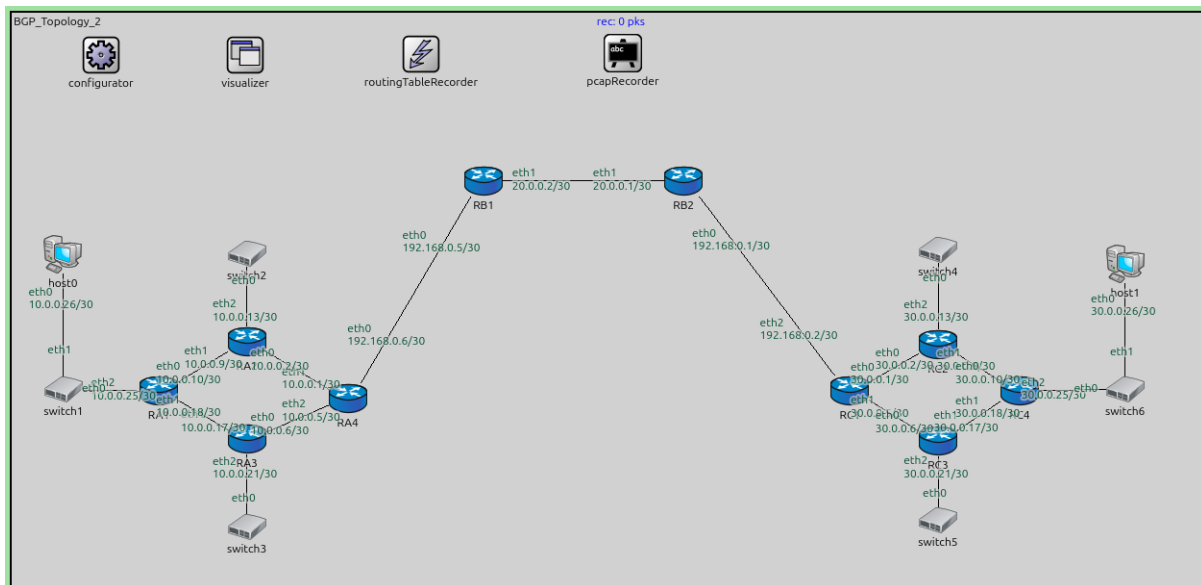
6.5.1 Goals

Step 4a provides a solution to the unreachable next hop issue observed in Step 4. Within AS 64600, the border routers RB1 and RB2 are configured with the `nextHopSelf` attribute enabled for their I-BGP session.

The goal of this configuration is to ensure that when a border router advertises a route learned from an external peer (E-BGP) to its internal peer (I-BGP), it updates the `NEXT_HOP` attribute to its own address. This makes the route reachable for the internal peer, provided there is internal connectivity (IGP or direct link) between them.

Configuration

This step uses the same network as Step 4 (BGP_Topology_2.ned).



The configuration in `omnetpp.ini` enables the feature:

```
*.RA4.bgp.nextHopSelf = true
*.RB1.bgp.nextHopSelf = true
*.RB2.bgp.nextHopSelf = true
*.RC1.bgp.nextHopSelf = true
```

The BGP configuration uses a specific XML file:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPCfg xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
  ↪ "BGP.xsd">

  <TimerParams>
    <connectRetryTime> 120 </connectRetryTime>
    <holdTime> 180 </holdTime>
    <keepAliveTime> 60 </keepAliveTime>
    <startDelay> 60 </startDelay>
  </TimerParams>

  <AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1" />
  </AS>
```

(continues on next page)

(continued from previous page)

```

<AS id="64600">
  <!--router RB1-->
  <Router interAddr="20.0.0.2" >
    <Network address='20.0.0.0' />
    <Neighbor address='20.0.0.1' nextHopSelf='true' />
  </Router>

  <!--router RB2-->
  <Router interAddr="20.0.0.1" >
    <Network address='20.0.0.0' />
    <Neighbor address='20.0.0.2' nextHopSelf='true' />
  </Router>
</AS>

<AS id="64700">
  <!--router RC1-->
  <Router interAddr="30.0.0.1" />
</AS>

<!--bi-directional E-BGP session between RA4 and RB1-->
<Session id="1">
  <Router exterAddr="192.168.0.6"/>
  <Router exterAddr="192.168.0.5"/>
</Session>

<!--bi-directional E-BGP session between RB2 and RC1-->
<Session id="2">
  <Router exterAddr="192.168.0.1"/>
  <Router exterAddr="192.168.0.2"/>
</Session>

</BGPCfg>

```

Results

With `nextHopSelf` enabled, the BGP behavior changes as follows:

1. RB1 receives the RA network (10.0.0.0/30) route from RA4.
2. When RB1 advertises this route to RB2 via I-BGP, it now replaces RA4's interface address with its own address (the address of RB1's interface on the link to RB2).
3. RB2 receives the advertisement. Since RB1 is directly connected, RB2 can resolve the next hop and successfully installs the route into its IP routing table.

Checking `step4a.rt`, we can see the successful installation:

- RB2 now has routes to RA's networks (10.0.0.x) with the next hop set to RB1 (20.0.0.2).
- Similarly, RB1 now has routes to RC's networks (30.0.0.x) with the next hop set to RB2 (20.0.0.1).

Full bi-directional reachability across the transit AS 64600 is now achieved.

Sources: `BGP_Topology_2.ned`, `omnetpp.ini`, `BGPConfig_IBGP_NextHopSelf.xml`

6.5.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.6 Step 5. BGP Scenario with I-BGP over not directly-connected BGP speakers

6.6.1 Goals

This step explores Internal BGP (I-BGP) in a more realistic scenario where I-BGP peers are not directly connected. The topology (`BGP_Topology_3.ned`) features a transit Autonomous System, AS 64600 (RB), consisting of three routers: RB1, RB3, and RB2.

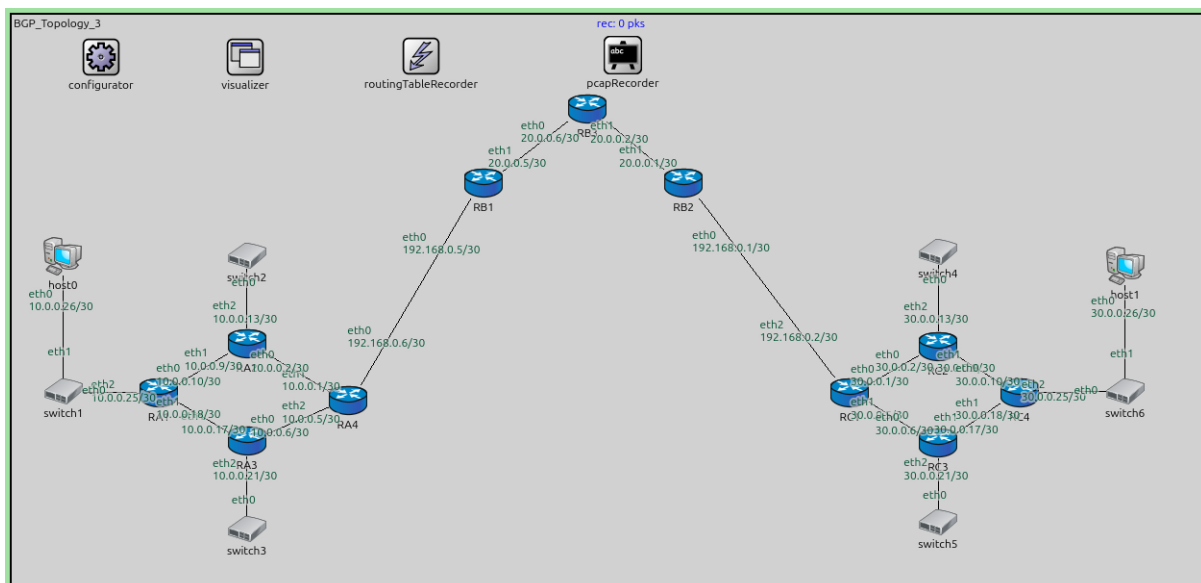
The setup is as follows:

- **RB1 and RB2** are border routers running BGP. They are I-BGP peers.
- **RB3** is an interior router that **does not run BGP**.
- The routers are connected in a chain: RB1 <-> RB3 <-> RB2.
- OSPF is running on all routers within AS 64600 to provide internal reachability.

The primary goal is to demonstrate the “BGP Hole” or “BGP Synchronization” problem. While RB1 and RB2 can establish an I-BGP session over the multi-hop OSPF path, the interior router RB3 remains unaware of the external routes learned via BGP.

Configuration

This step uses the following network:



```
network BGP_Topology_3
{
  @display("bgp=1238.76,594.405");

  submodules:
    configurator: Ipv4NetworkConfigurator {
      @display("p=93,44");
    }
    visualizer: IntegratedMultiCanvasVisualizer {
      @display("p=243.2025,43.536247");
    }
}
```

(continues on next page)

(continued from previous page)

```

}
routingTableRecorder: RoutingTableRecorder {
    @display("p=426,43");
}
pcapRecorder: PcapRecorder {
    @display("p=636,42");
}
RB2: Router {
    @display("p=699.3,174.825");
}
RC1: Router {
    @display("p=872.46,392.94");
}
RC2: Router {
    @display("p=964.035,344.655");
}
RC3: Router {
    @display("p=964.035,444.555");
}

RB1: Router {
    @display("p=491.175,174.825");
}
RA4: Router {
    @display("p=349.79123,400.83374");
}
RA2: Router {
    @display("p=244.70375,342.285");
}
RA3: Router {
    @display("p=244.70375,442.86874");
}

RA1: Router {
    @display("p=153.1275,393.32748");
}
RC4: Router {
    @display("p=1048.95,392.94");
}
switch1: EthernetSwitch {
    @display("p=54.045,391.82623");
}
switch2: EthernetSwitch {
    @display("p=244.70375,250.70874");
}
switch3: EthernetSwitch {
    @display("p=244.70375,534.445");
}
switch4: EthernetSwitch {
    @display("p=964.035,246.42");
}
switch5: EthernetSwitch {
    @display("p=964.035,527.805");
}
switch6: EthernetSwitch {
    @display("p=1158.84,391.275");
}

```

(continues on next page)

(continued from previous page)

```

    }
    host0: StandardHost {
        @display("p=53.28,251.415");
    }
    host1: StandardHost {
        @display("p=1158.84,261.405");
    }
    RB3: Router {
        @display("p=599.265,97.8075");
    }
    connections:
    RC1.ethg++ <--> Eth100M <--> RC2.ethg++;
    RC1.ethg++ <--> Eth100M <--> RC3.ethg++;
    RB2.ethg++ <--> Eth100M <--> RC1.ethg++;
    RB1.ethg++ <--> Eth100M <--> RA4.ethg++;
    RA4.ethg++ <--> Eth100M <--> RA2.ethg++;
    RA3.ethg++ <--> Eth100M <--> RA4.ethg++;
    RA2.ethg++ <--> Eth100M <--> RA1.ethg++;
    RA1.ethg++ <--> Eth100M <--> RA3.ethg++;
    RC4.ethg++ <--> Eth100M <--> RC2.ethg++;
    RC3.ethg++ <--> Eth100M <--> RC4.ethg++;
    switch1.ethg++ <--> Eth100M <--> RA1.ethg++;
    RA2.ethg++ <--> Eth100M <--> switch2.ethg++;
    RA3.ethg++ <--> Eth100M <--> switch3.ethg++;
    RC2.ethg++ <--> Eth100M <--> switch4.ethg++;
    RC3.ethg++ <--> Eth100M <--> switch5.ethg++;
    RC4.ethg++ <--> Eth100M <--> switch6.ethg++;
    switch1.ethg++ <--> Eth100M <--> host0.ethg++;
    switch6.ethg++ <--> Eth100M <--> host1.ethg++;
    RB1.ethg++ <--> Eth100M <--> RB3.ethg++;
    RB3.ethg++ <--> Eth100M <--> RB2.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step5]
description = "BGP Scenario with I-BGP over not directly-connected BGP speakers"
network = BGP_Topology_3
*.routingTableRecorder.logfile = "step5.rt"

*.pcapRecorder.pcapFile = "step5.pcap"

# adding default routes in RA4 and RC1 and ask OSPF to distribute it within the AS
*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RB1' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    \
    <interface hosts='RB2' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RC1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x'/> \
    \
    <interface among='host0 RA*' address='10.x.x.x'
↪netmask='255.x.x.x'/> \

```

(continues on next page)

(continued from previous page)

```

        <interface hosts='RA*' address='10.x.x.x' netmask=
↪ '255.x.x.x' /> \
        \
        <interface among='RB1 RB2 RB3' address='20.x.x.x'
↪ netmask='255.x.x.x' /> \
        \
        <interface among='host1 RC*' address='30.x.x.x'
↪ netmask='255.x.x.x' /> \
        <interface hosts='RC*' address='30.x.x.x' netmask=
↪ '255.x.x.x' /> \
        \
        <route hosts='RA4' destination='*' netmask='0.0.0.0'
↪ interface='eth0' /> \
        <route hosts='RC1' destination='*' netmask='0.0.0.0'
↪ interface='eth2' /> \
        <route hosts='host*' destination='*' netmask='0.0.0.0'
↪ ' interface='eth0' /> \
        </config>")

# OSPF configuration
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig_Multi.xml")

*.RA4.ipv4.routingTable.routerId = "10.0.0.5"
*.RC1.ipv4.routingTable.routerId = "30.0.0.5"
*.RB1.ipv4.routingTable.routerId = "20.0.0.5"
*.RB2.ipv4.routingTable.routerId = "20.0.0.1"

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB2.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPConfig_Multi.xml")

# enable OSPF redistribution
*.RA4.bgp.redistributeOspf = "O IA"
*.RC1.bgp.redistributeOspf = "O IA"
*.RB1.bgp.redistributeOspf = "O IA"
*.RB2.bgp.redistributeOspf = "O IA"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
↪ "BGP.xsd">

    <TimerParams>
        <connectRetryTime> 120 </connectRetryTime>
        <holdTime> 180 </holdTime>
        <keepAliveTime> 60 </keepAliveTime>
        <startDelay> 60 </startDelay>
    </TimerParams>

    <AS id="64500">
        <!--router RA4-->

```

(continues on next page)

(continued from previous page)

```

    <Router interAddr="10.0.0.1" />
</AS>

<AS id="64600">
    <!--router RB1-->
    <Router interAddr="20.0.0.5" >
        <Neighbor address='20.0.0.1' nextHopSelf='true' />
    </Router>

    <!--router RB2-->
    <Router interAddr="20.0.0.1" >
        <Neighbor address='20.0.0.5' nextHopSelf='true' />
    </Router>
</AS>

<AS id="64700">
    <!--router RC1-->
    <Router interAddr="30.0.0.1" />
</AS>

<!--bi-directional E-BGP session between RA4 and RB1-->
<Session id="1">
    <Router exteAddr="192.168.0.6"/>
    <Router exteAddr="192.168.0.5"/>
</Session>

<!--bi-directional E-BGP session between RB2 and RC1-->
<Session id="2">
    <Router exteAddr="192.168.0.1"/>
    <Router exteAddr="192.168.0.2"/>
</Session>

</BGPCfg>

```

Results

In this simulation, the following happens:

1. OSPF converges, allowing RB1 and RB2 to ping each other's interface addresses via RB3.
2. RB1 and RB2 establish an I-BGP session.
3. RB1 learns RA's networks (10.0.0.x) from RA4 and advertises them to RB2.
4. RB2 learns RC's networks (30.0.0.x) from RC1 and advertises them to RB1.
5. **The Problem:** While RB1 and RB2 have these routes in their BGP tables, the transit router **RB3 has no knowledge of these external networks.**

If you were to trace a packet from AS 64500 to AS 64700:

- RA4 sends it to RB1.
- RB1 knows the destination is reachable via RB2 and sends it towards RB3.
- RB3 receives the packet, looks up the destination (e.g., 30.0.0.26), finds no match in its OSPF-only routing table, and drops the packet.

This demonstrates that simply having I-BGP peering between border routers is insufficient for a transit AS if the intermediate routers are not running BGP or receiving redistributed routes.

Sources: BGP_Topology_3.ned, omnetpp.ini, OSPFConfig_Multi.xml BGPConfig_Multi.xml

Results

With internal redistribution enabled, we can observe the following in `step5a.rt`:

- RB1 and RB2 continue to exchange routes via I-BGP.
- RB1 redistributes the routes it learns from RA into OSPF.
- **RB3 receives these routes via OSPF.** Check RB3's routing table (20.0.0.6) and you will see entries for the 10.0.0.x and 30.0.0.x networks.
- Reachability is now established: packets from AS 64500 to AS 64700 can successfully transit RB3 because RB3 now has OSPF routes to those external subnets.

While this approach works for small networks, it is generally discouraged in large-scale internet routing because it can overwhelm the IGP with a massive number of external routes.

Sources: `BGP_Topology_3.ned`, `omnetpp.ini`, `OSPFConfig_Multi.xml`, `BGPConfig_Multi.xml`

6.7.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.8 Step 5b. Enabling BGP on RB3

6.8.1 Goals

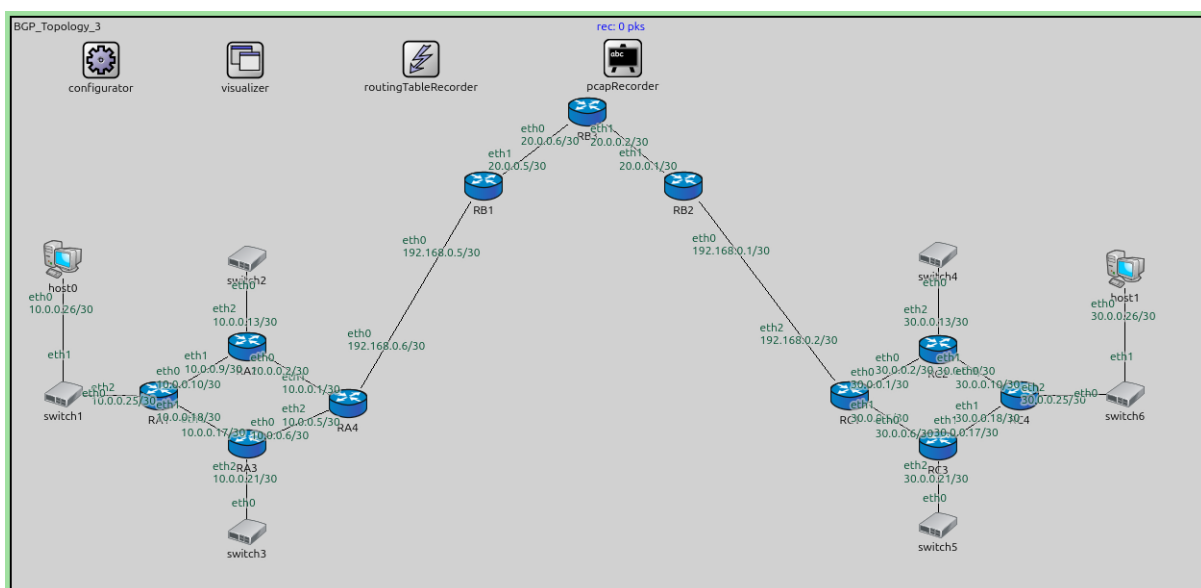
Step 5b demonstrates the second (and preferred) solution to the transit reachability problem: running BGP on all routers within the transit AS (Full-mesh I-BGP).

In this scenario:

- **RB3** is now configured to run BGP.
- BGP sessions are established between all pairs of routers in AS 64600 (RB1-RB2, RB1-RB3, RB2-RB3).
- Every router learns external routes directly via BGP, eliminating the need to redistribute internal BGP routes into OSPF.

Configuration

This step extends Step 5.



The configuration in `omnetpp.ini` enables BGP on RB3:

```
*.RB3.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_Multi_RB3.xml")
```

The configuration in `omnetpp.ini` is the following:

```
[Config Step5b]
description = "Enabling BGP on RB3"
extends = Step5
*.routingTableRecorder.logfile = "step5b.rt"

*.pcapRecorder.pcapFile = "step5b.pcap"

# enable BGP on RB3
*.RB3.hasBgp = true

*.R*.bgp.bgpConfig = xmldoc("BGPCfg_Multi_RB3.xml")
```

The BGP configuration:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPCfg xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
  ↪ "BGP.xsd">

  <TimerParams>
    <connectRetryTime> 120 </connectRetryTime>
    <holdTime> 180 </holdTime>
    <keepAliveTime> 60 </keepAliveTime>
    <startDelay> 60 </startDelay>
  </TimerParams>

  <AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1" />
  </AS>

  <AS id="64600">
    <!--router RB1-->
    <Router interAddr="20.0.0.5" >
      <Neighbor address='20.0.0.1' nextHopSelf='true' />
      <Neighbor address='20.0.0.6' nextHopSelf='true' />
    </Router>

    <!--router RB2-->
    <Router interAddr="20.0.0.1" >
      <Neighbor address='20.0.0.5' nextHopSelf='true' />
      <Neighbor address='20.0.0.6' nextHopSelf='true' />
    </Router>

    <!--router RB3-->
    <Router interAddr="20.0.0.6" />
  </AS>

  <AS id="64700">
    <!--router RC1-->
    <Router interAddr="30.0.0.1" />
  </AS>
```

(continues on next page)

(continued from previous page)

```

<!--bi-directional E-BGP session between RA4 and RB1-->
<Session id="1">
  <Router exteAddr="192.168.0.6"/>
  <Router exteAddr="192.168.0.5"/>
</Session>

<!--bi-directional E-BGP session between RB2 and RC1-->
<Session id="2">
  <Router exteAddr="192.168.0.1"/>
  <Router exteAddr="192.168.0.2"/>
</Session>

</BGPConfig>

```

Results

In this simulation, the “BGP hole” is resolved through a full-mesh I-BGP configuration:

1. **BGP Adjacencies:** RB3 establishes I-BGP sessions with both RB1 and RB2.
2. **Direct Route Learning:** Unlike Step 5, where RB3 was unaware of external routes, it now receives BGP UPDATE messages from RB1 (about RA’s networks) and RB2 (about RC’s networks).
3. **Successful Transit:** Because RB3 has these routes in its BGP table, it can successfully forward packets between RB1 and RB2.

This demonstrates the standard approach in large networks: every router in the transit path between BGP border routers should also run BGP to ensure proper packet forwarding without overwhelming the IGP with external routes.

Sources: BGP_Topology_3.ned, omnetpp.ini, BGPConfig_Multi_RB3.xml

6.8.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.9 Step 6. BGP Scenario and using loopbacks

6.9.1 Goals

Step 6 introduces a more complex, three-AS hierarchical topology (BGP_Topology_4.ned) and demonstrates a complete BGP-based transit scenario. This step focuses on establishing a full-mesh I-BGP configuration within the transit Autonomous System (AS 64600) and using BGP to provide end-to-end connectivity between external ASes.

The network consists of:

- **AS 64500 (RA):** A multi-router AS running OSPF internally. RA4 is the border router.
- **AS 64600 (RB):** The transit AS. It contains four routers (RB1, RB2, RB3, RB4) running OSPF. RB1 and RB4 are border routers.
- **AS 64700 (RC):** Another multi-router AS running OSPF. RC1 is the border router.

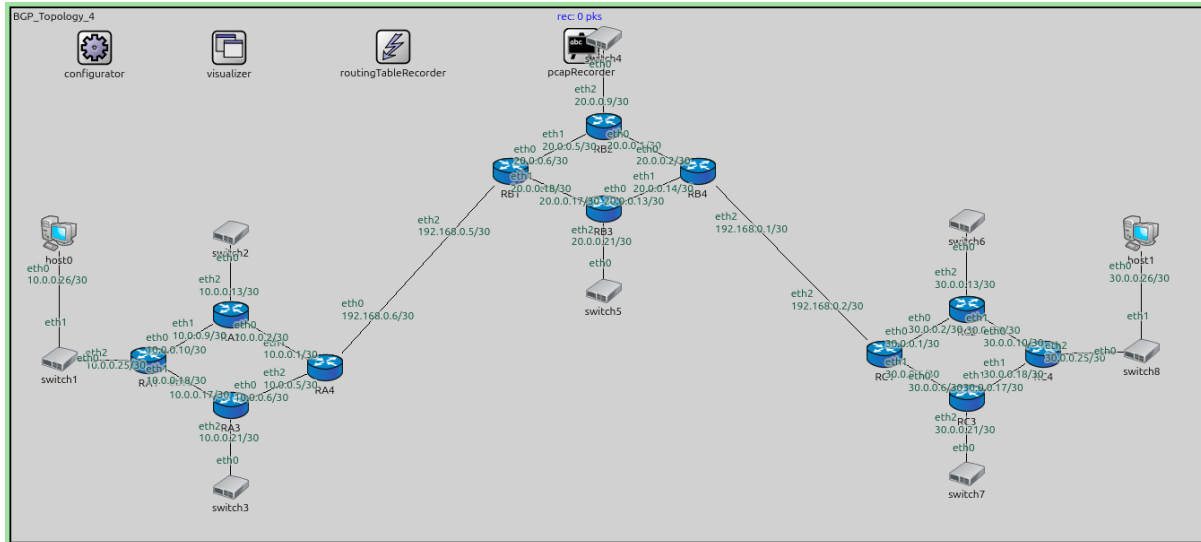
Key features demonstrated:

- **E-BGP Peering:** RA4 peers with RB1, and RB4 peers with RC1.
- **Full-Mesh I-BGP:** Within AS 64600, all routers (RB1, RB2, RB3, RB4) run BGP and are configured as I-BGP peers.
- **Multi-hop I-BGP:** Note that some I-BGP sessions (e.g., RB1 to RB4) are not directly connected. They rely on the IGP (OSPF) to provide reachability to the peering addresses.

- **Reachability:** The network is configured to allow hosts in AS 64500 (host0) to reach hosts in AS 64700 (host1).

Configuration

This step uses the BGP_Topology_4.ned network, which includes three distinct ASes with several routers and hosts.



The configuration in `omnetpp.ini`:

[Config Step6]

```
description = "BGP Scenario and using loopbacks"
network = BGP_Topology_4
*.routingTableRecorder.logfile = "step6.rt"

*.pcapRecorder.pcapFile = "step6.pcap"

# adding default routes in RA4 and RC1 and ask OSPF to distribute it within the AS
*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168. \
    ↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RB1' names='eth2' address='192.168. \
    ↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RB4' names='eth2' address='192.168. \
    ↪x.x' netmask='255.x.x.x'/> \
    <interface hosts='RC1' names='eth2' address='192.168. \
    ↪x.x' netmask='255.x.x.x'/> \
    \
    <interface among='host0 RA*' address='10.x.x.x' \
    ↪netmask='255.x.x.x'/> \
    <interface hosts='RA*' address='10.x.x.x' netmask= \
    ↪'255.x.x.x'/> \
    \
    <interface hosts='RB*' address='20.x.x.x' netmask= \
    ↪'255.x.x.x'/> \
    \
    <interface among='host1 RC*' address='30.x.x.x' \
    ↪netmask='255.x.x.x'/> \
    <interface hosts='RC*' address='30.x.x.x' netmask= \
    ↪'255.x.x.x'/> \
    \
    </config>")
```

(continues on next page)

(continued from previous page)

```

\
<route hosts='host*' destination='*' netmask='0.0.0.0
↪ ' interface='eth0' /> \
\
<route hosts='RA4' destination='*' netmask='0.0.0.0'
↪ interface='eth0' /> \
<route hosts='RC1' destination='*' netmask='0.0.0.0'
↪ interface='eth2' /> \
</config>")

# OSPF configuration
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig.xml")

*.RA4.ipv4.routingTable.routerId = "10.0.0.5"
*.RC1.ipv4.routingTable.routerId = "30.0.0.5"
*.RB1.ipv4.routingTable.routerId = "20.0.0.18"
*.RB4.ipv4.routingTable.routerId = "20.0.0.14"

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB4.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_Redist.xml")
*.R*.bgp.redistributeOspf = "O IA"

# BGP routes are distributed internally within the AS
*.*.RB1.bgp.redistributeInternal = true
*.*.RB4.bgp.redistributeInternal = true

# enable BGP on RB2 and RB3
*.RB2.hasBgp = true
*.RB3.hasBgp = true

```

The BGP configuration (BGPCfg_Redist.xml) defines the ASes and the full-mesh sessions within AS 64600. It also uses nextHopSelf="true" to ensure I-BGP peers can reach external next hops.

Results

In the simulation, we observe the convergence of the entire network:

1. **IGP Convergence:** OSPF converges within each AS, providing internal reachability. In AS 64600, this allows multi-hop BGP sessions (like RB1-RB4) to establish.
2. **BGP Session Establishment:**
 - EBGp sessions are established: RA4 <-> RB1 and RB4 <-> RC1.
 - A full mesh of IBGP sessions is established among RB1, RB2, RB3, and RB4.
3. **Route Propagation:**
 - RB1 learns RA's networks (10.x.x.x) and redistributes them into the RB mesh.
 - RB4 learns RC's networks (30.x.x.x) and redistributes them into the RB mesh.
 - Due to the full mesh, every router in AS 64600 learns the path to both external ASes.
4. **End-to-End Reachability:** step6.rt shows that all routers eventually possess routes to all subnets across all three ASes. Host0 can successfully communicate with Host1 through the transit AS 64600.

This step validates the use of a BGP full mesh within a transit AS as a robust solution for internet-scale routing.

Sources: BGP_Topology_4.ned, omnetpp.ini, OSPFConfig.xml BGPConfig_Redist.xml

6.9.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.10 Step 7. BGP with RIP redistribution

6.10.1 Goals

Step 7 demonstrates BGP route redistribution from the Routing Information Protocol (RIP). While OSPF is a common IGP in modern networks, BGP is flexible enough to interact with several other routing protocols, including RIP.

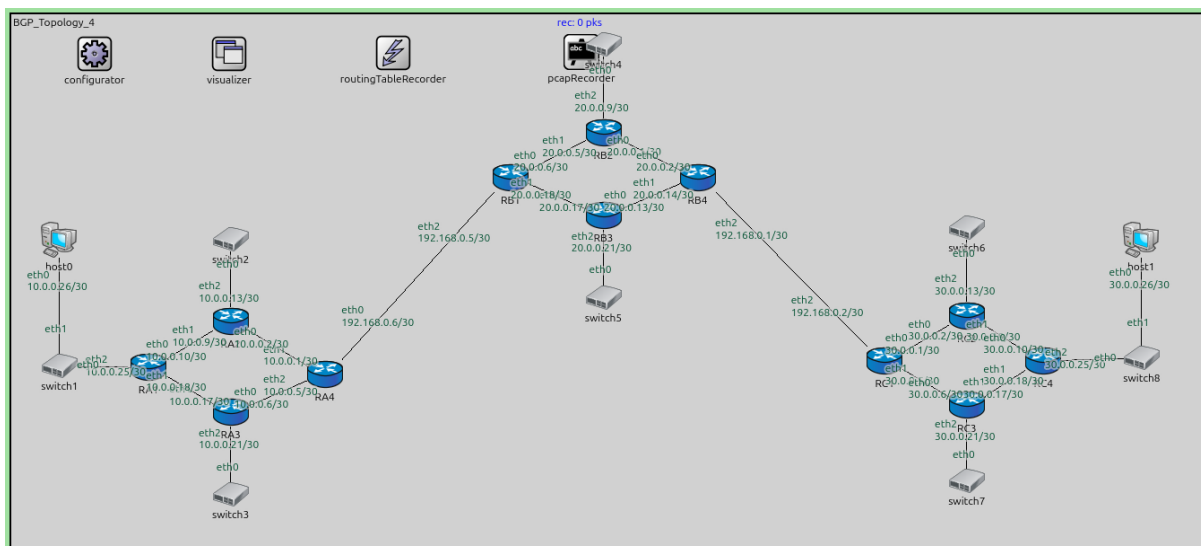
In this scenario (BGP_Topology_4.ned), each Autonomous System uses RIP as its internal routing protocol instead of OSPF. The border routers (RA4, RB1, RB4, RC1) are configured to take the routes they learn from RIP and advertise them to their BGP peers.

Key features demonstrated:

- **RIP as an IGP:** All routers within each AS run RIP to discover internal subnets.
- **RIP-to-BGP Redistribution:** The `redistributeRip` parameter is set to `true` in the BGP configuration, allowing border routers to inject RIP-learned routes into the BGP table for advertisement to external ASes.
- **Multiprotocol Interworking:** Verifying that end-to-end connectivity is maintained when BGP is used to bridge multiple RIP-based domains.

Configuration

The topology remains the same as in Step 6 (BGP_Topology_4.ned).



The configuration in `omnetpp.ini` enables RIP and disables OSPF:

```
[Config Step7]
description = "BGP with RIP redistribution"
network = BGP_Topology_4
*.routingTableRecorder.logfile = "step7.rt"

*.pcapRecorder.pcapFile = "step7.pcap"
```

(continues on next page)

```

*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB4' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface among='host0 RA*' address='10.x.x.x' _
↪netmask='255.x.x.x' /> \
    <interface hosts='RA*' address='10.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface hosts='RB*' address='20.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface among='host1 RC*' address='30.x.x.x' _
↪netmask='255.x.x.x' /> \
    <interface hosts='RC*' address='30.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
    \
    <route hosts='RA4' destination='*' netmask='0.0.0.0' _
↪interface='eth0' /> \
    <route hosts='RC1' destination='*' netmask='0.0.0.0' _
↪interface='eth2' /> \
    </config>")

# RIP configuration
*.R*.hasRip = true
*.R*.rip.ripConfig = xmldoc("RIPConfig.xml")

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB4.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_Redist.xml")
*.R*.bgp.redistributeRip = true

# enable BGP on RB2 and RB3
*.RB2.hasBgp = true
*.RB3.hasBgp = true

```

The BGP configuration uses `redistributeRip = true`:

```
*.R*.bgp.redistributeRip = true
```


Results

The simulation results in `step7.rt` confirm that RIP-learned routes successfully propagate via BGP:

1. **RIP Convergence:** Routers within each AS exchange RIP updates. For example, in AS 64700, RC1 learns about the networks connected to RC2, RC3, and RC4 via RIP.
2. **Redistribution:** RC1 takes these RIP routes (30.0.0.8/30, 30.0.0.12/30, etc.) and advertises them to its E-BGP peer, RB4.
3. **BGP Propagation:** RB4 receives these routes and advertises them via I-BGP to RB1, who then advertises them via E-BGP to RA4.
4. **End-to-End Connectivity:** Like in previous steps, all routers eventually learn routes to all possible destinations in the network. Routing table entries for 30.x.x.x networks appear in AS 64500 routers, and vice-versa, with BGP serving as the conduit.

This step shows that BGP's redistribution mechanism is protocol-agnostic, making it a powerful tool for connecting diverse network islands.

Sources: `BGP_Topology_4.ned`, `omnetpp.ini`, `RIPConfig.xml`, `BGPConfig_Redist.xml`

6.10.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.11 Step 8. BGP with OSPF and RIP redistribution

6.11.1 Goals

Step 8 demonstrates BGP's ability to act as a unified routing plane over a heterogeneous internal landscape. In this scenario (`BGP_Topology_4.ned`), different Autonomous Systems use different IGPs, and BGP is responsible for providing seamless connectivity between them.

The setup is as follows:

- **AS 64500 (RA):** Uses **RIP** as its internal routing protocol.
- **AS 64700 (RC):** Also uses **RIP** internally.
- **AS 64600 (RB):** Uses **OSPF** as its internal routing protocol.

Key features demonstrated:

- **Hybrid Redistribution:** Border routers are configured to redistribute routes from both RIP and OSPF into BGP.
- **Protocol Interworking:** BGP carries routes from a RIP-based domain, across an OSPF-based transit domain, to another RIP-based domain.
- **Unified Reachability:** Demonstrating that end-to-end connectivity (e.g., host0 to host1) is independent of the specific IGPs used within each AS, provided BGP is correctly configured.

Configuration

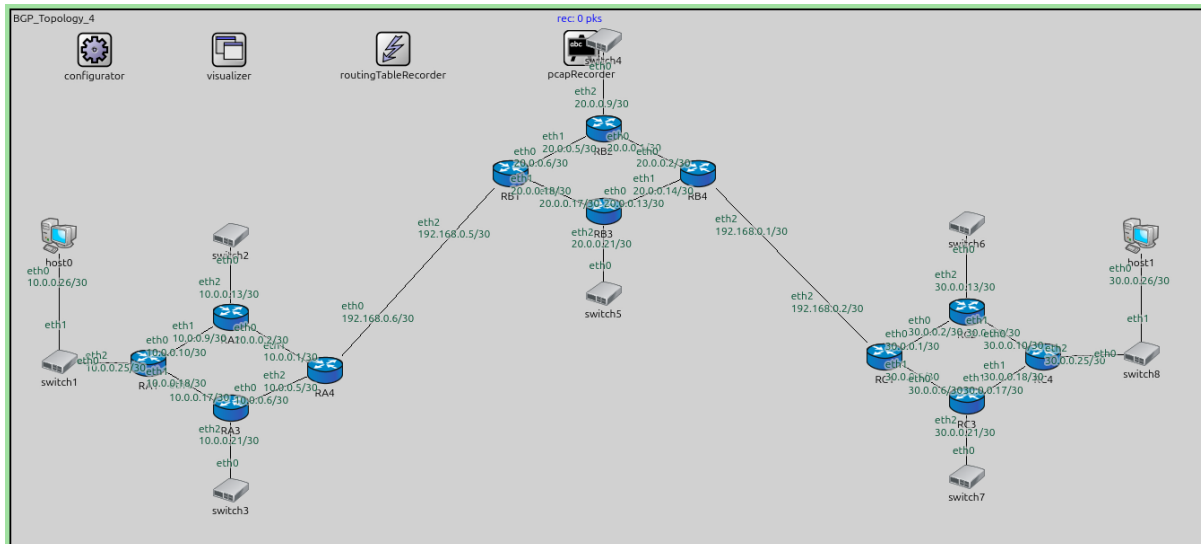
The topology is the same as in Step 6 (`BGP_Topology_4.ned`).

The configuration in `omnetpp.ini` defines the protocol mix:

```
[Config Step8]
description = "BGP with OSPF and RIP redistribution"
network = BGP_Topology_4
*.routingTableRecorder.logfile = "step8.rt"

*.pcapRecorder.pcapFile = "step8.pcap"
```

(continues on next page)



(continued from previous page)

```
# adding default routes in RA4 and RC1. RIP and OSPF will distribute it within the AS
*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
    ↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth2' address='192.168.
    ↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB4' names='eth2' address='192.168.
    ↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC1' names='eth2' address='192.168.
    ↪x.x' netmask='255.x.x.x' /> \
    \
    <interface among='host0 RA*' address='10.x.x.x' \
    ↪netmask='255.x.x.x' /> \
    <interface hosts='RA*' address='10.x.x.x' netmask=
    ↪'255.x.x.x' /> \
    \
    <interface hosts='RB*' address='20.x.x.x' netmask=
    ↪'255.x.x.x' /> \
    \
    <interface among='host1 RC*' address='30.x.x.x' \
    ↪netmask='255.x.x.x' /> \
    <interface hosts='RC*' address='30.x.x.x' netmask=
    ↪'255.x.x.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
    ↪' interface='eth0' /> \
    \
    <route hosts='RA4' destination='*' netmask='0.0.0.0' \
    ↪interface='eth0' /> \
    <route hosts='RC1' destination='*' netmask='0.0.0.0' \
    ↪interface='eth2' /> \
    </config>")

# RIP configuration
*.RA*.hasRip = true
*.RC*.hasRip = true
```

(continues on next page)

(continued from previous page)

```

*.R*.rip.ripConfig = xmldoc("RIPConfig.xml")

# OSPF configuration
*.RB*.hasOspf = true
*.RB*.ospf.ospfConfig = xmldoc("OSPFConfig.xml")

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB4.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPConfig_Redist.xml")
*.R*.bgp.redistributeRip = true
*.R*.bgp.redistributeOspf = "O IA"

# enable BGP on RB2 and RB3
*.RB2.hasBgp = true
*.RB3.hasBgp = true

```

The BGP configuration enables redistribution for both protocols:

```

*.R*.bgp.redistributeRip = true
*.R*.bgp.redistributeOspf = "O IA"

```

The BGP configuration:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<BGPConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
  ↪ "BGP.xsd">

  <TimerParams>
    <connectRetryTime> 120 </connectRetryTime>
    <holdTime> 180 </holdTime>
    <keepAliveTime> 60 </keepAliveTime>
    <startDelay> 50 </startDelay> <!--long enough for the intra-AS_
  ↪ routing protocol to converge-->
  </TimerParams>

  <AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1"/>
  </AS>

  <AS id="64600">
    <!--router RB1-->
    <Router interAddr="20.0.0.6">
      <Neighbor address='20.0.0.5' nextHopSelf='true' />
      <Neighbor address='20.0.0.2' nextHopSelf='true' />
      <Neighbor address='20.0.0.17' nextHopSelf='true' />
    </Router>

    <!--router RB4-->
    <Router interAddr="20.0.0.2">
      <Neighbor address='20.0.0.6' nextHopSelf='true' />
      <Neighbor address='20.0.0.5' nextHopSelf='true' />
      <Neighbor address='20.0.0.17' nextHopSelf='true' />
    </Router>

```

(continues on next page)

(continued from previous page)

```

    <!--router RB2-->
    <Router interAddr="20.0.0.5"/>

    <!--router RB3-->
    <Router interAddr="20.0.0.17"/>
  </AS>

  <AS id="64700">
    <!--router RC1-->
    <Router interAddr="30.0.0.1"/>
  </AS>

  <!--bi-directional E-BGP session between RA4 and RB1-->
  <Session id="1">
    <Router exterAddr="192.168.0.5"/>
    <Router exterAddr="192.168.0.6"/>
  </Session>

  <!--bi-directional E-BGP session between RB4 and RC1-->
  <Session id="2">
    <Router exterAddr="192.168.0.1"/>
    <Router exterAddr="192.168.0.2"/>
  </Session>
</BGPConfig>

```

Results

The simulation results in `step8.rt` show that BGP successfully bridges the different protocols:

1. IGP Convergence:

- Routers in RA and RC reach internal convergence using RIP.
- Routers in RB reach internal convergence using OSPF.

2. Redistribution into BGP:

- RA4 and RC1 redistribute their RIP-learned routes into BGP.
- RB1 and RB4 redistribute their OSPF-learned routes (including paths to IBGP peers) into BGP.

3. BGP Propagation:

- Routes from RA (10.x.x.x) are carried across RB via OSPF-enabled IBGP sessions and delivered to RC.
- Routes from RC (30.x.x.x) are carried across RB and delivered to RA.

4. End-to-End Success: Host0 in the RIP-based AS 64500 can successfully ping Host1 in the RIP-based AS 64700, transiting through the OSPF-based AS 64600.

This step highlights BGP's role as the "glue" of the Internet, enabling communication between networks with entirely different internal architectures.

Sources: `BGP_Topology_4.ned`, `omnetpp.ini`, `RIPConfig.xml`, `OSPFConfig.xml`, `BGPConfig_Redist.xml`

6.11.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.12 Step 9. Using Network attribute to advertise specific networks

6.12.1 Goals

Step 9 focuses on the `Network` attribute in BGP, which provides a manual and selective way to advertise network prefixes. In previous steps (6, 7, and 8), we relied on automatic redistribution from IGP (OSPF/RIP), which often advertises all known internal routes to BGP peers.

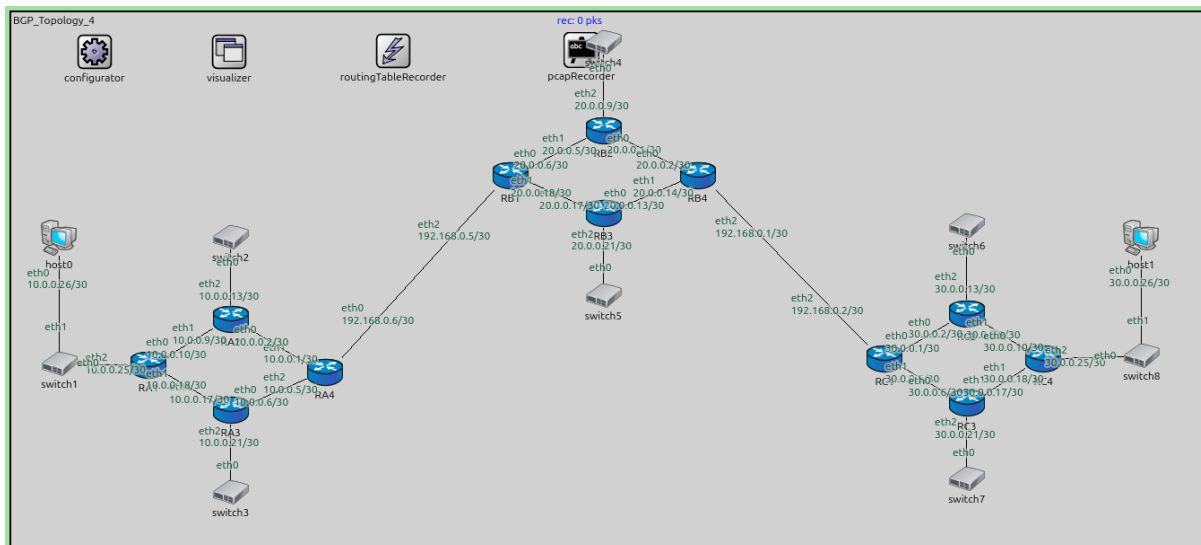
In a real-world scenario, a network administrator might want to advertise only specific prefixes to the public Internet or to a specific partner AS, while keeping other internal subnets private. The `Network` attribute allows for this fine-grained control.

Key features demonstrated:

- **Selective Advertisement:** Using the `Network` element in the BGP configuration XML to explicitly list the subnets to be advertised.
- **Independence from IGP Redistribution:** In this step, OSPF is running for internal connectivity, but the automatic redistribution into BGP is disabled. Only the prefixes listed in the XML are propagated via BGP.
- **Complex Topology Verification:** Verifying that even in a multi-AS transit topology (BGP_Topology_4.net), manually advertised routes can still provide end-to-end reachability for the selected networks.

Configuration

The topology is the same as in Step 6 (BGP_Topology_4.net).



The configuration in `omnetpp.ini` enables OSPF but does not set `redistributeOspf`:

```
[Config Step9]
description = "Using Network attribute to advertise specific networks"
network = BGP_Topology_4
*.routingTableRecorder.logfile = "step9.rt"

*.pcapRecorder.pcapFile = "step9.pcap"

# this example shows how to advertise selective networks in BGP using the 'Network'
↳ attribute
```

(continues on next page)

(continued from previous page)

```

*.configurator.config = xml("<config> \
    <interface hosts='RA4' names='eth0' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB4' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC1' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    \
    <interface among='host0 RA*' address='10.x.x.x' \
↪netmask='255.x.x.x' /> \
    <interface hosts='RA*' address='10.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface hosts='RB*' address='20.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <interface among='host1 RC*' address='30.x.x.x' \
↪netmask='255.x.x.x' /> \
    <interface hosts='RC*' address='30.x.x.x' netmask=
↪'255.x.x.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
    </config>")

# OSPF configuration
*.R*.hasOspf = true
*.R*.ospf.ospfConfig = xmldoc("OSPFConfig.xml")

# BGP configuration
*.RA4.hasBgp = true
*.RB1.hasBgp = true
*.RB4.hasBgp = true
*.RC1.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPConfig.xml")

```

The BGP configuration (BGPConfig.xml) uses Network elements to define the advertised prefixes for each AS:

```

<AS id="64500">
    <!--router RA4-->
    <Router interAddr="10.0.0.1">
        <Network address='10.0.0.0' />
        <Network address='10.0.0.4' />
    </Router>
</AS>

<AS id="64600">
    <!--router RB1-->
    <Router interAddr="20.0.0.6">
        <Network address='20.0.0.4' />
        <Network address='20.0.0.16' />
    </Router>

```

(continues on next page)

(continued from previous page)

```

<!--router RB4-->
<Router interAddr="20.0.0.2">
  <Network address='20.0.0.0' />
  <Network address='20.0.0.12' />
</Router>
</AS>

<AS id="64700">
  <!--router RC1-->
  <Router interAddr="30.0.0.1">
    <Network address='30.0.0.0' />
    <Network address='30.0.0.4' />
  </Router>
</AS>

```

Results

The simulation results in `step9.rt` confirm that prefix propagation is limited to the defined subnets:

1. **Internal Convergence:** OSPF ensures that routers within each AS can reach all internal subnets.
2. **Selective BGP Injection:** Border routers (like RA4 and RC1) only inject the prefixes specifically mentioned in `BGPConfig.xml` (e.g., 10.0.0.0/30, 30.0.0.0/30) into their BGP sessions.
3. **Restricted Routing Tables:** Unlike Step 6, where all OSPF-learned routes were visible across the entire network, we now observe that routers only learn BGP routes for the manually advertised prefixes.
4. **Targeted Reachability:** Connectivity is verified only for the subnets included in BGP. For example, if a subnet in RA was omitted from the `Network` list, it would remain unreachable for RC, even if RA's internal routers knew about it via OSPF.

This step demonstrates how BGP's `Network` attribute is fundamental for implementing routing policies and controlling the information shared with the outside world.

Sources: `BGP_Topology_4.ned`, `omnetpp.ini`, `OSPFConfig.xml`, `BGPConfig.xml`

6.12.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.13 Step 10. LOCAL_PREF Attribute

6.13.1 Goals

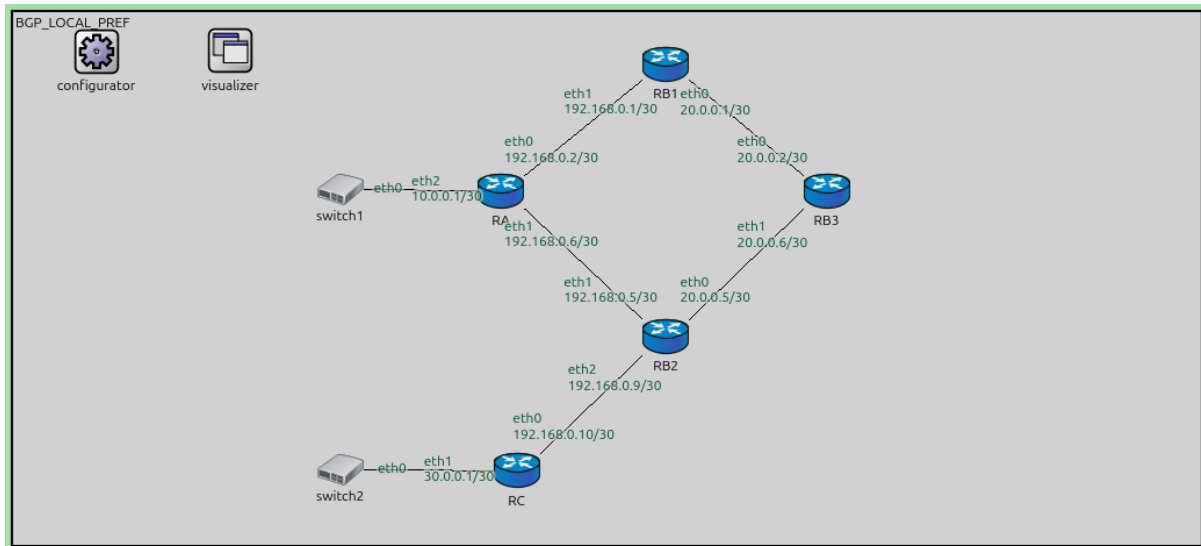
Step 10 demonstrates the **LOCAL_PREF** (Local Preference) BGP attribute. Local Preference is a discretionary attribute used to influence the selection of the preferred exit point from an Autonomous System (AS). When an AS has multiple connections to an external AS, the Local Preference attribute allows internal routers to agree on which border router should be used for outbound traffic.

Key features demonstrated:

- **Inbound Influence on Outbound Traffic:** Setting Local Preference on routes learned from external peers to control how traffic leaves the local AS.
- **Path Selection Hierarchy:** Local Preference is evaluated early in the BGP best-path selection algorithm, typically before `AS_PATH` length.
- **AS-Wide Consistency:** Local Preference is a transitive attribute within an AS, ensuring all internal routers (I-BGP peers) share the same preference information.

Configuration

This step uses the following network (BGP_LOCAL_PREF.ned):



```
network BGP_LOCAL_PREF
{
    @display("bgp=1328.6062,594.495");

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=93,44");
        }
        visualizer: IntegratedMultiCanvasVisualizer {
            @display("p=243.2025,43.536247");
        }
        routingTableRecorder: RoutingTableRecorder {
            @display("p=426,43");
        }
        pcapRecorder: PcapRecorder {
            @display("p=636,42");
        }
        RB1: Router {
            @display("p=730.935,58.275");
        }
        RB2: Router {
            @display("p=730.935,362.97");
        }
        RB3: Router {
            @display("p=910.755,199.8");
        }
        RA: Router {
            @display("p=546.12,199.8");
        }
        switch1: EthernetSwitch {
            @display("p=366.3,198.135");
        }
        RC: Router {
            @display("p=564.435,512.82");
        }
        switch2: EthernetSwitch {
```

(continues on next page)

(continued from previous page)

```

        @display("p=366.3,511.155");
    }
    connections:
        RB1.ethg++ <--> Eth100M <--> RB3.ethg++;
        RB2.ethg++ <--> Eth100M <--> RB3.ethg++;
        RA.ethg++ <--> Eth100M <--> RB1.ethg++;
        RA.ethg++ <--> Eth100M <--> RB2.ethg++;
        RC.ethg++ <--> Eth100M <--> RB2.ethg++;
        switch1.ethg++ <--> Eth100M <--> RA.ethg++;
        RC.ethg++ <--> Eth100M <--> switch2.ethg++;
}

```

The topology features AS 64600 (RB) with two border routers (RB1, RB2) and one interior router (RB3). Both border routers are connected to RA (AS 64500).

The configuration in `omnetpp.ini` defines the setup:

```

[Config Step10]
description = "LOCAL_PREF Attribute"
network = BGP_LOCAL_PREF
*.routingTableRecorder.logfile = "step10.rt"

*.pcapRecorder.pcapFile = "step10.pcap"

*.configurator.config = xml("<config> \
    <interface hosts='RA' names='eth0' address='192.168.x.
↪x' netmask='255.x.x.x' /> \
    <interface hosts='RA' names='eth1' address='192.168.x.
↪x' netmask='255.x.x.x' /> \
    <interface hosts='RB1' names='eth1' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB2' names='eth1' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RB2' names='eth2' address='192.168.
↪x.x' netmask='255.x.x.x' /> \
    <interface hosts='RC' names='eth0' address='192.168.x.
↪x' netmask='255.x.x.x' /> \
    \
    <interface hosts='RA' names='eth2' address='10.x.x.x' _
↪netmask='255.x.x.x' /> \
    <interface among='RB*' address='20.x.x.x' netmask=
↪'255.x.x.x' /> \
    <interface hosts='RC' names='eth1' address='30.x.x.x' _
↪netmask='255.x.x.x' /> \
    </config>")

# OSPF configuration
*.RB*.hasOspf = true
*.RB*.ospf.ospfConfig = xmldoc("OSPFConfig_LOCAL_PREF.xml")

*.R*.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPConfig_LOCAL_PREF.xml")

*.visualizer.routingTableVisualizer[1].displayRoutingTables = false
#*.visualizer.routingTableVisualizer[*].lineShift = 80
*.visualizer.routingTableVisualizer[*].destinationFilter = ""
*.visualizer.routingTableVisualizer[*].lineColor = "black"

```

In the BGP configuration (BGPCfg_LOCAL_PREF.xml), RB1 and RB2 are configured to assign different localPreference values to their sessions with RB3:

```
<Neighbor address='20.0.0.2' nextHopSelf='true' localPreference='100' />
</Router>

<!--router RB2-->
<Router interAddr="20.0.0.5">
  <Neighbor address='20.0.0.1' nextHopSelf='true' />
  <Neighbor address='20.0.0.2' nextHopSelf='true' localPreference='600' />
```

- **RB1** sets localPreference='100' for the route to RA.
- **RB2** sets localPreference='600' for the same route.

Results

The simulation results in `step10.rt` demonstrate how RB3 chooses its path to RA's network (10.0.0.0/30):

1. **E-BGP Discovery:** Both RB1 and RB2 learn the route to 10.0.0.0/30 from RA via their respective E-BGP sessions.
2. **I-BGP Propagation:** RB1 and RB2 advertise this route to RB3 via I-BGP.
3. **Initial Route Selection:** Initially, RB3 installs the route learned from RB1 (next hop 20.0.0.1) at approximately 60.0s.
4. **Best Path Update:** Shortly after (at 62.0s), RB3 receives the update from RB2. Because RB2's advertisement carries a higher Local Preference (600 vs 100), RB3 **updates its routing table** to use RB2 (next hop 20.0.0.5) as the preferred exit point.

You can verify this in `step10.rt` by observing the route change at the 62s mark for router RB3.

This step illustrates how network administrators can use Local Preference to enforce routing policies, such as preferring a high-bandwidth link or a specific upstream provider for outbound traffic.

Sources: `BGP_LOCAL_PREF.ned`, `omnetpp.ini`, `BGPCfg_LOCAL_PREF.xml`

6.13.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.14 Step 11. Multi-hop E-BGP

6.14.1 Goals

Step 11 introduces **Multi-hop E-BGP**. By default, External BGP (E-BGP) assumes that peers are directly connected (on the same physical link) and uses an IP Time-To-Live (TTL) of 1 for its BGP packets. If border routers are separated by one or more intermediate routers, the default E-BGP session closure will fail.

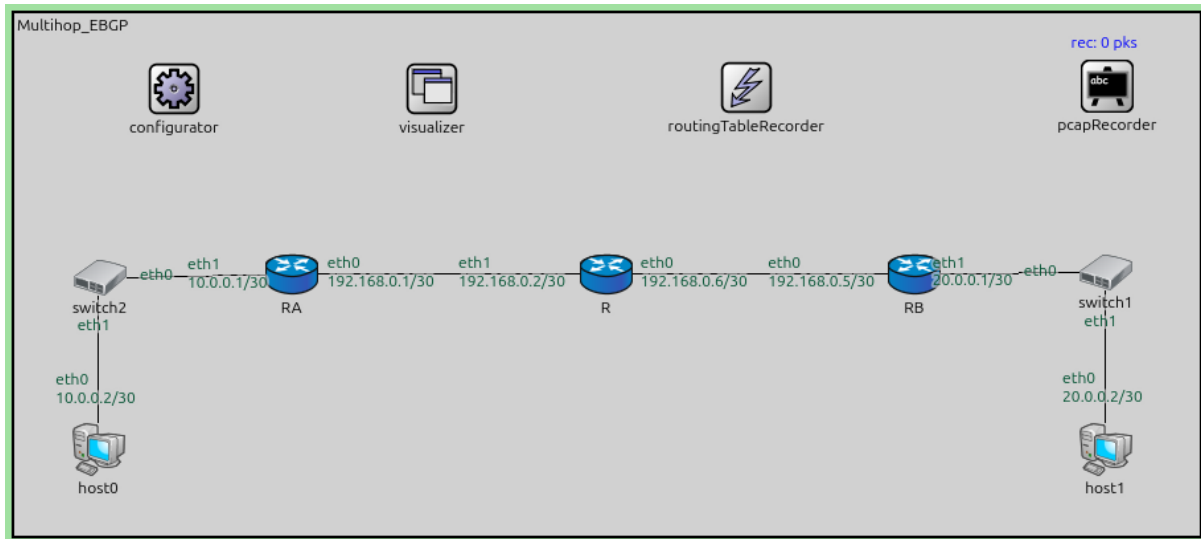
This scenario (`MultiHop_EBGP.ned`) demonstrates how to establish an E-BGP session between two border routers (RA and RB) that are separated by an intermediate router (R) belonging to a different infrastructure or acting as a simple forwarder.

Key features demonstrated:

- **E-BGP Multihop:** Using the `ebgpMultihop` attribute in the BGP configuration to increase the packet TTL, allowing the BGP session to traverse intermediate hops.
- **Peering Reachability:** In a multi-hop setup, peers must know how to reach each other's IP addresses *before* BGP can start. This is often achieved through static routes or an IGP.
- **Transit Routing:** Verifying that the intermediate router (R) successfully forwards BGP control traffic and subsequent data traffic between ASes.

Configuration

This step uses the following network:



```
network Multihop_EBGP
{
    @display("bgb=690,303");

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=93,44");
        }
        visualizer: IntegratedMultiCanvasVisualizer {
            @display("p=243.2025,43.536247");
        }
        routingTableRecorder: RoutingTableRecorder {
            @display("p=426,43");
        }
        pcapRecorder: PcapRecorder {
            @display("p=636,42");
        }
        RA: Router {
            @display("p=161.33,151.385");
        }
        RB: Router {
            @display("p=523.77,151.385");
        }
        R: Router {
            @display("p=344.76,151.385");
        }

        switch2: EthernetSwitch {
            @display("p=49.725,153.595");
        }
        switch1: EthernetSwitch {
            @display("p=635.375,150.28");
        }
        host0: StandardHost {
            @display("p=49,253");
        }
}
```

(continues on next page)

(continued from previous page)

```

    host1: StandardHost {
        @display("p=635,253");
    }
    connections:
    RB.ethg++ <--> Eth100M <--> R.ethg++;
    RA.ethg++ <--> Eth100M <--> R.ethg++;
    switch2.ethg++ <--> Eth100M <--> RA.ethg++;
    RB.ethg++ <--> Eth100M <--> switch1.ethg++;
    host1.ethg++ <--> Eth100M <--> switch1.ethg++;
    host0.ethg++ <--> Eth100M <--> switch2.ethg++;
}

```

The topology is a simple chain: RA <--> R <--> RB.

- **RA** belongs to AS 64520.
- **RB** belongs to AS 64530.
- **R** is an intermediate node.

The omnetpp.ini configuration sets up static routes so RA and RB can reach each other's peering addresses:

```

[Config Step11]
description = "Multi-hop E-BGP"
network = Multihop_EBGP
*.routingTableRecorder.logfile = "step11.rt"

*.pcapRecorder.pcapFile = "step11.pcap"

*.configurator.config = xml("<config> \
                                <interface among='RA host0' address='10.0.x.x' \
↳netmask='255.x.x.x' /> \
                                <interface among='RB host1' address='20.0.x.x' \
↳netmask='255.x.x.x' /> \
                                <interface among='RA R RB' address='192.168.x.x' \
↳netmask='255.x.x.x' /> \
                                <route hosts='RA' destination='192.168.0.5' netmask=
↳'255.255.255.255' interface='eth0' /> \
                                <route hosts='RB' destination='192.168.0.1' netmask=
↳'255.255.255.255' interface='eth0' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0
↳' interface='eth0' /> \
                                </config>")

*.RA.hasBgp = true
*.RB.hasBgp = true
*.R*.bgp.bgpConfig = xmldoc("BGPCfg_MultiHopEBGP.xml")

```

The BGP configuration (BGPCfg_MultiHopEBGP.xml) specifies ebgpMultihop='2' for both neighbors:

```

<AS id="64520">
    <!--router RA-->
    <Router interAddr="10.0.0.1">
        <Network address='10.0.0.0' />
        <Neighbor address='192.168.0.5' ebgpMultihop='2' />
    </Router>
</AS>

<AS id="64530">

```

(continues on next page)

(continued from previous page)

```

<!--router RB-->
<Router interAddr="20.0.0.1">
  <Network address='20.0.0.0' />
  <Neighbor address='192.168.0.1' ebgpMultihop='2' />
</Router>
</AS>

```

Results

The simulation results in `step11.rt` show the successful establishment of the multi-hop session:

1. **Static Convergence:** RA and RB use the configured static routes to reach each other via R.
2. **BGP Session Establishment:** Because `ebgpMultihop='2'` is set, the BGP packets sent by RA can reach RB (and vice-versa) even though they pass through R. The TCP connection is established successfully.
3. **Route Exchange:**
 - RA advertises its 10.0.0.0/30 network to RB.
 - RB advertises its 20.0.0.0/30 network to RA.
4. **Data Reachability:** Once BGP converges, hosts behind RA can reach hosts behind RB. `step11.rt` confirms that RA eventually has a BGP route to 20.0.0.0/30 with RB's address as the next hop.

This step demonstrates a common real-world scenario where border routers might not be physically adjacent, requiring multihop capabilities to form the BGP adjacency.

Sources: `Multihop_EBGP.ned`, `omnetpp.ini`, `BGPConfig_MultiHopEBGP.xml`

6.14.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

6.15 Conclusion

Congratulations, you've made it! You should now be able to set up BGP routing for networks in the INET Framework. Use the link below if you have questions or you'd like to help us improve this tutorial.

6.15.1 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

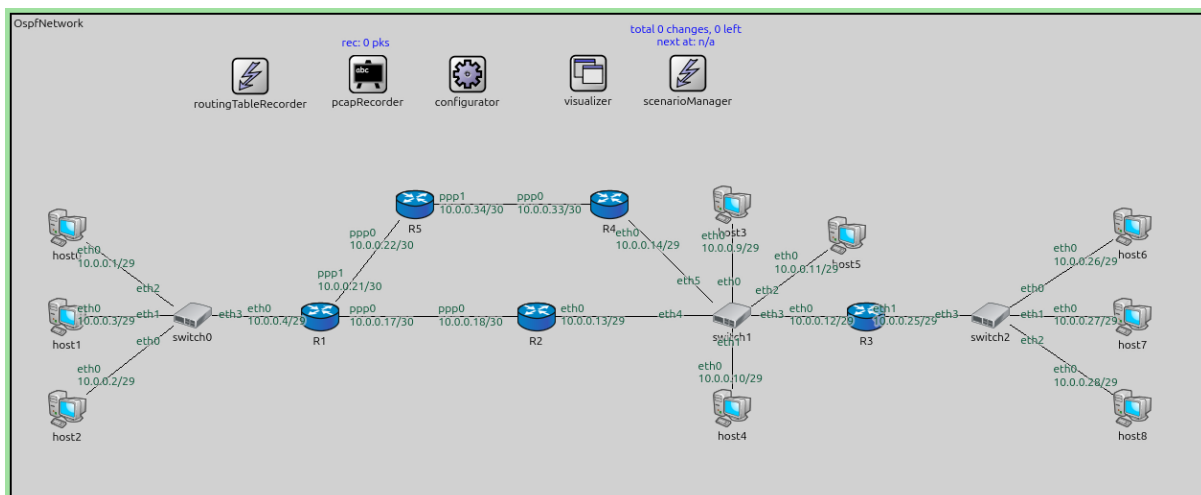
Contents:

7.1 Step 1. Pinging after OSPF convergence

The goal of this step is to demonstrate basic OSPF operation in a single-area network and verify that hosts can communicate after OSPF convergence.

This step shows OSPF operating in a single area (Area 0.0.0.0), with routers establishing adjacencies, exchanging LSAs, building their LSDBs, and computing routes. After convergence, hosts can ping each other across the network using the OSPF-computed routes.

This step uses the *OspfNetwork* topology with five routers (R1-R5) and nine hosts (host0-host8) connected via switches and point-to-point links.



(continues on next page)

(continued from previous page)

```

types:
  channel PppLink100M extends DatarateChannel
  {
    delay = 5us;
    datarate = 100Mbps;
  }

  channel PppLink10M extends DatarateChannel
  {
    delay = 5us;
    datarate = 10Mbps;
  }

submodules:
  configurator: Ipv4NetworkConfigurator {
    @display("p=804,105");
  }
  visualizer: IntegratedMultiCanvasVisualizer {
    @display("p=1017,103");
  }
  routingTableRecorder: RoutingTableRecorder {
    @display("p=421,108");
  }
  pcapRecorder: PcapRecorder {
    @display("p=628,105");
  }
  scenarioManager: ScenarioManager {
    @display("p=1193,103");
  }
  host0: StandardHost {
    @display("p=97.81249,375.59998");
  }
  host1: StandardHost {
    @display("p=97.81249,532.1");
  }
  host2: StandardHost {
    @display("p=97.81249,694.46875");
  }
  host3: StandardHost {
    @display("p=1271.5625,334.51874");
  }
  R1: Router {
    @display("p=543.83746,532.1");
  }
  R3: Router {
    @display("p=1508.2687,532.1");
  }
  R2: Router {
    @display("p=925.3062,532.1");
  }
  host4: StandardHost {
    @display("p=1271.5625,692.51245");
  }
  host5: StandardHost {
    @display("p=1473.0562,389.29373");
  }

```

(continues on next page)

(continued from previous page)

```

host6: StandardHost {
    @display("p=1977.7687,373.64374");
}
host7: StandardHost {
    @display("p=1977.7687,532.1");
}
host8: StandardHost {
    @display("p=1977.7687,692.51245");
}
switch0: EthernetSwitch {
    @display("p=318.86874,530.14374");
}
switch2: EthernetSwitch {
    @display("p=1727.3687,530.14374");
}
switch1: EthernetSwitch {
    @display("p=1271.5625,530.14374");
}
R4: Router {
    @display("p=1054.4187,334.51874");
}
R5: Router {
    @display("p=712.07495,334.51874");
}

connections:
host2.ethg++ <--> Eth100M <--> switch0.ethg++;
host1.ethg++ <--> Eth100M <--> switch0.ethg++;
host0.ethg++ <--> Eth100M <--> switch0.ethg++;
switch0.ethg++ <--> Eth100M <--> R1.ethg++;

switch1.ethg++ <--> Eth100M <--> host3.ethg++;
switch1.ethg++ <--> Eth100M <--> host4.ethg++;
switch1.ethg++ <--> Eth100M <--> host5.ethg++;
switch1.ethg++ <--> Eth100M <--> R3.ethg++;

switch2.ethg++ <--> Eth100M <--> host6.ethg++;
switch2.ethg++ <--> Eth100M <--> host7.ethg++;
switch2.ethg++ <--> Eth100M <--> host8.ethg++;

R3.ethg++ <--> Eth100M <--> switch2.ethg++;
R1.pppg++ <--> PppLink10M <--> R2.pppg++;
R2.ethg++ <--> Eth10M <--> switch1.ethg++;
R1.pppg++ <--> PppLink100M <--> R5.pppg++;
R5.pppg++ <--> PppLink100M <--> R4.pppg++;
R4.ethg++ <--> Eth100M <--> switch1.ethg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step1]
description = "Pinging after OSPF convergence"
network = OspfNetwork

*.configurator.config = xml("<config> \
                                <interface hosts='*' address='10.x.x.x' netmask='255.

```

(continues on next page)

(continued from previous page)

```

↪x.x.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
                                </config>")

# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 60s

```

Results

When the simulation starts:

1. All OSPF routers discover their neighbors and establish adjacencies.
2. Routers exchange Hello packets, then Database Description packets, Link State Request/Update packets to synchronize their LSDBs.
3. Each router runs the SPF algorithm to compute lowest cost paths (running Dijkstra's weighted shortest path algorithm) to all subnetworks in the area.
4. The routing tables are populated with OSPF-learned routes. For example, R1 learns routes to the 10.0.0.24/29 network (behind R3) via the path through R2 and R5->R4.
5. The R5->R4 path turns out to be higher hop-count but lower cost than the R2 path due to cost associated with link datarates.
6. At t=60s, **host0** begins pinging **host6**. The ping succeeds because OSPF has computed valid routes between all subnetworks.

Sources: `omnetpp.ini`, `OspfNetwork.ned`

7.1.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.2 Step 2. Change link cost

7.2.1 Goals

The goal of this step is to demonstrate how manually configuring OSPF interface costs affects route selection.

OSPF uses link costs to determine the best path to each destination. By default, the cost is calculated based on the link bandwidth, but it can also be manually configured. When multiple paths exist to a destination, OSPF selects the path with the lowest total cost. Changing the cost of a link can influence which path OSPF chooses.

Configuration

This configuration is based on Step 1. The OSPF configuration manually overrides the output cost of R1's `ppp1` interface using `ASConfig_cost.xml`.

The configuration in `omnetpp.ini` is the following:

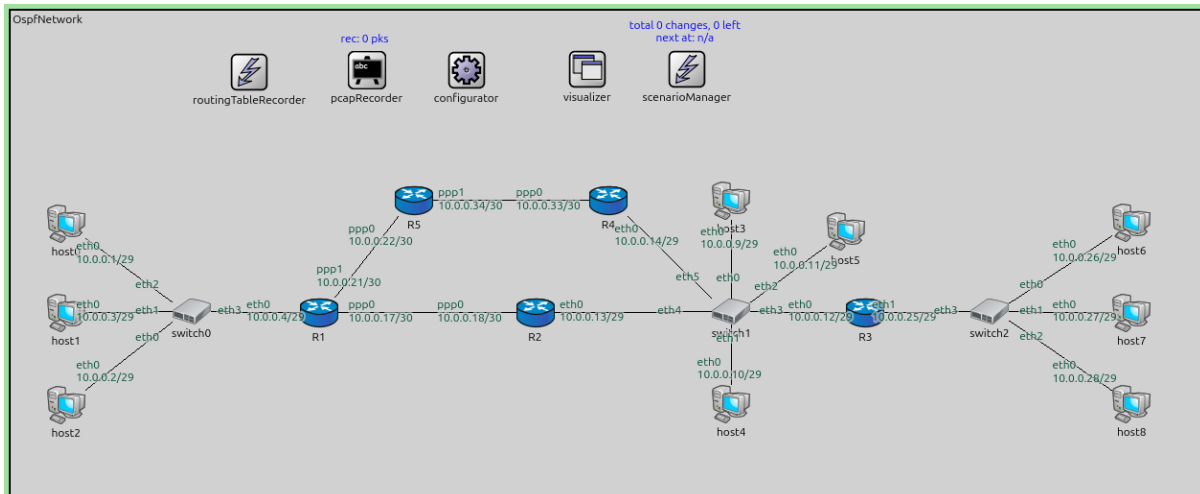
```

[Config Step2]
description = "Change link cost"
extends = Step1

# manually overriding the 'output cost' of ppp1 on router R1
# note that the route of ping request will be affected and not ping reply

```

(continues on next page)



(continued from previous page)

```
*.R*.ospf.ospfConfig = xmldoc("ASConfig_cost.xml")
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" />
    <PointToPointInterface ifName="ppp0" interfaceOutputCost='0' />
    <PointToPointInterface ifName="ppp1" interfaceOutputCost="20" />
  </Router>

  <Router name="*" RFC1583Compatible="true">
    <BroadcastInterface ifName='eth[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
    <PointToPointInterface ifName='ppp[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
  </Router>
</OSPFASConfig>
```

Results

The modified cost on R1's ppp1 interface affects routing decisions:

1. R1's ppp1 interface is assigned a higher cost than the default.
2. This influences the path selection for traffic originating from hosts behind R1.
3. In Step 1, traffic from host0 to host6 used the path through R5->R4. With the increased cost on R1's ppp1, OSPF now prefers the alternative path through R2.
4. Note that OSPF costs are directional. The modified cost affects routes computed by R1 (outbound direction) but not routes computed by other routers for reaching R1.

The routing table changes show how adjusting link costs allows network administrators to influence traffic engineering and load distribution in OSPF networks.

Sources: omnetpp.ini, OspfNetwork.ned, ASConfig_cost.xml

7.2.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.3 Step 2a. Reroute after link breakage

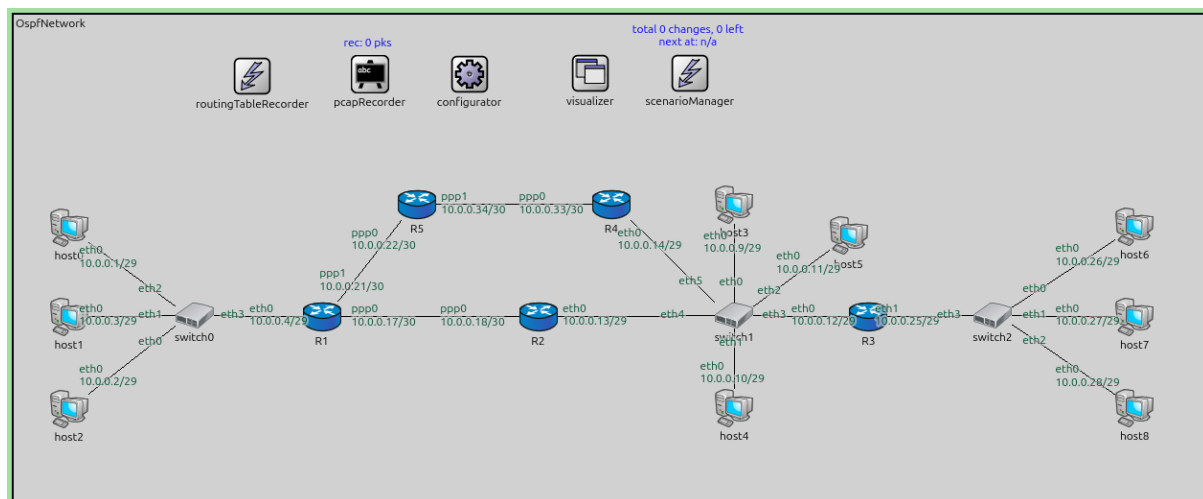
7.3.1 Goals

The goal of this step is to demonstrate OSPF's ability to automatically reroute traffic when a link fails.

OSPF continuously monitors link states. When a link fails, the affected router generates a new LSA reflecting the topology change and floods it throughout the area. Other routers update their LSDBs, rerun the SPF algorithm, and update their routing tables to use alternative paths (if available).

Configuration

This configuration is based on Step 1. The scenario manager script disconnects the link between **R5** and **R4** at $t=60s$.



The configuration in `omnetpp.ini` is the following:

```
[Config Step2a]
description = "Reroute after link breakage"
extends = Step1

# the link between R5 and R4 breaks at t = 60s, and OSPF finds a new route

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='70' src-module='R5' dest-module='R4"
                                ↪' /> \
                                </scenario>")
```

Results

When the link between R5 and R4 breaks at $t=70s$:

1. Both **R5** and **R4** detect the link down event on their respective interfaces.
2. R5 and R4 each generate new Router LSAs that no longer include the link between them.
3. These LSAs are flooded throughout Area 0.0.0.0.
4. All routers receive the updated LSAs, update their LSDBs, and rerun the SPF algorithm.

5. Routers that were using the R5-R4 link in their shortest paths now compute alternative paths. For example, traffic from R1 to networks behind R4 now routes through R2 instead of through R5.
6. Applications may experience brief packet loss during the reconvergence period, but connectivity is restored once OSPF converges on the new topology. In this example, no ping packet is lost.

The routing table changes demonstrate OSPF's resilience: when a link fails, the protocol automatically discovers and uses alternative paths, ensuring continued network connectivity.

Sources: `omnetpp.ini`, `OspfNetwork.ned`

7.3.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.4 Step 3. OSPF full adjacency establishment and LSDB sync

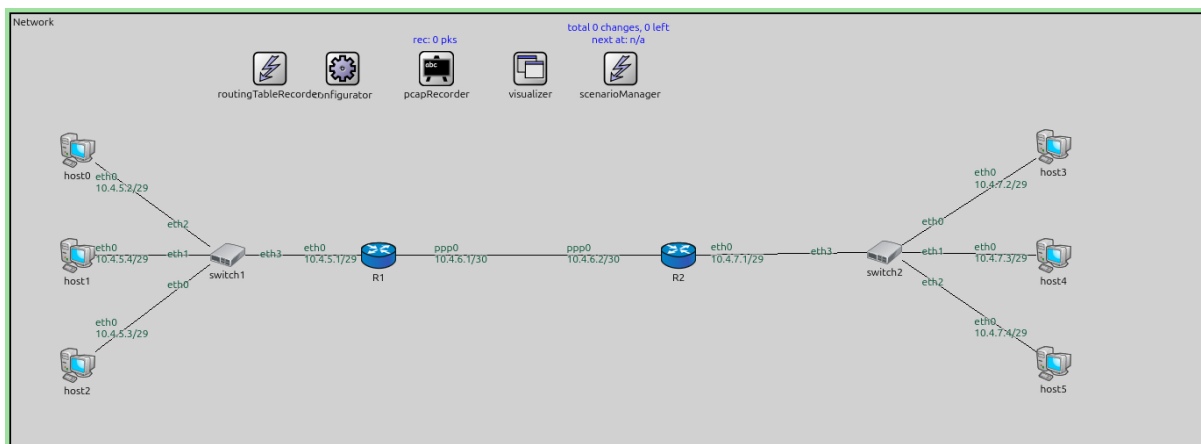
7.4.1 Goals

The goal of this step is to examine the process of OSPF adjacency establishment and Link State Database (LSDB) synchronization between two routers.

OSPF routers go through several states when forming an adjacency: Down → Init → 2-Way → ExStart → Exchange → Loading → Full. The Exchange and Loading states involve exchanging Database Description (DBD) packets, Link State Request (LSR) packets, and Link State Update (LSU) packets to synchronize the LSDBs. Only after reaching the Full state do routers have identical LSDBs and can use each other for SPF calculations.

Configuration

This step uses a simple two-router topology to clearly observe the adjacency process.



The configuration in `omnetpp.ini` is the following:

```
[Config Step3]
description = "OSPF full adjacency establishment and LSDB sync"
network = Network

*.configurator.config = xml("<config> \
    <interface hosts='R1' names='eth0' address='10.4.5.x' \
    ↪netmask='255.255.255.x' /> \
    <interface hosts='R1' names='ppp0' address='10.4.6.x' \
    ↪netmask='255.255.255.x' /> \
    <interface hosts='R2' names='eth0' address='10.4.7.x' \
    ↪netmask='255.255.255.x' /> \
    <interface hosts='*' address='10.4.x.x' netmask='255.
(continues on next page)
```

(continued from previous page)

```

↪255.255.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
                                </config>")

*.visualizer.routingTableVisualizer[*].displayRoutingTables = false

# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host3"
*.host0.app[0].startTime = 60s

```

Results

When the simulation starts, the two OSPF routers (R1 and R2) establish an adjacency on their point-to-point link:

1. **Init state:** Each router receives a Hello packet from the other and transitions to Init.
2. **2-Way state:** When each router sees its own Router ID in the neighbor's Hello packet, they transition to 2-Way.
3. **ExStart state:** The routers determine which one will be the master (higher Router ID) for the DBD exchange.
4. **Exchange state:** The routers exchange DBD packets describing the LSAs in their LSDBs.
5. **Loading state:** If a router finds that its neighbor has LSAs it doesn't have (or newer versions), it sends LSR packets to request them. The neighbor responds with LSU packets containing the requested LS As.
6. **Full state:** Once the LSDBs are synchronized, the adjacency reaches the Full state.

The PCAP and OSPF module logs show the detailed packet exchange during these state transitions, demonstrating the LSDB synchronization mechanism that ensures all OSPF routers in an area have consistent topology information.

Sources: `omnetpp.ini`, `Network.ned`

7.4.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.5 Step 4. Router LSA

7.5.1 Goals

The goal of this step is to examine the structure and content of Router LSAs (Type-1 LSAs).

Router LSAs are the fundamental building blocks of OSPF's link-state database. Each router generates a Router LSA describing its own links (connections to networks and other routers). The Router LSA includes information about each link's type, ID, metric (cost), and other attributes. By collecting all Router LSAs, each OSPF router builds a complete graph of the area topology.

Configuration

This step uses the RouterLSA network topology.

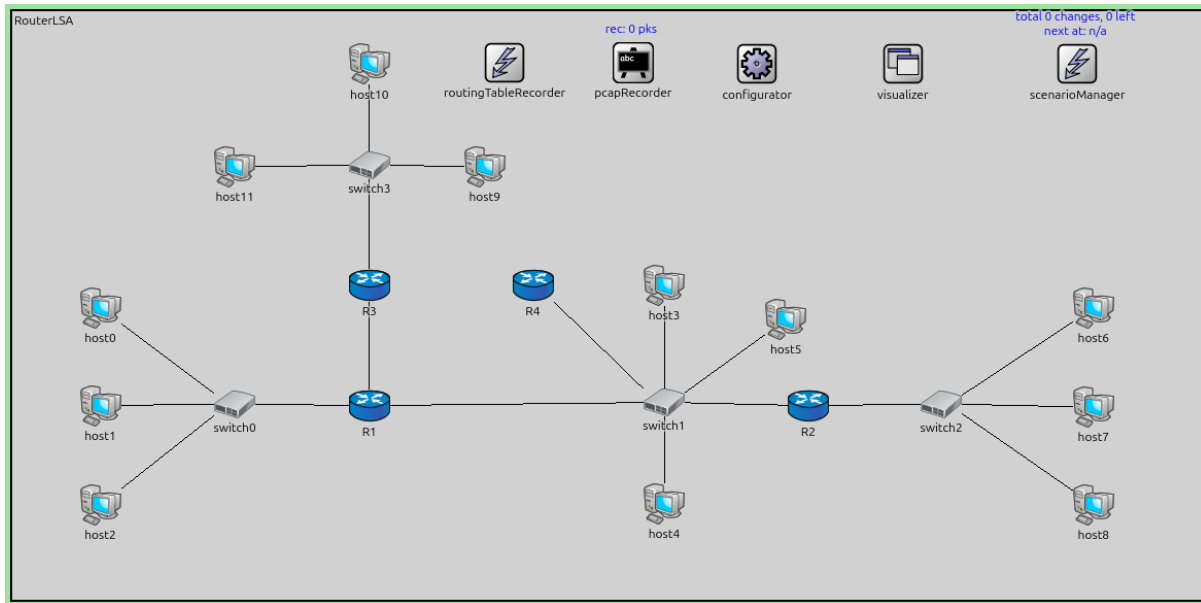
The configuration in `omnetpp.ini` is the following:

```

[Config Step4]
description = "Router LSA"
network = RouterLSA

```

(continues on next page)



(continued from previous page)

```
# what does a router LSA look like?

*.configurator.config = xml("<config> \
                                <interface hosts='**' address='10.x.x.x' netmask='255.
↪x.x.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
                                </config>")

# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 60s
```

Results

Each router in the network generates a Router LSA describing its links:

- **Point-to-point links:** Described with the neighbor's Router ID and the link cost.
- **Transit network links** (Ethernet/multi-access): Described with the Designated Router's IP address and the link cost.
- **Stub network links:** Described with the network address, netmask, and cost (e.g., for host routes or passive interfaces).

The OSPF module logs show the Router LSA contents, including:

- LSA header (age, Router ID, sequence number)
- Number of links
- For each link: type, Link ID, Link Data, metric

By examining the Router LSAs in the LSDB, you can see exactly how each router views its local topology, and how all these LSAs together form a complete picture of the area.

Sources: `omnetpp.ini`, `RouterLSA.ned`

7.5.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.6 Step 4a. Advertising loopback interface

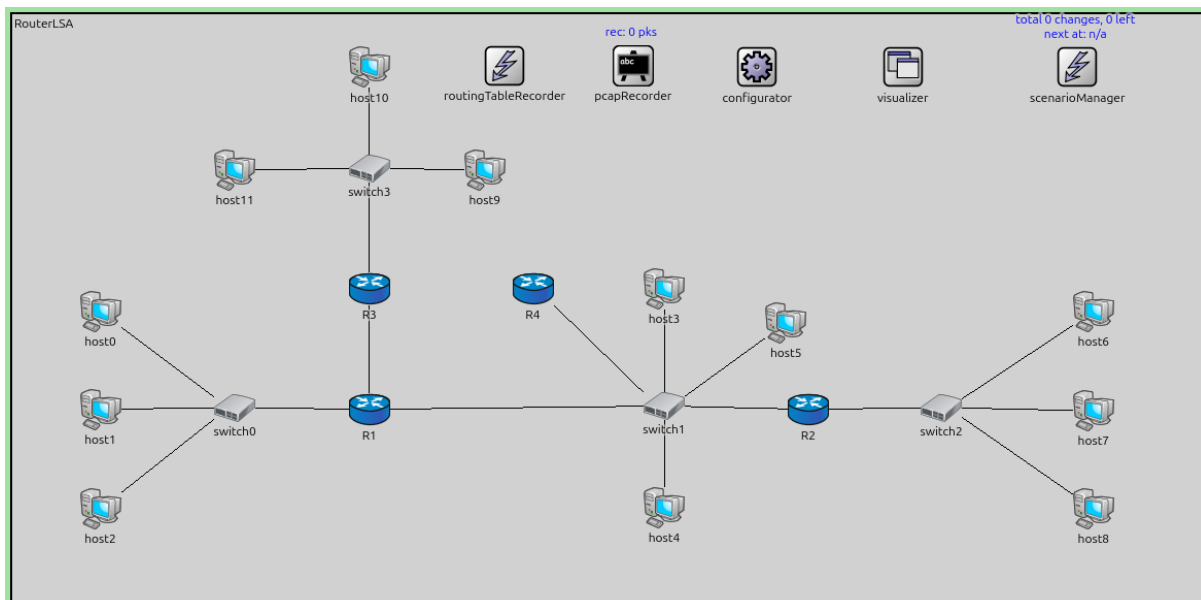
7.6.1 Goals

The goal of this step is to demonstrate how OSPF advertises loopback interfaces.

Loopback interfaces are virtual interfaces that are always up (unless administratively disabled). In OSPF, loopback interfaces are treated as stub networks and are advertised as host routes with a /32 netmask, regardless of the actual configured netmask. This behavior ensures that the loopback address is reachable as a specific host route, which is useful for management purposes and as a stable Router ID.

Configuration

This configuration is based on Step 4. R1 is configured with a second loopback interface (lo1), and the OSPF configuration includes this interface.



The configuration in `omnetpp.ini` is the following:

```
[Config Step4a]
description = "Advertising loopback interface"
network = RouterLSA

# Loopbacks are considered host routes in OSPF, and they are advertised as /32.
# advertising lo1 in router R1

*.configurator.config = xml("<config> \
                                <interface hosts='*' address='10.x.x.x' netmask='255.\
↵x.x.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0\
↵' interface='eth0' /> \
                                </config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Loopback.xml")

*.R1.numLoInterfaces = 2
```

(continues on next page)

(continued from previous page)

```
# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 60s
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceOutputCost='0' />
    <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
    <PointToPointInterface ifName="ppp0" interfaceOutputCost='0' />
    <LoopbackInterface ifName="lo1" />
  </Router>

  <Router name="*" RFC1583Compatible="true">
    <BroadcastInterface ifName='eth[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
    <PointToPointInterface ifName='ppp[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
  </Router>

</OSPFASConfig>
```

Results

R1's Router LSA now includes an additional stub network link for the loopback interface lo1:

- The link is advertised as a /32 host route (255.255.255.255 netmask).
- The metric is typically 0 or a small value for loopback interfaces.

Other routers in the area receive R1's Router LSA and install a /32 route to R1's loopback address. This demonstrates that:

1. Loopback interfaces are always advertised as /32 regardless of configuration.
2. Loopback addresses provide stable, reachable IP addresses for router management.
3. Loopback interfaces are commonly used as the Router ID source in OSPF.

Sources: omnetpp.ini, RouterLSA.ned, ASConfig_Loopback.xml

7.6.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.7 Step 5. Effect of network type of an interface on routing table routes

7.7.1 Goals

The goal of this step is to demonstrate how the OSPF network type of an interface affects the routing table entries.

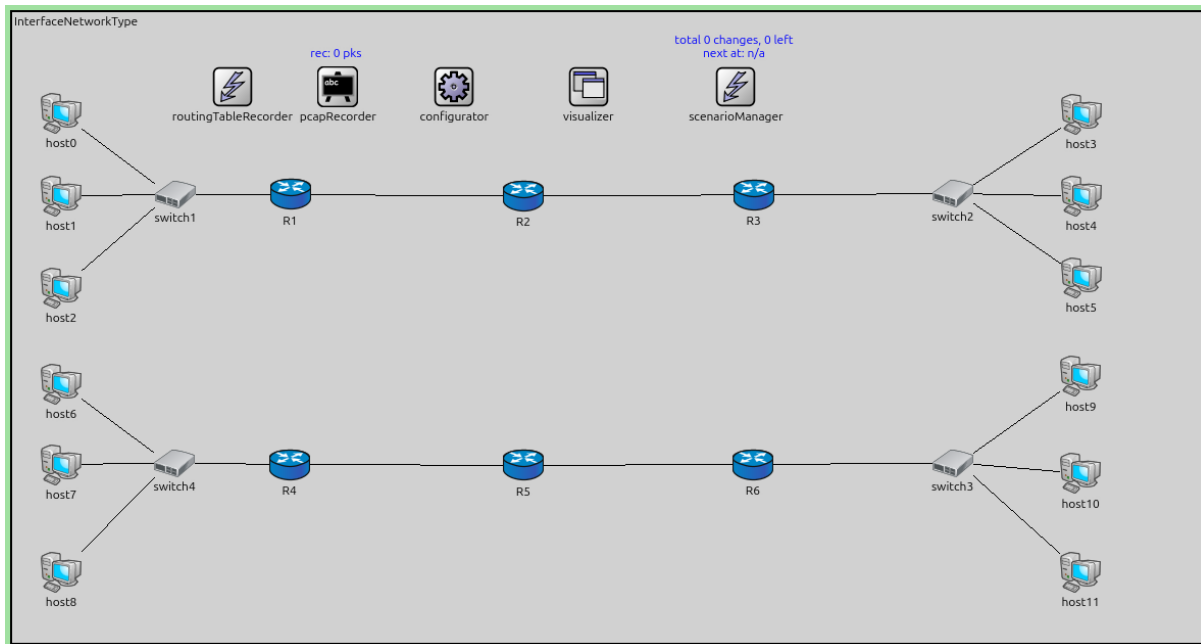
OSPF supports different network types: Point-to-Point, Broadcast, NBMA (Non-Broadcast Multi-Access), Point-to-Multipoint, and others. The network type determines how OSPF behaves on that interface, including whether a Designated Router (DR) is elected and how the network is represented in LSAs and routing tables.

- **Broadcast networks** (e.g., Ethernet): Use DR/BDR election, described by Network LSAs.
- **Point-to-Point networks** (e.g., PPP links): No DR election, direct adjacencies.

The network type affects both the OSPF protocol operation and how routes appear in the routing table.

Configuration

This step uses the `InterfaceNetworkType` network to demonstrate different network types.



The configuration in `omnetpp.ini` is the following:

```
[Config Step5]
description = "Effect of network type of an interface on routing table routes"
network = InterfaceNetworkType

*.configurator.configureIsolatedNetworksSeparately = true

*.configurator.config = xml("<config> \
                                <interface hosts='**' address='10.x.x.x' netmask='255.\
->x.x.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0\
->' interface='eth0' /> \
                                </config>")

**.visualizer.interfaceTableVisualizer.displayInterfaceTables = true
**.visualizer.interfaceTableVisualizer.nodeFilter = "not *.N*"

# application parameters
*.host8.numApps = 1
*.host8.app[0].typename = "PingApp"
*.host8.app[0].destAddr = "host11"
*.host8.app[0].startTime = 60s
```

Results

The simulation shows how different OSPF network types result in different routing table entries and OSPF behavior:

- **Broadcast interfaces:** OSPF elects a DR and BDR. The network is represented by a Network LSA originated by the DR. Routes to the network show the DR's interface as the next hop.
- **Point-to-Point interfaces:** No DR election occurs. The link is represented directly in Router LSAs from both ends. Routes show direct next-hop addresses.

The OSPF module logs and routing tables demonstrate how network type configuration affects:

1. Neighbor discovery and adjacency formation
2. LSA generation and flooding
3. Routing table entries and next-hop selection

Understanding network types is crucial for correctly configuring OSPF on different media types and troubleshooting adjacency issues.

Sources: `omnetpp.ini`, `InterfaceNetworkType.ned`

7.7.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.8 Step 5a. Mismatched Parameters between two OSPF neighbors

7.8.1 Goals

The goal of this step is to demonstrate that OSPF routers will not form a full adjacency if certain parameters are mismatched.

For OSPF neighbors to successfully form an adjacency, several parameters must match:

- **Area ID:** Must be the same on both sides of the link.
- **Hello interval and Dead interval:** Must match.
- **Authentication:** Type and keys must match.
- **Network type:** While not always required to match exactly, mismatches can prevent proper adjacency formation.
- **Stub area flag:** Must match if the area is configured as a stub.

When these parameters mismatch, routers may get stuck in states like 2-Way or Init, unable to reach the Full state.

Configuration

This configuration is based on Step 5. The OSPF configuration file introduces a parameter mismatch:

- **R1 and R2:** Mismatched Hello intervals

The configuration in `omnetpp.ini` is the following:

```
[Config Step5a]
description = "Mismatched Parameters between two OSPF neighbor"
network = InterfaceNetworkType

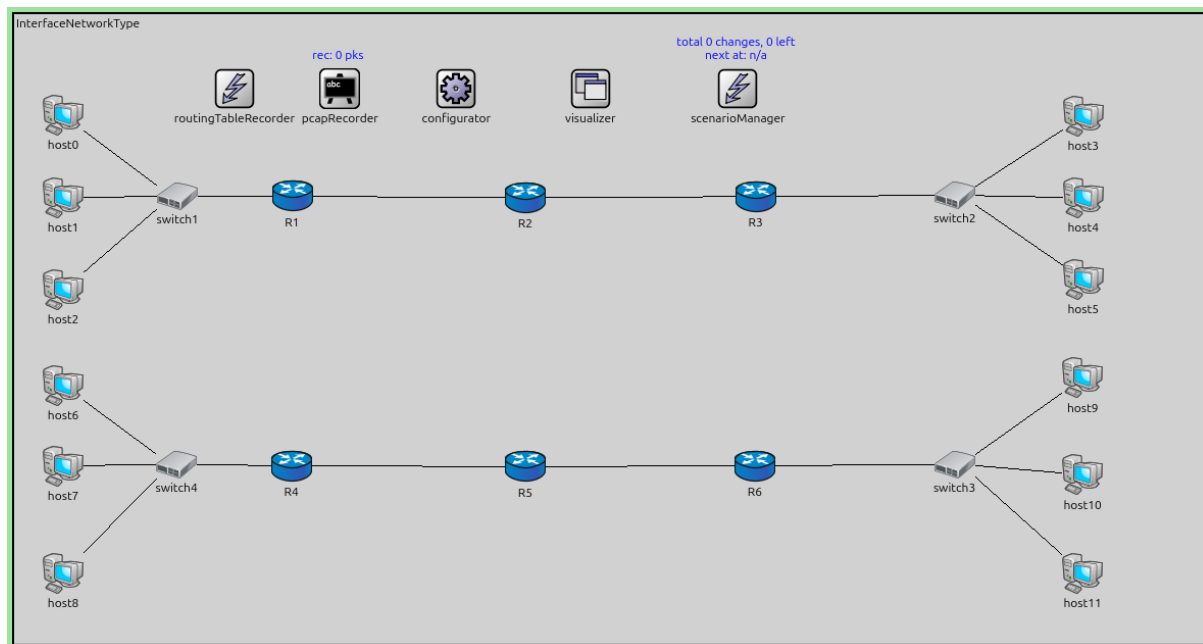
# R1 and R2 will not establish full adjacency because of mismatch Hello interval

*.configurator.configureIsolatedNetworksSeparately = true

*.configurator.config = xml("<config> \\"

```

(continues on next page)



(continued from previous page)

```

<interface hosts='*' address='10.x.x.x' netmask='255.
↪x.x.x' /> \
<route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
</config>")

**.visualizer.interfaceTableVisualizer.displayInterfaceTables = true
**.visualizer.interfaceTableVisualizer.nodeFilter = "not *.N*"

*.visualizer.routingTableVisualizer[0].destinationFilter = "host5"
*.visualizer.routingTableVisualizer[1].destinationFilter = "host0"

*.R*.ospf.ospfConfig = xmldoc("ASConfig_mismatch.xml")

# application parameters
*.host8.numApps = 1
*.host8.app[0].typename = "PingApp"
*.host8.app[0].destAddr = "host11"
*.host8.app[0].startTime = 60s

# TODO: full adjacency forms when it shouldn't: R4 and R5 will not establish full_
↪adjacency because of mismatch OSPF network type

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↪xsi:schemaLocation="OSPF.xsd">

  <Router name="R1" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" interfaceOutputCost='0' helloInterval="10" />
    <BroadcastInterface ifName="eth0" interfaceOutputCost='0' />
  </Router>

```

(continues on next page)

(continued from previous page)

```

<Router name="R2" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" interfaceOutputCost='0' helloInterval="15" />
  <PointToPointInterface ifName="ppp1" interfaceOutputCost='0' />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" interfaceOutputCost='0' />
  <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" interfaceOutputCost='0' />
<!-- <PointToPointInterface ifName="eth1" interfaceOutputCost='0' /> TODO: this_
      should prevent full adjacency, but it doesn't-->
  <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
</Router>

<Router name="*" RFC1583Compatible="true">
  <BroadcastInterface ifName='eth[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
  <PointToPointInterface ifName='ppp[*]' areaID='0.0.0.0' interfaceOutputCost='0' /
-->
</Router>

</OSPFASConfig>

```

Results

The simulation demonstrates adjacency formation failures:

1. **R1 and R2:** Because they have different Hello intervals, they cannot properly synchronize their neighbor discovery. They may detect each other but fail to maintain a stable adjacency or have timing issues.

The OSPF module logs show the routers detecting neighbors but failing to reach the Full state. The routing tables reflect the missing adjacencies - routes that would normally use these links are either absent or use alternative paths.

This step highlights the importance of consistent OSPF configuration across interfaces forming an adjacency.

Sources: omnetpp.ini, InterfaceNetworkType.ned, ASConfig_mismatch.xml

7.8.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.9 Step 6. OSPF DR/BDR election in a multi-access network (Ethernet)

7.9.1 Goals

The goal of this step is to demonstrate Designated Router (DR) and Backup Designated Router (BDR) election on a multi-access network.

On multi-access networks like Ethernet, OSPF elects a DR and BDR to reduce the number of adjacencies and LSA flooding overhead. Instead of every router forming adjacencies with every other router ($N \times (N-1)/2$ adjacencies), all routers form adjacencies only with the DR and BDR, reducing complexity to $2N$ adjacencies.

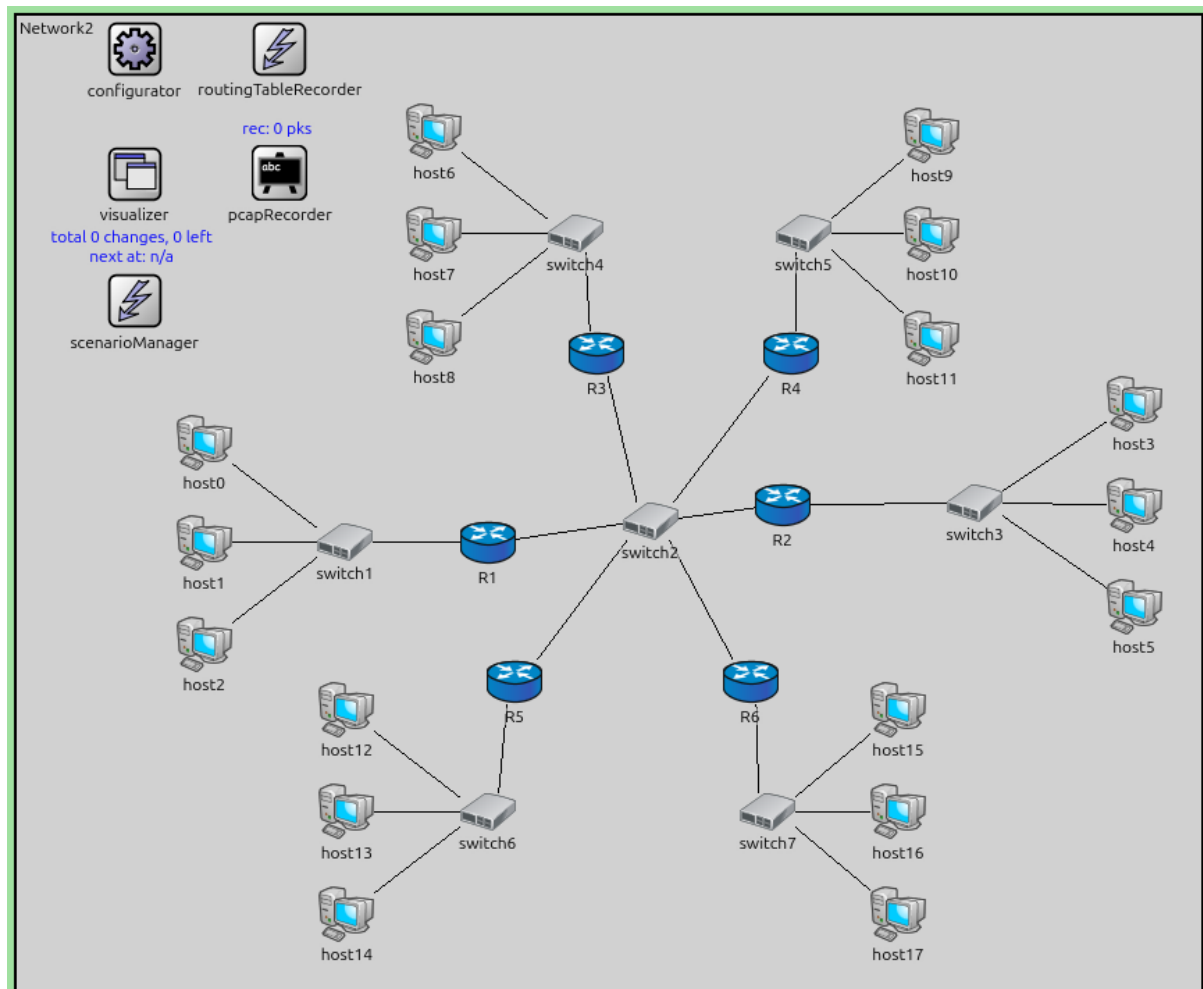
The DR is responsible for:

- Generating Network LSAs (Type-2) for the multi-access network
- Coordinating LSA flooding on the segment

The election is based on router priority (higher wins) and Router ID (as a tiebreaker).

Configuration

This step uses the Network2 topology with multiple routers connected to an Ethernet switch.



The configuration in `omnetpp.ini` is the following:

```
[Config Step6]
description = "OSPF DR/BDR election in a multi-access network (Ethernet)"
network = Network2

*.configurator.config = xml("<config> \
    <interface hosts='**' address='10.x.x.x' netmask='255.\
↪255.255.x' /> \
    <route hosts='host*' destination='*' netmask='0.0.0.0\
↪' interface='eth0' /> \
    </config>")

# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
```

(continues on next page)

(continued from previous page)

```
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 60s
```

Results

When the simulation starts:

1. All routers on the Ethernet segment exchange Hello packets.
2. Each router sees which other routers are present and their priorities.
3. The router with the highest priority becomes the DR. The router with the second-highest priority becomes the BDR. (If priorities are equal, the highest Router ID wins.)

In this simulation, all routers have the default priority of 1. R6 has the highest Router ID, so it becomes the DR. R5 has the second-highest Router ID, so it becomes the BDR.

4. Routers with priority 0 cannot become DR or BDR.
5. Other routers (called DROthers) form Full adjacencies only with the DR and BDR, remaining in 2-Way state with each other.
6. The DR generates a Network LSA describing all routers attached to the multi-access network.

The OSPF module logs show the election process and the final DR/BDR assignments. The routing tables and LSDBs reflect the hierarchical adjacency structure enabled by the DR/BDR mechanism.

Sources: `omnetpp.ini`, `Network2.ned`

7.9.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.10 Step 7. Influencing OSPF DR/BDR election

7.10.1 Goals

The goal of this step is to demonstrate how to influence the DR/BDR election by configuring router priorities.

The OSPF DR/BDR election is based primarily on the interface priority value (0-255). Network administrators can use priorities to control which router becomes DR and BDR. This is useful for ensuring that the most capable router (e.g., with better hardware or connectivity) serves as the DR.

Configuration

This configuration is based on Step 6. The OSPF configuration assigns different priorities to R3, making it more likely to become the DR.

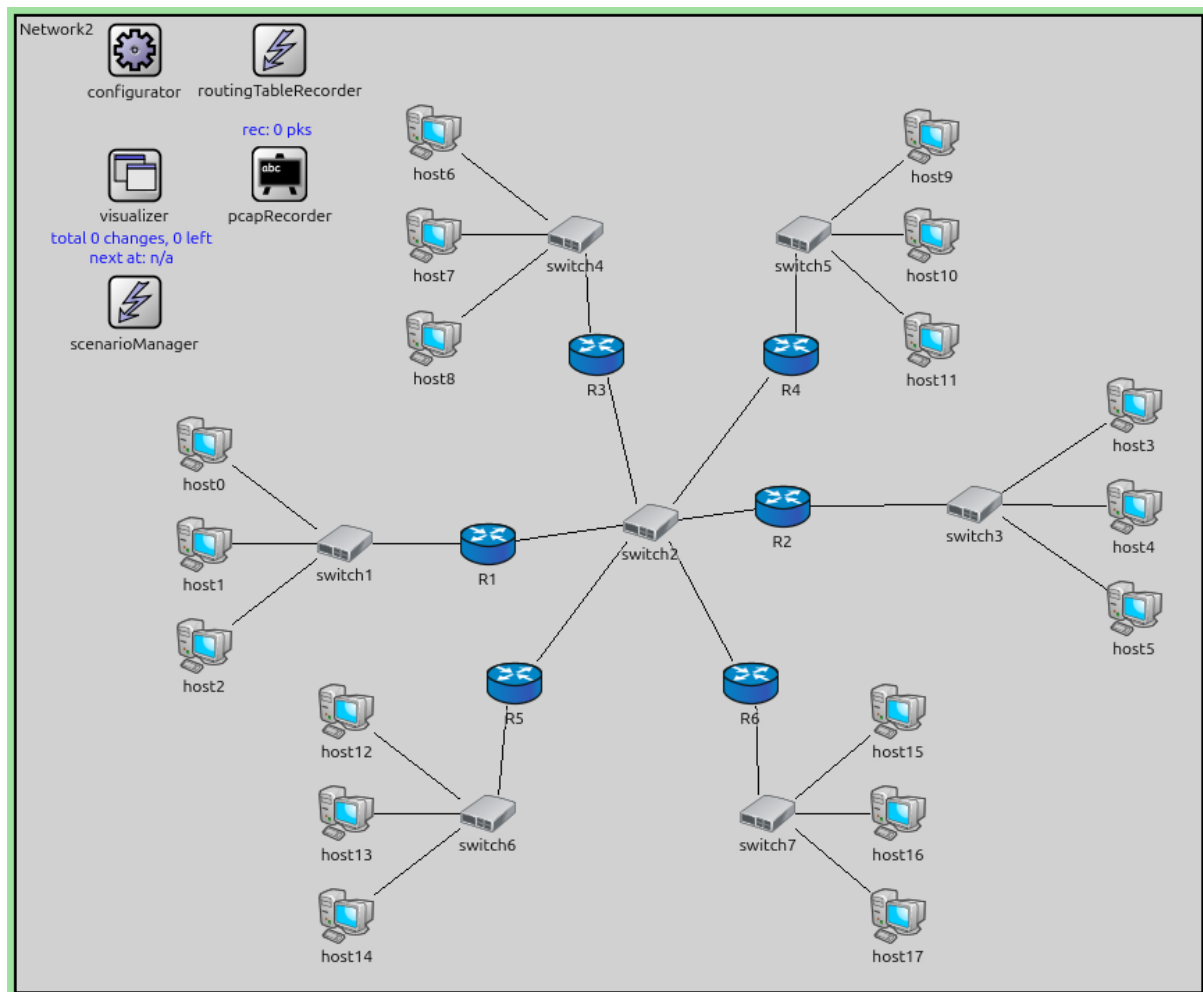
The configuration in `omnetpp.ini` is the following:

```
[Config Step7]
description = "Influencing OSPF DR/BDR election"
extends = Step6

# increasing the router priority of R3

*.R*.ospf.ospfConfig = xmldoc("ASConfig_priority.xml")
```

The OSPF configuration:




```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <Router name="R3" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceOutputCost='0' routerPriority="10" />
    <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
  </Router>

  <Router name="*" RFC1583Compatible="true">
    <BroadcastInterface ifName='eth[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
    <PointToPointInterface ifName='ppp[*]' areaID='0.0.0.0' interfaceOutputCost='0' /
  >
  </Router>

</OSPFASConfig>
```

Results

With the modified priorities:

1. **R3** has a higher priority than the other routers.
2. During the DR/BDR election, R3 is elected as the DR (or BDR) based on its higher priority.
3. Comparing with Step 6 (where all routers had default priorities), the DR/BDR assignment changes to reflect the priority configuration.

In this simulation, R3 becomes the DR because it has the highest priority (10). Among the remaining routers (which all have priority 1), R6 has the highest Router ID, so it becomes the BDR.

This demonstrates that network administrators have control over the DR/BDR election and can engineer the network to ensure specific routers take on these roles.

Important: The DR/BDR election is NOT preemptive. If a router with higher priority joins the network after a DR/BDR have already been elected, it will not automatically replace them. The election only occurs when there is no DR/BDR, or when the current DR/BDR fails.

Sources: omnetpp.ini, Network2.ned, ASConfig_priority.xml

7.10.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.11 Step 8. Setting all router priorities to zero

7.11.1 Goals

The goal of this step is to demonstrate what happens when all routers on a multi-access network have priority 0.

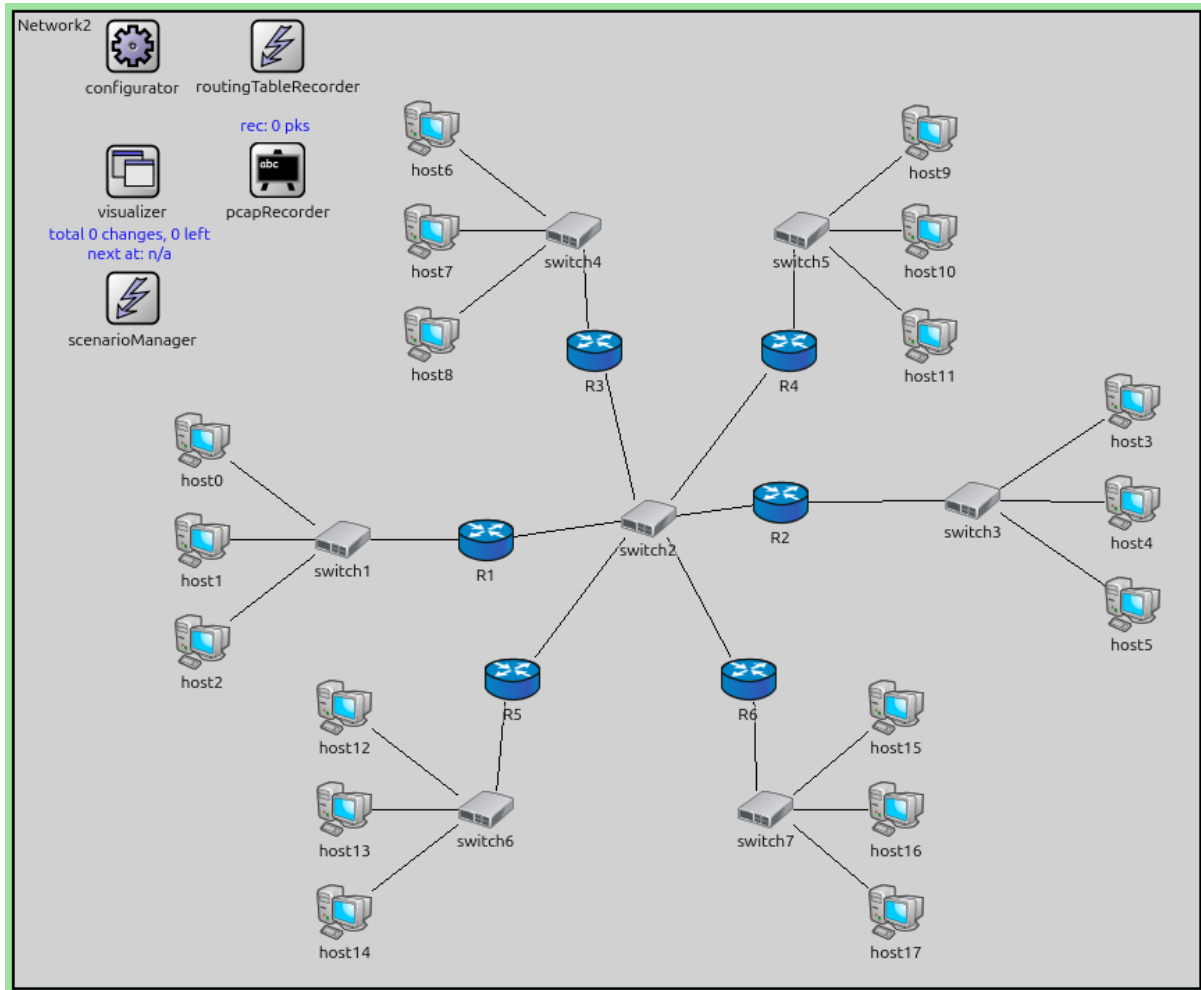
A router with OSPF interface priority 0 cannot become DR or BDR. If all routers on a multi-access network have priority 0, no DR or BDR will be elected, and:

- No Network LSA will be generated for that network.
- The network will not be properly recognized in the OSPF topology.

This effectively disables OSPF routing for that network segment.

Configuration

This configuration is based on Step 6. All routers have their priorities set to 0 for the multi-access interface.



The configuration in `omnetpp.ini` is the following:

[Config Step8]

```
description = "Setting all router ids to zero"
```

```
extends = Step6
```

```
# when the router id of a multi-access interface is zero then no Wait timer is_
↳ started.
# this means that no DR/BDR election will be performed and no Network LSA is_
↳ exchanged.
# this also means that the Ethernet network will not be recognized by any routers.
```

```
*.R*.ospf.ospfConfig = xmldoc("ASConfig_zero_priority.xml")
```

```
*.visualizer.routingTableVisualizer.destinationFilter = "R*"
```

```
*.visualizer.routingTableVisualizer.nodeFilter = "R* and not *.N*"
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↳ xsi:schemaLocation="OSPF.xsd">
```

(continues on next page)

(continued from previous page)

```

<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" />
  <BroadcastInterface ifName="eth1" routerPriority="0" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" />
  <BroadcastInterface ifName="eth1" routerPriority="0" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" routerPriority="0" />
  <BroadcastInterface ifName="eth1" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" routerPriority="0" />
  <BroadcastInterface ifName="eth1" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" routerPriority="0" />
  <BroadcastInterface ifName="eth1" />
</Router>

<Router name="R6" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" routerPriority="0" />
  <BroadcastInterface ifName="eth1" />
</Router>

</OSPFASConfig>

```

Results

When all routers have priority 0:

1. The DR/BDR election completes, but no router is elected as DR or BDR.
2. No Network LSA is generated for the Ethernet LAN between the routers.
3. The OSPF topology is incomplete - the multi-access network is not visible to the routing protocol.
4. Routing between networks may fail or use suboptimal paths since the multi-access segment is not properly integrated into the OSPF topology.

This demonstrates the critical role of the DR in multi-access networks and shows what happens when the DR/BDR election mechanism is effectively disabled.

Sources: omnetpp.ini, Network2.ned, ASConfig_zero_priority.xml

7.11.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.12 Step 9. High-priority OSPF router joins after OSPF DR/BDR election

7.12.1 Goals

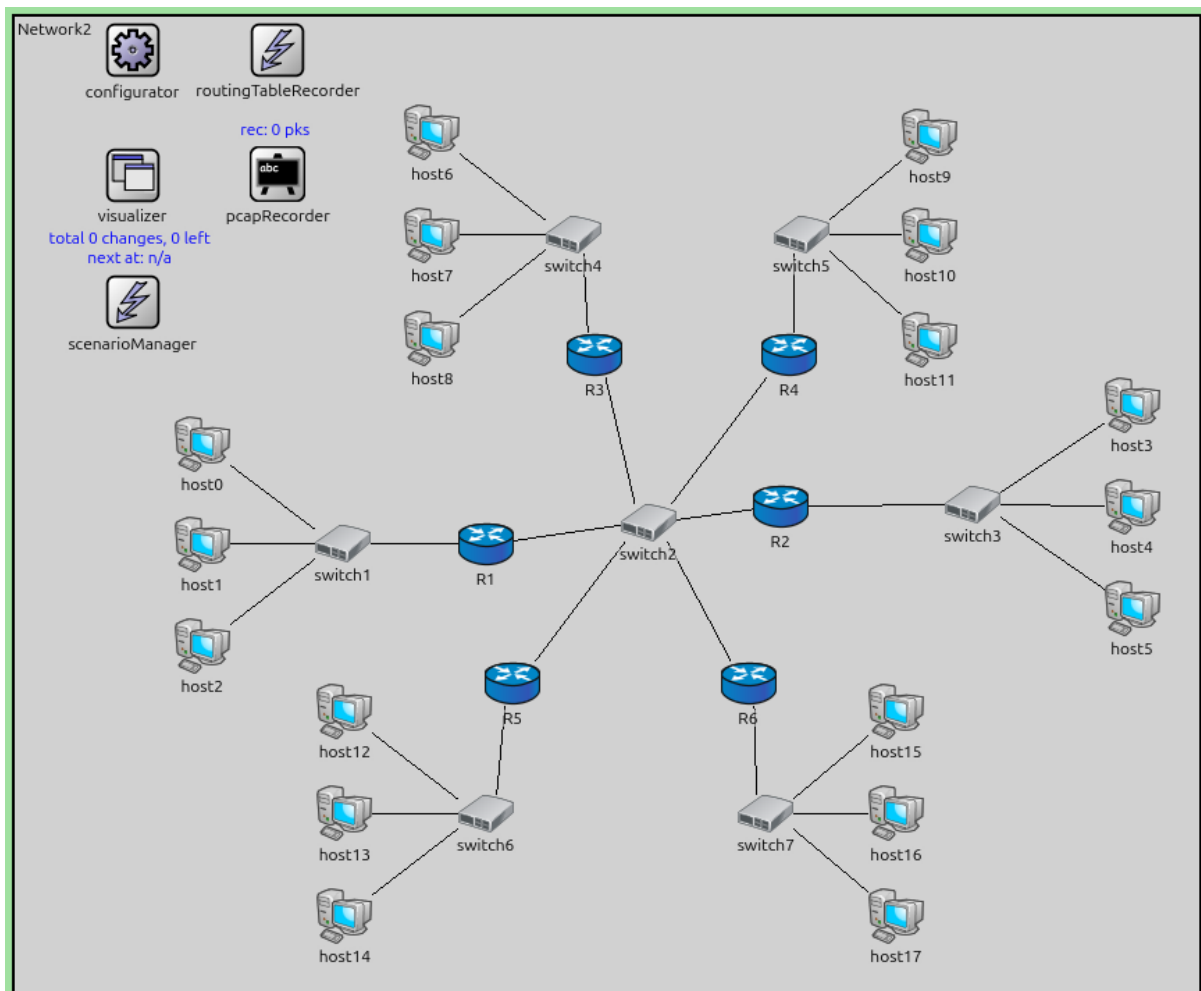
The goal of this step is to demonstrate that OSPF DR/BDR elections are not preemptive.

Once a DR and BDR have been elected on a multi-access network, they remain in those roles even if a router with higher priority joins the network later. The new router will not trigger a re-election. A new election only occurs when the current DR or BDR fails or is removed from the network.

This behavior promotes stability in the OSPF network, preventing frequent re-elections that could cause routing disruptions.

Configuration

This configuration is based on Step 6. Router R6 has a higher priority than the other routers but is configured to start OSPF later (at $t=60s$), after the other routers have already completed their DR/BDR election.



The configuration in `omnetpp.ini` is the following:

```
[Config Step9]
description = "High-priority OSPF router joins after OSPF DR/BDR election"
extends = Step6

# although R6 has the highest priority, but it does not trigger a DR/BDR re-election
```

(continues on next page)

(continued from previous page)

```
*.R6.ospf.startupTime = 60s
```

Results

The simulation demonstrates non-preemptive election:

1. Initially, routers (excluding R6) elect a DR and BDR based on their priorities/Router IDs.
2. At t=60s, R6 starts OSPF and joins the multi-access network.
3. R6 has the highest priority and could theoretically become the DR.
4. However, since a DR and BDR already exist, R6 does NOT trigger a re-election.
5. R6 becomes a DROther and forms adjacencies with the existing DR and BDR.
6. The DR and BDR remain unchanged despite R6's higher priority.

This behavior ensures OSPF network stability. If a re-election was needed, the administrator would need to manually restart OSPF on the current DR/BDR or take other administrative action.

The OSPF logs show R6 joining the network and accepting the existing DR/BDR without triggering an election.

Sources: omnetpp.ini, Network2.ned

7.12.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.13 Step 10. Network Topology Changes

7.13.1 Goals

The goal of this step is to examine how OSPF reacts to network topology changes.

OSPF routers monitor the state of their links. When a link state changes (e.g., up/down), the router generates a new LSA and floods it throughout the area. Other routers receive the update, update their LSDBs, and run the SPF algorithm to calculate new routes.

Configuration

This step uses the TopologyChange network. The OSPF configuration in ASConfig_tp_priority.xml assigns different priorities to the routers connected to the central switch (Switch2) to deterministically select the DR and BDR.

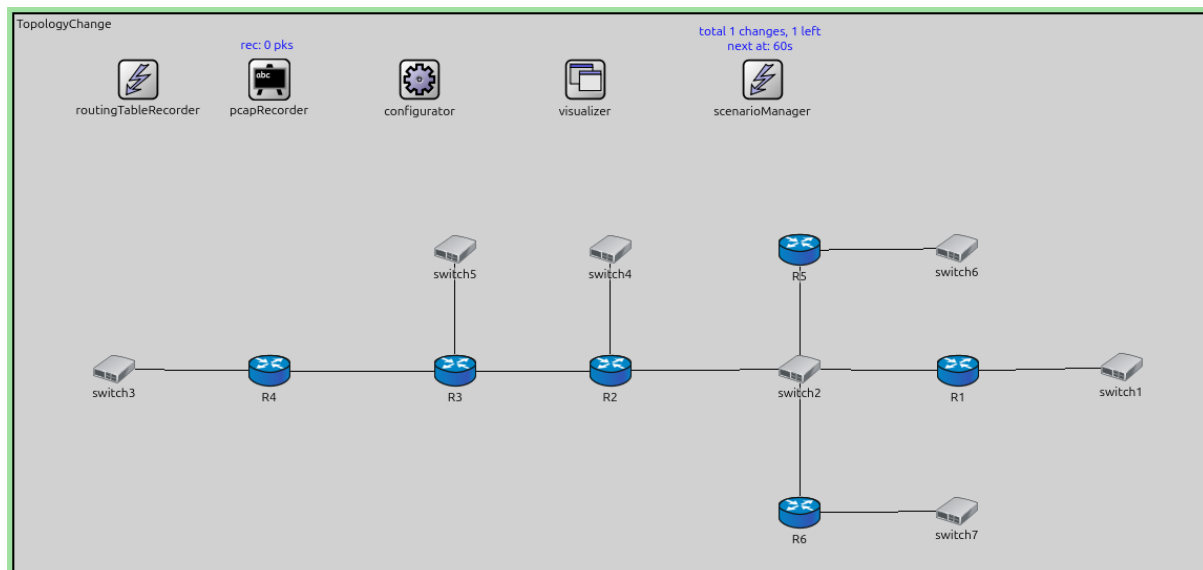
- **R1**: Priority 10 (DR)
- **R5**: Priority 9 (BDR)
- **R2, R6**: Priority 1 (DROthers)

The simulation script disconnects the link between **R4** and **Switch3** at t=60s.

```
network TopologyChange
{
    @display("bgb=1914.2025,894.73376");

    types:
        channel PppLink100M extends DatarateChannel
        {
            delay = 5us;
            datarate = 100Mbps;
        }
}
```

(continues on next page)



(continued from previous page)

```

    }

    channel PppLink10M extends DatarateChannel
    {
        delay = 5us;
        datarate = 10Mbps;
    }

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=652.46,105.545");
        }
        visualizer: IntegratedCanvasVisualizer {
            @display("p=918.72125,105.545");
        }
        routingTableRecorder: RoutingTableRecorder {
            @display("p=199,105");
        }
        pcapRecorder: PcapRecorder {
            @display("p=410,105");
        }
        scenarioManager: ScenarioManager {
            @display("p=1204.1725,105.545");
        }
        R2: Router {
            @display("p=959.4937,573.0675");
        }
        switch3: EthernetSwitch {
            @display("p=160.35374,570.4387");
        }
        R4: Router {
            @display("p=410.085,573.0675");
        }
        R3: Router {
            @display("p=709.7625,573.0675");
        }

```

(continues on next page)

(continued from previous page)

```

switch2: EthernetSwitch {
    @display("p=1264.4287,570.4387");
}
switch5: EthernetSwitch {
    @display("p=709.7625,378.54");
}
R1: Router {
    @display("p=1519.4175,573.0675");
}
switch4: EthernetSwitch {
    @display("p=959.4937,378.54");
}
switch1: EthernetSwitch {
    @display("p=1782.2925,567.81");
}
R5: Router {
    @display("p=1264.1412,379.0025");
}
R6: Router {
    @display("p=1264.1412,801.1825");
}
switch6: EthernetSwitch {
    @display("p=1517.6176,376.4075");
}
switch7: EthernetSwitch {
    @display("p=1517.6176,798.3675");
}
}
connections:
R2.pppg++ <--> PppLink100M <--> R3.pppg++;
R3.pppg++ <--> PppLink100M <--> R4.pppg++;
R4.ethg++ <--> Eth100M <--> switch3.ethg++;
R3.ethg++ <--> Eth100M <--> switch5.ethg++;
R2.ethg++ <--> Eth100M <--> switch2.ethg++;
R1.ethg++ <--> Eth100M <--> switch2.ethg++;
switch4.ethg++ <--> Eth100M <--> R2.ethg++;
switch1.ethg++ <--> Eth100M <--> R1.ethg++;
switch2.ethg++ <--> Eth100M <--> R5.ethg++;
R6.ethg++ <--> Eth100M <--> switch2.ethg++;
switch6.ethg++ <--> Eth100M <--> R5.ethg++;
switch7.ethg++ <--> Eth100M <--> R6.ethg++;

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step10]
description = "Network Topology Changes"
network = TopologyChange

*.R*.ospf.ospfConfig = xmldoc("ASConfig_tp_priority.xml")

*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "*"
*.visualizer.routingTableVisualizer.nodeFilter = "R*"

*.*.hasStatus = true

```

(continues on next page)

(continued from previous page)

```

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='60' src-module='R4' dest-module=
↪ 'switch3' /> \
                                </scenario>")

# application parameters
*.R3.numApps = 1
*.R3.app[0].typename = "PingApp"
*.R3.app[0].destAddr = "R1"
*.R3.app[0].startTime = 60s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↪ xsi:schemaLocation="OSPF.xsd">

  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceOutputCost='0' routerPriority="10" />
    <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceOutputCost='0' routerPriority="9" />
    <BroadcastInterface ifName="eth1" interfaceOutputCost='0' />
  </Router>

  <Router name="*" RFC1583Compatible="true">
    <BroadcastInterface ifName='eth[*]' areaID='0.0.0.0' interfaceOutputCost='0' />
    <PointToPointInterface ifName='ppp[*]' areaID='0.0.0.0' interfaceOutputCost='0' /
↪ >
  </Router>

</OSPFASConfig>

```

Results

When the link between R4 and Switch3 breaks at t=60s:

1. **R4** detects the link down event.
2. R4 generates a new Router LSA that *excludes* the link to Switch3's network.
3. R4 floods this new LSA to its neighbors (R3).
4. The LSA propagates through the network (R3 → R2 → Switch2 → others).
5. All routers update their LSDB and remove the route to the network that was behind Switch3 (if it's no longer reachable).

This demonstrates the basic mechanism of LSA flooding and database synchronization upon topology change.

Sources: omnetpp.ini, TopologyChange.ned, ASConfig_tp_priority.xml

7.13.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.14 Step 10a. Router R4 goes down

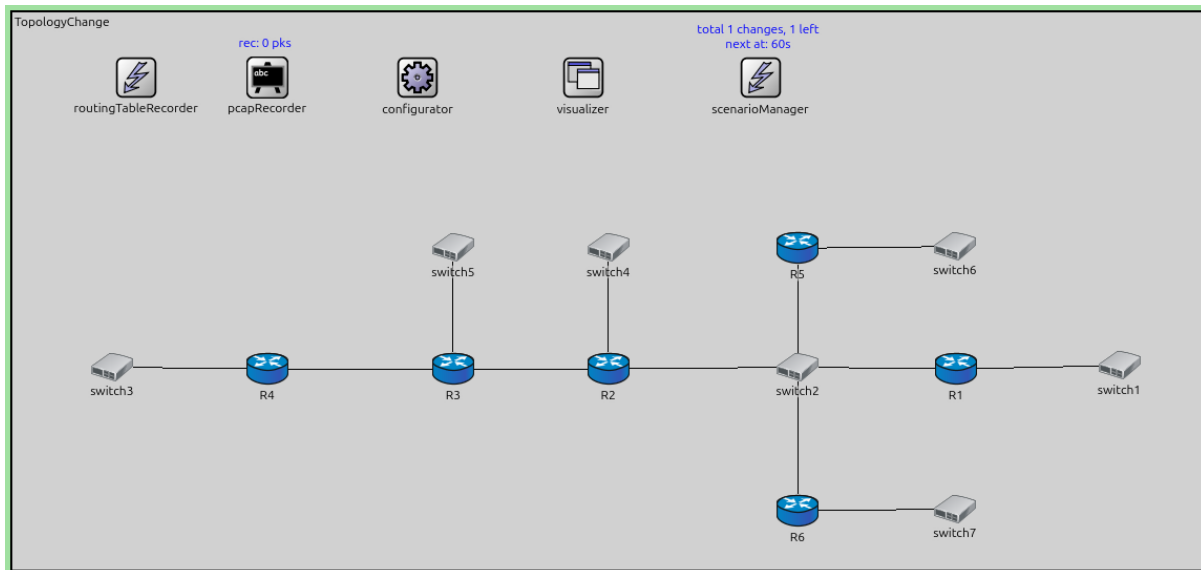
7.14.1 Goals

The goal of this step is to demonstrate OSPF's reaction when an entire router fails.

When a router completely fails (as opposed to just a single link), all of its adjacencies are lost simultaneously. Neighboring routers detect the failure and update the topology accordingly.

Configuration

This configuration is based on Step 10. The simulation script shuts down router **R4** at t=60s.



The configuration in `omnetpp.ini` is the following:

```
[Config Step10a]
description = "Router R4 goes down"
extends = Step10

*.scenarioManager.script = xml("<scenario> <at t='60'> <shutdown module='R4' /> </at>
↵</scenario>")
```

Results

When R4 is shutdown at t=60s:

1. All routers adjacent to **R4** (R3, and any others) detect the adjacency loss when they stop receiving Hello packets.
2. After the Dead Interval expires, these neighbors declare R4 dead and remove it from their neighbor lists.
3. Routers that had adjacencies with R4 generate new LSAs reflecting the topology change (removing links to R4).
4. These LSAs are flooded throughout the area.
5. All routers update their LSDBs, removing R4's Router LSA and any links to R4.

In General, routes that used a failed router as a next-hop or transit router are recomputed, using alternative paths if available.

Sources: `omnetpp.ini`, `TopologyChange.ned`, `ASConfig_tp_priority.xml`

7.14.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.15 Step 10b. Router R2 (DROTHER) goes down

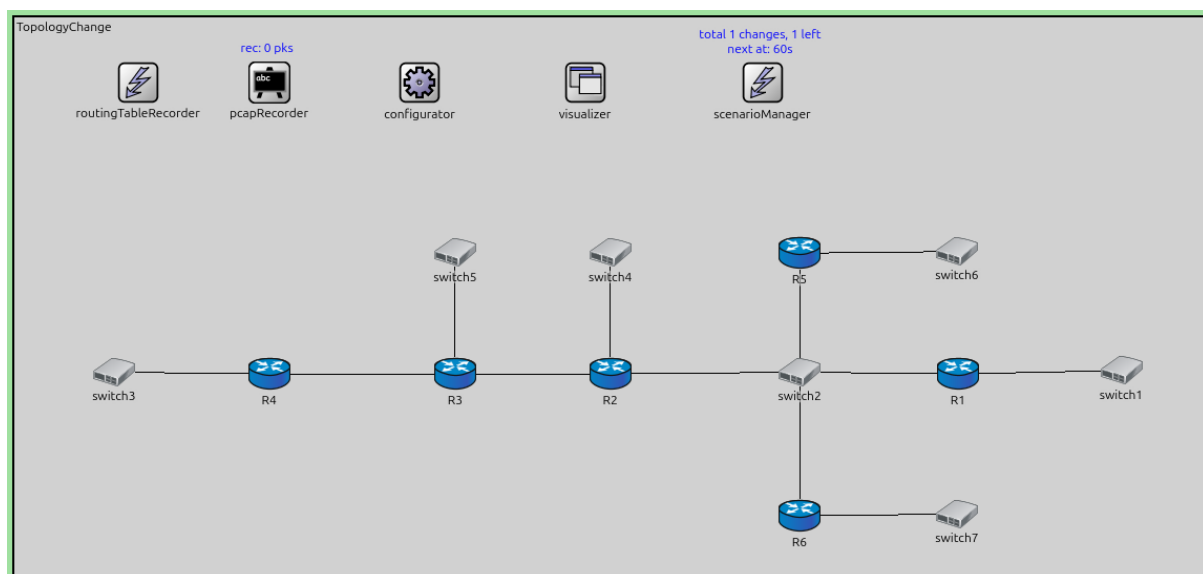
7.15.1 Goals

The goal of this step is to demonstrate what happens when a DROther router fails on a multi-access network.

When a DROther (a router that is neither DR nor BDR) fails, the impact is limited since DROthers do not have special responsibilities in the OSPF protocol operation. The DR and BDR are unaffected, and the multi-access network continues to operate normally.

Configuration

This configuration is based on Step 10. The simulation script shuts down router **R2** (a DROther) at t=60s.



The configuration in `omnetpp.ini` is the following:

```
[Config Step10b]
description = "Router R2 (DROTHER) goes down"
extends = Step10

*.scenarioManager.script = xml("<scenario> <at t='60'> <shutdown module='R2' /> </at>
↵</scenario>")

# application parameters
*.R3.numApps = 1
*.R3.app[0].typename = "PingApp"
*.R3.app[0].destAddr = "R4"
*.R3.app[0].startTime = 60s

*.R1.numApps = 1
*.R1.app[0].typename = "PingApp"
*.R1.app[0].destAddr = "R6"
*.R1.app[0].startTime = 60s
```

Results

When R2 (a DROther) is shutdown at t=60s:

1. The DR and BDR on Switch2 detect that R2 has failed when Hello packets stop arriving.
2. The DR generates a new Network LSA for Switch2 that no longer lists R2 as an attached router.
3. Other routers adjacent to R2 on other interfaces also detect the failure and update their LSAs accordingly.
4. The topology change is flooded throughout the area.
5. Importantly, **no DR/BDR re-election occurs** on Switch2. The existing DR and BDR continue their roles.

In general, Routing around the failed router is recomputed using alternative paths, if available.

This demonstrates that DROther failures have minimal impact on the OSPF protocol operation on multi-access networks.

Sources: `omnetpp.ini`, `TopologyChange.ned`, `ASConfig_tp_priority.xml`

7.15.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.16 Step 10d. Router R1 (DR) goes down

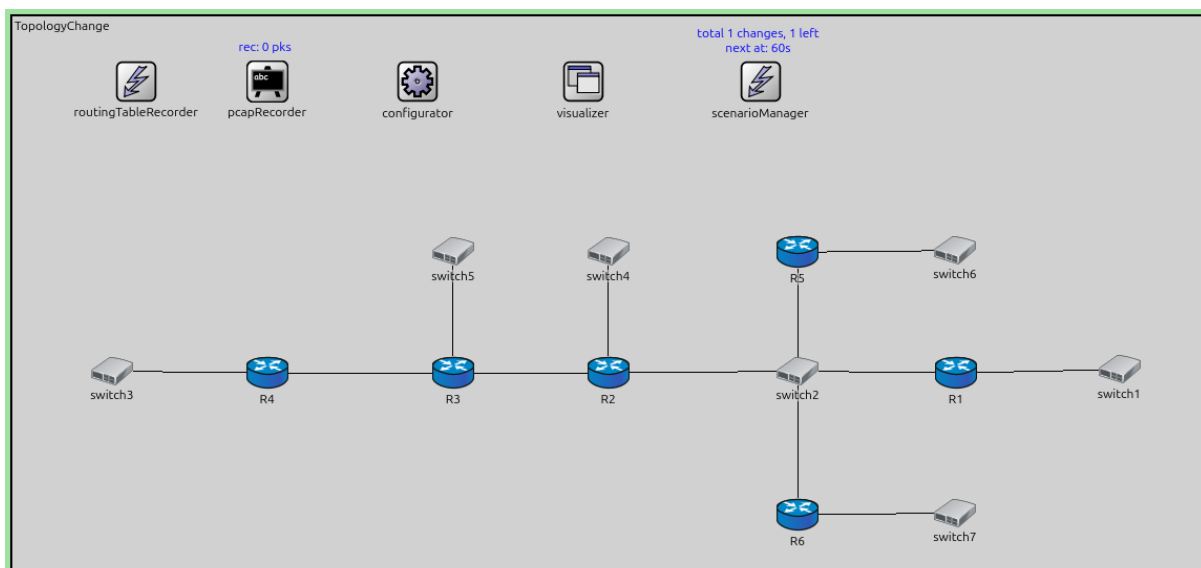
7.16.1 Goals

The goal of this step is to demonstrate what happens when the Designated Router (DR) fails on a multi-access network.

When the DR fails, the BDR immediately assumes the DR role, and a new BDR is elected from the remaining DROthers. This ensures minimal disruption to the network.

Configuration

This configuration is based on Step 10. The simulation script shuts down router **R1** (the DR) at t=60s.



The configuration in `omnetpp.ini` is the following:

```
[Config Step10d]
description = "Router R1 (DR) goes down"
extends = Step10
```

(continues on next page)

(continued from previous page)

```
*.scenarioManager.script = xml("<scenario> <at t='60'> <shutdown module='R1'/> </at>
↳</scenario>")

# application parameters
*.R3.numApps = 1
*.R3.app[0].typename = "PingApp"
*.R3.app[0].destAddr = "R6"
*.R3.app[0].startTime = 60s
```

Results

When R1 (the DR) is shutdown at t=60s:

1. The BDR and other routers on Switch2 detect that R1 (the DR) has failed.
2. **The BDR (R5) immediately becomes the new DR.** This is a critical OSPF feature - the BDR is pre-synchronized and ready to take over.
3. **A new BDR is elected** from the remaining DROthers (R6). Since all remaining DROthers have priority 1, the new DBR is R4.
4. The new DR (formerly the BDR) begins generating Network LSAs for the multi-access network.
5. All routers update their adjacencies to reflect the new DR/BDR assignments.
6. Routing is recomputed to account for the lost R1 and any topology changes.

The quick promotion of the BDR to DR ensures minimal disruption. The Network LSA generation continues with minimal delay, and the multi-access network remains fully functional.

This demonstrates OSPF's high availability design for multi-access networks through the DR/BDR redundancy mechanism.

Sources: `omnetpp.ini`, `TopologyChange.ned`, `ASConfig_tp_priority.xml`

7.16.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.17 Step 11. Configure an interface as NoOSPF

7.17.1 Goals

The goal of this step is to demonstrate the effect of excluding an interface from OSPF using the NoOSPF interface mode.

Setting an interface to NoOSPF mode effectively removes it from OSPF processing. The interface will not:

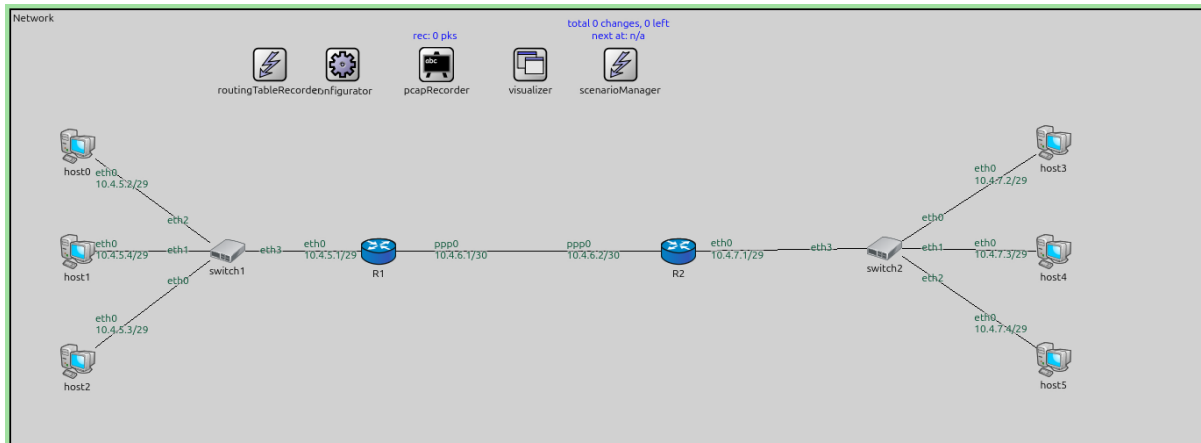
- Send or receive OSPF packets
- Form adjacencies
- Be advertised in OSPF LSAs

This is equivalent to not including the interface in the OSPF configuration at all.

Configuration

This configuration is based on Step 3. Each router's eth0 interface (the one facing the switch) is configured with `interfaceMode="NoOSPF"`.

The configuration in `omnetpp.ini` is the following:



[Config Step11]

description = "Configure an interface as NoOSPF"

extends = Step3

setting interfaceMode to "NoOSPF" is equal to removing the interface from the XML
↪ config

.R.ospf.ospfConfig = xmldoc("ASConfig_NoOspf.xml")

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  ↪xsi:schemaLocation="OSPF.xsd">

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceMode="NoOSPF" />
    <PointToPointInterface ifName="ppp0" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceMode="NoOSPF" />
    <PointToPointInterface ifName="ppp0" />
  </Router>

</OSPFASConfig>
```

Results

With the interface configured as NoOSPF:

1. The interface does not participate in OSPF at all.
2. No Hello packets are sent on the interface.
3. The network connected to this interface is not advertised in Router LSAs.
4. Other routers are unaware of this network and cannot route to it via OSPF.

This mode is useful when you want OSPF running on a router but need to exclude specific interfaces from OSPF processing, perhaps because they connect to non-OSPF networks or for security reasons.

Sources: omnetpp.ini, Network.ned, ASConfig_NoOspf.xml

7.17.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.18 Step 12. Configure an interface as Passive

7.18.1 Goals

The goal of this step is to demonstrate the Passive interface mode in OSPF.

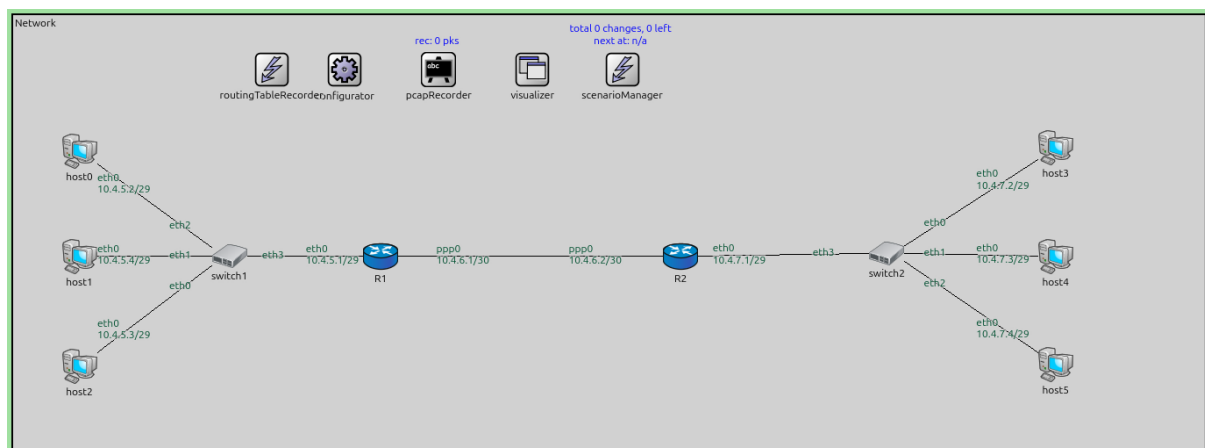
A passive interface in OSPF is one that:

- **Does NOT** send or receive Hello packets (no adjacencies are formed)
- **IS** advertised in Router LSAs (the network is visible to OSPF)

Passive mode is useful for stub networks (networks with only hosts, no other OSPF routers). It reduces OSPF overhead by not running the protocol where it's unnecessary, while still advertising the network so it's reachable from other parts of the OSPF domain.

Configuration

This configuration is based on Step 3. One of the router's interfaces is configured with `interfaceMode="Passive"`.



The configuration in `omnetpp.ini` is the following:

```
[Config Step12]
description = "Configure an interface as Passive"
extends = Step3

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Passive.xml")
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" />
  </Router>
```

(continues on next page)

(continued from previous page)

```

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" />
</Router>

</OSPFASConfig>

```

Results

With the interface configured as Passive:

1. No OSPF Hello packets are sent on the interface.
2. No OSPF adjacencies can form on this interface.
3. However, the network is still advertised in the router's Router LSA as a stub network.
4. Other OSPF routers learn about this network and can route to it.
5. Hosts on the passive network can communicate with the rest of the OSPF domain.

Passive mode is commonly used on interfaces connected to end-user networks or networks where no other OSPF routers are expected, reducing unnecessary protocol overhead.

Sources: `omnetpp.ini`, `Network.ned`, `ASConfig_Passive.xml`

7.18.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.19 Step 13. Freshness of a LSA

7.19.1 Goals

The goal of this step is to demonstrate LSA aging and the MaxAge mechanism in OSPF.

Every LSA has an age field that starts at 0 when originated and increments over time (up to MaxAge = 3600 seconds). LSAs are periodically refreshed by their originating router before they reach MaxAge. If a router becomes unreachable and cannot refresh its LSAs, those LSAs age to MaxAge and are then flushed from the LSDB.

When a link or router disappears, the affected router can immediately set the LSA age to MaxAge and flood it, accelerating the removal of stale information from the network.

Configuration

This step uses the Freshness network. The simulation script disconnects a link at `t=60s`.

The configuration in `omnetpp.ini` is the following:

```

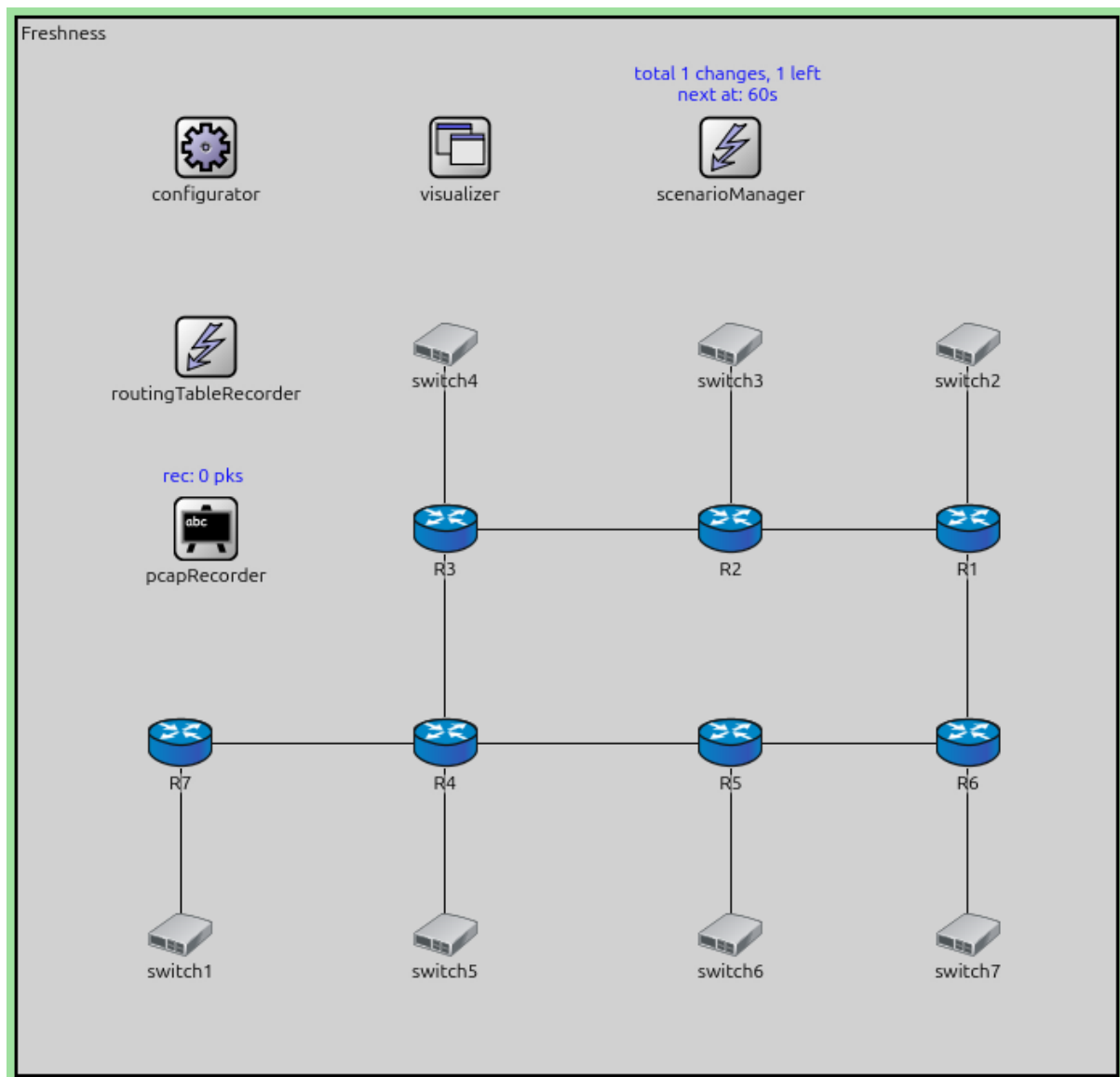
[Config Step13]
description = "Freshness of a LSA"
network = Freshness

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='60' src-module='R1' dest-module=
↪ 'switch2' /> \
                                </scenario>")

# application parameters
*.R7.numApps = 1
*.R7.app[0].typename = "PingApp"

```

(continues on next page)



(continued from previous page)

```
*.R7.app[0].destAddr = "R1"
*.R7.app[0].startTime = 60s
```

Results

The simulation demonstrates LSA aging:

1. During normal operation, routers periodically refresh their LSAs (every LSRefreshTime, typically 1800 seconds) to prevent them from aging out.
2. When the link between R1 and switch2 is disconnected at t=60s:
 - R1 detects the link down event
 - R1 generates a new Router LSA that no longer includes this link
 - The old LSA (which included the link) is effectively replaced
3. The OSPF module logs show LSA age values incrementing over time.

If a router were to fail completely (unable to refresh its LSAs), its LSAs would eventually age to MaxAge (3600s) and be flushed from all routers' LSDBs.

By flooding an LSA with age=MaxAge, a router can rapidly remove obsolete information from the network rather than waiting for natural aging.

This mechanism ensures that the OSPF database remains current and that stale information is eventually removed even if routers fail silently.

Sources: `omnetpp.ini`, `Freshness.ned`

7.19.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.20 Step 14. Hierarchical OSPF topology and summary LSA

7.20.1 Goals

The goal of this step is to demonstrate hierarchical OSPF with multiple areas and to examine how Area Border Routers (ABRs) generate and flood Summary LSAs to provide inter-area reachability.

OSPF uses a hierarchical design to improve scalability. The network is divided into areas, with Area 0 (the backbone area) connecting all other areas. Routers with interfaces in multiple areas are called ABRs. ABRs generate Summary LSAs (Type-3 LSAs) to advertise networks from one area into another area, allowing routers to learn about networks outside their own area without needing to know the detailed topology of other areas.

Configuration

This step uses the `OSPF_AreaTest` network with three OSPF areas:

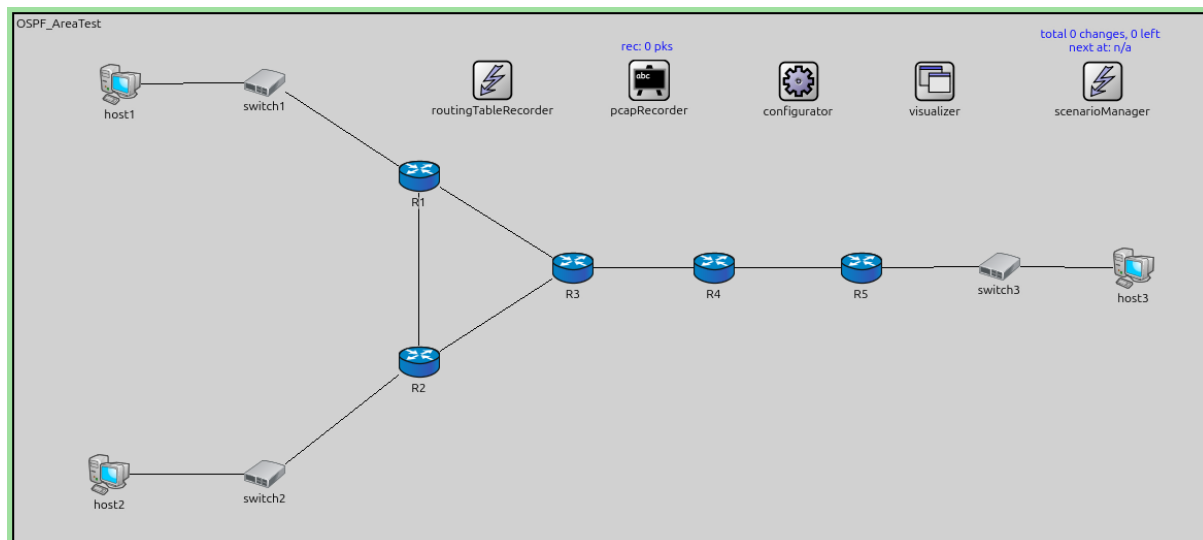
- **Area 0.0.0.0** (backbone): Contains the link between **R3** and **R4**
- **Area 0.0.0.1**: Contains **R1**, **R2**, **R3**, **host1**, and **host2**
- **Area 0.0.0.2**: Contains **R4**, **R5**, and **host3**

R3 and **R4** are ABRs because they have interfaces in multiple areas.

```
network OSPF_AreaTest
{
    @display("bgb=1153.1,510.9");

    types:
```

(continues on next page)



(continued from previous page)

```

channel PppLink100M extends DatarateChannel
{
    delay = 5us;
    datarate = 100Mbps;
}

submodules:
    configurator: Ipv4NetworkConfigurator {
        @display("p=760.5,65");
    }
    visualizer: IntegratedCanvasVisualizer {
        @display("p=893.1,65");
    }
    routingTableRecorder: RoutingTableRecorder {
        @display("p=464,64");
    }
    pcapRecorder: PcapRecorder {
        @display("p=616,64");
    }
    scenarioManager: ScenarioManager {
        @display("p=1055.6,65");
    }
    host1: StandardHost {
        @display("p=102.7,67.6");
    }
    host2: StandardHost {
        @display("p=92.299995,445.9");
    }
    R1: Router {
        @display("p=392.6,157.3");
    }
    R2: Router {
        @display("p=392.6,336.69998");
    }
    R3: Router {
        @display("p=542.1,245.7");
    }

```

(continues on next page)

(continued from previous page)

```

switch1: EthernetSwitch {
    @display("p=243.09999,66.299995");
}
switch2: EthernetSwitch {
    @display("p=243.09999,445.9");
}
host3: StandardHost {
    @display("p=1085.5,245.7");
}
R4: Router {
    @display("p=678.6,245.7");
}
R5: Router {
    @display("p=821.6,245.7");
}
switch3: EthernetSwitch {
    @display("p=955.5,244.4");
}

connections:
host1.ethg++ <--> Eth100M <--> switch1.ethg++;
switch1.ethg++ <--> Eth100M <--> R1.ethg++;
R1.pppg++ <--> PppLink100M <--> R3.pppg++;
switch2.ethg++ <--> Eth100M <--> host2.ethg++;
R1.pppg++ <--> PppLink100M <--> R2.pppg++;
R2.pppg++ <--> PppLink100M <--> R3.pppg++;
R2.ethg++ <--> Eth100M <--> switch2.ethg++;
R4.pppg++ <--> PppLink100M <--> R5.pppg++;
R5.ethg++ <--> Eth100M <--> switch3.ethg++;
switch3.ethg++ <--> Eth100M <--> host3.ethg++;
R3.pppg++ <--> PppLink100M <--> R4.pppg++;
}

```

The configuration in `omnetpp.ini` is the following:

```

[Config Step14]
description = "Hierarchical OSPF topology and summary LSA"
network = OSPF_AreaTest

*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    \
    <interface among='host1 host2 R1 R2 R3' address='192.
↳168.11.x' netmask='255.255.255.x' /> \
    <interface among='R3 R4 R5 host3' address='192.168.22.
↳x' netmask='255.255.255.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↳' interface='eth0' /> \
    </config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area.xml")

```

(continues on next page)

(continued from previous page)

```

*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.routingTableVisualizer.destinationFilter = "R*"
*.visualizer.routingTableVisualizer.nodeFilter = "R*"

# application parameters
*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[0].destAddr = "host3"
*.host1.app[0].startTime = 60s

```

The OSPF configuration in `ASConfig_Area.xml` assigns interfaces to areas:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">

```

(continues on next page)

(continued from previous page)

```

<PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
<PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
<PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
</Router>

</OSPFASConfig>

```

Results

After OSPF convergence, the network exhibits the following behavior:

1. **Intra-area routing:** Within Area 0.0.0.1, routers **R1**, **R2**, and **R3** exchange Router LSAs to build a complete view of the area topology. Similarly, **R4** and **R5** exchange Router LSAs within Area 0.0.0.2.
2. **Summary LSA generation:**
 - **R3** (ABR between Area 0.0.0.1 and Area 0.0.0.0) generates Summary LSAs to advertise networks from Area 0.0.0.1 into Area 0.0.0.0, and vice versa. For example, R3 advertises the 192.168.11.x/30 networks into the backbone.
 - **R4** (ABR between Area 0.0.0.2 and Area 0.0.0.0) generates Summary LSAs to advertise the 192.168.22.x/30 networks from Area 0.0.0.2 into Area 0.0.0.0.
 - Both ABRs then propagate Summary LSAs from the backbone into their respective non-backbone areas.
3. **Inter-area routing:** Routers use Summary LSAs to install inter-area routes in their routing tables. For example:
 - **R1** learns about the 192.168.22.x networks via Summary LSAs from **R3**
 - **R5** learns about the 192.168.11.x networks via Summary LSAs from **R4**

The routing table shows inter-area routes (learned via Summary LSAs) alongside intra-area routes.

Sources: omnetpp.ini, OSPF_AreaTest.ned, ASConfig_Area.xml

7.20.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.21 Step 15. Set advertisement of a network to ‘false’

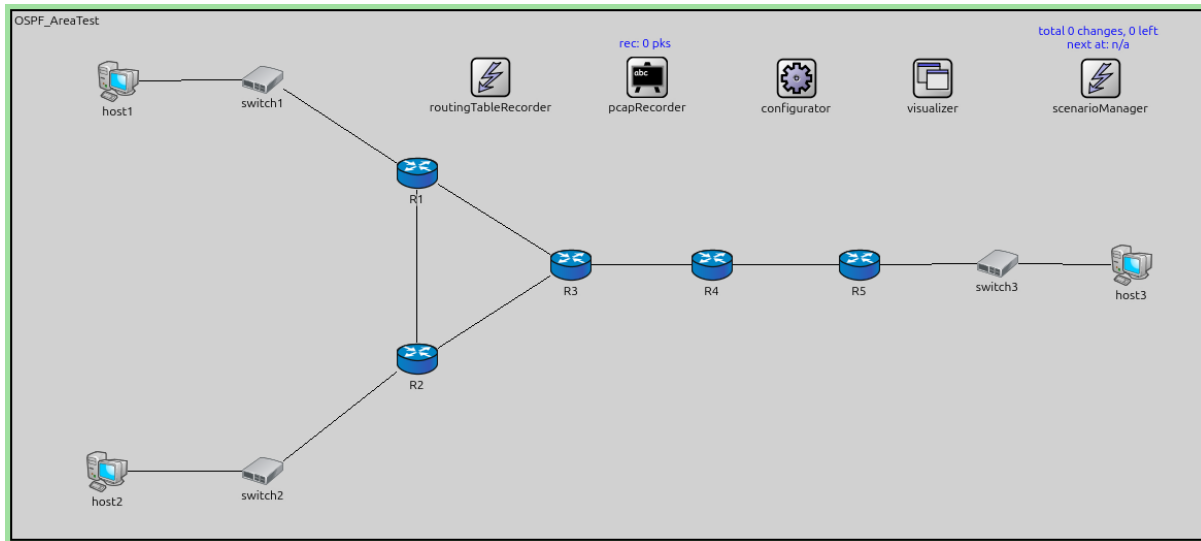
7.21.1 Goals

The goal of this step is to demonstrate how to suppress the advertisement of a network using the `advertise="false"` attribute in the OSPF configuration.

By default, OSPF routers advertise all networks configured in their areas. However, in some scenarios, you may want to prevent a specific network from being advertised to other areas. This can be achieved by setting the `advertise` attribute to `false` for a specific `AddressRange` in the OSPF area configuration. When this is set, the ABR will not generate a Summary LSA for that network, effectively hiding it from routers in other areas.

Configuration

This configuration is based on step 14. The only change is in the OSPF configuration file, where the network connected to **R5**'s eth0 interface (192.168.22.0/30, which includes **host3**) is configured with `advertise="false"`.



The configuration in `omnetpp.ini` is the following:

```
[Config Step15]
description = "Set advertisement of a network to 'false'"
extends = Step14

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_NoAdvertisement.xml")

# application parameters
*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[0].destAddr = "host2"
*.host1.app[0].startTime = 60s
```

The OSPF configuration shows the `advertise="false"` attribute on line 27:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />
```

(continues on next page)

(continued from previous page)

```

<AddressRange address="R3>R1" mask="R3>R1" />
<AddressRange address="R3>R2" mask="R3>R2" />
</Area>

<Area id="0.0.0.2">
  <AddressRange address="R4>R5" mask="R4>R5" />

  <AddressRange address="R5>R4" mask="R5>R4" />
  <AddressRange address="R5>switch3" mask="R5>switch3" advertise="false" />
</Area>

<!-- Routers -->
<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
</Router>

</OSPFASConfig>

```

Results

When `advertise="false"` is set for the R5->switch3 network (192.168.22.0/30):

1. **R5** still learns about this directly connected network and installs it in its own routing table.
2. **R4** (the ABR for Area 0.0.0.2) **does not generate a Summary LSA** for the 192.168.22.0/30 network to advertise it into Area 0.0.0.0 or into other areas.
3. As a result, routers in Area 0.0.0.1 (**R1, R2, R3**) **do not learn** about the 192.168.22.0/30 network and cannot reach **host3**.
4. Routers in Area 0.0.0.1 can still reach the 192.168.22.4/30 network (the R4-R5 link), because that network is still being advertised normally.

Comparing the routing tables from Step 14 and Step 15 confirms that the 192.168.22.0/30 network is absent from routers in Area 0. 0.0.1 in this step, while it was present in Step 14.

This feature is useful when you want to keep certain networks local to an area and prevent them from being visible to the rest of the OSPF domain.

Sources: `omnetpp.ini`, `OSPF_AreaTest.ned`, `ASConfig_Area_NoAdvertisement.xml`

7.21.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.22 Step 16. OSPF topology change in multi-area OSPF

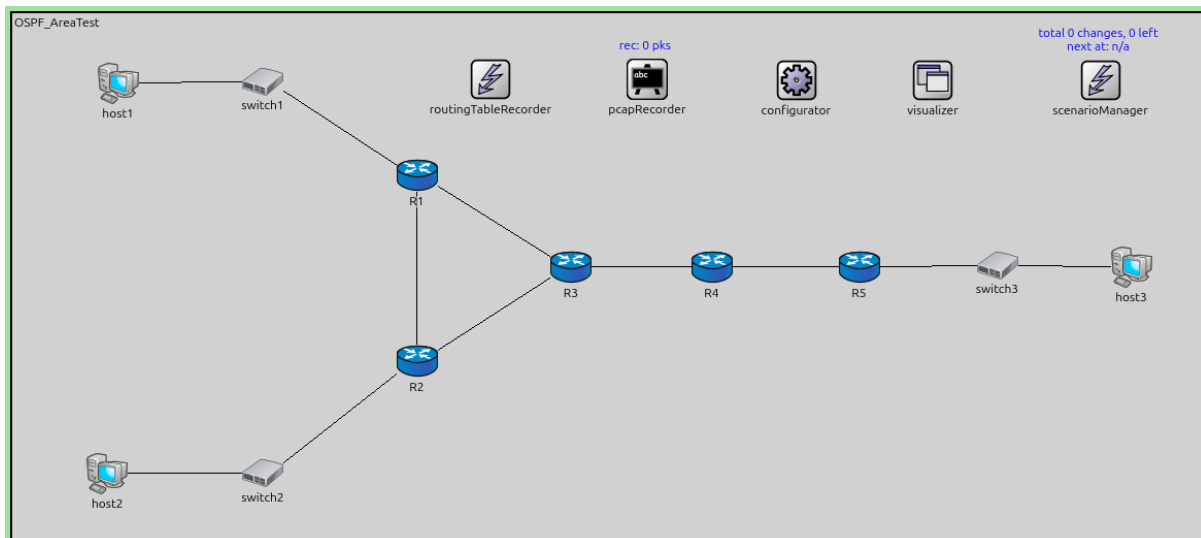
7.22.1 Goals

The goal of this step is to examine how OSPF reacts to network topology changes in a multi-area environment and how LSA updates propagate across area boundaries.

When a link fails in a multi-area OSPF network, the affected router generates new LSAs to reflect the topology change. These LSAs are flooded within the area, and ABRs generate updated Summary LSAs to propagate the change to other areas. This step demonstrates the cascading effect of topology changes through the hierarchical OSPF structure.

Configuration

This configuration is based on step 14. The simulation script disconnects the link between **R1** and **switch1** at $t=60s$, which isolates **host1** from the rest of the network.



The configuration in `omnetpp.ini` is the following:

```
[Config Step16]
description = "OSPF topology change in multi-area OSPF"
extends = Step14

*.scenarioManager.script = xml("<scenario> \
                                <disconnect t='60' src-module='R1' dest-module=
↪ 'switch1' /> \
                                </scenario>")

*.visualizer.routingTableVisualizer.destinationFilter = "R* or host1"

# application parameters
*.host2.numApps = 1
```

(continues on next page)

(continued from previous page)

```
*.host2.app[0].typename = "PingApp"
*.host2.app[0].destAddr = "host3"
*.host2.app[0].startTime = 60s
```

Results

When the link between **R1** and **switch1** breaks at t=60s:

1. **R1** detects the link down event on its eth0 interface.
2. **R1** generates a new Router LSA that no longer includes the link to the 192.168.11.0/30 network (the network containing **host1**).
3. This Router LSA is flooded within Area 0.0.0.1 to **R2** and **R3**.
4. **R2** and **R3** update their LSDBs and run the SPF algorithm. They remove the route to 192.168.11.0/30 from their routing tables because it was reachable only via **R1**.
5. **R3** (as an ABR) regenerates Summary LSAs for Area 0.0.0.1 that it advertises into Area 0.0.0.0. The Summary LSA for 192.168.11.0/30 is withdrawn because no other path exists.
6. **R4** (the other ABR) receives the updated Summary LSAs from **R3** via the backbone and updates its LSDB and routing table accordingly.
7. **R4** then generates new Summary LSAs to advertise into Area 0.0.0.2, informing **R5** about the topology change.
8. **R5** updates its routing table to remove the route to 192.168.11.0/30.

The routing table visualizer output shows that at t=60s, the 192.168.11.0/30 network becomes unreachable from all other routers in the network. The routing table changes propagate through the multi-area hierarchy: first within Area 0.0.0.1 (intra-area), then across the backbone (inter-area), and finally to Area 0.0.0.2 (inter-area).

This demonstrates how OSPF's hierarchical design efficiently propagates topology changes across area boundaries using Summary LSAs, while containing the detailed topology changes (Router LSAs) within their originating area.

Sources: `omnetpp.ini`, `OSPF_AreaTest.ned`

7.22.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.23 Step 17. Loop avoidance in multi-area OSPF topology

7.23.1 Goals

The goal of this step is to demonstrate how OSPF prevents routing loops in multi-area topologies.

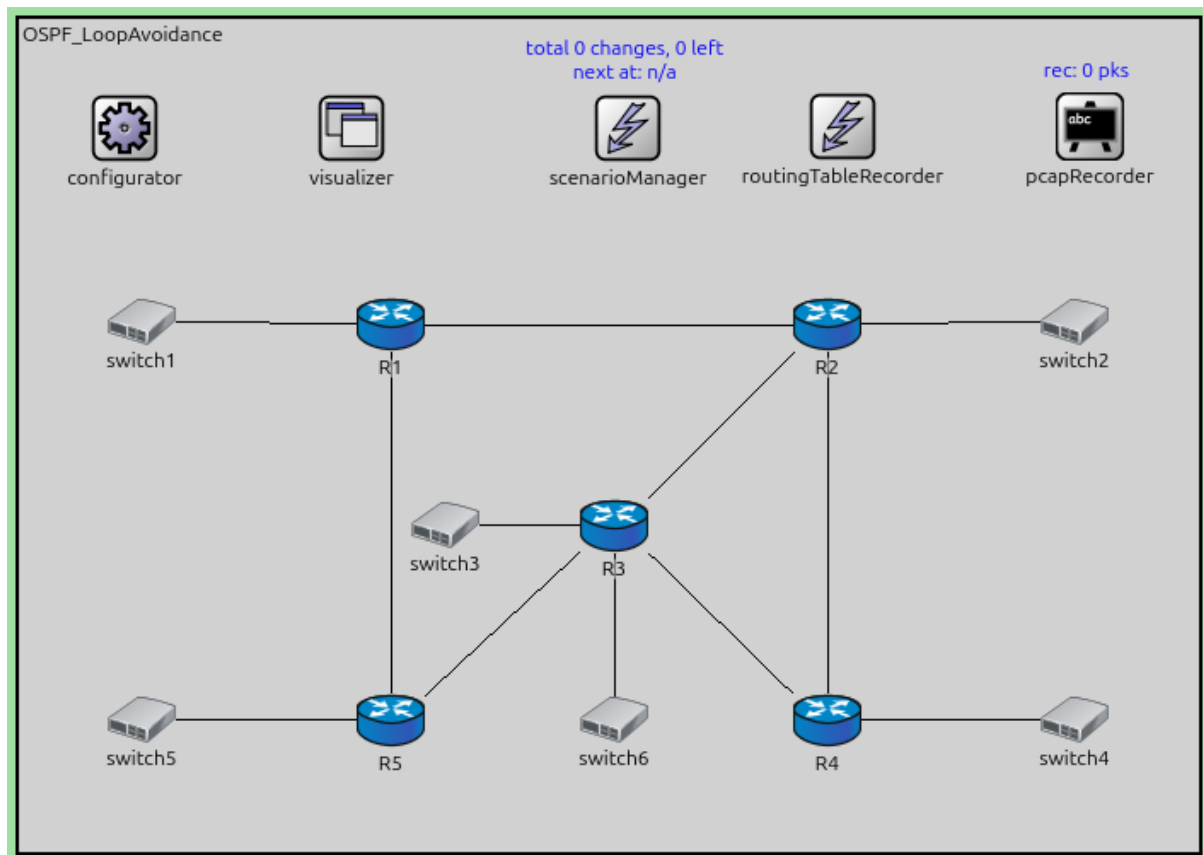
In multi-area OSPF, there is a risk of routing loops if areas are not properly connected to the backbone (Area 0). OSPF has a rule that inter-area routes (learned via Summary LSAs) are only accepted from the backbone area. This ensures a hierarchical topology and prevents loops.

If an area is not directly connected to the backbone, it must use a virtual link to logically connect to Area 0.

Configuration

This step uses the `OSPF_LoopAvoidance` network with a topology that could create loops if not properly configured.

The configuration in `omnetpp.ini` is the following:



[Config Step17]

```
description = "Loop avoidance in multi-area OSPF topology"
```

```
network = OSPF_LoopAvoidance
```

```
*.configurator.config = xml("<config> \n
                                <interface hosts='R1' names='eth0' address='10.0.0.9' \n
↪netmask='255.255.255.252' /> \n
                                <interface hosts='R1' towards='R2' address='10.0.0.5' \n
↪netmask='255.255.255.252' /> \n
                                <interface hosts='R1' towards='R5' address='10.0.0.1' \n
↪netmask='255.255.255.252' /> \n
                                \n
                                <interface hosts='R2' names='eth1' address='10.0.0.21\n
↪' netmask='255.255.255.252' /> \n
                                <interface hosts='R2' towards='R1' address='10.0.0.6' \n
↪netmask='255.255.255.252' /> \n
                                <interface hosts='R2' towards='R3' address='10.0.0.13\n
↪' netmask='255.255.255.252' /> \n
                                <interface hosts='R2' towards='R4' address='10.0.0.17\n
↪' netmask='255.255.255.252' /> \n
                                \n
                                <interface hosts='R3' names='eth3' address='10.0.0.33\n
↪' netmask='255.255.255.252' /> \n
                                <interface hosts='R3' names='eth4' address='10.0.0.37\n
↪' netmask='255.255.255.252' /> \n
                                <interface hosts='R3' towards='R2' address='10.0.0.14\n
↪' netmask='255.255.255.252' /> \n
                                <interface hosts='R3' towards='R5' address='10.0.0.29\n
                                \n
                                </interface>\n
                                </config>\n")
```

(continues on next page)

(continued from previous page)

```

↪ ' netmask='255.255.255.252' /> \
    <interface hosts='R3' towards='R4' address='10.0.0.25
↪ ' netmask='255.255.255.252' /> \
    \
    <interface hosts='R4' names='eth2' address='10.0.0.41
↪ ' netmask='255.255.255.252' /> \
    <interface hosts='R4' towards='R2' address='10.0.0.18
↪ ' netmask='255.255.255.252' /> \
    <interface hosts='R4' towards='R3' address='10.0.0.26
↪ ' netmask='255.255.255.252' /> \
    \
    <interface hosts='R5' names='eth0' address='10.0.0.45
↪ ' netmask='255.255.255.252' /> \
    <interface hosts='R5' towards='R1' address='10.0.0.2'
↪ netmask='255.255.255.252' /> \
    <interface hosts='R5' towards='R3' address='10.0.0.30
↪ ' netmask='255.255.255.252' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↪ ' interface='eth0' /> \
    </config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_Loop.xml")

# application parameters
*.R5.numApps = 1
*.R5.app[0].typename = "PingApp"
*.R5.app[0].destAddr = "R2"
*.R5.app[0].startTime = 60s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↪ xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>R5" mask="R1>R5" />
    <AddressRange address="R1>switch1" mask="R1>switch1" />

    <AddressRange address="R5>R1" mask="R5>R1" />
    <AddressRange address="R5>R3" mask="R5>R3" />
    <AddressRange address="R5>switch5" mask="R5>switch5" />

    <AddressRange address="R3>R5" mask="R3>R5" />
    <AddressRange address="R3>switch3" mask="R3>switch3" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R2>switch2" mask="R2>switch2" />

```

(continues on next page)

(continued from previous page)

```

<AddressRange address="R2>R4" mask="R2>R4" />
<AddressRange address="R2>R3" mask="R2>R3" />

<AddressRange address="R4>switch4" mask="R4>switch4" />
<AddressRange address="R4>R2" mask="R4>R2" />
<AddressRange address="R4>R3" mask="R4>R3" />

<AddressRange address="R3>R2" mask="R3>R2" />
<AddressRange address="R3>R4" mask="R3>R4" />
<AddressRange address="R3>switch6" mask="R3>switch6" />
</Area>

<!-- Routers -->
<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.0" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth1" areaID="0.0.0.2" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth3" areaID="0.0.0.0" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth3" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth4" areaID="0.0.0.2" interfaceMode="Passive" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.2" interfaceMode="Passive" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
</Router>

</OSPFASConfig>

```

Results

The configuration demonstrates potential loop scenarios:

1. Areas are configured in a way that could create routing loops.
2. OSPF's loop prevention mechanisms ensure that:
 - Summary LSAs from non-backbone areas are ignored by ABRs
 - Only Summary LSAs received via the backbone are accepted for inter-area routing

3. If the topology violates OSPF's hierarchical requirements (non-backbone area not connected to Area 0), routing may be suboptimal or certain networks may be unreachable.
4. The routing tables show how OSPF enforces the hierarchical structure to prevent loops.

This demonstrates the importance of proper area design and the backbone's central role in OSPF multi-area networks.

Sources: `omnetpp.ini`, `OSPF_LoopAvoidance.ned`, `ASConfig_Area_Loop.xml`

7.23.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.24 Step 17a. Make R3 an ABR - advertise its loopback to backbone

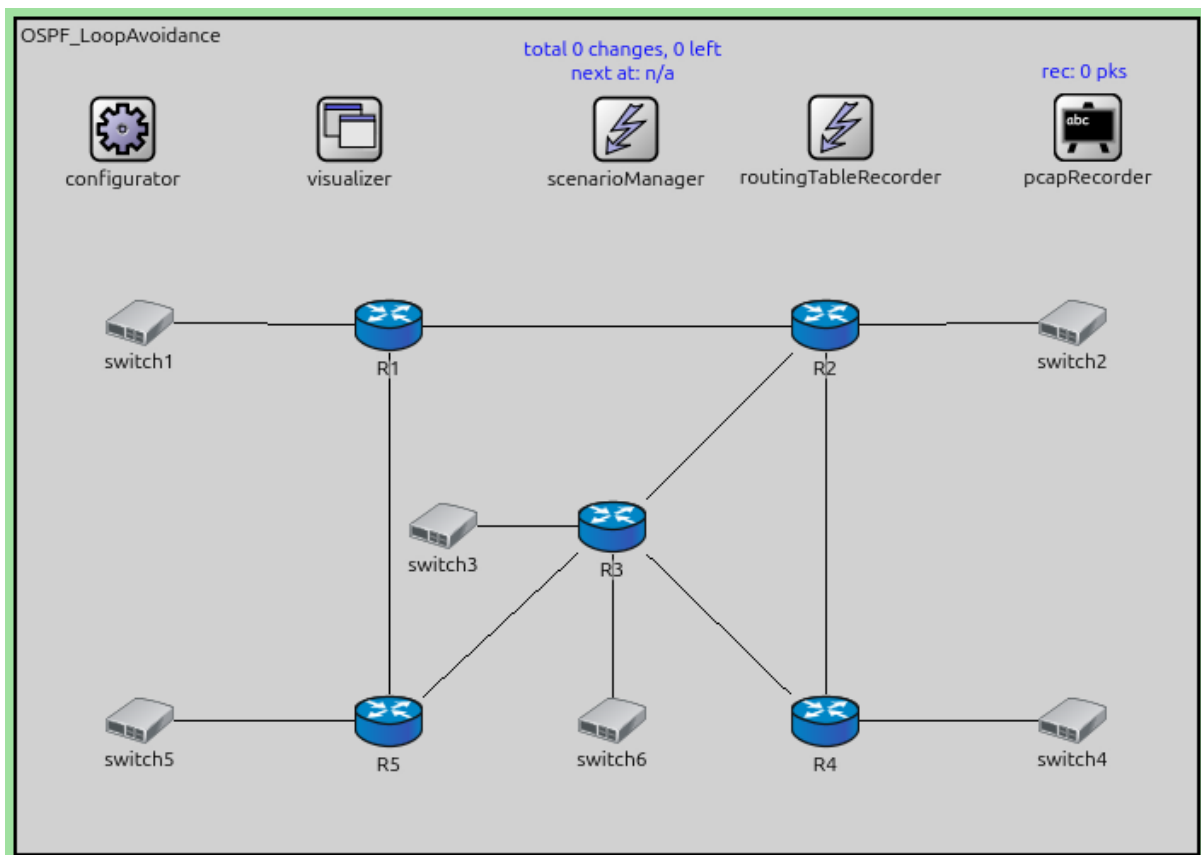
7.24.1 Goals

The goal of this step is to demonstrate one solution to connectivity issues in disconnected areas: making a router an ABR by assigning one of its interfaces (e.g., a loopback) to the backbone area.

When an area is not directly connected to Area 0, connectivity problems arise. One solution is to create an ABR by assigning one of the router's interfaces to Area 0, effectively connecting the isolated area to the backbone.

Configuration

This configuration is based on Step 17. Router R3 is made an ABR by advertising its loopback interface in Area 0.



The configuration in `omnetpp.ini` is the following:

[Config Step17a]

```
description = "Make R3 an ABR - advertise its loopback to backbone"
extends = Step17
```

```
*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_Loop_ABR.xml")
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="127.0.0.1" mask="255.0.0.0" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>R5" mask="R1>R5" />
    <AddressRange address="R1>switch1" mask="R1>switch1" />

    <AddressRange address="R5>R1" mask="R5>R1" />
    <AddressRange address="R5>R3" mask="R5>R3" />
    <AddressRange address="R5>switch5" mask="R5>switch5" />

    <AddressRange address="R3>R5" mask="R3>R5" />
    <AddressRange address="R3>switch3" mask="R3>switch3" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R4" mask="R2>R4" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R4>switch4" mask="R4>switch4" />
    <AddressRange address="R4>R2" mask="R4>R2" />
    <AddressRange address="R4>R3" mask="R4>R3" />

    <AddressRange address="R3>R2" mask="R3>R2" />
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R3>switch6" mask="R3>switch6" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.0" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth1" areaID="0.0.0.2" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth3" areaID="0.0.0.0" />
```

(continues on next page)

(continued from previous page)

```

</Router>

<Router name="R3" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth3" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth4" areaID="0.0.0.2" interfaceMode="Passive" />
  <LoopbackInterface ifName="lo0" areaID="0.0.0.0" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.2" interfaceMode="Passive" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
</Router>

</OSPFASConfig>

```

Results

By making R3 an ABR (with presence in both Area 0 and another area):

1. R3 now has interfaces in the backbone area (its loopback) and in another area.
2. R3 can generate Summary LSAs for both areas, providing inter-area connectivity.
3. Routes that were previously unreachable or suboptimal are now properly computed.
4. This solves some connectivity issues but may not be the optimal solution for all topologies.

This demonstrates one approach to connecting areas, though virtual links (shown in later steps) are often more appropriate for complex topologies.

Sources: `omnetpp.ini`, `OSPF_LoopAvoidance.ned`, `ASConfig_Area_Loop_ABR.xml`

7.24.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.25 Step 17b. Make R3 an ABR - create a virtual link between R1 and R3

7.25.1 Goals

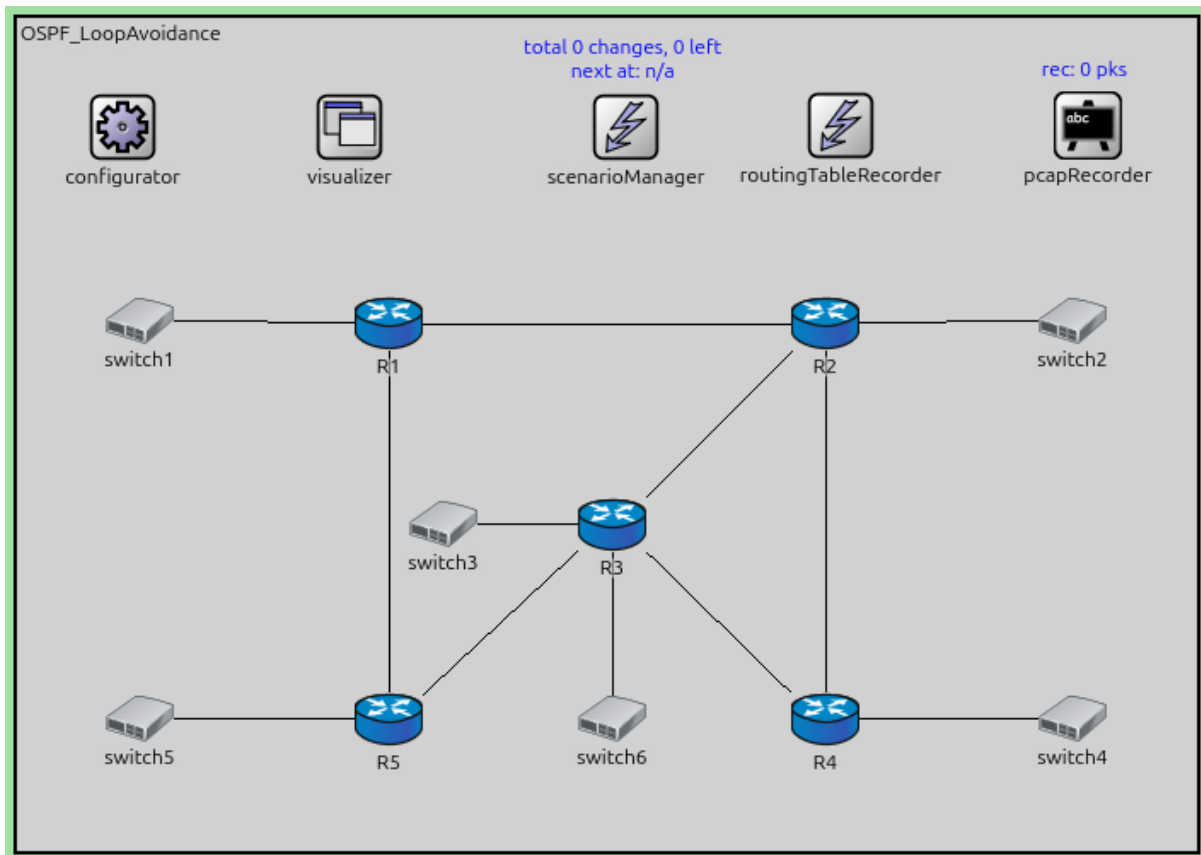
The goal of this step is to demonstrate the use of OSPF virtual links to logically connect an area to the backbone.

A virtual link is a logical connection through a transit area that allows two ABRs to appear directly connected in Area 0. Virtual links are used when:

- An area cannot be physically connected to the backbone
- The backbone itself is partitioned and needs to be reconnected

Configuration

This configuration is based on Step 17. A virtual Link is configured between R1 and R3 through a transit area.



The configuration in `omnetpp.ini` is the following:

```
[Config Step17b]
description = "Make R3 an ABR - create a virtual link between R1 and R3"
extends = Step17

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_Loop_ABR_Virtual.xml")
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>R5" mask="R1>R5" />
    <AddressRange address="R1>switch1" mask="R1>switch1" />

    <AddressRange address="R5>R1" mask="R5>R1" />
    <AddressRange address="R5>R3" mask="R5>R3" />
    <AddressRange address="R5>switch5" mask="R5>switch5" />
  </Area>
</OSPFASConfig>
```

(continues on next page)

(continued from previous page)

```

    <AddressRange address="R3>R5" mask="R3>R5" />
    <AddressRange address="R3>switch3" mask="R3>switch3" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R4" mask="R2>R4" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R4>switch4" mask="R4>switch4" />
    <AddressRange address="R4>R2" mask="R4>R2" />
    <AddressRange address="R4>R3" mask="R4>R3" />

    <AddressRange address="R3>R2" mask="R3>R2" />
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R3>switch6" mask="R3>switch6" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.0" />
    <VirtualLink endPointRouterID="R3%routerId" transitAreaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth1" areaID="0.0.0.2" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth3" areaID="0.0.0.0" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth3" areaID="0.0.0.1" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth4" areaID="0.0.0.2" interfaceMode="Passive" />
    <VirtualLink endPointRouterID="R1%routerId" transitAreaID="0.0.0.1" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.2" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.2" interfaceMode="Passive" />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
  </Router>
</OSPFASConfig>

```

Results

With the virtual link configured:

1. R1 and R3 establish a virtual adjacency through the transit area.
2. The virtual link is treated as part of Area 0 (the backbone).
3. Inter-area routing works correctly as if the areas were physically connected to the backbone.
4. The routing tables show improved connectivity compared to Step 17.

Virtual links provide flexibility in network design but should be used sparingly as they add complexity and can make troubleshooting more difficult.

Sources: `omnetpp.ini`, `OSPF_LoopAvoidance.ned`, `ASConfig_Area_Loop_ABR_Virtual.xml`

7.25.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.26 Step 18. AS-External LSAs of ‘type 1 metric’ with different advertised destination

7.26.1 Goals

The goal of this step is to demonstrate AS-External LSAs (Type-5 LSAs) with Type-1 metrics.

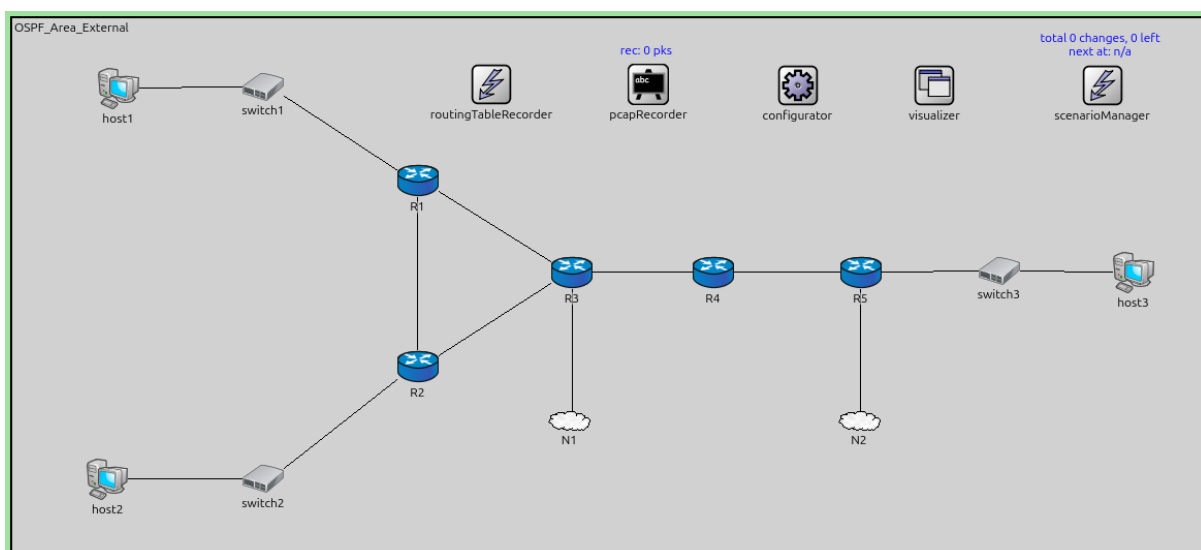
OSPF routers can advertise routes to destinations outside the OSPF domain using AS-External LSAs. These external routes can use two metric types:

- **Type-1 (E1):** The metric is the sum of the external cost plus the internal OSPF cost to reach the ASBR (Autonomous System Boundary Router). This allows OSPF to select the closest exit point.
- **Type-2 (E2):** The metric is only the external cost; internal cost is ignored. Type-2 is the default.

This step demonstrates Type-1 metrics with different external destinations.

Configuration

This step uses the `OSPF_Area_External` network with ASBRs advertising external routes.



The configuration in `omnetpp.ini` is the following:

[Config Step18]

```

description = "AS-External LSAs of 'type 1 metric' with different advertised_
↳destination"
network = OSPF_Area_External

# ToDo: R5 has a cost of 13 to the external network

*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    \
    <interface hosts='R3' towards='N1.*' address='192.168.
↳9.x' netmask='255.255.255.x' /> \
    <interface hosts='R5' towards='N2.*' address='192.168.
↳10.x' netmask='255.255.255.x' /> \
    \
    <interface among='host1 host2 R1 R2 R3' address='192.
↳168.11.x' netmask='255.255.255.x' /> \
    <interface among='R3 R4 R5 host3' address='192.168.22.
↳x' netmask='255.255.255.x' /> \
    \
    <interface hosts='N1.
↳host[*]' address='192.168.9.x' netmask='255.255.255.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↳' interface='eth0' /> \
    </config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Type1.xml")

# application parameters
*.host*.numApps = 1
*.host*.app[0].typename = "UdpBasicApp"
*.host*.app[0].destAddresses = "N1.host[0]"
*.host*.app[0].destPort = 5000
*.host*.app[0].messageLength = 1024B
*.host*.app[0].sendInterval = 1s
*.host*.app[0].startTime = 10s
*.host*.app[0].stopTime = 100s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFAConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↳xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />
  </Area>

```

(continues on next page)

```

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
    <ExternalInterface toward="N1" advertisedExternalNetworkAddress="R3>N1"
    ↪advertisedExternalNetworkMask="R3>N1" externalInterfaceOutputCost="10"
    ↪externalInterfaceOutputType="Type1" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
    <ExternalInterface toward="N2" advertisedExternalNetworkAddress="R5>N2"
    ↪advertisedExternalNetworkMask="R5>N2" externalInterfaceOutputCost="5"
    ↪externalInterfaceOutputType="Type1" />
  </Router>

</OSPFASConfig>

```

Results

With Type-1 external routes:

1. ASBRs (e.g., R3, R5) generate AS-External LSAs for external networks.
2. The LSAs specify Type-1 metric (E1).
3. When routers calculate routes to these external destinations, they use: **Total Cost = External Cost + Internal Cost to ASBR**
4. Routers select the ASBR with the lowest total cost. This may result in choosing a closer ASBR even if it advertises a higher external cost, as long as the total cost remains lower.
5. Different routers may choose different ASBRs for the same destination based on their location in the topology.

Type-1 metrics are useful when the external cost is comparable to internal OSPF costs and you want traffic to exit the AS at the nearest point.

Sources: `omnetpp.ini`, `OSPF_Area_External.ned`, `ASConfig_Area_ExternalRoute_Type1.xml`

7.26.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.27 Step 18a. AS-External LSAs of ‘type 1 metric’ with the same advertised destination

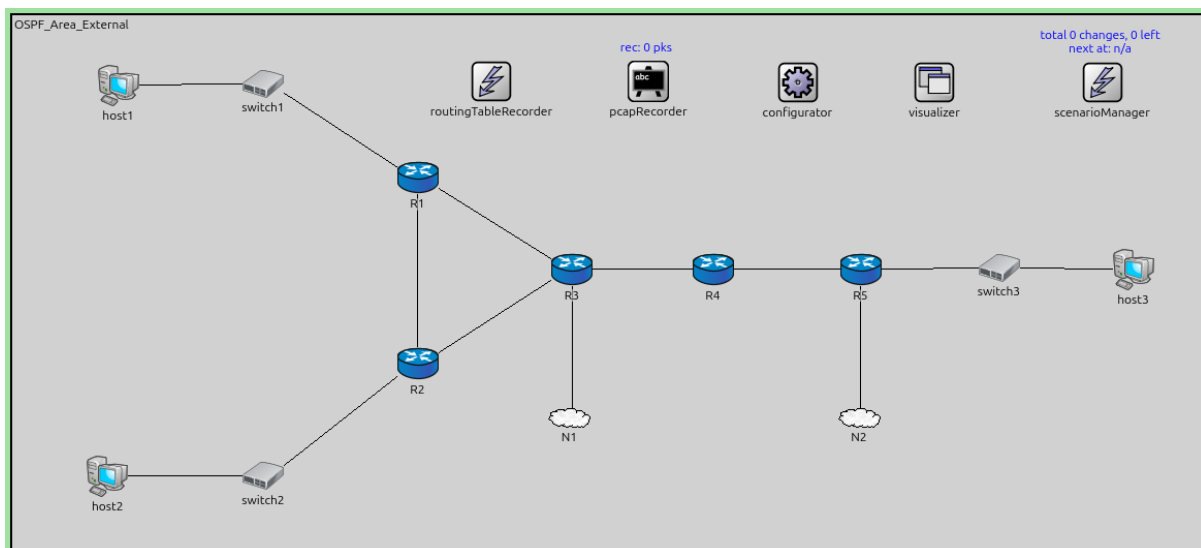
7.27.1 Goals

The goal of this step is to demonstrate how OSPF selects between multiple ASBRs advertising the same external destination with Type-1 metrics.

When multiple ASBRs advertise the same external network using Type-1 metrics, each router selects the best ASBR based on the total cost (external cost + internal cost to ASBR).

Configuration

This configuration is based on Step 18. Multiple ASBRs advertise the same external destination.



The configuration in `omnetpp.ini` is the following:

```

[Config Step18a]
description = "AS-External LSAs of 'type 1 metric' with the same advertised_
↳destination"
extends = Step18
network = OSPF_Area_External_Same_Dest

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Type1_Dest.xml")

*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    \
    <interface hosts='R3' towards='N1.*' address='192.168.
↳9.x' netmask='255.255.255.x' /> \
    <interface hosts='R5' towards='N1.*' address='192.168.
↳10.x' netmask='255.255.255.x' /> \
    \
    <interface among='host1 host2 R1 R2 R3' address='192.
↳168.11.x' netmask='255.255.255.x' /> \
    <interface among='R3 R4 R5 host3' address='192.168.22.
↳x' netmask='255.255.255.x' /> \
    \
    <interface hosts='N1.
↳host[*]' address='120.100.90.20' netmask='255.255.255.x' /> \
    \
    \
    <route hosts='host*'
↳destination='*' netmask='0.0.0.0' interface='eth0' /> \
    </config>")

*.host*.app[0].destAddresses = "N1.host[0]"

*.configurator.assignDisjunctSubnetAddresses = false

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↳xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />

```

(continues on next page)

(continued from previous page)

```

</Area>

<Area id="0.0.0.2">
  <AddressRange address="R4>R5" mask="R4>R5" />

  <AddressRange address="R5>R4" mask="R5>R4" />
  <AddressRange address="R5>switch3" mask="R5>switch3" />
</Area>

<!-- Routers -->
<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="120.100.90.0"
→advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="10"
→externalInterfaceOutputType="Type1" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="120.100.90.0"
→advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="5"
→externalInterfaceOutputType="Type1" />
</Router>

</OSPFASConfig>

```

Results

With multiple ASBRs advertising the same destination:

1. Each ASBR originates an AS-External LSA for the same network prefix.
2. The LSAs may have different external costs.
3. Each router calculates the total cost to reach the destination via each ASBR: **Total Cost via ASBR_X = External Cost (ASBR_X) + Internal Cost to ASBR_X**
4. The router selects the ASBR with the lowest total cost.

5. Routers in different parts of the topology may select different ASBRs based on their internal costs to each ASBR.

This demonstrates OSPF's ability to perform intelligent load balancing and path selection for external routes when using Type-1 metrics.

Sources: `omnetpp.ini`, `OSPF_Area_External.ned`, `ASConfig_Area_ExternalRoute_Type1_Dest.xml`

7.27.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.28 Step 18b. AS-External LSAs of 'type 2 metric' with different advertised destination

7.28.1 Goals

The goal of this step is to demonstrate AS-External LSAs with Type-2 metrics.

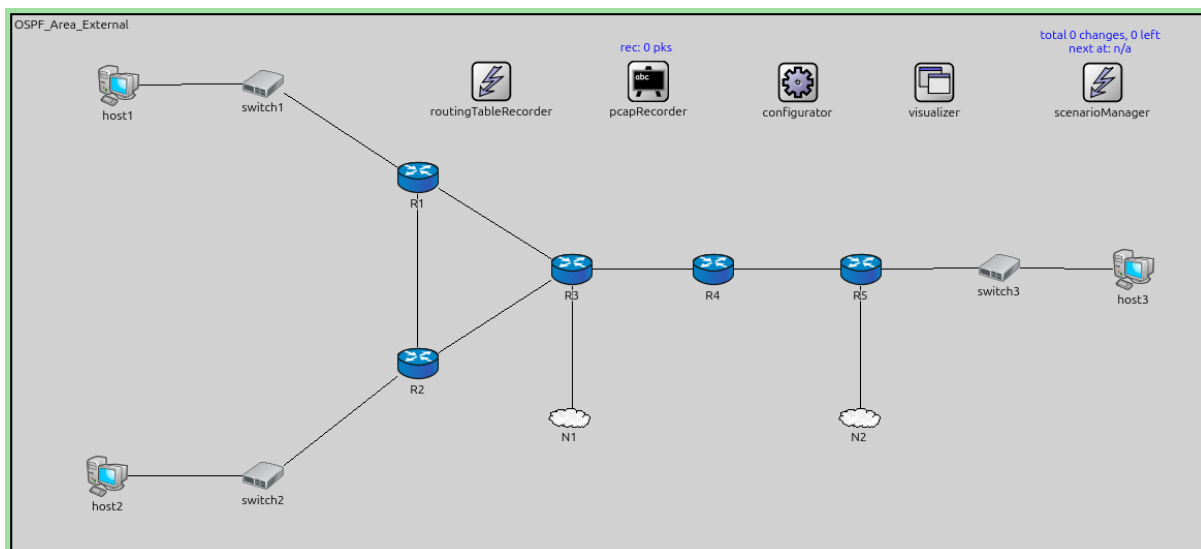
Type-2 (E2) is the default external metric type in OSPF. With Type-2 metrics:

- The cost to the external destination is **ONLY** the external cost advertised by the ASBR
- Internal OSPF cost to reach the ASBR is **NOT** added to the metric
- If multiple ASBRs advertise the same destination with the same external cost, the ASBR with the lowest internal cost is preferred (used as a tiebreaker)

Type-2 is appropriate when external costs are much larger than internal OSPF costs.

Configuration

This configuration is based on Step 18, using Type-2 metrics instead of Type-1.



The configuration in `omnetpp.ini` is the following:

```
[Config Step18b]
description = "AS-External LSAs of 'type 2 metric' with different advertised_
↪destination"
extends = Step18

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Type2.xml")
```


The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
    <ExternalInterface toward="N1" advertisedExternalNetworkAddress="R3>N1"
      advertisedExternalNetworkMask="R3>N1" externalInterfaceOutputCost="10"
      externalInterfaceOutputType="Type2" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
  </Router>
```

(continues on next page)

(continued from previous page)

```

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <ExternalInterface toward="N2" advertisedExternalNetworkAddress="R5>N2"
↪advertisedExternalNetworkMask="R5>N2" externalInterfaceOutputCost="5"
↪externalInterfaceOutputType="Type2" />
</Router>
</OSPFASConfig>

```

Results

With Type-2 external routes:

1. ASBRs generate AS-External LSAs with Type-2 metric (E2).
2. Routers calculate routes using ONLY the external cost.
3. The route cost displayed in routing tables is the external cost, regardless of how far the ASBR is from the router.
4. All routers see the same cost for a given external route.
5. Internal topology has minimal impact on external route selection (only used as tiebreaker).

Type-2 is useful when external metrics represent costs in a different domain (e.g., BGP AS-path length) that shouldn't be mixed with internal OSPF costs.

Sources: `omnetpp.ini`, `OSPF_Area_External.ned`, `ASConfig_Area_ExternalRoute_Type2.xml`

7.28.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.29 Step 18c. AS-External LSAs of 'type 2 metric' with the same advertised destination

7.29.1 Goals

The goal of this step is to demonstrate how OSPF selects between multiple ASBRs advertising the same external destination with Type-2 metrics.

When multiple ASBRs advertise the same destination with Type-2 metrics, the selection process is:

1. **Primary:** Compare external costs - lowest wins
2. **Tiebreaker** (if external costs are equal): Use internal cost to ASBR - lowest wins

Configuration

This configuration is based on Step 18b. Multiple ASBRs advertise the same external destination with Type-2 metrics.

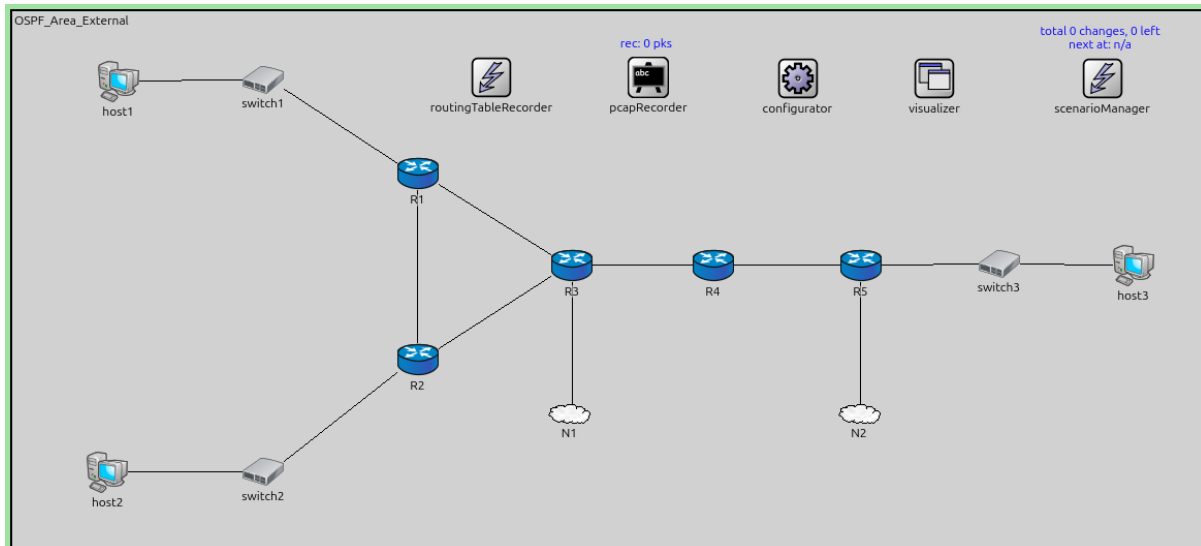
The configuration in `omnetpp.ini` is the following:

```

[Config Step18c]
description = "AS-External LSAs of 'type 2 metric' with the same advertised_
↪destination"
extends = Step18
network = OSPF_Area_External_Same_Dest

```

(continues on next page)



(continued from previous page)

```
*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Type2_Dest.xml")
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
```

(continues on next page)

(continued from previous page)

```

<BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
<PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
<PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="R3>N1"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="10"
↪externalInterfaceOutputType="Type2" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="R5>N1"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="5"
↪externalInterfaceOutputType="Type2" />
</Router>

</OSPFASConfig>

```

Results

With multiple ASBRs using Type-2 metrics for the same destination:

1. Each router compares the external costs from all ASBRs.
2. The ASBR with the lowest external cost is preferred.
3. If external costs are equal, the ASBR with the lowest internal cost (closest to the router) is selected.
4. All routers in the network see the same external cost, but may select different ASBRs based on internal costs if external costs are equal.

This demonstrates Type-2's preference for external cost over internal topology, while still using internal cost for intelligent ASBR selection when needed.

Sources: `omnetpp.ini`, `OSPF_Area_External.ned`, `ASConfig_Area_ExternalRoute_Type2_Dest.xml`

7.29.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.30 Step 18d. AS-External LSAs of mixed 'type 1/type 2 metric' with the same advertised destination

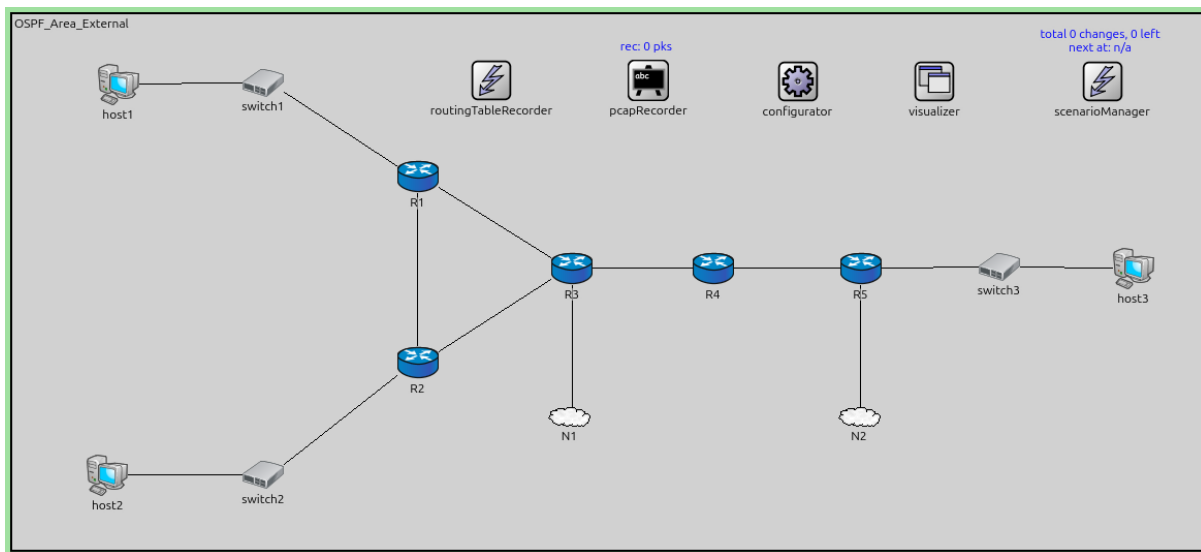
7.30.1 Goals

The goal of this step is to demonstrate OSPF's behavior when multiple ASBRs advertise the same external destination using different metric types (Type-1 and Type-2).

OSPF always prefers Type-1 (E1) routes over Type-2 (E2) routes for the same destination, regardless of the metric values. This is because E1 routes include internal topology in the cost calculation, making them more accurate.

Configuration

This configuration is based on Step 18. Some ASBRs use Type-1 metrics while others use Type-2 metrics for the same destination.



The configuration in `omnetpp.ini` is the following:

```
[Config Step18d]
description = "AS-External LSAs of mixed 'type 1/type 2 metric' with the same_
↳advertised destination"
extends = Step18
network = OSPF_Area_External_Same_Dest

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Mixed_Dest.xml")

*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    \
    <interface hosts='R3' towards='N1.*' address='192.168.
↳9.x' netmask='255.255.255.x' /> \
    <interface hosts='R5' towards='N1.*' address='192.168.
↳9.x' netmask='255.255.255.x' /> \
    \
    <interface among='host1 host2 R1 R2 R3' address='192.
↳168.11.x' netmask='255.255.255.x' /> \
    <interface among='R3 R4 R5 host3' address='192.168.22.
```

(continues on next page)

(continued from previous page)

```

↪x' netmask='255.255.255.x' /> \
                                                    <interface hosts='N1.
↪host[*]' address='192.168.9.x' netmask='255.255.255.x' /> \
                                                    \
                                                    <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
                                                    </config>")

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" _
↪xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />

```

(continues on next page)

(continued from previous page)

```

<PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
<ExternalInterface toward="N1" advertisedExternalNetworkAddress="192.168.9.0"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="10"
↪externalInterfaceOutputType="Type1" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="192.168.9.0"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="5"
↪externalInterfaceOutputType="Type2" />
</Router>

</OSPFASConfig>

```

Results

When ASBRs advertise the same destination with mixed metric types:

1. OSPF **always prefers E1 (Type-1) routes** over E2 (Type-2) routes.
2. Even if an E2 route has a lower advertised cost, the E1 route will be selected.
3. This preference reflects OSPF's design: E1 routes provide more accurate end-to-end cost calculation by including internal topology.
4. All routers will select an ASBR advertising E1, even if they are closer to an ASBR advertising E2.

The routing tables clearly show the preference for E1 routes, demonstrating OSPF's route selection hierarchy: Intra-area > Inter-area > E1 > E2.

Sources: `omnetpp.ini`, `OSPF_Area_External.ned`, `ASConfig_Area_ExternalRoute_Mixed_Dest.xml`

7.30.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.31 Step 18e. Address Forwarding

7.31.1 Goals

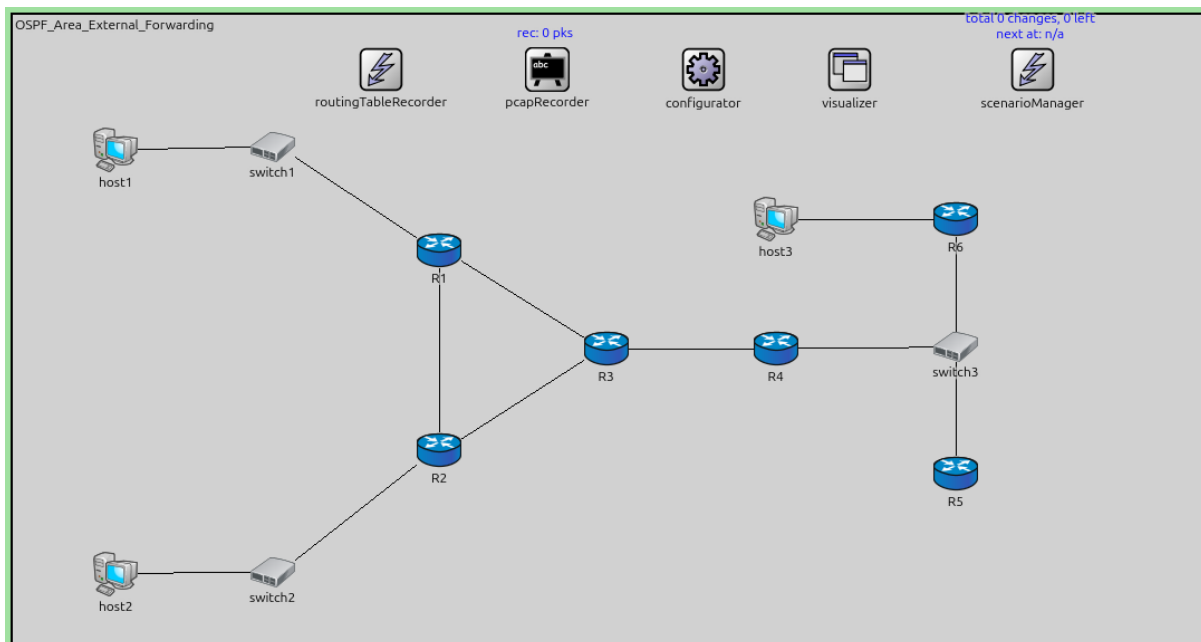
The goal of this step is to demonstrate the use of the Forwarding Address field in AS-External LSAs.

By default, when an ASBR advertises an external route, the Forwarding Address is set to 0.0.0.0, meaning traffic should be sent to the ASBR itself. However, if the external network is reachable through another router on a shared network, the ASBR can set the Forwarding Address to that router's IP, allowing packets to be forwarded directly to the actual next-hop router, saving a hop.

Configuration

This step demonstrates a scenario where R6 (not running OSPF) is the actual gateway to an external network, and R5 (an ASBR) sets the Forwarding Address to R6's IP.

The configuration in `omnetpp.ini` is the following:

**[Config Step18e]**

```
description = "Address Forwarding"
```

```
network = OSPF_Area_External_Forwarding
```

```
# R6 is not an OSPF router.
```

```
# forwardingAddress is set to 192.168.22.3 in R5. Other routers forward packets with_
```

```
↳ destination 192.168.22.8
```

```
# to R6. If forwardingAddress is unset, packets are forwarded to R5 and then to R6_
```

```
↳ (packet delivery takes one extra hop).
```

```
*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↳x' netmask='255.255.255.x' /> \
    \
    <interface among='host1 host2 R1 R2 R3' address='192.
↳168.11.x' netmask='255.255.255.x' /> \
    <interface among='R3 R4 R5 R6 host3' address='192.168.
↳22.x' netmask='255.255.255.x' /> \
    \
    <route hosts='host*' destination='*' netmask='0.0.0.0
↳' interface='eth0' /> \
    <route hosts='R6' destination='*' netmask='0.0.0.0'_
↳interface='eth0' gateway='192.168.22.1' /> \
    </config>")
```

```
*.R6.hasOspf = false
```

```
*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_ExternalRoute_Forwarding.xml")
```

```
# application parameters
```

```
*.host1.numApps = 1
```

```
*.host1.app[0].typename = "PingApp"
```

```
*.host1.app[0].destAddr = "host3"
```

```
*.host1.app[0].startTime = 60s
```


The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>switch3" mask="R4>switch3" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.0" />
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" />
    <ExternalInterface ifName="eth0" advertisedExternalNetworkAddress="192.168.22.8"
  </advisedExternalNetworkMask="255.255.255.252" externalInterfaceOutputCost="10" />
  </Router>
</OSPFASConfig>
```

(continues on next page)

(continued from previous page)

```

↪externalInterfaceOutputType="Type1" forwardingAddress="192.168.22.3" />
</Router>

</OSPFASConfig>

```

Results

With the Forwarding Address configured:

1. R5 (ASBR) advertises an external route with Forwarding Address set to R6's IP (192.168.22.3).
2. Other OSPF routers learn this external route.
3. When sending packets to the external destination, routers forward them directly to R6 (via OSPF routing to 192.168.22.3), not to R5.
4. This saves one hop compared to routing through R5 first.
5. The ping from host1 to host3 demonstrates the direct forwarding path.

The Forwarding Address field allows OSPF to optimize forwarding when the ASBR is not the actual next-hop to the external destination.

Sources: `omnetpp.ini`, `OSPF_Area_External_Forwarding.ned`, `ASConfig_Area_ExternalRoute_Forwarding.xml`

7.31.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.32 Step 19. Default-route distribution in OSPF

7.32.1 Goals

The goal of this step is to demonstrate how OSPF can distribute a default route (0.0.0.0/0) throughout the AS.

An ASBR can advertise a default route using an AS-External LSA. This allows routers throughout the OSPF domain to learn a default route for reaching destinations outside the AS, without needing specific routes to every external network.

Default routes are particularly useful for providing Internet connectivity in enterprise networks.

Configuration

This step configures an ASBR to advertise a default route into the OSPF domain.

The configuration in `omnetpp.ini` is the following:

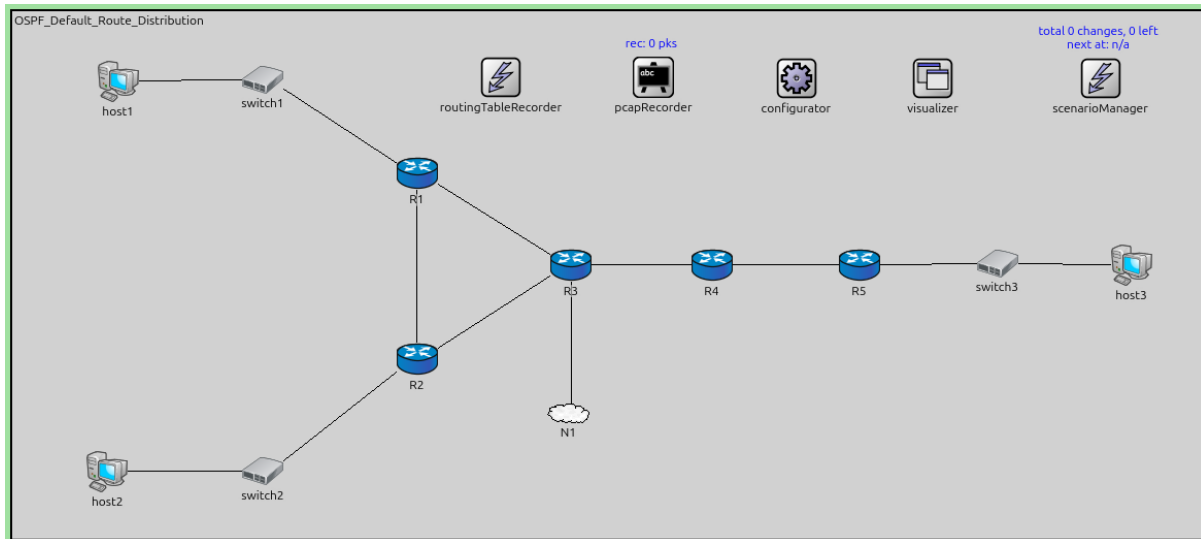
```

[Config Step19]
description = "Default-route distribution in OSPF"
network = OSPF_Default_Route_Distribution

*.configurator.config = xml("<config> \
                                <interface hosts='R3' towards='R4' address='192.168.0.
↪x' netmask='255.255.255.x' /> \
                                <interface hosts='R4' towards='R3' address='192.168.0.
↪x' netmask='255.255.255.x' /> \
                                \
                                <interface hosts='R3' towards='N1.*' address='192.168.
↪9.x' netmask='255.255.255.x' /> \
                                \

```

(continues on next page)



(continued from previous page)

```

<interface among='host1 host2 R1 R2 R3' address='192.
↪168.11.x' netmask='255.255.255.x' /> \
<interface among='R3 R4 R5 host3' address='192.168.22.
↪x' netmask='255.255.255.x' /> \
<route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
<route hosts='R3' destination='*' netmask='0.0.0.0'
↪interface='eth0' /> \
</config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Area_Default_Route.xml")

# application parameters
*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[0].destAddr = "host3"
*.host1.app[0].startTime = 60s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↪xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
  </Area>
</OSPFASConfig>

```

(continues on next page)

(continued from previous page)

```

    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R4>R5" mask="R4>R5" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true" DistributeDefaultRoute="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  </Router>

</OSPFASConfig>

```

Results

When an ASBR advertises a default route:

1. The ASBR generates an AS-External LSA with destination 0.0.0.0/0.
2. This LSA is flooded throughout the OSPF AS.
3. All routers install a default route pointing to the ASBR (or its Forwarding Address).
4. Traffic to unknown destinations is forwarded to the ASBR for external routing.
5. The routing tables show 0.0.0.0/0 routes pointing toward the ASBR.

Default route distribution simplifies configuration and reduces the number of external routes that must be advertised into OSPF, particularly useful when the AS has limited external connectivity points.

Sources: omnetpp.ini, OSPF_Default_Route_Distribution.ned, ASConfig_Area_Default_Route.xml

7.32.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.33 Step 20. Stub area

7.33.1 Goals

The goal of this step is to demonstrate OSPF stub areas.

A stub area is an area that does not receive AS-External LSAs (Type-5). Instead, the ABR injects a default route into the stub area, reducing the LSDB size and memory requirements for routers in that area.

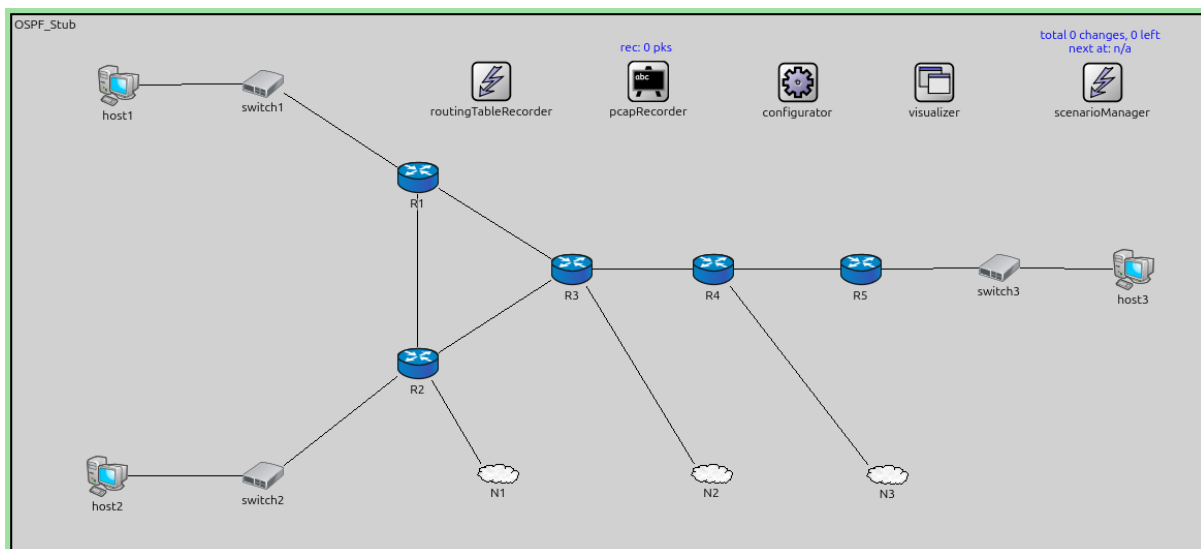
Stub areas are useful for:

- Reducing memory and CPU usage in routers with limited resources
- Simplifying routing in areas with limited external connectivity (single exit point)

All routers in a stub area must be configured as stub; otherwise, adjacencies will not form.

Configuration

This step configures an area as a stub area.



The configuration in omnetpp.ini is the following:

```
[Config Step20]
description = "Stub area"
network = OSPF_Stub

*.configurator.config = xml("<config> \
    <interface hosts='R3' towards='R4' address='192.168.0.
↵x' netmask='255.255.255.x' /> \
    <interface hosts='R4' towards='R3' address='192.168.0.
↵x' netmask='255.255.255.x' /> \
    \
    <interface hosts='R2' towards='N1.*' address='192.168.
↵8.x' netmask='255.255.255.x' /> \
    <interface hosts='R3' towards='N2.*' address='192.168.
```

(continues on next page)

(continued from previous page)

```

↪9.x' netmask='255.255.255.x' /> \
                                <interface hosts='R4' towards='N3.*' address='192.168.
↪10.x' netmask='255.255.255.x' /> \
                                \
                                <interface among='host1 host2 R1 R2 R3' address='192.
↪168.11.x' netmask='255.255.255.x' /> \
                                <interface among='R3 R4 R5 host3' address='192.168.22.
↪x' netmask='255.255.255.x' /> \
                                                                <interface hosts='**' \
↪address='192.168.x.x' /> \
                                                                \
                                                                <route hosts='host*' destination='*' netmask='0.0.0.0
↪' interface='eth0' /> \
                                                                </config>")

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Stub_Area.xml")

# application parameters
*.host1.numApps = 1
*.host1.app[0].typename = "PingApp"
*.host1.app[0].destAddr = "N3.host[0]"
*.host1.app[0].startTime = 60s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" \
↪xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R3>R4" mask="R3>R4" />
    <AddressRange address="R4>R3" mask="R4>R3" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R1>R3" mask="R1>R3" />

    <AddressRange address="R2>switch2" mask="R2>switch2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />

    <AddressRange address="R3>R1" mask="R3>R1" />
    <AddressRange address="R3>R2" mask="R3>R2" />
  </Area>

  <Area id="0.0.0.2">
    <Stub defaultCost="1" />

    <!-- area 2 is a normal area -->
    <AddressRange address="R4>R5" mask="R4>R5" />
    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />
  </Area>

```

(continues on next page)

(continued from previous page)

```

<!-- Routers -->
<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <ExternalInterface toward="N1" advertisedExternalNetworkAddress="R2>N1"
→advertisedExternalNetworkMask="R2>N1" externalInterfaceOutputCost="20"
→externalInterfaceOutputType="Type1" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <PointToPointInterface ifName="ppp2" areaID="0.0.0.0" />
  <ExternalInterface toward="N2" advertisedExternalNetworkAddress="R3>N2"
→advertisedExternalNetworkMask="R3>N2" externalInterfaceOutputCost="10"
→externalInterfaceOutputType="Type1" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.0" />
  <ExternalInterface toward="N3" advertisedExternalNetworkAddress="R4>N3"
→advertisedExternalNetworkMask="R4>N3" externalInterfaceOutputCost="5"
→externalInterfaceOutputType="Type1" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.2" />
</Router>

</OSPFASConfig>

```

Results

In a stub area configuration:

1. The ABR does NOT flood AS-External LSAs (Type-5) into the stub area.
2. Instead, the ABR generates a default route Summary LSA (0.0.0.0/0) into the stub area.
3. Routers in the stub area have smaller LSDBs (no external LSAs).
4. Traffic destined for external networks uses the default route to the ABR (R5 forwards external traffic to R4, which then routes it appropriately to the external networks, N1, N2, or N3.)
5. Intra-area and inter-area routing works normally.

The stub area flag is negotiated in Hello packets. If there's a mismatch, routers will not form adjacencies.

Stub areas provide significant scalability benefits for areas that don't need detailed external routing information.

Sources: omnetpp.ini, OSPF_Stub.ned, ASConfig_Stub_Area.xml

7.33.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.34 Step 21. Virtual link - connect two separate parts of a discontinuous backbone

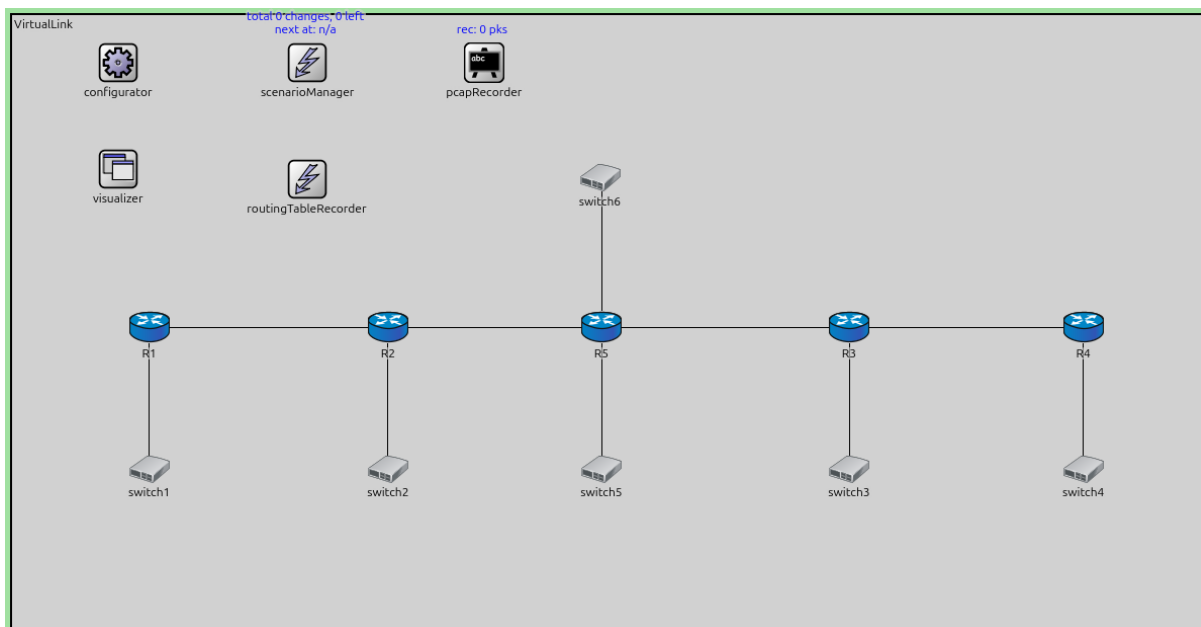
7.34.1 Goals

The goal of this step is to demonstrate using virtual links to connect a partitioned backbone.

If the backbone area (Area 0) becomes partitioned (split into disconnected parts), OSPF inter-area routing fails because Summary LSAs can only be exchanged through a contiguous backbone. Virtual links can reconnect the backbone partitions through a transit area, restoring full connectivity.

Configuration

This step simulates a partitioned backbone and uses a virtual link to reconnect it.



The configuration in `omnetpp.ini` is the following:

```
[Config Step21]
description = "Virtual link - connect two separate parts of a discontinuous backbone"
network = VirtualLink

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Virtual_Discontinuous.xml")

*.R1.numApps = 1
*.R1.app[0].typename = "PingApp"
*.R1.app[0].destAddr = "R4"
*.R1.app[0].startTime = 60s
```

The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">
```

(continues on next page)

(continued from previous page)

```

<!-- Areas -->
<Area id="0.0.0.0">
  <AddressRange address="R1>switch1" mask="R1>switch1" />
  <AddressRange address="R1>R2" mask="R1>R2" />

  <AddressRange address="R2>switch2" mask="R2>switch2" />
  <AddressRange address="R2>R1" mask="R2>R1" />

  <AddressRange address="R3>switch3" mask="R3>switch3" />
  <AddressRange address="R3>R4" mask="R3>R4" />

  <AddressRange address="R4>switch4" mask="R4>switch4" />
  <AddressRange address="R4>R3" mask="R4>R3" />
</Area>

<Area id="0.0.0.1">
  <AddressRange address="R2>R5" mask="R2>R5" />

  <AddressRange address="R3>R5" mask="R3>R5" />

  <AddressRange address="R5>R2" mask="R5>R2" />
  <AddressRange address="R5>R3" mask="R5>R3" />
  <AddressRange address="R5>switch5" mask="R5>switch5" />
  <AddressRange address="R5>switch6" mask="R5>switch6" />
</Area>

<!-- Routers -->
<Router name="R1" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.0" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.0" />
</Router>

<Router name="R2" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.0" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.0" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <VirtualLink endPointRouterID="R3%routerId" transitAreaID="0.0.0.1" />
</Router>

<Router name="R3" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.0" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.0" />
  <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  <VirtualLink endPointRouterID="R2%routerId" transitAreaID="0.0.0.1" />
</Router>

<Router name="R4" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.0" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.0" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" interfaceMode="Passive" />
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />

```

(continues on next page)

(continued from previous page)

```
<PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
</Router>

</OSPFASConfig>
```

Results

With a partitioned backbone:

1. Without virtual links, the two parts of Area 0 cannot exchange Summary LSAs.
2. Inter-area routing between the partitioned sections fails.
3. A virtual link is configured between two ABRs, one in each partition, through a transit area.
4. The virtual link acts as a logical Area 0 connection.
5. Summary LSAs can now be exchanged between the partitions.
6. Full inter-area routing is restored.

The routing tables show that routes are successfully computed across the previously partitioned backbone using the virtual link as a bridge.

Sources: `omnetpp.ini`, `VirtualLink.ned`, `ASConfig_Virtual_Discontinuous.xml`

7.34.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.35 Step 22. Virtual link - connect a disconnected area to the backbone

7.35.1 Goals

The goal of this step is to demonstrate using virtual links to connect an area that is not directly attached to the backbone.

OSPF requires all areas to be connected to Area 0 (the backbone). If an area cannot be physically connected to the backbone, a virtual link through a transit area can provide the logical connection required for proper OSPF operation.

Configuration

This step demonstrates an area that is not directly connected to Area 0, and a virtual link is used to connect it.

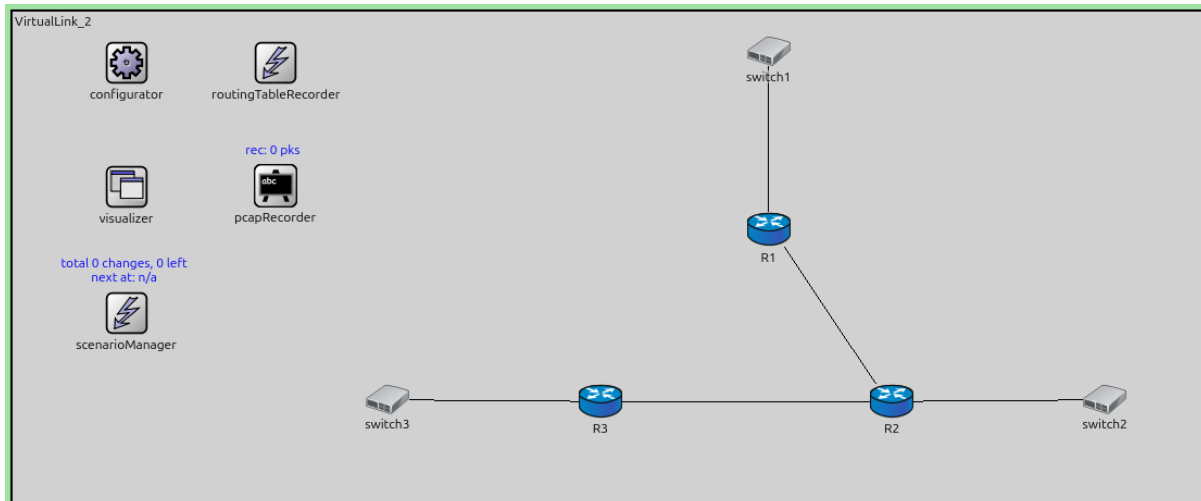
The configuration in `omnetpp.ini` is the following:

```
[Config Step22]
description = "Virtual link - connect a disconnected area to the backbone"
network = VirtualLink_2

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Virtual_Disconnected.xml")

*.R1.numApps = 1
*.R1.app[0].typename = "PingApp"
*.R1.app[0].destAddr = "R3"
*.R1.app[0].startTime = 60s
```

The OSPF configuration:



```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.0">
    <AddressRange address="R1>switch1" mask="R1>switch1" />
  </Area>

  <Area id="0.0.0.1">
    <AddressRange address="R1>R2" mask="R1>R2" />
    <AddressRange address="R2>R1" mask="R2>R1" />
    <AddressRange address="R2>R3" mask="R2>R3" />
    <AddressRange address="R2>switch2" mask="R2>switch2" />
  </Area>

  <Area id="0.0.0.2">
    <AddressRange address="R3>switch3" mask="R3>switch3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.0" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <VirtualLink endPointRouterID="R3%routerId" transitAreaID="0.0.0.1" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.2" interfaceMode="Passive" />
    <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
    <VirtualLink endPointRouterID="R1%routerId" transitAreaID="0.0.0.1" />
  </Router>
</OSPFASConfig>
```

(continues on next page)

(continued from previous page)

</OSPFASConfig>

Results

When an area is disconnected from Area 0:

1. Without a connection to the backbone, the disconnected area cannot exchange Summary LSAs properly.
2. Routers in the disconnected area may have incomplete routing information.
3. A virtual link is configured between an ABR in the disconnected area and an ABR in the backbone, through a transit area.
4. The virtual link provides the required logical connection to Area 0.
5. Summary LSAs can now flow between the previously disconnected area and the backbone.
6. Full inter-area routing is established.

This demonstrates how virtual links maintain OSPF's hierarchical design principles even when physical topology constraints prevent direct backbone connectivity.

Sources: `omnetpp.ini`, `VirtualLink_2.ned`, `ASConfig_Virtual_Disconnected.xml`

7.35.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.36 Step 23. OSPF Path Selection

7.36.1 Goals

The goal of this step is to demonstrate OSPF's route selection hierarchy when multiple paths to the same destination exist with different route types.

OSPF prioritizes routes in the following order:

1. **Intra-area routes** (routes within the same area)
2. **Inter-area routes** (routes learned via Summary LSAs from other areas)
3. **External Type-1 routes** (E1)
4. **External Type-2 routes** (E2)

This hierarchy ensures that OSPF prefers routes with the most accurate cost information and maintains the hierarchical area structure.

Configuration

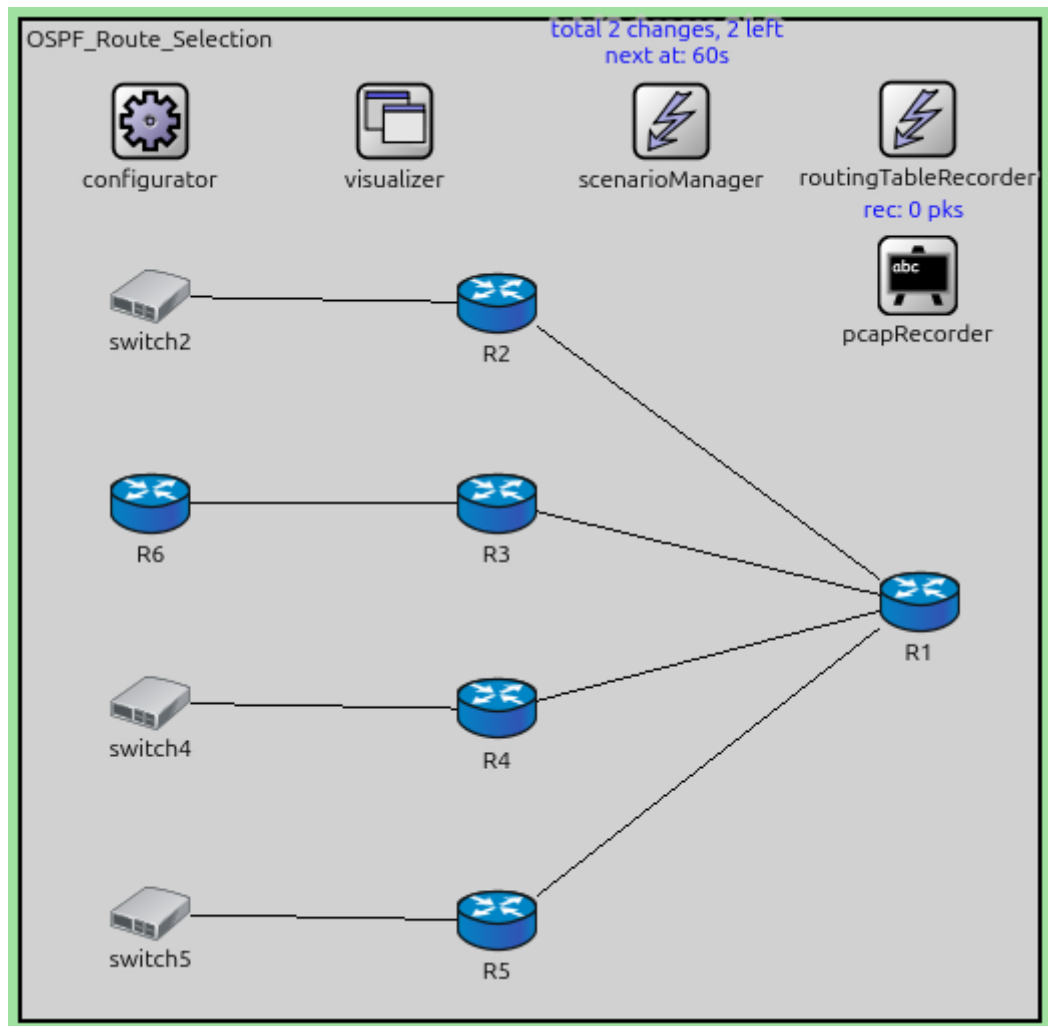
This step uses a topology where the same destination can be reached via different route types.

The configuration in `omnetpp.ini` is the following:

```
[Config Step23]
description = "OSPF Path Selection"
network = OSPF_Route_Selection

*.configurator.config = xml("<config> \
                                <interface hosts='R2' names='eth0' address='1.1.1.1' \
↪netmask='255.255.255.0' /> \
                                <interface hosts='R3' names='ppp1' address='1.1.1.1' \
↪netmask='255.255.255.0' /> \
```

(continues on next page)



(continued from previous page)

```

        <interface hosts='R6' names='ppp0' address='1.1.1.2'
↳netmask='255.255.255.0' /> \
        <interface hosts='R4' names='eth0' address='1.1.1.1'
↳netmask='255.255.255.0' /> \
        <interface hosts='R5' names='eth0' address='1.1.1.1'
↳netmask='255.255.255.0' /> \
        <interface hosts='*' address='10.x.x.x' netmask='255.
↳x.x.x' /> \
        </config>")

*.configurator.assignUniqueAddresses = false
*.configurator.assignDisjunctSubnetAddresses = false

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Route_Selection.xml")

*.*.hasStatus = true
*.scenarioManager.script = xml("<scenario> \
        <disconnect t='60' src-module='R1' dest-module='R2
↳' /> \
        <disconnect t='70' src-module='R1' dest-module='R3
↳' /> \
        </scenario>")

*.R1.numApps = 1
*.R1.app[0].typename = "PingApp"
*.R1.app[0].destAddr = "1.1.1.1"
*.R1.app[0].startTime = 0s

```

The OSPF configuration:

```

<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
↳xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.1">
    <AddressRange address="R3>R6" mask="R3>R6" />
    <AddressRange address="R6>R3" mask="R6>R3" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" />
    <PointToPointInterface ifName="ppp1" />
    <PointToPointInterface ifName="ppp2" />
    <PointToPointInterface ifName="ppp3" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" />
    <BroadcastInterface ifName="eth0" areaID="0.0.0.0" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <PointToPointInterface ifName="ppp0" />
    <PointToPointInterface ifName="ppp1" areaID="0.0.0.1" />

```

(continues on next page)

(continued from previous page)

```

</Router>

<Router name="R4" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" />
  <ExternalInterface ifName="eth0" advertisedExternalNetworkAddress="1.1.1.1"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="10"
↪externalInterfaceOutputType="Type1" />
</Router>

<Router name="R5" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" />
  <ExternalInterface ifName="eth0" advertisedExternalNetworkAddress="1.1.1.1"
↪advertisedExternalNetworkMask="255.255.255.0" externalInterfaceOutputCost="2"
↪externalInterfaceOutputType="Type2" />
</Router>

<Router name="R6" RFC1583Compatible="true">
  <PointToPointInterface ifName="ppp0" areaID="0.0.0.1" />
</Router>

</OSPFASConfig>

```

Results

The simulation demonstrates route selection hierarchy:

1. Multiple routers advertise the same destination (1.1.1.0/24) using different methods:
 - R2: Intra-area (RouterLsa), cost 1
 - R3: Inter-area (SummaryLsa), cost 2
 - R4: External Type-1 (NetworkLsa), cost 10
 - R5: External Type-2 (NetworkLsa), cost 1
2. At t=60s, the link between R1 and R2 breaks, changing available paths.
3. At t=70s, another link breaks, further changing available paths.
4. Throughout these changes, OSPF consistently applies its route preference hierarchy.
5. Intra-area routes are always preferred over inter-area routes, even if the inter-area route has lower cost.
6. Similarly, inter-area routes are preferred over external routes.

The routing tables clearly show OSPF selecting routes based on type hierarchy, not just cost, demonstrating the protocol's design for stability and hierarchical routing.

Sources: omnetpp.ini, OSPF_Route_Selection.ned, ASConfig_Route_Selection.xml

7.36.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.37 Step 24. OSPF Path Selection - Suboptimal routes

7.37.1 Goals

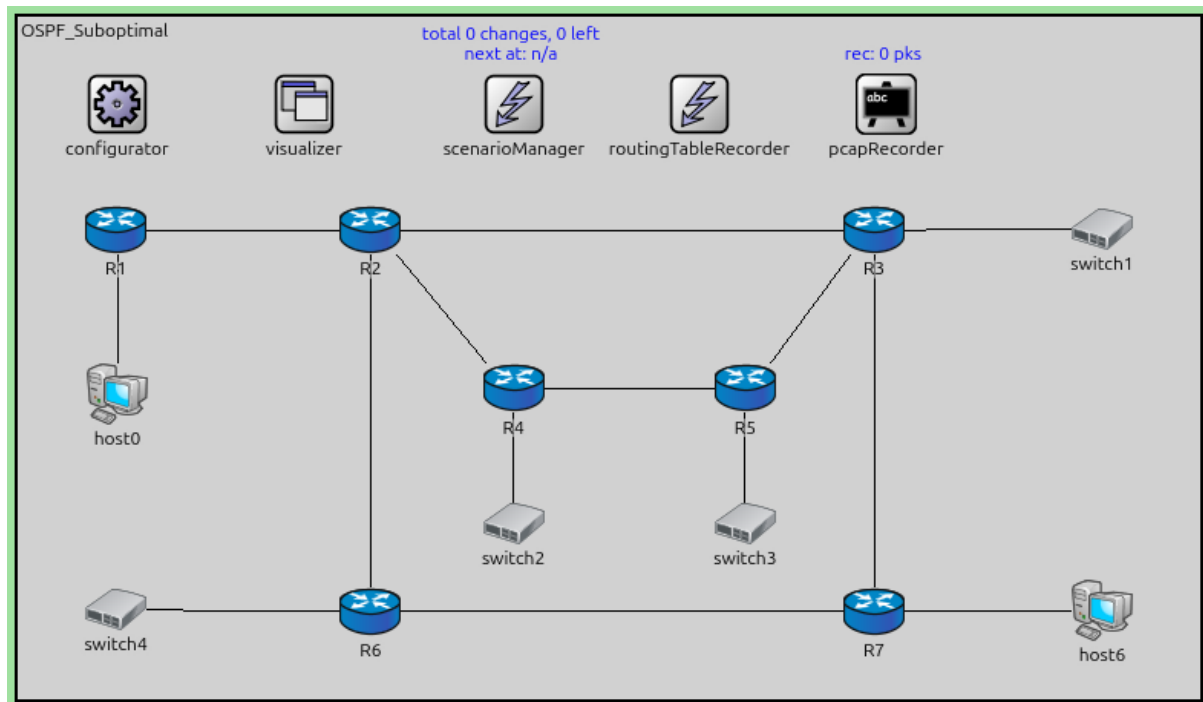
The goal of this step is to demonstrate scenarios where OSPF's route selection hierarchy can lead to suboptimal routing.

Because OSPF always prefers intra-area routes over inter-area routes (regardless of cost), there are situations where the protocol selects a longer path within an area instead of a shorter path that crosses area boundaries.

This is an intentional design trade-off: OSPF prioritizes routing stability and hierarchical structure over always finding the absolute shortest path.

Configuration

This step uses a topology where an intra-area path is longer than an available inter-area path.



The configuration in `omnetpp.ini` is the following:

```
[Config Step24]
description = "OSPF Path Selection - Suboptimal routes"
network = OSPF_Suboptimal

# host0 pings host6 and the following route is used by the OSPF that is not optimal:
# R1--> R2 --> R4 --> R5 --> R3 --> R7 --> 10.0.0.52
# this is because intra-area routes are always preferred by inter-area routes even if
# they have higher total cost.

*.configurator.config = xml("<config> \
                                <interface hosts='**' address='10.x.x.x' netmask='255.\
->x.x.x' /> \
                                <route hosts='host*' destination='*' netmask='0.0.0.0\
->' interface='eth0' /> \
                                </config>")

# application parameters
*.host0.numApps = 1
*.host0.app[0].typename = "PingApp"
*.host0.app[0].destAddr = "host6"
*.host0.app[0].startTime = 60s

*.R*.ospf.ospfConfig = xmldoc("ASConfig_Suboptimal.xml")
```


The OSPF configuration:

```
<?xml version="1.0"?>
<OSPFASConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="OSPF.xsd">

  <!-- Areas -->
  <Area id="0.0.0.1">
    <AddressRange address="R2>R4" mask="R2>R4" />
    <AddressRange address="R2>R6" mask="R2>R6" />

    <AddressRange address="R3>R5" mask="R3>R5" />
    <AddressRange address="R3>R7" mask="R3>R7" />

    <AddressRange address="R4>R2" mask="R4>R2" />
    <AddressRange address="R4>R5" mask="R4>R5" />
    <AddressRange address="R4>switch2" mask="R4>switch2" />

    <AddressRange address="R5>R4" mask="R5>R4" />
    <AddressRange address="R5>R3" mask="R5>R3" />
    <AddressRange address="R5>switch3" mask="R5>switch3" />

    <AddressRange address="R6>R2" mask="R6>R2" />
    <AddressRange address="R6>R7" mask="R6>R7" />
    <AddressRange address="R6>switch4" mask="R6>switch4" />

    <AddressRange address="R7>R6" mask="R7>R6" />
    <AddressRange address="R7>R3" mask="R7>R3" />
    <AddressRange address="R7>host6" mask="R7>host6" />
  </Area>

  <!-- Routers -->
  <Router name="R1" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.0" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.0" interfaceMode="Passive" />
  </Router>

  <Router name="R2" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.0" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.0" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.1" interfaceOutputCost="10" />
    <BroadcastInterface ifName="eth3" areaID="0.0.0.1" />
  </Router>

  <Router name="R3" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.0" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth3" areaID="0.0.0.0" interfaceMode="Passive" />
  </Router>

  <Router name="R4" RFC1583Compatible="true">
    <BroadcastInterface ifName="eth0" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
    <BroadcastInterface ifName="eth2" areaID="0.0.0.1" interfaceMode="Passive" />
  </Router>

  <Router name="R5" RFC1583Compatible="true">
```

(continues on next page)

(continued from previous page)

```

<BroadcastInterface ifName="eth0" areaID="0.0.0.1" />
<BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
<BroadcastInterface ifName="eth2" areaID="0.0.0.1" interfaceMode="Passive" />
</Router>

<Router name="R6" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" interfaceMode="Passive" />
</Router>

<Router name="R7" RFC1583Compatible="true">
  <BroadcastInterface ifName="eth0" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth1" areaID="0.0.0.1" />
  <BroadcastInterface ifName="eth2" areaID="0.0.0.1" interfaceMode="Passive" />
</Router>

</OSPFASConfig>

```

Results

The simulation demonstrates suboptimal routing:

1. host0 pings host6.
2. An inter-area path R1 → R7 exists with low cost.
3. However, an intra-area path R1 → R2 → R4 → R5 → R3 → R7 also exists.
4. OSPF selects the longer intra-area path because intra-area routes are always preferred over inter-area routes.
5. The traffic follows the suboptimal path: R1 → R2 → R4 → R5 → R3 → R7 → 10.0.0.52

This demonstrates that OSPF's route preference hierarchy can lead to suboptimal paths in certain topologies, which is an acceptable trade-off for the benefits of hierarchical routing (scalability, stability, summarization).

Sources: `omnetpp.ini`, `OSPF_Suboptimal.ned`, `ASConfig_Suboptimal.xml`

7.37.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.38 Step 25. PCAP recording

7.38.1 Goals

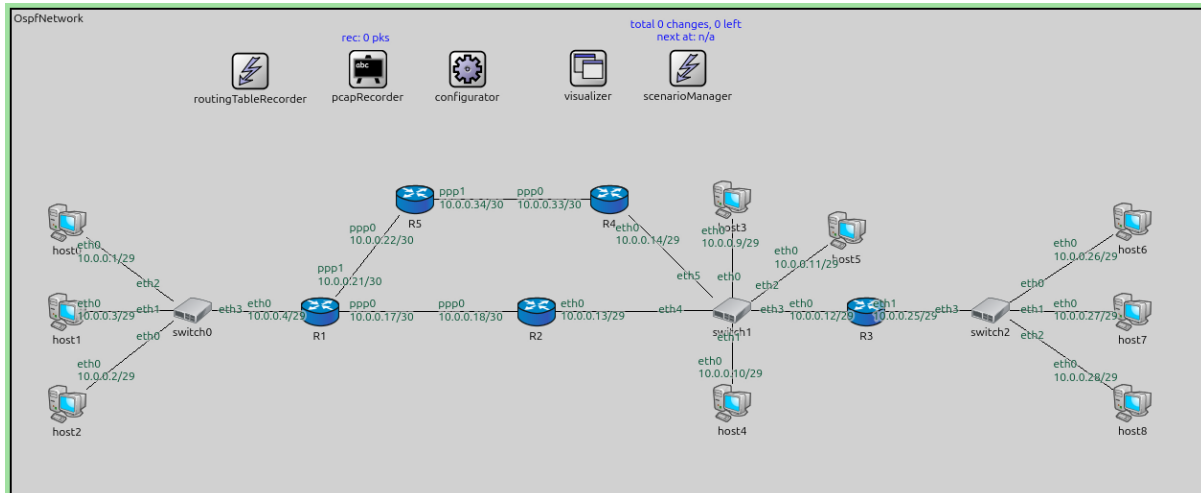
The goal of this step is to demonstrate PCAP recording capabilities for capturing OSPF traffic.

INET's PCAP recorder module can capture network traffic at specific interfaces and save it in PCAP format, which can later be analyzed with tools like Wireshark. This is useful for:

- Debugging OSPF protocol issues
- Understanding packet-level protocol behavior
- Verifying LSA contents and exchanges
- Analyzing timing and sequencing of OSPF messages

Configuration

This configuration is based on Step 1. PCAP recorders are configured on specific interfaces of selected routers.



The configuration in `omnetpp.ini` is the following:

```
[Config Step25]
description = "PCAP recording"
extends = Step1

**.crcMode = "computed"
**.fcsMode = "computed"

*.R1.numPcapRecorders = 1
*.R1.pcapRecorder[0].moduleNamePatterns = "ppp[0]"
*.R1.pcapRecorder[0].pcapNetwork = 204 # ppp
*.R1.pcapRecorder[0].pcapFile = "results/R1.ppp0.pcap"

*.R5.numPcapRecorders = 1
*.R5.pcapRecorder[0].moduleNamePatterns = "ppp[1]"
*.R5.pcapRecorder[0].pcapNetwork = 204 # ppp
*.R5.pcapRecorder[0].pcapFile = "results/R5.ppp1.pcap"

*.R4.numPcapRecorders = 1
*.R4.pcapRecorder[0].moduleNamePatterns = "eth[0]"
*.R4.pcapRecorder[0].pcapNetwork = 1 # ethernet
*.R4.pcapRecorder[0].pcapFile = "results/R4.eth0.pcap"
```

Results

With PCAP recording enabled:

1. PCAP recorder modules capture all traffic on the specified interfaces.
2. Separate PCAP files are created for each configured interface:
 - *R1.ppp0.pcap* - Traffic on R1's ppp0 interface
 - *R5.ppp1.pcap* - Traffic on R5's ppp1 interface
 - *R4.eth0.pcap* - Traffic on R4's eth0 interface
3. These PCAP files can be opened in Wireshark to examine:
 - OSPF Hello packets

- Database Description (DBD) packets
 - Link State Request/Update/Acknowledgment packets
 - LSA contents (Router LSAs, Network LSAs, Summary LSAs, etc.)
4. The PCAP files show the timing and sequencing of OSPF protocol messages, useful for understanding adjacency formation, LSDB synchronization, and LSA flooding.

PCAP recording is a powerful tool for learning OSPF protocol details and troubleshooting complex scenarios.

Sources: `omnetpp.ini`, `OspfNetwork.ned`

7.38.2 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.

7.39 Conclusion

Congratulations, you've made it! You should now be able to set up OSPF routing for networks in the INET Framework. Use the link below if you have questions or you'd like to help us improve this tutorial.

7.39.1 Discussion

Use [this page](#) in the GitHub issue tracker for commenting on this tutorial.